

SILAB[®]

Version 7.1

Referenz Verkehssimulation
TRFX

INHALTSVERZEICHNIS

INHALTSVERZEICHNIS	2
1 ÜBERBLICK.....	5
1.1 Generelle Funktionsweise	5
1.2 Allgemeine Konzepte	6
1.2.1 Programmmodule	6
1.2.2 Modulgrenzen, globaler und lokaler Verkehr	6
1.2.3 Course und Area2 Streckenabschnitte	7
1.2.4 Fahrspuren	8
1.2.5 Verhaltensmodelle	9
1.2.5.1 Überblick	9
1.2.5.2 Behaviour Schemes	10
1.2.5.3 Parameter von Verhaltensmodellen	10
1.3 Verkehrsdefinition im Skript	10
2 MODULGEBUNDENER VERKEHR	14
2.1 Prinzipielle Vorgehensweisen	14
2.2 Generelle Verkehrsdefinitionen	14
2.3 Verkehrsteilnehmer	17
2.3.1 Course Streckenabschnitte	17
2.3.2 Area2 Streckenabschnitte	19
2.4 Verhaltenssteuerung	25
2.4.1 Course Streckenabschnitte	25
2.4.1.1 Streckenbezogene Flowpoints	26
2.4.1.2 Bedingte Flowpoints	29
2.4.2 Area2 Streckenabschnitte	30
2.5 Verkehrsflüsse	40
2.5.1 Traffic Flow in Course Streckenabschnitten	40
2.5.2 Quellen in Courses und Areas	44
2.6 Ampelanlagen	51
2.7 Beispiele	54
2.7.1 Vorausfahrendes Fahrzeug auf einem Course Streckenabschnitt	54
2.7.2 Plötzliches Bremsen eines vorausfahrenden Fahrzeugs	55
2.7.3 Kopplung eines vorausfahrenden Fahrzeugs	56
2.7.4 Stau	57
2.7.5 Vorausfahrendes und abbiegendes Fahrzeug in einer Area2	65

3 VERKEHRSDEFINITION IN MODULEN MIT GEMISCHTEN COURSE UND AREA2 STRECKENABSCHNITTEN	69
4 MODULÜBERGREIFENDER VERKEHR.....	77
4.1 Generelle Verkehrsdefinitionen	77
4.2 Course Streckenabschnitte	78
4.2.1 GlobalConstVehicleX	78
4.2.2 GlobalX	78
4.2.3 GlobalSourceX	80
4.3 Area2 Streckenabschnitte	83
4.4 Verhaltenssteuerung	87
4.5 Verkehr in Modulübergängen	90
5 REFERENZEN	95
5.1 Generelle Verkehrsdefinitionen, SILAB_TRFX_600.cfg	95
5.1.1 Vordefinierte Behaviour Schemes	95
5.1.2 Witterungsabhängige Geschwindigkeitsprofile	96
5.1.3 Ampelprogramme	97
5.1.3.1 Vordefinierte Programme	97
5.1.3.1.1 Programm 1: schnelle Schaltzeiten ohne Fußgängerampel	97
5.1.3.1.2 Programm 2: schnelle Schaltzeiten mit Fußgängerampel	98
5.1.3.1.3 Programm 3: nicht weiterschaltendes Programm mit Fußgängerampel	99
5.1.3.1.4 Programm 4: langsame Schaltzeiten mit Fußgängerampel	100
5.1.3.1.5 Programm 5: gleichlange Schaltzeiten mit Fußgängerampel	101
5.1.3.1.6 Programm 6: langsame Schaltzeiten mit Fußgängerampel	102
5.1.3.1.7 Programm 7: Fußgängerampel für Überwege	103
5.1.3.2 Benutzerdefinierte Programme	104
5.1.4 RoadUserTable	106
5.1.4.1 RoadUserTable aus „SILAB_TRFX_600.cfg“	106
5.1.4.2 ColourTable aus „SILAB_TRFX_600.cfg“	114
5.1.4.3 Benutzerdefinierte RoadUserTable	117
5.2 Grundlegendes Verhalten von Verkehrsteilnehmern	119
5.3 Verhaltensmodelle	123
5.3.1 Überblick	123
5.3.2 Definition von Verhaltensmodellen für Verkehrsteilnehmer	128
5.3.3 Allgemeine Parameter der Verhaltensmodelle	129
5.3.3.1 Parameter zur Längsgeschwindigkeit	130
5.3.3.2 Parameter zur Abstandsregelung	130
Parameter zum lateralen Abstandsverhalten	130
5.3.3.3 Parameter zur streckenabhängigen Geschwindigkeitsanpassung	131
5.3.3.4 Faktoren für die maximale Geschwindigkeit (siehe 5.1.2) in Abhängigkeit von Umweltparameter	131
5.3.3.5 Parameter zur Gassenbildung	131
5.3.3.6 Verhaltensmodellunabhängiger Parameter zur Verkehrsdichtenregelung	133
5.3.4 Setzen von Defaultparametern für Verhaltensmodelle	133

5.3.5	Verhalten der Verkehrsteilnehmer	136
5.3.5.1	Verhaltensmodell: EXSTD1	136
5.3.5.2	Verhaltensmodell: EXSTD2	139
5.3.5.3	Verhaltensmodell: EXSTD3	140
5.3.5.4	Verhaltensmodell: EXSTDEMG	143
5.3.5.5	Verhaltensmodell: EXSTDGlobalS	144
5.3.5.6	Verhaltensmodell: LaneCrossing	145
5.3.6	Verhalten der Verkehrsteilnehmer speziell in Course-Instanzen	146
5.3.6.1	Verhaltensmodell: HazardAvoidance2	146
5.3.6.2	Verhaltensmodell: LaneChangeAvoidance	147
5.3.6.3	Verhaltensmodell: SignalSpeed	149
5.3.6.4	Verhaltensmodell: SimCarLocked	152
5.3.6.5	Verhaltensmodell: SimCarLocked2	153
5.3.6.6	Verhaltensmodell: CommandAction	155
5.3.6.7	Verhaltensmodell: EXWDM	160
5.3.6.8	Verhaltensmodell: EXIDM	167
5.3.6.9	Verhaltensmodell: HazardFree	169
5.3.7	Obsolete Verhaltensmodelle	170
5.3.7.1	Verhaltensmodell: HazardAvoidance	171
5.3.7.2	Verhaltensmodell: FreeDriving	171
5.4	Generierung von Messwerten	173

1 ÜBERBLICK

1.1 Generelle Funktionsweise

Die Verkehrssimulation TRFX von SILAB bietet Ihnen die Möglichkeit, Verkehrsteilnehmer (Fahrzeuge, Motorräder und Fahrräder) in ein Streckennetz einzufügen und diese einzeln oder in Gruppen in Abhängigkeit vom Verhalten des Testfahrers oder vom Verhalten anderer Verkehrsteilnehmer zu steuern. Sie können Messwerte (Position, Geschwindigkeit, Beschleunigung usw.) jedes einzelnen Verkehrsteilnehmers aufzeichnen oder zur weiteren Verwendung mit anderen DPUs verknüpfen.

Einem Verkehrsteilnehmer kann eines von zahlreichen Fahrzeugen zugewiesen werden. Im Lieferumfang von SILAB sind als Fahrzeuge verschiedene Modelle von PKWs, Kleintransporter und LKWs in mehreren Farben sowie Motorräder und Fahrräder enthalten.

Die simulierten Verkehrsteilnehmer von TRFX fahren auf den Streckennetzen, die mit der SILAB Datenbasis SCN/SCNX entworfen wurden (s. „Referenz Datenbasis SCNX“). Sie nutzen u.a. die in SCNX definierten Spurinformatoren und Verkehrsregeln, um sich auf den Straßen zu bewegen.

In TRFX haben Sie die Möglichkeit, sowohl einzelne Verkehrsteilnehmer als auch ganze Verkehrsflüsse zu definieren. Die Verkehrsflüsse nutzen sog. Quellen, um Verkehrsteilnehmer nach bestimmten Regeln im Straßennetz zu platzieren und Senken, um die Verkehrsteilnehmer wieder zu deaktivieren. Quellen und Senken können sowohl statisch platziert als auch mit dem EGO-Fahrzeug bewegt werden.

Gesteuert werden die Verkehrsteilnehmer über sog. Verhaltensmodelle, die individuell zu sog. Behaviour Schemes kombiniert werden können. Auf das Verhalten eines Verkehrsteilnehmers können Sie über sog. Flowpoints Einfluss nehmen. Ähnlich wie bei den Events („Hedgehogs“) in SCNX handelt es sich dabei zum einen um unsichtbare Punkte auf Fahrspuren, bei deren Überfahren Parameter und Unterprogramme der Verhaltensmodelle verändert werden. Zum anderen gibt es Flowpoints, die beim Eintreten einer zusätzlichen Bedingung auslösen (z.B. mit einer bestimmten zeitlichen Verzögerung). Wahlweise können Flowpoints vom EGO-Fahrzeug oder von anderen Verkehrsteilnehmern überfahren und ausgelöst werden.

Lichtzeichenanlagen nehmen eine besondere Stellung in der Datenbasis ein. Während der statische Anteil einer Ampel in SCNX definiert wird (Position, Aussehen etc.), übernimmt TRFX die Steuerung der Ampelanlage für den Verkehr und auch für die Fußgänger des Moduls MOV.

1.2 Allgemeine Konzepte

1.2.1 Programmmodule

Aus Sicht der DPU-Konfiguration besteht die Verkehrssimulation TRFX von SILAB hauptsächlich aus den Programmmodulen DPUTRFX, DPUTRFClient und DPUTRFSGE. Alle drei DPUs benötigen die Ausführung der DPUSCNX auf dem Computer, auf dem sie selbst berechnet werden.

Die DPUTRFX positioniert und bewegt die Verkehrsteilnehmer im Streckennetz basierend auf den jeweiligen Verhaltensmodellen. Die DPUTRFClient verbindet sich über TCP/IP mit der DPUTRFX und synchronisiert die Positionen der Verkehrsteilnehmer über das Netz. Die Visualisierung (DPUTRFSGE) greift auf die von der DPUTRFClient bzw. DPUTRFX berechneten Positionen zu und visualisiert die Fahrzeuge mit der Grafikengine SGE. Die beschriebene DPU-Konfiguration ist Bestandteil der Standard-Konfigurationsarchitektur, die sich im Lieferumfang von SILAB befindet (s. „Referenz SILAB-Konfiguration und Bedienung.pdf“). Normalerweise muss sie nicht verändert werden.

Zur Generierung von Messwerten für die simulierten Verkehrsteilnehmer (z.B. für die Datenaufzeichnung mit der DPUDataFile oder zur Simulation von Sensoren für Assistenzsysteme) verwenden Sie die DPUs DPUTRFDDataSingle, DPUTRFTarget, DPUTRFDDataFull und DPUTRFDDataSmall. Diese DPUs liefern Messwerte ausgewählter Verkehrsteilnehmer bzw. Messwerte von Verkehrsteilnehmer im Umfeld des EGO-Fahrzeugs. Darauf wird detailliert in Abschnitt 5.4 eingegangen.

1.2.2 Modulgrenzen, globaler und lokaler Verkehr

Der simulierte Verkehr setzt sich aus zwei grundsätzlich unterschiedlichen Arten zusammen. Es gibt den modulgebundenen (auch „lokalen“) Verkehr (s. Abschnitt 2), der innerhalb eines Szenario-Moduls von SCNX definiert wird und auch nur dann aktiv ist, wenn sich das EGO-Fahrzeug in diesem Modul befindet. Fährt das EGO-Fahrzeug aus dem Modul heraus, dann werden die zu diesem Modul gehörigen modulgebundenen Verkehrsteilnehmer deaktiviert. Dies hat den Vorteil, dass der Fahrer in jedem Modul in eine klar definierte Verkehrssituation gerät. Der Ablauf eines Szenarios ist damit nicht abhängig von Verkehrssituationen in vorher durchfahrenen Szenario-Modulen, denn Verkehrsteilnehmer aus vorhergehenden Szenario-Modulen können keinen Einfluss auf zukünftige Situationen nehmen.

Daneben gibt es den sogenannten modulübergreifenden (auch „globalen“) Verkehr, der den Zweck erfüllt, Fahrzeuge zu erzeugen, die unabhängig von Modulgrenzen das EGO-Fahrzeug umgeben und mit ihm interagieren (siehe Abschnitt 4). Auch wenn das EGO-Fahrzeug die Grenze eines Moduls überfährt, bleiben modulübergreifende Verkehrsteilnehmer aktiv. Sie können den Fahrer somit über mehrere Module hinweg begleiten und mit ihm interagieren.

Obwohl es bzgl. der Reproduzierbarkeit von Verkehrssituationen in Szenario-Modulen sehr vorteilhaft ist, ausschließlich mit modulgebundenem Verkehr zu arbeiten, hat dies jedoch den Nachteil, dass Sie den modulgebundene Verkehr bei jedem Modul-

anfang neu aufbauen müssen. Am Ende des Moduls müssen Sie die lokalen Verkehrsteilnehmer stets wieder unauffällig verschwinden lassen (wenn Sie das nicht selbst machen, dann verschwinden die Verkehrsteilnehmer einfach, sobald der Fahrer die Modulgrenze überfährt). Um zu verhindern, dass Fahrzeuge plötzlich im Sichtbereich des Fahrers auftauchen bzw. innerhalb des Sichtbereichs plötzlich verschwinden, werden Sie üblicherweise bestimmte Wegstrecken und Sichtverdeckungen am Anfang und am Ende des Moduls einfügen (z.B. Kurven mit hoher Randbebauung, Kuppen). Dadurch entsteht jedoch zwangsläufig eine Lücke in der Verkehrssimulation, in der sich der Fahrer ohne umgebenden Verkehr bewegt. Eine sinnvolle Verwendung des modulübergreifenden Verkehrs ist, diese Lücke zwischen den Modulen zu schließen. Diese „Lückenfüller“-Verkehrsteilnehmer gehören zum sog. „Ausschmückungsverkehr“. Der Sachverhalt ist in der folgenden Grafik veranschaulicht.

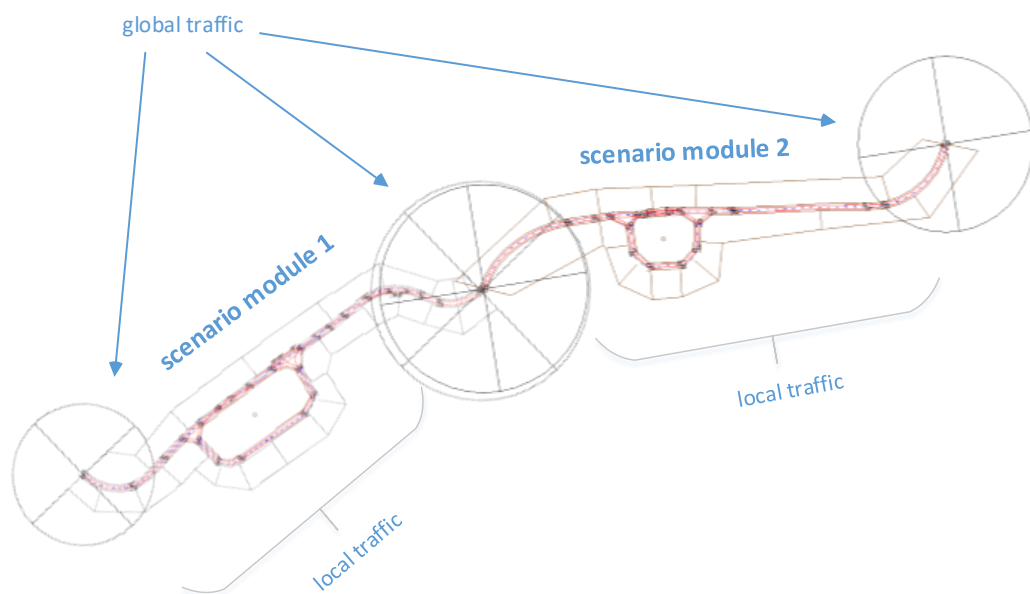


Abbildung 1: Innerhalb der beiden Szenario-Module wird lokaler Verkehr verwendet. Die Übergänge werden mit globalen Verkehrsteilnehmern ausgeschmückt. Die Kreise an den Szenario-Ein- und Ausgängen repräsentieren Sichtbarkeitsbereiche.

Um zu verhindern, dass Verkehrsteilnehmer des Ausschmückungsverkehrs die Verkehrsszenarien in nachfolgenden Modulen stören, gibt es in TRFX Mechanismen, mit denen sie unauffällig entfernt werden können (z.B. durch Einparken oder Abbiegen, s. Abschnitt [□](#)).

1.2.3 Course und Area2 Streckenabschnitte

Bei der Definition von Straßennetzwerken mit SCNX haben Sie bereits die Unterscheidung zwischen Course und Area2 Streckenabschnitten kennen gelernt: Streckenabschnitte vom Typ

- **Course** modellieren Straßen ohne Abzweigungen mit einheitlichem Querschnittsprofil. Sie werden mittels einer Skript-Sprache definiert.
- **Area2** modellieren Querschnittsveränderungen (z.B. Spurverbreiterungen, Autobahnauf- und abfahrten), Kreuzungen und ganze Stadtteile. Zur Modellierung verwenden Sie den grafischen Editor SILABAEEdit.

Indem Sie Course und Area2 Streckenabschnitte mit Hilfe der Skriptsprache verknüpfen, können Sie Szenario-Module konstruieren, deren Straßennetzwerk aus einer Mischung beider Typen besteht (z.B. Landstraße mit Kreuzung).

Das Konzept überträgt sich auf die Verkehrsdefinition: Auf Course Streckenabschnitten werden Verkehrsteilnehmer und deren Verhalten mittels der Skriptsprache definiert. In Area2 Streckenabschnitten verwenden Sie hierzu SILABAEEdit. Bei Szenario-Modulen mit einer Mischung aus Course und Area2 Streckenabschnitten, können Sie die Verkehrsteilnehmer folgendermaßen steuern:

- Aufsetzen eines Verkehrsteilnehmers und Steuerung innerhalb einer Area2: SILABAEEdit
- Aufsetzen eines Verkehrsteilnehmers und Steuerung innerhalb von Course Streckenabschnitten: Skriptsprache
- Steuerung eines Verkehrsteilnehmers, der in einer Area2 erzeugt wurde, in Course Streckenabschnitten: Skriptsprache
- Steuerung eines Verkehrsteilnehmers, der in einem Course erzeugt wurde, in einer Area2: SILABAEEdit

1.2.4 Fahrspuren

Alle Verkehrsteilnehmer bewegen sich entlang der in SCNX definierten Spuren. In Course Streckenabschnitten handelt es sich um die Spuren, die Sie im Querschnittsprofil des Course festgelegt haben. Die Spuren von Area2 Streckenabschnitten werden in SILABAEEdit direkt angezeigt. In solchen Streckenabschnitten kommt es vor, dass eine Spur sich in mehrere verzweigt (z.B. bei Abbiegespuren in Kreuzungen). Die Verkehrsteilnehmer wählen dann die Spur, die ihrer gewünschten Fahrtrichtung entspricht. Die Fahrtrichtung einer Spur wird in SILABAEEdit als Eigenschaft der Spur festgelegt („Heading“ einer Spur). Der Richtungswunsch eines Verkehrsteilnehmers kann über Flowpoints definiert werden.

Die Bewegung der Verkehrsteilnehmer erfolgt mittels eines Schleppkurven-Modells. Ähnlich wie bei einer Slotracing-Bahn wird die Mitte der Vorderachse entlang der Spur geführt. Der hintere Teil des Fahrzeugs wird „nachgeschleppt“. Mittels eines geeigneten Befehls können die Verkehrsteilnehmer auch Spurwechsel durchführen. Bei der Bewegung hierfür kommt ebenfalls das Schleppkurvenmodell zum Einsatz, wie folgende Abbildung verdeutlicht.

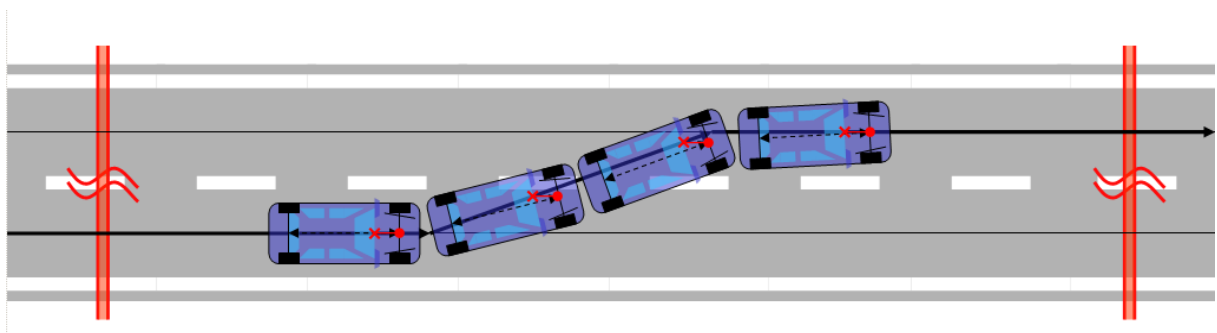


Abbildung 2: Spurwechsel mit Schleppkurven-Modell

Außerdem bewirkt die Schleppkurve, dass die Fahrzeuge leicht die Kurven schneiden bzw. bei längeren Fahrzeugen (LKW's), dass sie entsprechend weit ausholen, um in eine Kurve einzufahren.

1.2.5 Verhaltensmodelle

1.2.5.1 Überblick

Anhand eines Beispiels wird die Wirkungsweise der Verhaltensmodelle verdeutlicht:

Ein Verkehrsteilnehmer soll ein vorgegebenes Geschwindigkeitsprofil einnehmen, wenn es durch keinen anderen Verkehrsteilnehmer, sei es aus dem umliegenden Verkehr oder dem EGO Fahrzeug, ausgebremst wird. Spurwechsel werden dabei nicht unterstützt.

Dieses Verhalten kann in zwei grundlegende Aufgaben unterteilt werden:

1. Geschwindigkeitsprofil an den Verkehrsteilnehmer senden
2. Bremsen, falls der Abstand zum vorausfahrenden Verkehrsteilnehmer zu gering wird

Die Realisierung in TRFX erfolgt über zwei unterschiedliche Verhaltensmodelle:

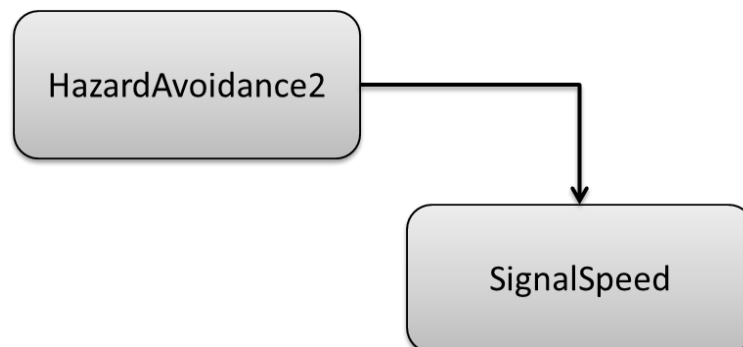


Abbildung 3 Hierarchie von zwei Verhaltensmodellen

Das Verhaltensmodell „HazardAvoidance2“ bremst den Verkehrsteilnehmer ab, falls ein bestimmter Sekundenabstand zum vorausfahrenden Verkehrsteilnehmer unterschritten wird. Das zweite Verhaltensmodell „SignalSpeed“ generiert ein Geschwindigkeitsprofil für den Verkehrsteilnehmer ohne Rücksicht auf Hindernisse oder andere Verkehrsteilnehmer.

Die Verhaltensmodelle, die einem Verkehrsteilnehmer zugewiesen werden, haben eine festgelegte Hierarchie. Ein Modell kann die nachfolgenden blockieren, so dass diese den Verkehrsteilnehmer nicht steuern können. In Abbildung 3 hat das HazardAvoidance2 - Modell die höchste Priorität. Wenn der Abstand zum vorausfahrenden Verkehrsteilnehmer unterschritten wird, schickt das HazardAvoidance2 - Modell eine negative Beschleunigung an das Fahrzeug, um es abzubremsen und blockiert dabei alle nachfolgenden Modelle. Ist der Abstand zum vorausfahrenden Verkehrsteilnehmer wieder groß genug, dann wird die Geschwindigkeit wieder über das SignalSpeed - Modell gesteuert.

Eine Referenz aller in TRFX zur Verfügung stehenden Verhaltensmodelle finden Sie in Abschnitt 5.3.

1.2.5.2 Behaviour Schemes

Die zahlreichen Verhaltensmodelle von TRFX modellieren i.d.R. sehr grundlegende Verhaltensweisen oder eingeschränkte Teilaspekte des Verhaltens von Verkehrsteilnehmern. In den meisten Fällen werden Sie deshalb die Verhaltensmodelle in den gleichen hierarchischen Kombinationen einsetzen (s. Beispiel im vorhergehenden Kapitel). Hierfür bietet TRFX den Mechanismus der sog. Behaviour Schemes: Ein Behaviour Scheme ist genau eine solche hierarchische Kombination von Verhaltensmodellen, zusammen mit bestimmten Voreinstellungen von Parametern der Verhaltensmodelle. Selbstverständlich können Sie diese Parameter im Verlauf von Szenarien verändern.

Im Lieferumfang von SILAB befinden sich vordefinierte Behaviour Schemes, über die Sie in Abschnitt 5.1.1 Näheres erfahren.

1.2.5.3 Parameter von Verhaltensmodellen

Jedes Verhaltensmodell von TRFX bietet zahlreiche Parameter, mit denen Sie seine Funktionsweise detailliert festlegen können. Sie können die Werte für diese Parameter auf drei Arten setzen:

- Wenn Sie Verhaltensmodelle zu einem Behaviour Scheme zusammenfassen, gehört dazu auch die Belegung der Parameter mit Standard-Werten.
- Wenn Sie einen Verkehrsteilnehmer definieren und ihm dabei ein Behaviour Scheme zuweisen, können Sie anschließend die im Behaviour Scheme festgelegten Standard-Werte neu setzen.
- Indem Sie Flowpoints definieren, mit denen die Werte von Parametern neu gesetzt werden.

Die Parameter der Verhaltensmodelle von TRFX werden in den Abschnitten 5.3.4 und 5.3.3 beschrieben.

1.3 Verkehrsdefinition im Skript

Ähnlich wie bei der Definition von Straßennetzwerken für Szenario-Module befindet sich die gesamte Verkehrsdefinition in einem Skript-Abschnitt, den Sie, ergänzt um Computer- und DPU-Konfiguration, zur Durchführung einer Simulation in der Anwendung SILAB laden. Auch Verkehrsdefinitionen, die Sie mit dem grafischen Editor SILABAEEdit erzeugt haben, werden letztendlich als Skript-Abschnitt exportiert und in das „Haupt-Skript“ integriert.

Die folgenden Grammatiken zeigen, an welchen Stellen im Skript Informationen stehen, die bei der Verkehrsdefinition eine Rolle spielen (rot markiert).

Auf der obersten Ebene im Skript finden sich (u.a.) zwei Abschnitte:

- **SILAB TRF { ... };** In diesem Abschnitt werden generelle Verkehrsdefinitionen vorgenommen (z.B. Tabellen von benötigten Fahrzeugen, Behaviour Schemes).
- **SILAB SCN { ... };** Durch diesen Abschnitt wird die Definition von Szenario-Modulen und deren Ablauf (Map) gekapselt.

Grammatik 1 Top Level of Skript

```

SILAB TRF
{
    <general TRF definitions> (→ Grammatik 5 Seite 11)
};
<SCNXSGE configuration (→ "Referenz Datenbasis SCNX")>
SILAB SCN
{
    <SCNX configuration (→ "Referenz Datenbasis SCNX")>
    <database (→ Grammatik 2 Seite 11)>
};

```

Innerhalb der <database> werden die benötigten Querschnittsprofile für Course Streckenabschnitte, die Typen der Szenario-Module und die Karten mit Modulabläufen definiert:

Grammatik 2 Database

```

<database>→
    (1...*)<cross section profile (→ "Referenz Datenbasis SCNX")>
    (1...*)<module type>
    (1...*)<map>
<module type>→
    define Module <name of module type>
    {
        <definition module type (→ Grammatik 3 Seite 11)>
    }; |
    <inclusion of Area2s as modules (→ "Referenz Datenbasis SCNX")>
<map>→
    Map <name of map>
    {
        (1...*)<module instance (→ "Referenz Datenbasis SCNX")>
        <module instance sequence (→ "Referenz Datenbasis SCNX")>
        <setup points (→ "Referenz Datenbasis SCNX")>
    };

```

Die Verkehrsdefinition erfolgt letztendlich in den Modultypen:

Grammatik 3 Defining a Module Type

```

<definition module type> →
  define Module <name of module type>
  {
    ModuleID = <index> ;
    (1...*)<road segment type (→"Referenz Datenbasis SCNX")>
    (1...*)<road segment instance (→"Referenz Datenbasis SCNX")>
    (0..*)<road network (→"Referenz Datenbasis SCNX")>
    (0..*)<events of road segments(→"Referenz Datenbasis SCNX")>
    (0,1)<traffic definition(→ Grammatik 4 Seite 11)>
  };

```

Die <traffic definition> in einem Modultyp ist folgendermaßen aufgebaut (auf die einzelnen Teile wird im Verlauf dieses Handbuchs detailliert eingegangen).

Grammatik 4 Traffic Definition

```

(0...*)<traffic definition> →
  Traffic <any name>
  {
    (0...*)<vehicle(→Grammatik 7 Seite 17)>
    (0...*)<traffic flow (→Grammatik 11Seite 41)>
    (0...*)<global const vehicle(→Grammatik 17 Seite 78)>
    (0...*)<global source(→Grammatik 18 Seite 79)>
    (0...*)<flowpoints(→Grammatik 10 Seite 25)>
  };

```

Folgendes wird in den einzelnen Abschnitten festgelegt:

- <vehicle> Definition von einzelnen modulgebundenen Verkehrsteilnehmern
- <traffic flow> Definition von modulgebundenen Verkehrsfüssen
- <global const vehicle> Definition von einzelnen modulübergreifenden Verkehrsteilnehmern
- <global source> Definition von modulübergreifenden Verkehrsflüssen.
- <flowpoints> öffentliche Flowpoints, die alle Arten von Verkehrsteilnehmer betreffen können

Bitte beachten Sie: Wenn Sie mit SILABAEEdit Verkehr in einer Area2 „XYZ“ definieren, dann wird beim Export als Modul-Typ ein Skript „XYZ.cfg“ erzeugt, das der Syntax von Grammatik 3 folgt. Dieses Skript bindet wiederum folgende Unterskripten ein:

- „XYZ_Area2.cfg“: Enthält den eigentlichen Area2 Streckenabschnitt.
- „XYZ_TRFTable.cfg“: Enthält generelle Verkehrsdefinitionen (<general TRF definitions> gem. Grammatik 5).
- „XYZ_TRF.cfg“: Verkehrsdefinition gemäß Grammatik 4.

Wenn Sie die Area2 nicht als Modultyp exportieren, wird nur das Unterskript XYZ_Area2.cfg und keine Verkehrsdefinition erzeugt.

2 MODULGEBUNDENER VERKEHR

2.1 Prinzipielle Vorgehensweisen

Um einen **einzelnen Verkehrsteilnehmer** des modulgebundenen Verkehrs auf einem **Course Streckenabschnitt** zu definieren, gehen Sie wie folgt vor:

1. Sorgen Sie dafür, dass der Fahrzeugtyp des Verkehrsteilnehmers, den Sie verwenden möchten, in der sog. RoadUserTable definiert wird (s. Abschnitt 5.1.4).
2. Definieren Sie im Skript auf der Ebene des Moduls, das den Course enthält, den Verkehrsteilnehmer. Unter anderem legen Sie dabei dessen Fahrzeugtyp, seine anfängliche Position und sein Behaviour Scheme (s. Abschnitt 2.3.1) fest.
3. Platzieren Sie Flowpoints an den Stellen der Course Streckenabschnitte, an denen Sie das Verhalten des Verkehrsteilnehmers ändern möchten (s. Abschnitt 2.4.1).
4. Deaktivieren Sie den Verkehrsteilnehmer manuell durch einen entsprechenden Flowpoint (s. Abschnitt 2.4.1). Wenn Sie das nicht machen, wird der Verkehrsteilnehmer entfernt, wenn das EGO-Fahrzeug das Modul verlässt.

Um einen **Verkehrsfluss** des modulgebundenen Verkehrs auf einem **Course Streckenabschnitt** zu definieren, gehen Sie wie folgt vor:

1. Sorgen Sie dafür, dass die Fahrzeugtypen, welche die Quelle erzeugen soll, in der sog. RoadUserTable definiert werden (s. Abschnitt 5.1.4).
2. Definieren Sie im Skript auf der Ebene des Moduls, das den Course enthält, den Verkehrsfluss. Platzieren Sie Flowpoints zum Aktivieren und Deaktivieren von Quellen und Senken (s. Abschnitt 2.5.1).
3. Platzieren Sie Flowpoints an den Stellen der Course Streckenabschnitte, an denen Sie das Verhalten des Verkehrsteilnehmers ändern möchten (s. Abschnitt 2.4.1).

Verwenden Sie SILABAEEdit, um einen **einzelnen Verkehrsteilnehmer** des modulgebundenen Verkehrs auf einem **Area2 Streckenabschnitt** zu definieren (s. Abschnitt 2.3.2). Innerhalb der Area2 können Sie mit SILABAEEdit auch Flowpoints definieren, um sein Verhalten zu beeinflussen (s. Abschnitt 2.4.2). Auch zur Definition von Ampeln verwenden Sie SILABAEEdit (s. Abschnitt 2.6).

Um einen **Verkehrsfluss** des modulgebundenen Verkehrs auf einem **Area2 Streckenabschnitt** zu definieren, verwenden Sie SILABAEEdit (s. Abschnitt 2.5.2).

Um modulgebundene Verkehrsteilnehmer in Modulen zu steuern, die aus einer **Mischung von Course und Area2 Streckenabschnitten** bestehen, lesen Sie bitte Abschnitt 3.

2.2 Generelle Verkehrsdefinitionen

Im Abschnitt *<general TRF definitions>* können Sie Einstellungen vornehmen, die den gesamten Verkehr in allen Modultypen Ihres Skripts betreffen. Er ist wie folgt aufgebaut:

Grammatik 5 General Traffic definitions

<general TRF definitions> →

SILAB TRF

{

(0...*)<behaviour schemes(→Grammatik 9, Seite 18)>

(0...*)<default parameters for behaviour models (→Grammatik 25 Seite 133)>

(0...*)<user defined programs for traffic lights (→Grammatik 21 Seite 104)>

(0...*)<weather dependant speed profiles (→Grammatik 20 Seite 96)>

(0...*)<road user table (→Grammatik 23 Seite 117)>

(0...*)<used road users table (→Grammatik 6 Seite 16)>

(0...1)<global vehicles(→Grammatik 16 Seite77)>

};

Folgendes wird in den einzelnen Abschnitten festgelegt:

- <behaviour schemes> : Zusammenfassung von einzelnen Verhaltensmodellen zu Behaviour Schemes, Initialisierungen von Default-Werten für Parameter der Verhaltensmodelle.
- <default parameters for behaviour models> : Alternative Möglichkeit, Default-Werte für Parameter von Verhaltensmodellen festzulegen.
- <user defined programs for traffic lights>: In diesem Abschnitt können Sie eigene Schaltprogramme für Ampelanlagen definieren.
- <weather dependant speed profiles> : Einige Verhaltensmodelle können die Geschwindigkeitswahl von den Witterungseinflüssen abhängig machen. Hierzu können Tabellen für die Maximalgeschwindigkeit in Abhängigkeit der Witterung angegeben werden.
- <road user table> : Tabelle mit Fahrzeugtypen und Gruppen von Fahrzeugen, die in diesem Skript gebraucht werden.
- <used road users table> : Optimierungen für sehr komplexe Szenarien, basierend auf der <road user table>.
- <global vehicles>: Dieser Abschnitt spielt bei der Definition von modulübergreifendem Verkehr eine Rolle. Darauf wird in Abschnitt 4.1 eingegangen.

Im Lieferumfang von SILAB ist die Datei „SILAB_TRFX_600.cfg“, in der Einstellungen für den <general TRF definitions> Abschnitt enthalten sind, mit denen Sie sofort arbeiten können. Definieren Sie hierzu folgendes im Skript:

```
SILAB TRF
{
    include SILAB_TRFX_600.cfg

    # insert your additional definitions here
};
```

Die in früheren Versionen verwendete Datei SILAB_TRFX.cfg wird in der aktuellen SILAB Version durch die Datei SILAB_TRFX_600.cfg ersetzt.

Normalerweise müssen Sie – bis auf eine Ausnahme (s.u.) - im SILAB TRF Abschnitt nichts ergänzen. Die mitgelieferten Definitionen sind für die meisten Verkehrssituationen, die Sie entwerfen werden, ausreichend. Wenn Sie als fortgeschrittener Benutzer dennoch Einstellungen anpassen möchten, dann machen Sie dies an der mit dem Kommentar markierten Stelle in obigem Beispiel. Eine Übersicht der Möglichkeiten sowie der vordefinierten Informationen in SILAB_TRFX_600.cfg finden Sie in Abschnitt 5.1.

Es gibt jedoch eine wichtige Ausnahme, die mit der Leistungsfähigkeit der Verkehrssimulation zu tun hat. Pro Szenario-Modul können zwar theoretisch beliebig viele Fahrzeuge gleichzeitig aktiv sein, praktisch ist die Gesamtzahl jedoch beschränkt, da sonst eine flüssige Wiedergabe der Simulation nicht mehr gewährleistet werden kann. Wie viele Fahrzeuge tatsächlich gleichzeitig aktiviert werden können, hängt von der Komplexität der Fahrzeuge, deren Verhaltensmodelle und der Leistungsfähigkeit der verwendeten Bildgeneratoren ab. Als maximale Obergrenze von gleichzeitig sichtbaren Fahrzeugen (lokale und globale Verkehrsteilnehmer) ist eine Anzahl von 40 Teilnehmern anzusehen. Wenn Sie feststellen, dass die Szenarien, die Sie entworfen haben,

- eine lange Lade- und Startzeit der Simulation verursachen,
- auf den Bildkanälen Ihres Simulators sehr ruckelig dargestellt werden,
- auf dem Rechner, der die DPUTRF berechnet (üblicherweise der Hauptrechner Ihres Simulators), Verzögerungen erzeugen,

dann fügen Sie für jedes Ihrer Szenario-Module einen Abschnitt *<used road users table>* mit folgender Syntax ein:

Grammatik 6 Actually Used Road Users

```

<used road users table > →
    UsedRoadUserTable <any name>
    {
        (0...*)Object = < vehicle type (→Grammatik 23 Seite 117)>;
    };

```

Der Name <any name> spielt keine Rolle, sollte aber sinnvollerweise der des zugehörigen Szenario-Moduls sein. Innerhalb einer **UsedRoadUserTable** führen Sie alle Fahrzeugtypen auf, die im Modul verwendet werden. Als <vehicle type> geben Sie bitte den Fahrzeugtyp an, der in der <road user table> definiert ist. Eine Übersicht der Fahrzeugtypen, die in der im Lieferumfang enthaltenen Datei „SILAB_TRFX_600.cfg“ definiert sind, finden Sie in Abschnitt 5.1.4. Bitte beachten Sie, dass SILABAEEdit beim Export als Modul automatisch eine **UsedRoadUserTable**

für das Szenario erzeugt. Wenn ein Szenario „XYZ“ heißt, denn findet sich die exportierte Tabelle in der Datei „XYZ_TRFTable.cfg“. Diese Datei können Sie einfach mit „include“ einbinden.

Das folgende Beispiel zeigt den SILAB TRF Abschnitt mit solchen Ergänzungen für zwei Szenarien. Das Szenario-Modul „Scenario1“ besteht rein aus Course Streckenabschnitten, in denen insgesamt drei Fahrzeuge verwendet werden. Das Szenario-Modul „Scenario2“ wurde von SILABAEEdit (in das Unterverzeichnis „parts“) exportiert:

```
SILAB TRF
{
    include SILAB_TRFX_600.cfg
    UsedRoadUserTable Scenario1
    {
        Object = BMWI8;
        Object = Car;
        Object = HyundaiI40;
    };
    include parts/Scenario2_TRFTable.cfg
    ...
};
```

Bitte beachten Sie:

- Wenn Sie **UsedRoadUserTables** verwenden wollen, dann müssen Sie sie konsequent für *alle* Szenario-Module erstellen.
- Wenn ein Fahrzeugtyp in einem Szenario-Modul verwendet wird, der nicht in einer **UsedRoadUserTable** aufgeführt wurde, dann wird dafür ein Platzhalterfahrzeug dargestellt. Der Typ des Platzhalterfahrzeuges kann über die default-Zuweisung in der RoadUserTable festgelegt werden. Der Standard ist ein blauer Golf II.

2.3 Verkehrsteilnehmer

2.3.1 Course Streckenabschnitte

In einem Course Streckenabschnitt wird ein Verkehrsteilnehmer folgendermaßen definiert:

Grammatik 7 Vehicle

```
<vehicle>→
    VehicleX <instance name of vehicle>
    {
        Vehicle = <vehicle type>;
        (0,1)UserID = <index>;
        <initial position (→Grammatik 8 Seite 18)>
        <behaviour (→Grammatik 9 Seite 18)>
        (0,1)<flowpoints (→Grammatik 10, Seite 25) >
        (0,1)MergeDefault = <1 ... 10 (→Abschnitt 5.3.4, Seite 133)>
        (0,*)<parameter> = <value>;
    };

```

Der *<instance name of vehicle>* ist ein Name, den Sie frei vergeben dürfen. Beachten Sie, dass er mit einem Buchstaben beginnen muss und keine Leerzeichen/Sonderzeichen außer dem Unterstrich enthalten darf.

Mit **Vehicle** = *<vehicle type>* legen Sie zunächst den Fahrzeugtyp für den Verkehrsteilnehmer fest. Verwenden Sie als *<vehicle type>* dabei den Namen, der in der **RoadUserTable** festgelegt wurde (s. Abschnitt 5.1.4).

Mit **UserID** = *<index>* weisen Sie dem Verkehrsteilnehmer eine (eindeutige) Kennnummer zu. Über diese UserID können Sie sein Verhalten an anderen Stellen der Verkehrsdefinition über öffentliche Flowpoints steuern (s. Kontext **SimCarUserID** und **TrafficUserID** in Grammatik 10).

Die *<initial position>* des Verkehrsteilnehmers wird folgendermaßen definiert:

Grammatik 8 Initial Position

<initial position> →
Position =
 (*<course type instance name>* , *<position along road>* ,
 <lane index>);

Dabei ist

- *<course type instance name>* der Name, den Sie in der Moduldefinition bei der Instanzierung des Course Streckenabschnitts vergeben haben.
- *<position along road>* die vom Beginn der Course Instanz entlang der Straße gemessene anfängliche Position des Verkehrsteilnehmers.
- *<lane index>* die Nummer der Spur. Die Spuren werden, vom Beginn der Course Instanz auf das Ende blickend, von links nach rechts beginnend mit 0 gezählt. Dies entspricht der Zählung bei der Definition von Querschnittsprofilen (s. „Referenz Datenbasis SCNX.pdf“).

Die Verhaltensmodelle, mit denen der Verkehrsteilnehmer gesteuert wird, werden folgendermaßen festgelegt:

Grammatik 9 Behaviour

<behaviour> →
BehaviourScheme = *<name>*; |
 <behaviour models (→Grammatik 24, Seite 128)>
 (0...*)*<tuple parameters of behaviour models (→Kapitel Grammatik 24, Seite 129)>*

Hier gibt es zwei Möglichkeiten:

1. Wählen Sie mit **BehaviourScheme** = <name> eines der Behaviour Schemes, die im Abschnitt mit den generellen Verkehrsdefinitionen konfiguriert wurden. Die vordefinierten Schemes finden Sie in Abschnitt 5.1.1.
2. Definieren Sie die einzelnen grundlegenden Verhaltensmodelle des Verkehrsteilnehmers mit <behaviour models>. Mit dieser Variante lassen sich sehr komplexe Verhaltensweisen von Verkehrsteilnehmern realisieren. Die Details werden in Abschnitt 5.3.2 erläutert. Typische Einsatzbeispiele finden Sie in den Beispielen von 2.7.

Im Abschnitt <parameters of behaviour models> können Sie die Parameter der Verhaltensmodelle, die im Behaviour Scheme enthalten sind, anpassen.

Nach diesen Definitionen ist der Verkehrsteilnehmer noch nicht sichtbar. Er muss erst über einen Flowpoint aktiviert werden. Darauf wird in Abschnitt 2.4 eingegangen. Beispiele für die Definition von Verkehrsteilnehmern finden Sie in Abschnitt 2.7.

Mit **MergeDefault** = <1 ... 10> können Default Parameter für die verwendeten Verhaltensmodelle gesetzt werden (siehe auch Abschnitt 5.3.4).

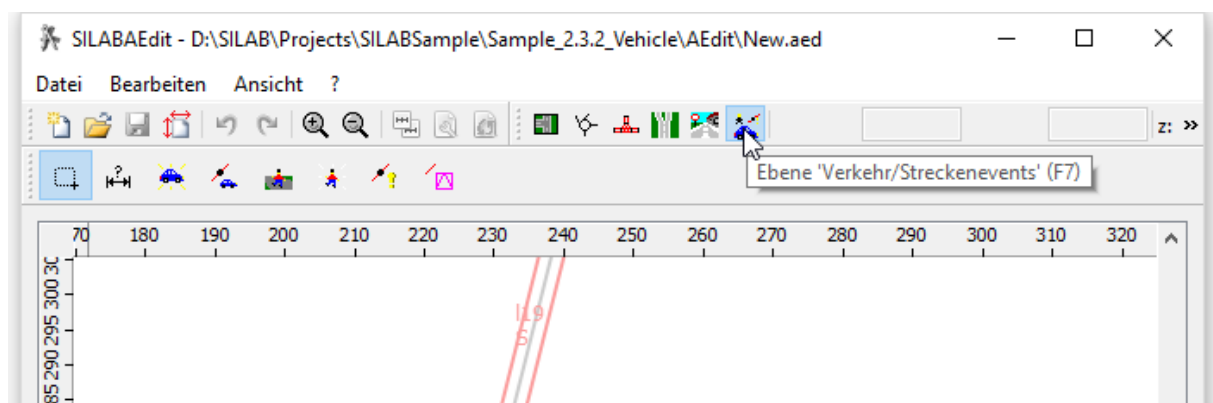
Einzelne Parameter können für das Verhaltensmodell auch direkt gesetzt werden. Die hier möglichen Parameter werden in den einzelnen Modellen bzw. auch in der Übersicht von gemeinsam genutzten Parametern (siehe 5.3.3) beschrieben.

2.3.2 Area2 Streckenabschnitte

Verwenden Sie den grafischen Editor SILABAEEdit, wenn Sie einen Verkehrsteilnehmer in einer Area2 erzeugen möchten. Die Bedienung von SILABAEEdit wird im Dokument „SILABAEEdit.pdf“ detailliert beschrieben. Im Folgenden werden die wesentlichen Schritte zum Einfügen eines Verkehrsteilnehmers anhand eines Beispiels erläutert.

Beispiel zum Einfügen eines Verkehrsteilnehmers:

Zunächst wechseln Sie im Editor SILABAEEdit in den Layer des Verkehrs:

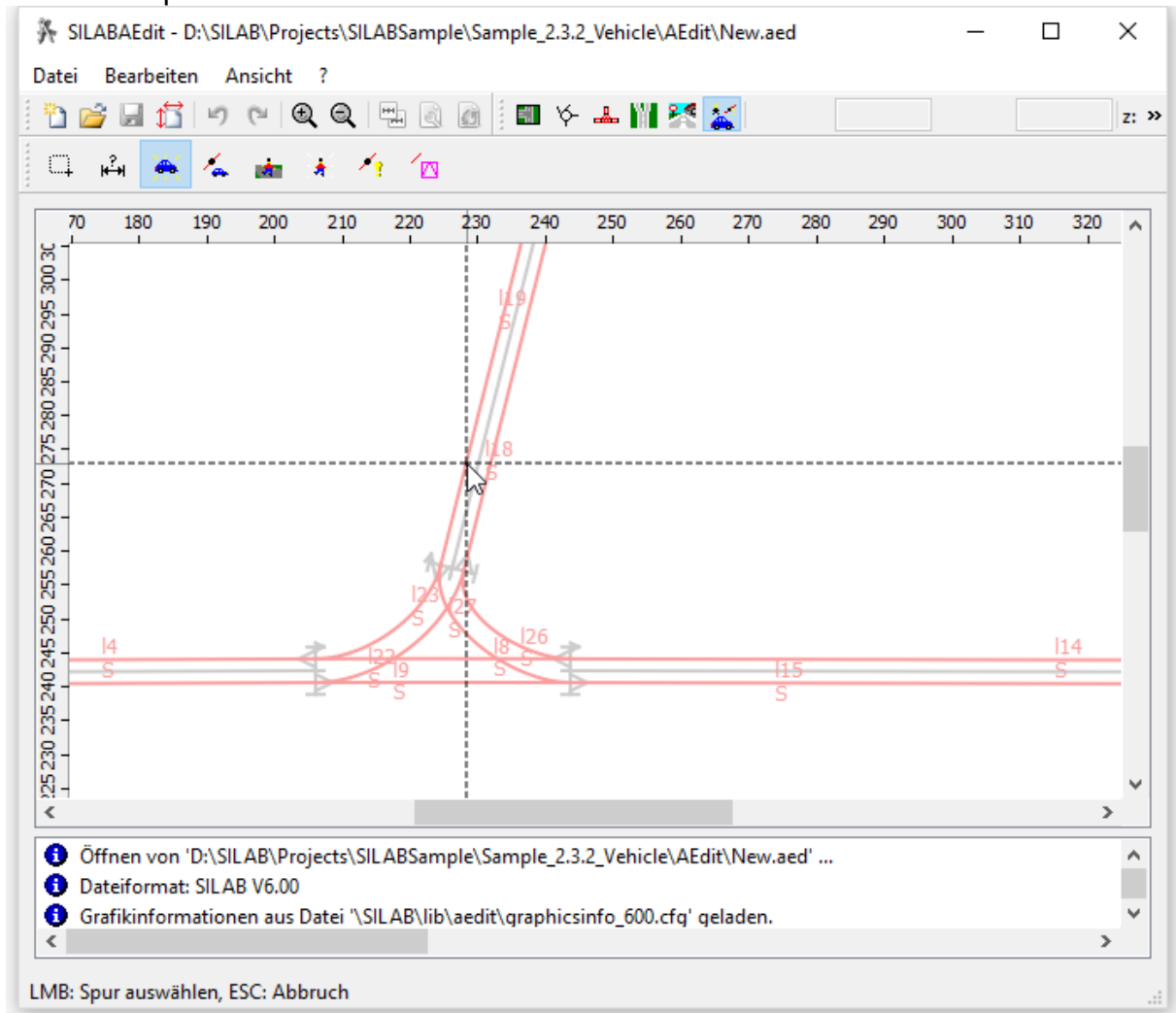


Im Verkehrslayer stehen die Werkzeuge

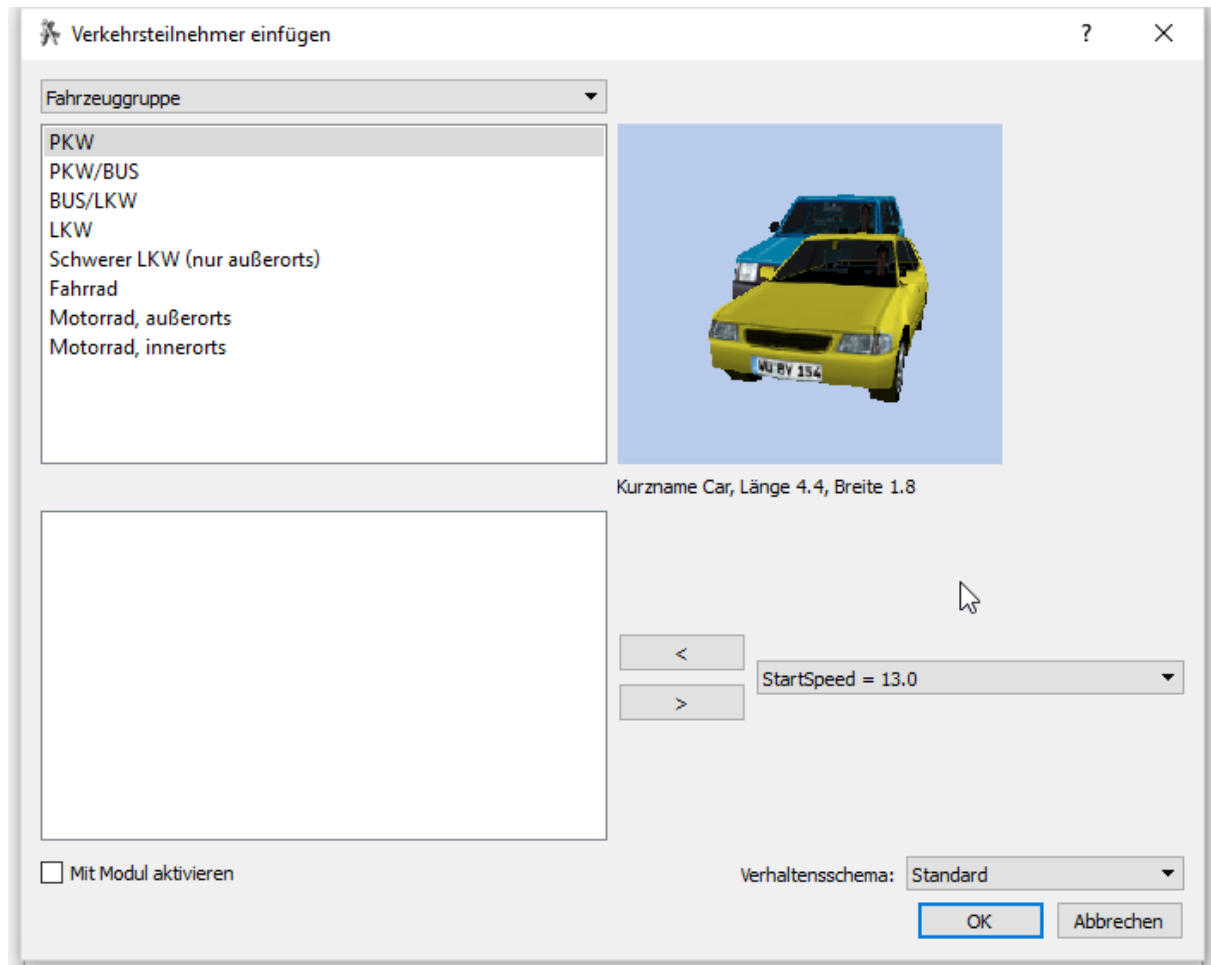
-  Markierungswerkzeug
-  Einfügen eines Verkehrsteilnehmers
-  Einfügen eines Flowpoints

für die Verkehrsdefinition zur Verfügung.

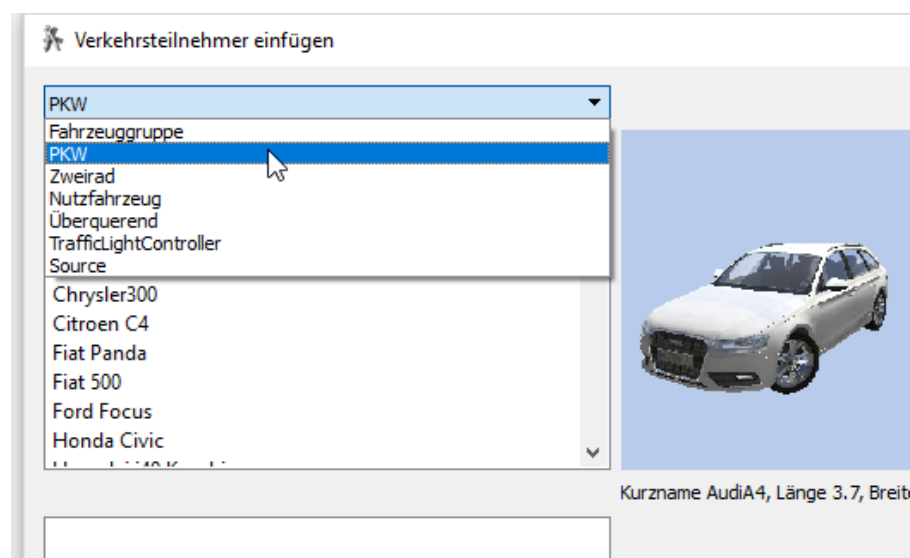
Um einen neuen Verkehrsteilnehmer in eine Area2 einzufügen, wählen Sie aus der Toolbar des Verkehrs das 2. Werkzeug von links und wählen in der Area2 die Spur der Aufsetzposition des Verkehrsteilnehmers mit dem Fadenkreuz aus.



Mit einem linken Mausklick auf eine Spur öffnet sich der Dialog zum Einfügen eines Verkehrsteilnehmers:



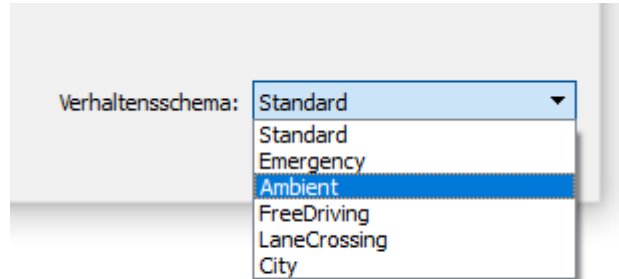
In der oberen Auswahlbox können Sie die Art des Verkehrsteilnehmers bestimmen.



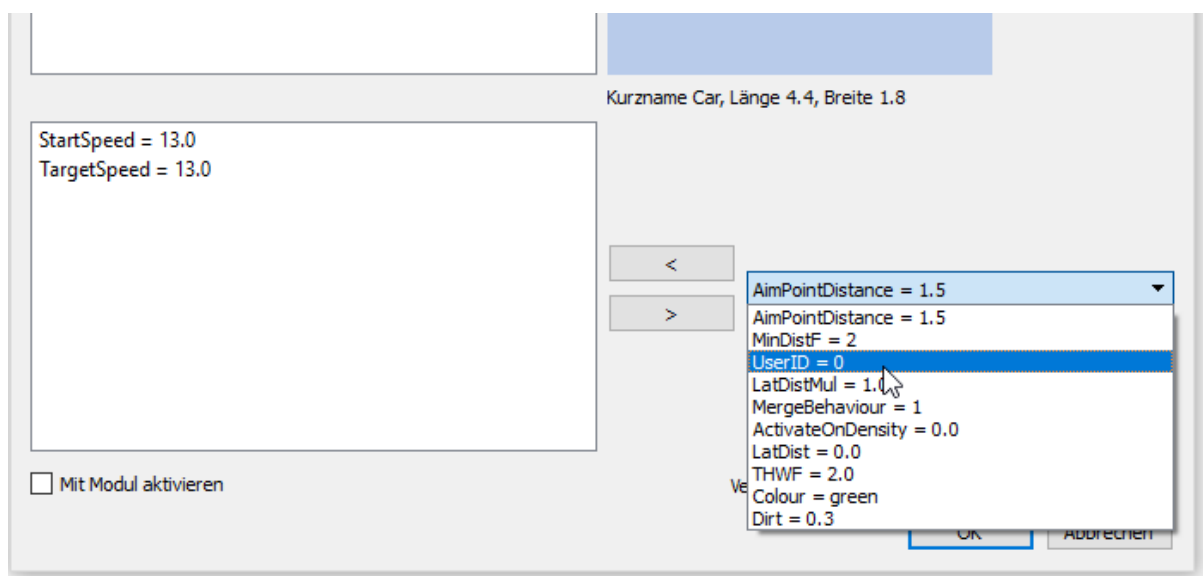
Für ein einzelnes Fahrzeug wählen Sie den Typ Vehicle aus. Unterhalb dieser Auswahlbox sehen Sie dann eine Liste der zur Verfügung stehen Fahrzeugtypen. Die

Gruppentypen (z.B. Car im obigen Bild) werden in dem Vorschauenfenster auf der rechten Seite als eine Abbildung mit mehreren Fahrzeugen dargestellt.

In der rechten unteren Auswahlbox wählen Sie eines der Behaviour Schemes für das Fahrzeug aus.



In dem mittleren Bereich können weitere Parameter für das Behaviour Scheme geändert werden:



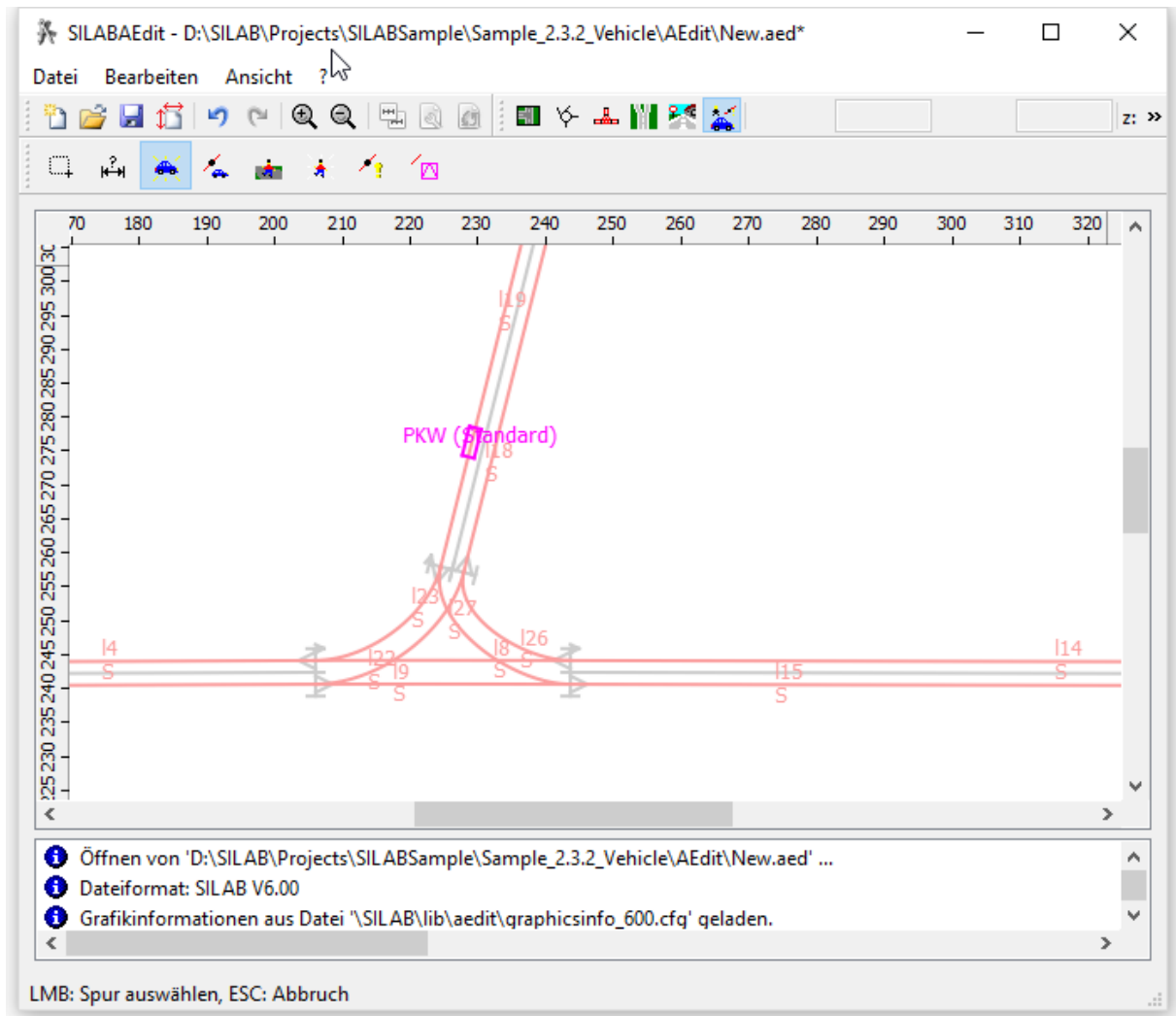
Dabei handelt es sich um Parameter der einzelnen Verhaltensmodelle des Behaviour Schemes. Sie können einen zusätzlichen Parameter dem Verkehrsteilnehmer hinzufügen indem Sie den Parameter aus der Liste auswählen und diesen mit der Pfeiltaste nach links in die linke Liste schieben.

The screenshot shows a software dialog box with a light gray background. On the left, there is a list box containing three parameters: 'StartSpeed = 13.0', 'TargetSpeed = 13.0', and 'UserID = 100'. The 'UserID' entry is selected, and its value '100' is visible in a text field to its right. Below the list box is a checkbox labeled 'Mit Modul aktivieren'. To the right of the list box are two buttons, '<' and '>'. Further right is a dropdown menu labeled 'LatDistMul = 1.0'. Below this is another dropdown menu labeled 'Verhaltensschema: Standard'. At the bottom right are two buttons: 'OK' and 'Abbrechen'.

Um den Wert eines Parameters zu ändern, wählen Sie den Parameter in der linken Liste mit einem Doppelklick aus und ändern den Wert in der Zeile.

Mit einem Haken an dem Punkt [] *Mit Modul aktivieren* kann festgelegt werden, ob der Verkehrsteilnehmer bei Modulstart aktiviert werden soll oder nicht. Wird kein Haken gesetzt, dann muss noch ein Flowpoint zur Aktivierung an den Verkehrsteilnehmer geheftet werden (siehe Abschnitt 2.4.2).

Mit der Taste [OK] beenden Sie den Dialog. Anschließend legen Sie in der Area2 die endgültige Aufsetzposition des Verkehrsteilnehmers auf der gewählten Spur fest. Diesen Vorgang beenden Sie mit einem linken Mausklick.



In der Area2 sehen Sie nun den Umriss des Verkehrsteilnehmers, den Typ (z.B. PKW) und das ausgewählte Behaviour Scheme (hier Standard).

Änderungen an den Einstellungen des Verkehrsteilnehmers können Sie mit dem Markierungswerkzeug vornehmen:

- Wählen Sie das Markierungswerkzeug.
- Selektieren Sie den Verkehrsteilnehmer mit einem linken Mausklick und drücken Sie die Taste [P] oder selektieren Sie den Verkehrsteilnehmer mit einem doppelten linken Mausklick.
- Nun öffnet sich der Dialog für diesen Verkehrsteilnehmer und Sie können die Änderungen vornehmen.

Wurde der Verkehrsteilnehmer mit einem einfachen linken Mausklick selektiert, kann auch die Aufsetzposition innerhalb der Spur neu festgelegt werden, indem man den Verkehrsteilnehmer mit einem linken Mausklick anhebt und mit einem weiteren linken Mausklick neu positioniert.

2.4 Verhaltenssteuerung

Alle Verkehrsteilnehmer können über sog. Flowpoints gesteuert werden. Streckenbezogene Flowpoints sind vergleichbar mit den Events („Hedgehogs“), die in der Datenbasis SCNX verwendet werden, beinhalten jedoch die Angabe eines Kontextes, der bestimmt, welche Fahrzeuge bzw. Fahrzeuggruppen sie auslösen können. Daneben gibt es sog. bedingte Flowpoints, die nicht streckenbezogen, sondern z.B. anhand von zeitlichen Bedingungen ausgelöst werden.

Flowpoints werden verwendet, um

- Verkehrsteilnehmer zu aktivieren und zu deaktivieren.
- Parameter von Verhaltensmodellen zu verändern. Beispiele hierfür sind Änderung der Wunschgeschwindigkeit oder des Abstands.
- unabhängig von den Verhaltensmodellen das grundlegende Verhalten eines Verkehrsteilnehmers zu beeinflussen. Hierzu gehört z.B. das Setzen von Fahrtrichtungswünschen und die gezielte Steuerung der Fahrzeuglichter (z.B. Aufblendlicht).

Das Arbeiten mit Flowpoints wird in den folgenden Abschnitten erläutert. Eine Übersicht aller Flowpoints für das grundlegende Verhalten finden Sie in Abschnitt 5.2. Die Flowpoints zur Änderung von Parametern von Verhaltensmodellen werden in den Abschnitten 5.3.4 und 5.3.3 detailliert beschrieben.

2.4.1 Course Streckenabschnitte

Im Skript gibt es zwei Stellen, an denen Sie Flowpoints einfügen können:

1. Direkt im Abschnitts **Traffic** *<any name> { ... }*; (s. Grammatik 4 als sog. öffentliche Flowpoints).
2. Innerhalb der Definition eines Verkehrsteilnehmers im Abschnitt **VehicleX** *<instance name of vehicle> { ... }*; (s. Grammatik 7) als sog. private Flowpoints.

Welche Fahrzeuge von einem Flowpoint betroffen sind, wird von der Stelle im Skript bestimmt. Die öffentlichen Flowpoints von Variante 1 gelten für alle Fahrzeuge des Verkehrs, die privaten von Variante 2 nur für den in diesem Abschnitt definierten Verkehrsteilnehmer.

Die Syntax ist in beiden Fällen wie folgt:

Grammatik 10 Flowpoints

```

<flowpoints>→
    Flowpoints =
    {
        (,)<flowpoint>
    };

<flowpoint>→
    (
        <course type instance name>,

```

```

        < position along road>,
        <lane index>,
        <context>,
        (,)<flowpoint information>
    )
    |
    (
        <condition>,<value>,(,)<flowpoint information>
    )

<context>→
    SimCar | SimCarOnLane | SimCarToAll | SimCarDelay,<value> |
    SimCarOnLaneDelay,<value> | IfSimCarEmergency,<value> |
    IfSimCarSpeedGreater,<value> | IfSimCarSpeedLess,<value> |
    SimCarUserID,<value> | TrafficUserID,<value>
    Traffic | TrafficDelay,<value> |
    Vehicle | VehicleUserID,<value> | VehicleDelay,<value> |
    ToUserID, <value> | FromToUserID, <value>, <value>

<condition>→
    LifeTime,<value> | SimulationTime,<value> |
    ModuleTime,<value> | DriveDistance,<value> |
    ExternalEvent,<value>, <value>

<flowpoint information>→
    <token>,(,)<value>

```

2.4.1.1 Streckenbezogene Flowpoints

Ein streckenbezogener Flowpoint wird ausgelöst, wenn die Stelle im Streckennetz, auf die er positioniert ist, überfahren wird. Die Definition der Stelle erfolgt über folgende Angaben:

- *<course type instance name>* der Name, den Sie in der Moduldefinition bei der Instanzierung des Course Streckenabschnitts vergeben haben.
- *<position along road>* die vom Beginn der Course Instanz entlang der Straße gemessene anfängliche Position des Flowpoints.
- *<lane index>* die Nummer der Spur. Die Spuren werden, vom Beginn der Course Instanz auf das Ende blickend, von links nach rechts beginnend mit 0 gezählt. Dies entspricht der Zählung bei der Definition von Querschnittsprofilen (s. „Referenz Datenbasis SCNX.pdf“).

Wer einen Flowpoint für welches Fahrzeug auslöst, hängt vom *<context>* ab. Es gibt folgende Kontexte:

SimCar

Spurunabhängige Auslösung des Flowpoints durch das EGO - Fahrzeug. Dieser Flowpoint wird auf allen Spuren eines Course Streckenabschnitts gesetzt, so dass er am entsprechenden Streckenmeter vom EGO - Fahrzeug in jedem Fall ausgelöst wird, egal auf welcher Spur es sich befindet. Bitte beachten Sie, dass trotzdem ein gültiger *<lane index>* angegeben werden muss.

SimCarOnLane

Auslösung durch EGO – Fahrzeug, spurabhängig. Im Gegensatz zum **SimCar** Kontext wird dieser Flowpoint nur auf die angegebene Fahrspur gesetzt und er wird deshalb auch nur dann ausgelöst, wenn sich das EGO - Fahrzeug auf der angegebenen Spur *<lane index>* befindet.

SimCarToAll

Das EGO – Fahrzeug löst diesen spurunabhängigen Flowpoint für alle Verkehrsteilnehmer aus. Dies ermöglicht einen öffentlichen Flowpoint vom EGO – Fahrzeug aus auszulösen.

SimCarDelay,<value>

Das EGO – Fahrzeug löst diesen spurunabhängigen Flowpoint verzögert aus. Der Wert direkt nach dem Kontext gibt die Verzögerungszeit in ms an. Erst nach dieser Verzögerungszeit beginnend mit dem Überfahren des Flowpoints wird der Flowpoint ausgelöst.

SimCarOnLaneDelay,<value>

Das EGO – Fahrzeug löst diesen Flowpoint nur auf der angegebenen Spur aus. Dabei wird der Flowpoint um die im nachfolgenden Wert angegebenen ms verzögert.

SimCarUserID,<value>

Das EGO – Fahrzeug löst diesen spurgebundenen Flowpoint für ein Fahrzeug aus, dessen UserID auf den Wert von *<value>* gesetzt wurde.

IfSimCarSpeedGreater,<value> / IfSimCarSpeedLess,<value>

Das EGO – Fahrzeug löst diesen spurgebundenen Flowpoint nur aus, wenn die Geschwindigkeit des EGO – Fahrzeuges an der Auslöseposition größer /kleiner als die im nachfolgenden Wert angegebene Geschwindigkeit in m/s ist.

IfSimCarEmergency,<value>

Das EGO – Fahrzeug löst diesen spurgebundenen Flowpoint für das Fahrzeug aus, wenn der Status der Sondersignale des EGO – Fahrzeug größer/gleich dem nachfolgenden Wert ist. Folgende Werte gelten für den Status: 0 = keine Sondersignale, 1 = Blaulicht, 2 = Blaulicht + Martinshorn mit geringer Reichweite, 3 = Blaulicht + Martinshorn mit hoher Reichweite

Traffic

Ein Verkehrsteilnehmer mit Ausnahme des EGO-Fahrzeugs löst diesen spurgebundenen Flowpoint für sich selbst aus, wenn er auf der angegebenen Spur fährt.

TrafficUserID,<value>

Trifft ein Verkehrsteilnehmer auf einen öffentlichen Flowpoint und seine eigene UserID stimmt mit den <value>-Wert überein, dann löst dieser Verkehrsteilnehmer den Flowpoint für sich aus.

ToUserID, <value>

Trifft ein Verkehrsteilnehmer auf diesen Flowpoint, dann wird das Fahrzeug mit der UserID <value> gesucht und - falls vorhanden - für dieses der FlowPoint ausgelöst.

FromToUserID <value1>, <value2>

Trifft ein Verkehrsteilnehmer mit der UserID <value1> auf einen solchen Flowpoint, dann wird für das Fahrzeug mit der UserID <value2> - falls vorhanden – der zugehörige Flowpoint ausgelöst.

TrafficDelay,<value>

Ein Verkehrsteilnehmer löst diesen spurgebundenen Flowpoint mit der im nachfolgenden Wert angegebene Verzögerung für sich selbst aus. Die Verzögerung wird in ms angegeben.

Vehicle

Alle anderen simulierten Verkehrsteilnehmer lösen diesen spurgebundenen Flowpoint für einen bestimmten Verkehrsteilnehmer aus.

VehicleUserID,<value>

Nur ein Verkehrsteilnehmer mit der UserID <value> kann diesen spurgebundenen Flowpoint für einen bestimmten Verkehrsteilnehmer auslösen.

VehicleDelay,<value>

Alle anderen simulierten Verkehrsteilnehmer lösen diesen spurgebundenen Flowpoint für einen bestimmten Verkehrsteilnehmer aus. Die Auslösung erfolgt um <value> ms verzögert.

Das folgende Beispiel zeigt die Anwendung von Flowpoints zur Aktivierung und Deaktivierung eines Verkehrsteilnehmers durch das EGO-Fahrzeug. Im Szenario-Modul „MyModule“ befindet sich der Verkehrsteilnehmer an Streckenmeter 200 der course type instance `c1` und wird aktiviert, sobald das EGO-Fahrzeug den Streckenmeter 100 passiert. Wenn der Verkehrsteilnehmer Streckenmeter 500 überfährt, dann wird er mit 2.5 m/s^2 auf eine Geschwindigkeit von 120 km/h (33.33 m/s) beschleunigt. An Streckenmeter 1000 wird er durch das EGO-Fahrzeug deaktiviert.

```
define Module MyModule
{
    # ... (definition of road network)

    Traffic MyTraffic
    {
        VehicleX MyVehicle
        {
            Vehicle = Car;
            Position = (c1, 200, 1);
```

```

BehaviourScheme = Standard;
Flowpoints =
{
    (c1, 100, 1, SimCar, Activate),
    (c1, 500, 1, Traffic, TargetSpeedAndAcceleration,
     33.33, 2.5),
    (c1, 1000, 1, SimCar, Deactivate)
};

# ... (definitions other traffic participants)
};
};

```

Weitere Beispiele für den Einsatz streckenbezogener Flowpoints finden Sie in Abschnitt 2.7.

2.4.1.2 Bedingte Flowpoints

Ein bedingter Flowpoint wird über die Festlegung der *<condition>* und einem zugehörigen Parameter definiert. Es gibt folgende Möglichkeiten:

LifeTime, *<value>*

Der Flowpoint wird nach *<value>* Sekunden seit Aktivierung des Verkehrsteilnehmers ausgelöst.

DriveDistance, *<value>*:

Hat der Verkehrsteilnehmer die angegebene Strecke *<value>* m zurückgelegt, dann wird der betreffende Flowpoint ausgelöst. Dies funktioniert nur bei einzelnen fahrenden Verkehrsteilnehmern (also nicht bei TrafficFlow und TrafficLight).

SimulationTime, *<value>*:

Der Flowpoint wird nach Ablauf von *<value>* Sekunden Simulationszeit ausgelöst.

ModuleTime, *<value>*:

Der Flowpoint wird nach Ablauf von *<value>* Sekunden seit der Einfahrt des EGO-Fahrzeugs in das Szenario-Modul ausgelöst.

ExternalEvent, *<value1>*, *<value2>*

Dieser Flowpoint wird einmalig ausgelöst, wenn am entsprechenden Eingang ExternTrigger*<value1>* der DPUTRFX der Wert *<value2>* (ganzzahlig) anliegt.

Alle Flowpoints des Typs ExternalEvent eines Moduls werden jeweils beim Aktivieren des zugehörigen Moduls reaktiviert und können somit auch mehrfach verwendet werden.

Das folgende Beispiel ist eine Modifikation des Beispiels für spurbezogene FlowPoints aus dem vorhergehenden Abschnitt. Es zeigt die Aktivierung des Verkehrsteilnehmers, sobald das EGO-Fahrzeug das Modul betritt, in dem er definiert ist. Er wird deaktiviert, sobald er eine Strecke von 2000 m zurückgelegt hat:

```

define Module MyModule
{

```

```

# ... (definition of road network)

Traffic MyTraffic
{
    VehicleX MyVehicle
    {
        Vehicle = Car;
        Position = (c1, 200, 1);
        BehaviourScheme = Standard;
        Flowpoints =
        {
            (ModuleTime, 0, Activate),
            (c1, 500, 1, Traffic, TargetSpeedAndAcceleration,
             33.33, 2.5),
            (DriveDistance, 2000, Deactivate)
        };
    };

    # ... (definitions other traffic participants)
};
};

```

Bitte beachten Sie:

Wird ein Verkehrsteilnehmer durch die Modulzeit (wie in obigem Beispiel) oder die Simulationszeit aktiviert, dann geht das nur, wenn zum Zeitpunkt der Aktivierung der Aufsetzpunkt im Sichtbarkeitsbereich des Fahrers liegt. Standardmäßig ist auf Course Streckenabschnitten eine Strecke von 2 km vor und hinter dem Fahrer sichtbar. Bei Area2 Streckenabschnitten wird die Sichtbarkeit über die Sichtbarkeitsradien an den Eintritts- und Austrittsports festgelegt.

Bedingte Flowpoints und Flowpoints, die mit IDs arbeiten können ausschließlich für Einzelfahrzeuge und nicht für Quellen/Senken eingesetzt werden.

Weitere Beispiele für den Einsatz bedingter Flowpoints finden Sie in Abschnitt 2.7.

2.4.2 Area2 Streckenabschnitte

In Area2 Streckenabschnitten definieren Sie Flowpoints mit dem grafischen Editor SILABAEEdit. Die Bedienung von SILABAEEdit ist im Dokument „SILABAEEdit.pdf“ beschrieben. Die wesentlichen Konzepte (z.B. streckenabhängige vs. bedingte Flowpoints), die für Course Streckenabschnitte erläutert wurden, sind aber gleich.

Im Folgenden werden die beiden Beispiele aus den Abschnitten 2.4.1.1 und 2.4.1.2 auf einen Area2 Streckenabschnitt übertragen.

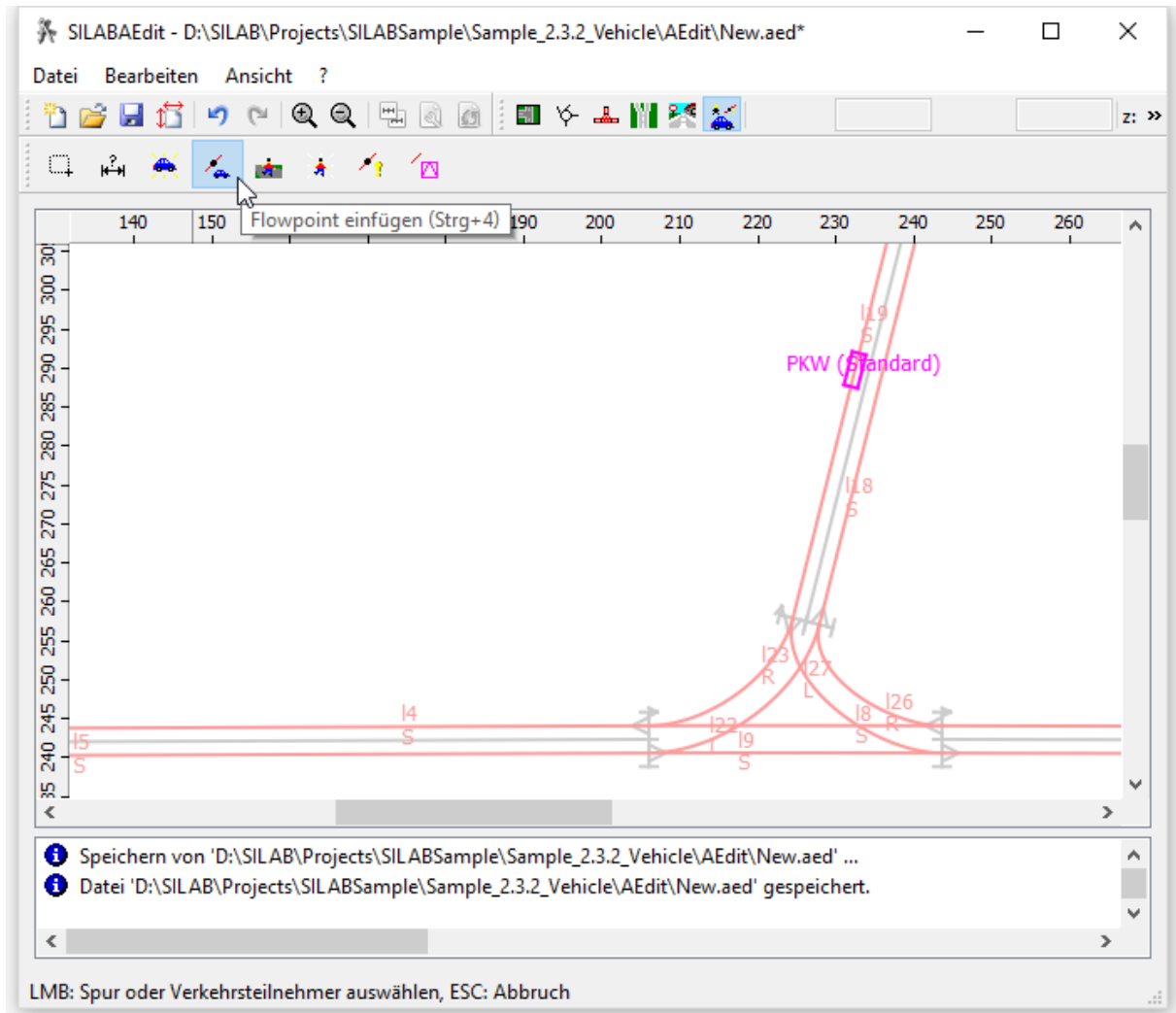
Bitte beachten Sie:

Die Flowpoints mit dem Kontext **SimCar** werden in Area2-Abschnitten nur ausgelöst, wenn sich das EGO-Fahrzeug genau auf dieser Spur befindet – dies entspricht somit dem Kontext **SimCarOnLane** in Course-Abschnitten. In Area2-Abschnitten müssen daher alle Flowpoints auch für das EGO-Fahrzeug immer auf allen in Frage kommenden Spuren einzeln im Editor SILABAEEdit gesetzt werden.

Das Beispiel aus Abschnitt 2.4.1.1 besteht aus einem einzelnen Fahrzeug aus der Typengruppe Car mit dem Behaviour Scheme Standard. Es wird an einem Punkt

durch das EGO – Fahrzeug aktiviert, ändert dann seine Zielgeschwindigkeit und wird durch das EGO – Fahrzeug wieder deaktiviert.

In der folgenden Übertragung des Beispiels auf eine Area2 kommt das Fahrzeug aus einer Seitenstraße (Spur I9 im Bild unten) und biegt rechts ab. Das EGO – Fahrzeug durchfährt die Area2 von links nach rechts über die Spuren I5, I9 und I15.

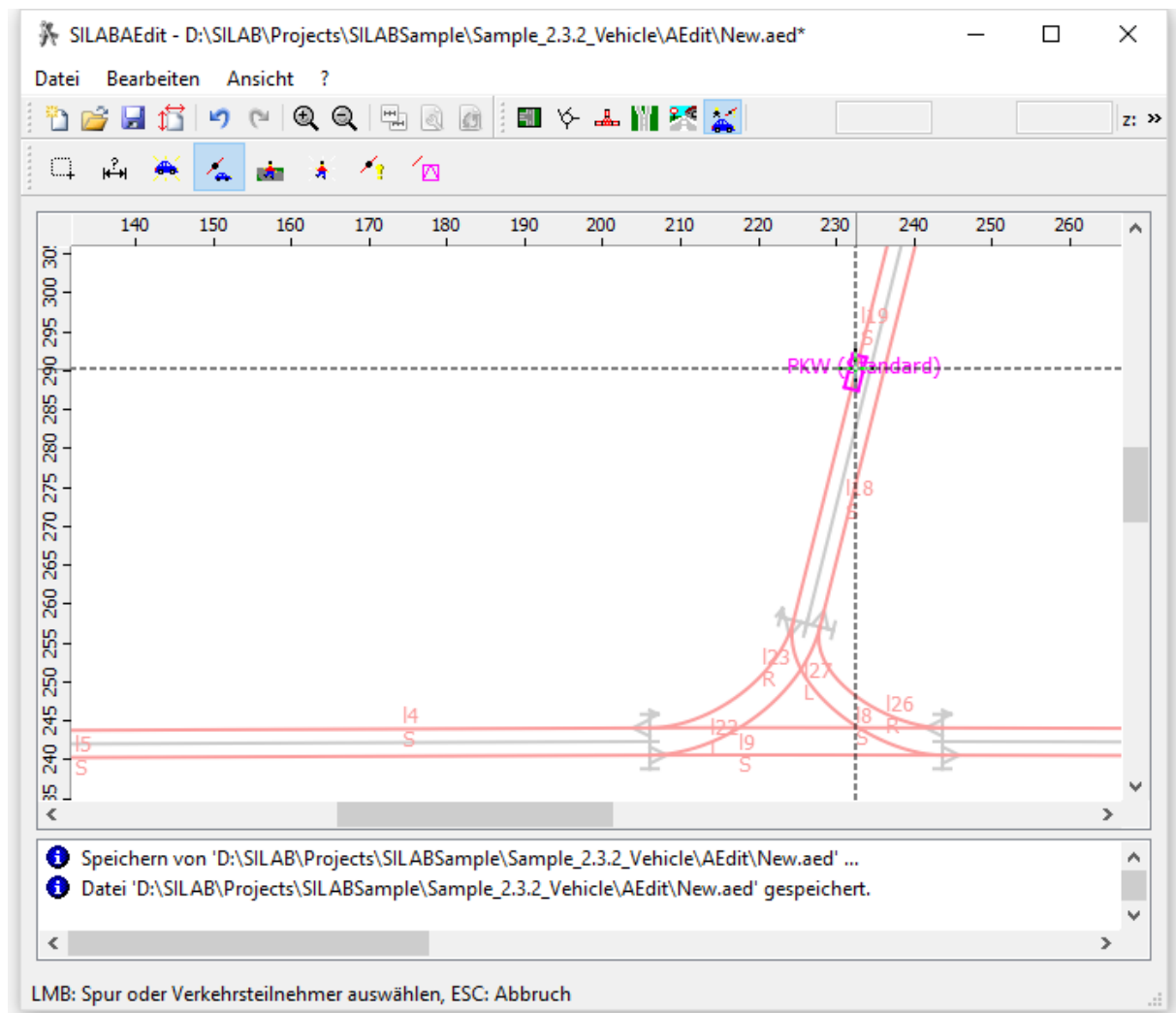


Der Verkehrsteilnehmer wurde wie im Abschnitt 2.3.2 beschrieben auf die Spur I19 eingefügt. Der Haken [] *Mit Modul aktivieren* wurde nicht gesetzt.

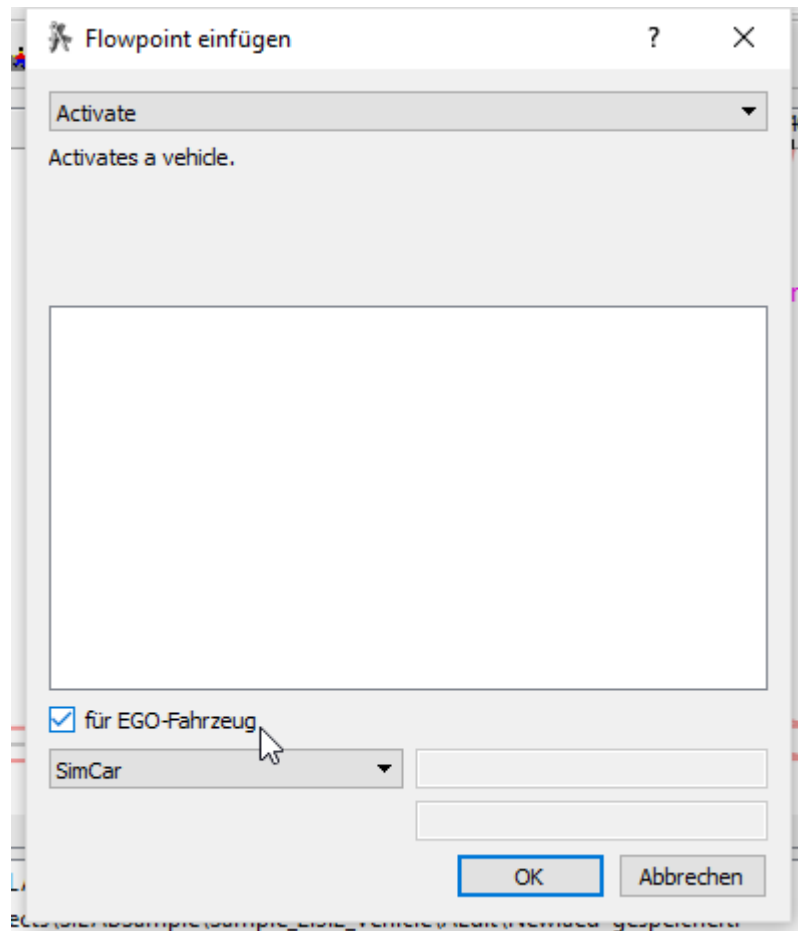
Bei allen drei Flowpoints für dieses Beispiel handelt es sich um private Flowpoints, die dem Verkehrsteilnehmer direkt zugeordnet sind.

Wählen Sie das Werkzeug zum Einfügen eines Flowpoints. Mit dem ersten Flowpoint soll der Verkehrsteilnehmer durch das EGO – Fahrzeug aktiviert werden:

Bewegen Sie das Fadenkreuz auf den Verkehrsteilnehmer:

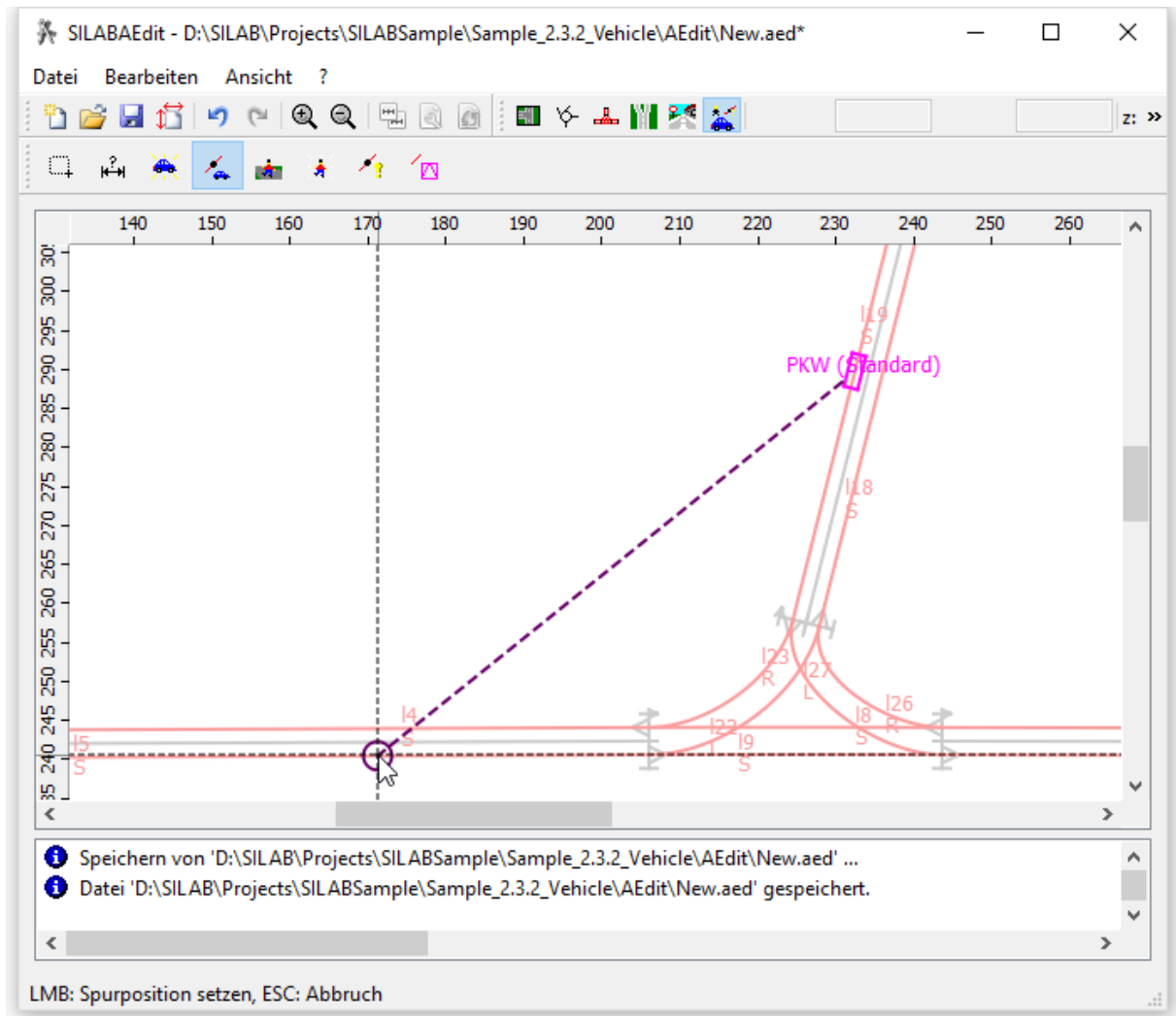


Öffnen Sie den Dialog zum Einfügen eines Flowpoints mit einem linken Mausklick auf den Verkehrsteilnehmer. Dadurch wird der Flowpoint diesem Teilnehmer als privater Flowpoint zugeordnet.



Wählen Sie in der oberen Auswahlbox den Flowpoint Activate aus. Danach wählen Sie in der unteren Auswahlbox den Kontext für die Auslösung des Flowpoints aus. Da sehr häufig Flowpoints vom EGO – Fahrzeug ausgelöst werden, gibt es eine Checkbox *[für EGO-Fahrzeug]*. Wenn Sie diese Option anwählen, wird der Kontext für das EGO-Fahrzeug gewählt.

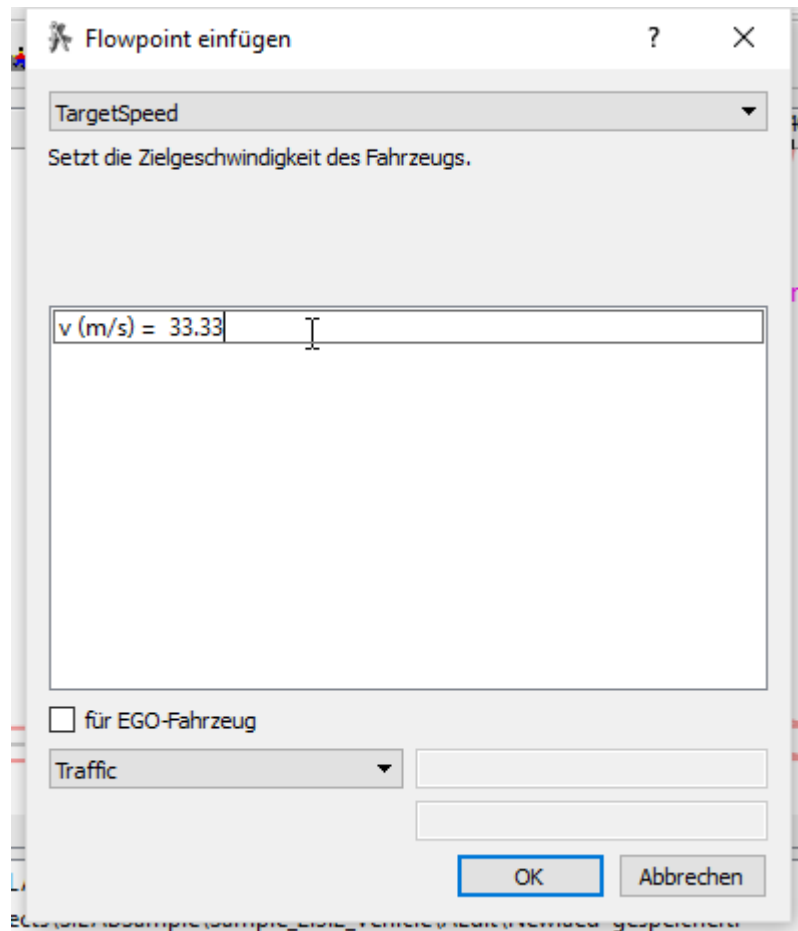
Mit der Taste [OK] bestätigen Sie die Auswahl des Flowpoints. Der Flowpoint wird jetzt auf der Spur I5 positioniert. Bewegen Sie das Fadenkreuz auf die gewünschte Spur und wählen diese mit einem linken Mausklick aus.



Der neue Flowpoint wird als Kreis dargestellt und mit einer Linie mit dem zugehörigen Verkehrsteilnehmer verbunden. Nun können Sie den Flowpoint auf der Spur positionieren. Schließen Sie den Vorgang mit einem linken Mausklick ab.

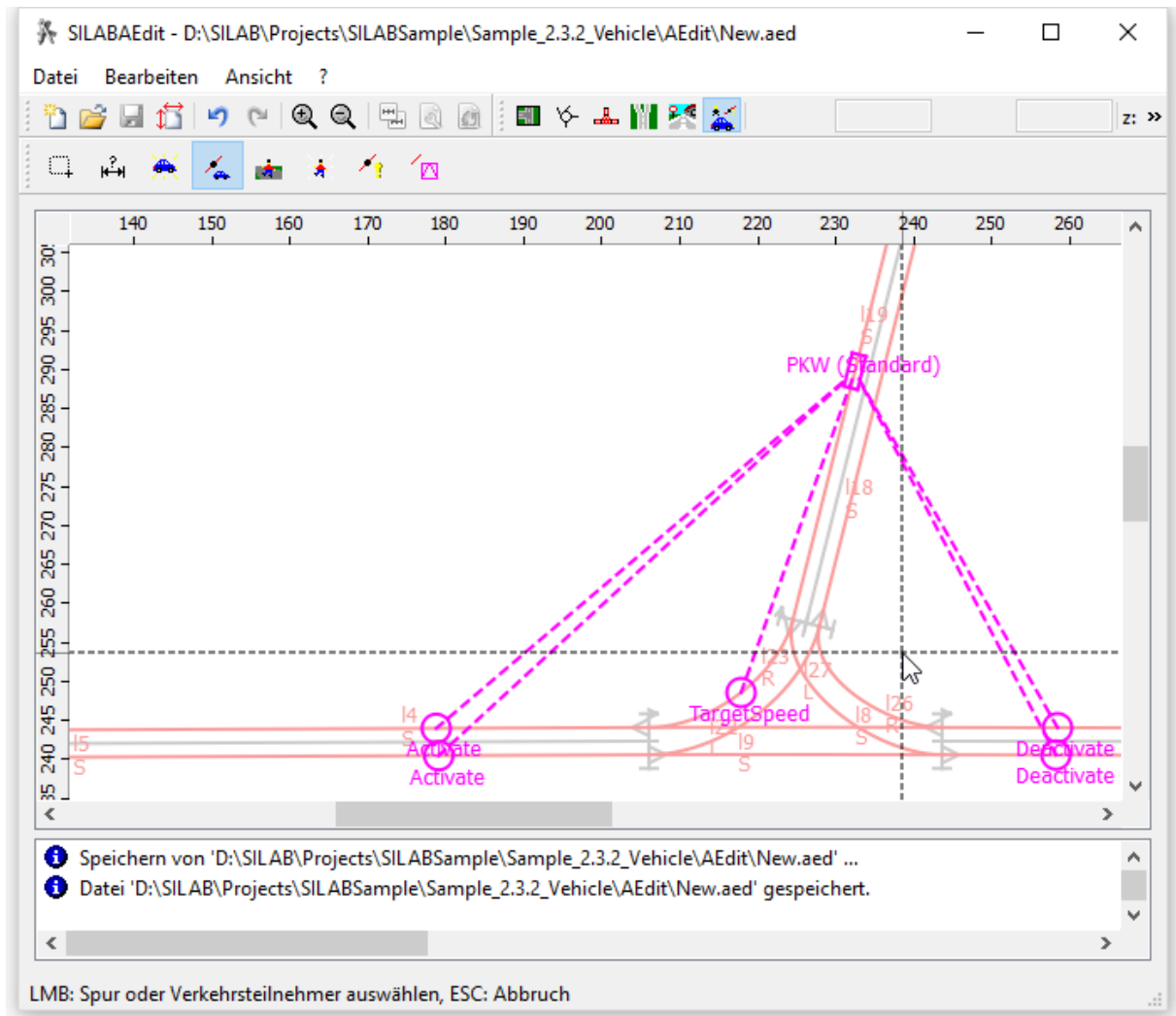
Da im Unterschied zu einem Course-Abschnitt ein Flowpoint des Kontexts SimCar nur für eine bestimmte Spur gilt, ist es sicherer, diesen Flowpoint auch auf Spur I4 zu setzen.

Der nächste Flowpoint setzt die Zielgeschwindigkeit neu. Da der Verkehrsteilnehmer diesen für sich selbst auslösen soll, muss er auf einer Spur liegen, auf die der Verkehrsteilnehmer fahren wird. Im obigen Beispiel fährt der Verkehrsteilnehmer von I10 über I8 nach I0. Sie fügen den Flowpoint analog zu den ersten beiden ein und wählen in der oberen Auswahlbox den Flowpoint TargetSpeed und in der unteren Auswahlbox als Kontext Traffic aus.



In der mittleren Liste werden alle Parameter des Flowpoints dargestellt. Wählen Sie in dieser Liste den Eintrag $v \text{ (m/s)}$ für den Parameter der Geschwindigkeit in mit einem Doppelklick m/s aus und ändern Sie in den aktuellen Wert in 33.33. Nun können Sie den Flowpoint auf Spur l8 positionieren.

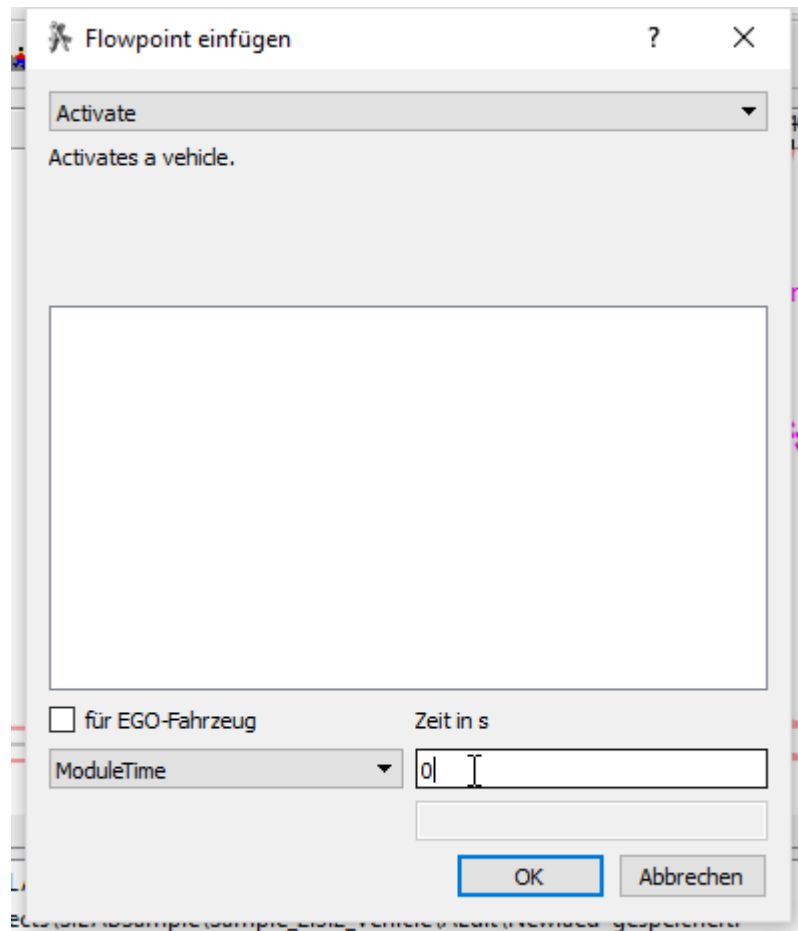
Der letzte Flowpoint wird mit dem Kontext SimCar zweimal parallel auf die rechten Spuren verteilt.



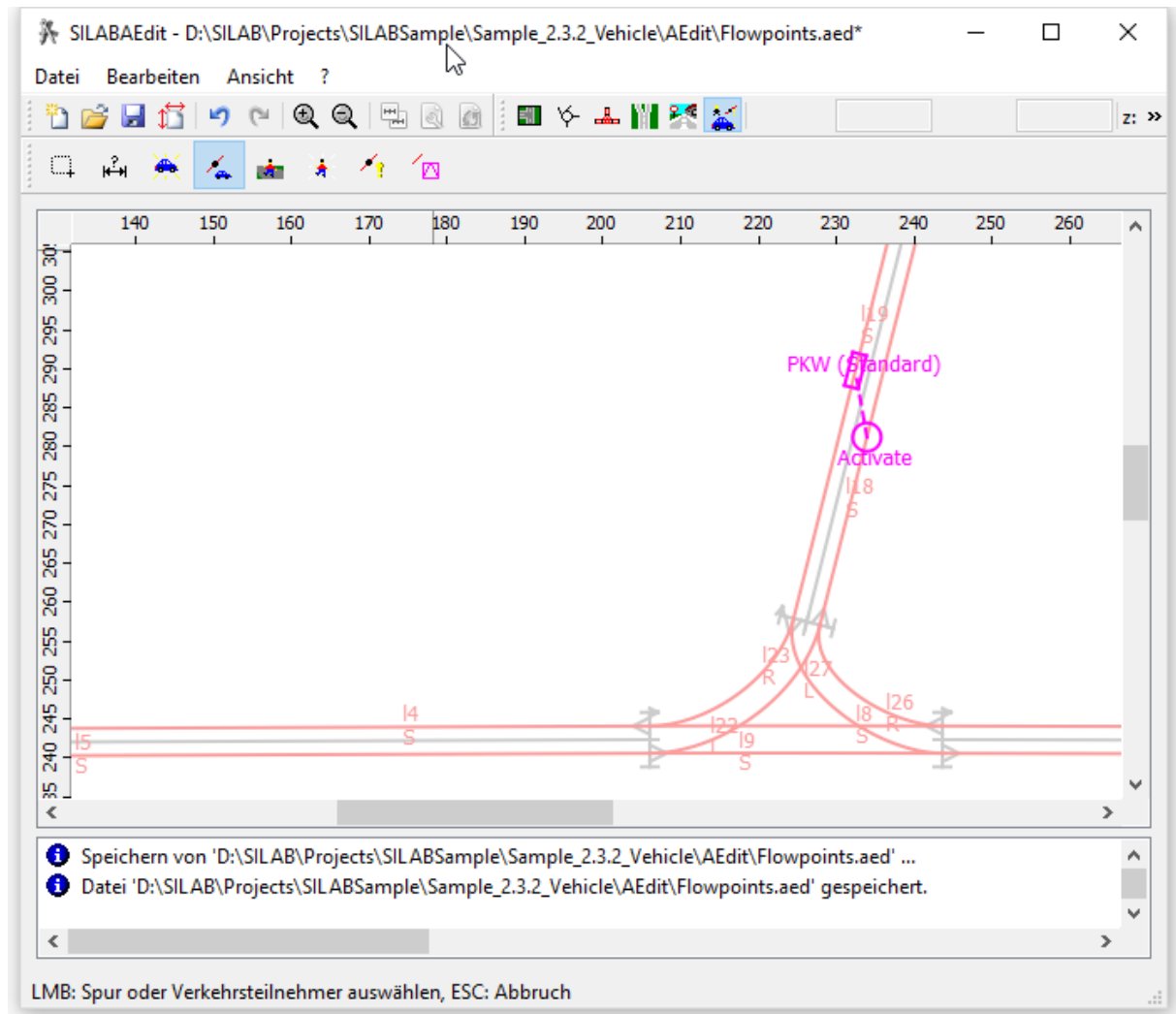
Es folgt eine weitere Variante des Beispiels:

Um den Verkehrsteilnehmer bei Modulstart zu aktivieren können Sie entweder den Verkehrsteilnehmer zur Bearbeitung selektieren und den Haken bei *[] Mit Modul aktivieren* setzen, oder Sie fügen einen bedingten Flowpoint ein:

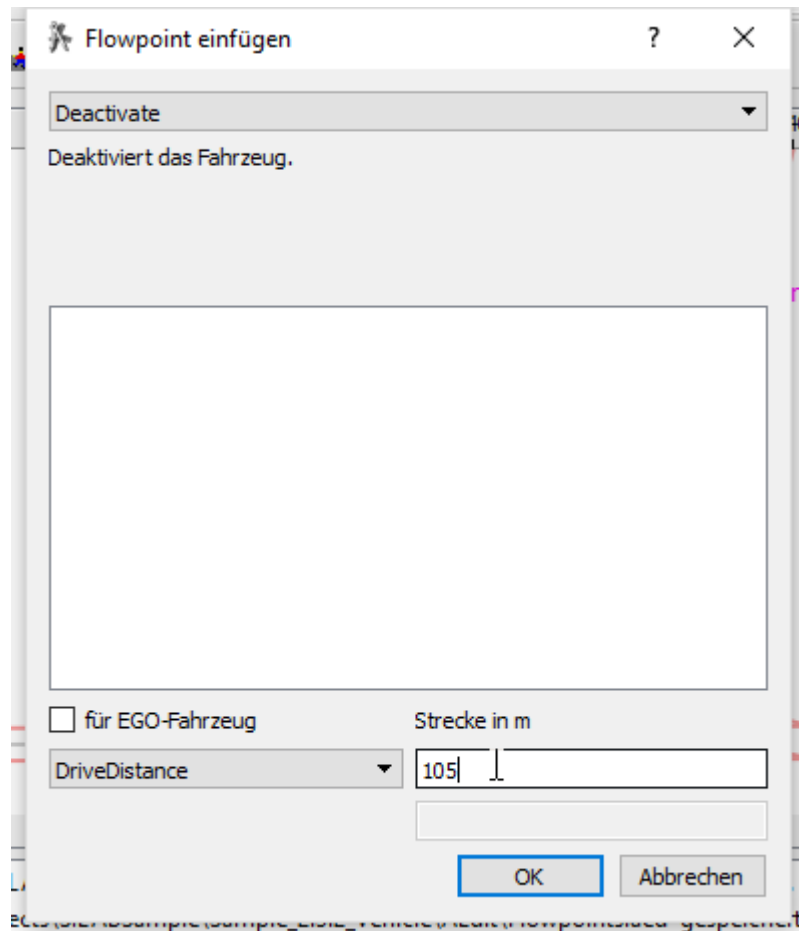
Bedingte Flowpoints werden wie spurgebundene Flowpoints eingefügt. Wählen Sie den Kontext *ModuleTime* und tragen in dem nun freigeschalteten Eingabefeld rechts neben dem Kontext die Zeit in Sekunden ein:



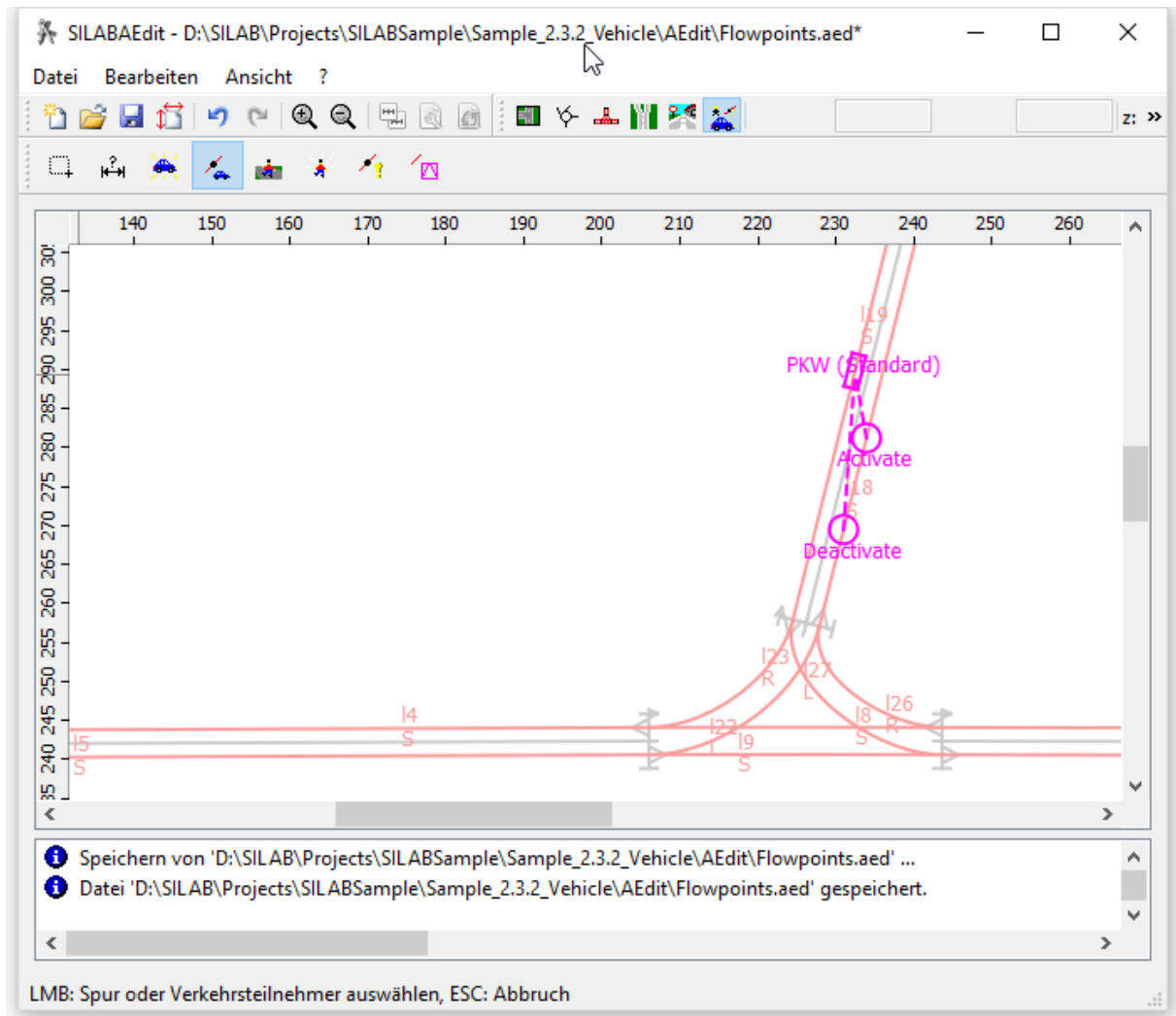
Dieser bedingte Flowpoint muss ebenfalls auf einer Spur platziert werden. In diesem Fall ist es egal auf welcher Spur, da ein bedingter Flowpoint nicht überfahren werden muss.



Der zweite Flowpoint wird auf die Spur I18 platziert. Bei dem letzten Flowpoint handelt es sich wieder um einen bedingten Flowpoint mit dem Kontext DriveDistance und der Fahrtstrecke in m. In dieser kompakten Area2 reichen 105m.



Auch dieser Flowpoint muss auf eine Spur platziert werden.



Bitte beachten Sie:

Die Flowpoints mit dem Kontext **SimCar** werden in Area2-Abschnitten nur ausgelöst, wenn sich das EGO-Fahrzeug genau auf dieser Spur befindet – dies entspricht somit dem Kontext **SimCarOnLane** in Course-Abschnitten. In Area2-Abschnitten müssen daher alle Flowpoints auch für das EGO-Fahrzeug immer auf allen in Frage kommenden Spuren einzeln im Editor SILABAEEdit gesetzt werden.

2.5 Verkehrsflüsse

Neben den bisher vorgestellten Methoden, einzelne Fahrzeuge zu definieren, gibt es die Möglichkeit, ganze Gruppen von Fahrzeugen zu aktivieren und zu deaktivieren. Eine solche Gruppe wird als Traffic Flow bezeichnet.

2.5.1 Traffic Flow in Course Streckenabschnitten

Der Gegenverkehr auf einer zweispurigen Überlandstrecke ist ein typisches Beispiel für die Verwendung eines Traffic Flows in Course Streckenabschnitten. In diesem Fall ist das Verhalten eines einzelnen Fahrzeugs des Gegenverkehrs nicht von Bedeutung. Es reicht, dem Fahrer eine Menge von Fahrzeugen mit gleicher Geschwindigkeit in variierenden Abständen entgegenkommen zu lassen. Genau dies ermög-

licht die unsichtbare Quelle eines Traffic Flows. Sie kann entweder an einer bestimmten Position statisch platziert werden oder an das EGO-Fahrzeug geheftet werden und ihm z.B. auf der linken Spur 500 m „vorausfahren“. An ihrer jeweiligen Position setzt die Quelle dann in definierten Abständen einen Verkehrsteilnehmer aus ihrer Menge auf, solange noch nicht aktive Verkehrsteilnehmer in ihrer Menge sind. Sind alle Verkehrsteilnehmer ihrer Menge aktiviert, unterbricht die Quelle ihren Verkehrsfluss, bis sich wieder inaktive Verkehrsteilnehmer in ihrer Menge befinden. Die Verkehrsteilnehmer werden von Senken des Traffic Flows (oder natürlich auch über Flowpoints) deaktiviert. Auch Senken können entweder statisch platziert werden oder an das EGO-Fahrzeug geheftet werden. Die Senken deaktivieren die Verkehrsteilnehmer des Traffic Flows, zu dem sie gehören, wenn diese sich außerhalb eines bestimmten Radius um das EGO-Fahrzeug befinden. Geschieht dies zu nah am EGO-Fahrzeug, besteht die Gefahr, dass der Fahrer die Deaktivierung von Verkehrsteilnehmer bemerkt. Verkehrsteilnehmer eines Traffic Flows, die sich innerhalb dieses Radius *um das EGO-Fahrzeug herum* befinden, werden von der Senke nicht deaktiviert.

Die Definition eines Traffic Flows im Skript erfolgt auf derselben Ebene wie einzelne Verkehrsteilnehmer (**VehicleX**, s. Grammatik 7) und sieht so aus:

Grammatik 11 Traffic Flow

```

<traffic flow> →
    TrafficFlow <instance name of traffic flow>
    {
        (0..*)<source>
        (0..*)<drain>
        (0,1)<flowpoints(→Grammatik 10 Seite 25) >
    };

```

Ein Traffic Flow kann mehrere Quellen und Senken enthalten. Die Quellen aktivieren Verkehrsteilnehmer an der Quellenposition nach einer vorgegebenen, zeitlichen Folge. Die Senken deaktivieren die Verkehrsteilnehmer, wenn diese die Senke erreichen.

Grammatik 12 Quellen

```

<source>→
    source <instance name of source>
    {
        <traffic flow position(→Grammatik 13 Seite 42) >
        <set of vehicles>
        Rotate = 0|1;
        Parameters = <time random parameters>;
    };

<time random parameters>→
    (Gauss,<μ>,<σ>,<period>,<seed>) |
    (Uniform,<min value>,<max value>,<period>,<seed>)

<set of vehicles>→
    Vehicles = { (,) <source vehicle definition> };

<source vehicle definition>→
    (<number> , <vehicle type(→Grammatik 23 Seite 117)> , (0,1) <colour dirt params> ,
    (<behavior model(→Grammatik 24 Seite 128)>))

<colour dirt params>→
    <colour name> , <dirt value>

```

Zunächst wird die Position <traffic flow position> der Quelle definiert:

Grammatik 13 Positions of a Traffic Flow

```

<traffic flow position>→
    Position =
        (<course type instance name>,<position along road>,<lane index>) |
        (SimCar,<distance>,<lane index>)

```

Wie Sie in obiger Grammatik sehen, kann eine Quelle ortsfest in der Datenbasis positioniert werden (Alternative 1 von **Position = ...**). In diesem Fall machen Sie folgende Angaben:

- <course type instance name> : Der Name, den Sie in der Moduldefinition bei der Instanzierung des Course Streckenabschnitts vergeben haben.
- <position along road> : Die vom Beginn der Course Instanz entlang der Straße gemessene Position der Quelle.
- <lane index> : Die Nummer der Spur. Die Spuren werden, vom Beginn der Course Instanz auf das Ende blickend, von links nach rechts beginnend mit 0 gezählt. Dies entspricht der Zählung bei der Definition von Querschnittsprofilen (s. „Referenz Datenbasis SCNX.pdf“).

Darüber hinaus kann die Quelle an das EGO-Fahrzeug geheftet werden (2. Alternative in Grammatik 13) und so mit dem EGO-Fahrzeug fahren. Geben Sie hierfür den Abstand *<distance>* in Meter zum EGO-Fahrzeug an. Positive Abstände liegen

- bei Fahrspuren, welche die gleiche Richtung der Fahrspur haben, auf der das EGO-Fahrzeug fährt, *vor* dem EGO-Fahrzeug. Negative Abstände liegen in diesem Fall *hinter* dem EGO-Fahrzeug.
- bei Fahrspuren, welche die entgegengesetzte Richtung der Fahrspur haben, auf der das EGO-Fahrzeug fährt, *hinter* dem EGO-Fahrzeug. Negative Abstände liegen dann entsprechend *vor* dem EGO-Fahrzeug.

Der *<lane index>* gibt auch hier die Spur an, auf der sich die Quelle befindet. Die Spuren werden, vom Beginn der Course Instanz auf das Ende blickend, von links nach rechts beginnend mit 0 gezählt.

Die Fahrzeugquelle hat eine Menge von Verkehrsteilnehmern (**Vehicles** = { ... }) mit den angegebenen Fahrzeugtypen und dem angegebenen Verhalten. Verwenden Sie als Fahrzeugtypen dabei diejenigen, die in der RoadUserTable definiert sind (s. Abschnitt 5.1.4). Die Farben werden in einer separaten **ColourTable** definiert (siehe Abschnitt 5.1.4.2). Der Verschmutzungsgrad hat einen Bereich von 0.0 (sauber) bis 1.0 (schmutzig).

Bei jedem neu generierten Fahrzeug sucht SILAB nach einem noch nicht verwendeten Fahrzeug in der **Vehicles**-Liste

- angefangen an der ersten Position für den Fall, dass der Parameter **Rotate** den Wert Null hat
- in aufsteigender Reihenfolge für den Fall, dass der Parameter **Rotate** den Wert Eins hat.

Die zeitliche Folge der erzeugten Verkehrsteilnehmer wird durch die *<time random parameters>* festgelegt. Hierzu wird in jedem Simulationsschritt die Distanz in Sekunden zwischen dem zuletzt erzeugten Verkehrsteilnehmer und der Quelle ermittelt. Abhängig von den *<time random parameters>* werden dann folgende Verkehrsteilnehmer erzeugt, deren Abstände folgendermaßen verteilt sind:

- **Gauss** : Die Abstände sind eine normal verteilte Zufallsfolge mit den angegebenen Parametern für Mittelwert $\langle \mu \rangle$ und Standard-Abweichung $\langle \sigma \rangle$.
- **Uniform** : Die Zufallsfolge ist gleich verteilt mit dem angegebenen Minimal- und Maximalwert *<min value>* bzw. *<max value>*.

Bei der Erzeugung der Zufallszahlen wird eine Zufallstabelle mit der angegebenen Anzahl von Zufallswerten *<period>* und dem angegebenen Samen *<seed>* für den Zufallsgenerator erzeugt. Ein Same ist eine beliebige ganze Zahl. Ein bestimmter Same erzeugt bei jedem Simulationsstart die gleiche Zufallsreihenfolge. Damit ist das Verhalten von Traffic Flows zwar zufällig aber zwischen verschiedenen Simulator-Fahrten (z.B. bei verschiedenen Durchläufen eines Experiments) reproduzierbar.

Die Definition von Senken sieht wie folgt aus:

Grammatik 14 Senken

```

<Drain>→
    Drain <instance name of drain>
    {
        <traffic flow position(→Grammatik 13 Seite 42)>
    };

```

Eine Senke deaktiviert nur Verkehrsteilnehmer aus dem zugehörigen Verkehrsfluss. Wie die Quellen können auch die Senken über *<traffic flow position>* ortsfest positioniert oder an das EGO - Fahrzeug geheftet werden. Bei mobilen Senken wird die Spurangabe ignoriert. Sie deaktivieren alle Verkehrsteilnehmer aus dem Verkehrsfluss, die eine größere Entfernung zum EGO - Fahrzeug haben als im Parameter *<distance>* in Grammatik 13 angegeben. Dabei gilt der euklidische Abstand zum EGO-Fahrzeug, der bei einer kurvigen Strecke kleiner ist als der Abstand entlang der Strecke. Die Deaktivierung erfolgt somit kreisförmig um das EGO-Fahrzeug herum.

Über die *<flowpoints>* des **TrafficFlow** Abschnitts werden nur die Quellen und Senken aktiviert bzw. deaktiviert. Auf das Verhalten der einzelnen, vom Traffic Flow erzeugten Verkehrsteilnehmer kann nur über öffentliche Flowpoints Einfluss genommen werden, die direkt im Abschnitts **Traffic** *<any name> { ... };* (s. Grammatik 5) definiert wurden. Folgende *<flowpoints>* im **TrafficFlow** Abschnitt sind zuässig:

ActivateSource,(*(,)<instance name of source>*)

Mit diesem Flowpoint wird die angegebene Quelle aktiviert.

DeactivateSource,(*(,)<instance name of source>*)

Mit diesem Flowpoint wird die angegebene Quelle deaktiviert.

ActivateDrain,(*(,)<instance name of drain>*)

Mit diesem Flowpoint wird die angegebene Senke aktiviert.

DeactivateDrain,(*(,)<Name der Senke>*)

Mit diesem Flowpoint wird die angegebene Senke deaktiviert.

Ein ausführliches Beispiel für den Einsatz eines Traffic Flows zur Erzeugung von Gegenverkehr auf einer Autobahn finden Sie in Abschnitt 2.7.4.

2.5.2 Quellen in Courses und Areas

Für die Realisierung eines Verkehrsflusses steht in Areas und Courseabschnitten Quellen zur Verfügung. In Areas steht hier die Source und die SequenceSource zur Verfügung. Source setzt Fahrzeuge mit einem konstanten Abstand zueinander auf. Die SequenceSource setzt Fahrzeuge in einer frei definierbaren Abstandsfolge auf. Beide Quellen werden von dem Editor SILABAedit in das gleiche Konstrukt (SourceX) überführt. Durch die Zuweisung des Parameters Sequence zu einem Tupel von Zahlen wird aus der Source eine SequenceSource.

Quellen in Course Abschnitten werden mit der folgenden Grammatik definiert:

Grammatik 15 SourceX

```

<traffic flow> →
    SourceX <instance name of SourceX>
    {
        <initial position (→Grammatik 8 Seite 18)>
        <behaviour (→Grammatik 9 Seite 18)>
        <flowpoints (→Grammatik 10 Seite 25) >
        NumberOfVehicles = <value>;
        (0,1) Distance = <value>;
        Group = <vehicle group type(→Grammatik 23 Seite 117)>;

        EGODistance = <value>;
        RandomSeed = <value>;
        DeactivationRadius = <value>;
        MaxNumberActivation = <value>;

        (0,1) MaxNumberOfVehicles = <value>;

        (0,1) Sequence = (<value>, (0...*)[,<value>]);

    };

```

Beispiel einer SourceX, die an das EGO-Fahrzeug geheftet wurde:
 Das Beispiel ist für eine zweispurige Straße und setzt die Fahrzeuge in 300m Entfernung auf der Gegenspur (Spur 0) vor dem EGO-Fahrzeug auf.

```

SourceX S01
{
    Position = (SimCar,-300,0);
    Group = Car;
    NumberOfVehicles = 10;
    Distance = 10;
    BehaviourScheme = Standard;
    Flowpoints =
    {
        (u1,2,1,SimCar,Activate)
    };
};

```

Die Quellen werden in SILABAEEdit mit dem Werkzeug zum Einfügen eines Verkehrsobjektes in die Area2 eingesetzt. Wählen Sie in der oberen Auswahlbox hierfür Source aus. Danach stehen zwei Arten von Quellen zur Verfügung:

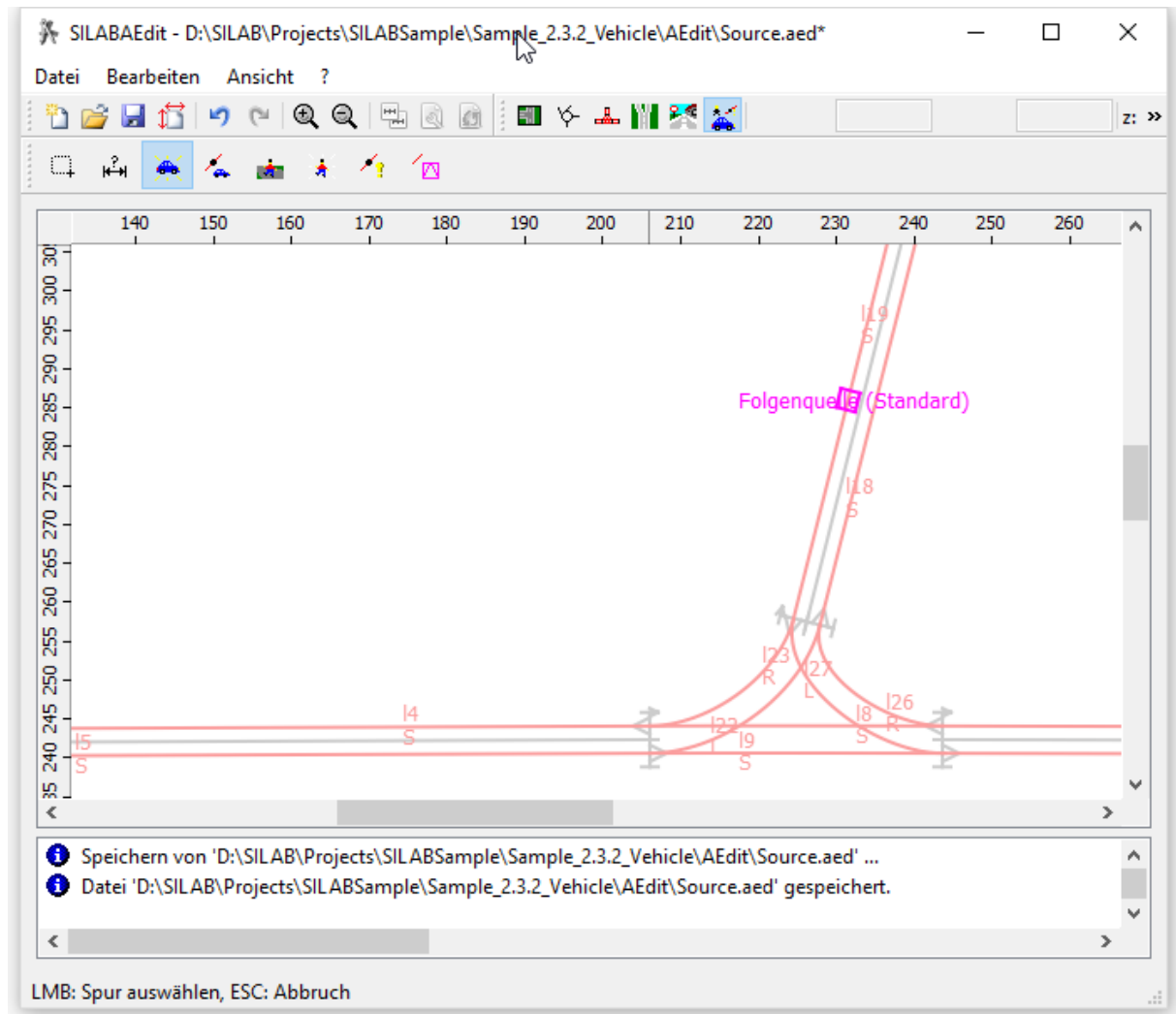
- **Source:** Diese Quelle setzt Fahrzeuge mit einem konstanten Abstand zueinander an der Position der Quelle ein. Die folgenden Parameter lassen sich verändern:

- **NumberOfVehicles**: Anzahl der Fahrzeuge, die der Quelle zur Verfügung stehen. Dies definiert die maximal gleichzeitig aktiven Fahrzeuge dieser Quelle. Sobald diese Zahl erreicht ist wird gewartet bis wieder Fahrzeug deaktiviert wird bevor ein neues Fahrzeug aufgesetzt wird
- **MaxNumberActivation**: Maximale Anzahl an aufgesetzten Fahrzeugen. Ist dieser Wert erreicht, dann deaktiviert sich die Quelle automatisch. Bei einem Wert von 0 ist dieses Feature deaktiviert. (Default: 0).
- **Behaviour or BehaviourScheme**
- **Group**: Gruppenname als Typ der verwendeten Fahrzeuge. Wird hier eine Gruppe angegeben, dann werden die Fahrzeuge der Gruppe zufällig gezogen.
- **Distance**: Abstand zwischen den Fahrzeugen in m. (Minimum 8m). Default: 10m Wurde das letzte Fahrzeug deaktiviert, dann ist der Abstand unendlich groß und es wird sofort ein neues aktiviert
- **EGODistance**: Nähert sich das EGO – Fahrzeug der Quelle und wird der bei diesem Parameter angegebene (euklidische) Abstand (in m) zur Quelle unterschritten, dann deaktiviert sich die Quelle. Damit wird verhindert, dass Fahrzeuge im Sichtbereich des Fahrers plötzlich erscheinen.
- **DeactivationRadius**: Der Radius d.h. die Entfernung eines Fahrzeugs zum EGO-Fahrzeugs muss einmal unterschritten sein und beim nächsten Überschreiten wird das zugehörige Fahrzeug deaktiviert.
- **RandomSeed**: Die verwendeten Fahrzeuge werden zufällig aus allen verfügbaren Fahrzeugen ausgewählt. Die Zufallsfolge wurde mit dem Samen RandomSeed initialisiert.
- **SequenceSource**: Die SequenceSource wird in Courses ausgewählt durch die Spezifikation des Parameters Sequence und im SILABAedit durch die Auswahl der SequenceSource im Auswahldialog. Diese Quelle setzt die Fahrzeuge in einer frei definierbaren Folge von zeitlichen Abständen (in Sekunden) auf. Diese Quelle kann über die folgenden Parameter verändert werden:
 - **NumberOfVehicles**: Anzahl der Fahrzeuge, die der Quelle zur Verfügung stehen
 - **Behaviour or BehaviourScheme**
 - **Group**: Gruppenname als Typ der verwendeten Fahrzeuge. Wird hier eine Gruppe angegeben, dann werden die Fahrzeuge der Gruppe zufällig gezogen.
 - **Distance**: Abstand zwischen den Fahrzeugen in m. (Minimum 8m). Default 10m; Nicht relevant bei der Spezifikation von Sequence d.h. die **Distance** wird in diesem Fall ignoriert.
 - **Sequence**: Die Folge der Abstände wird als Tupel angegeben. Das Tupel (2.0, 4.0 , 3 , 5.3, 3.3) setzt das zweite Fahrzeug 2 Sekunden nach dem ersten auf, das dritte 4 Sekunden nach dem zweiten usw. Die Folge wiederholt sich, wenn der letzte Wert des Tupels erreicht wurde. Zeitliche Abstände werden unter Annahme konstanter Geschwindigkeit als Abstände von Stoßstange zu Stoßstange umgesetzt. Sollte das letzte Fahrzeug nicht mehr aktiv sein, dann wird der zeitliche Abstand zwischen dem Aufsetzen des letzten Fahrzeugs und des nächsten Fahrzeugs entsprechend gesetzt.

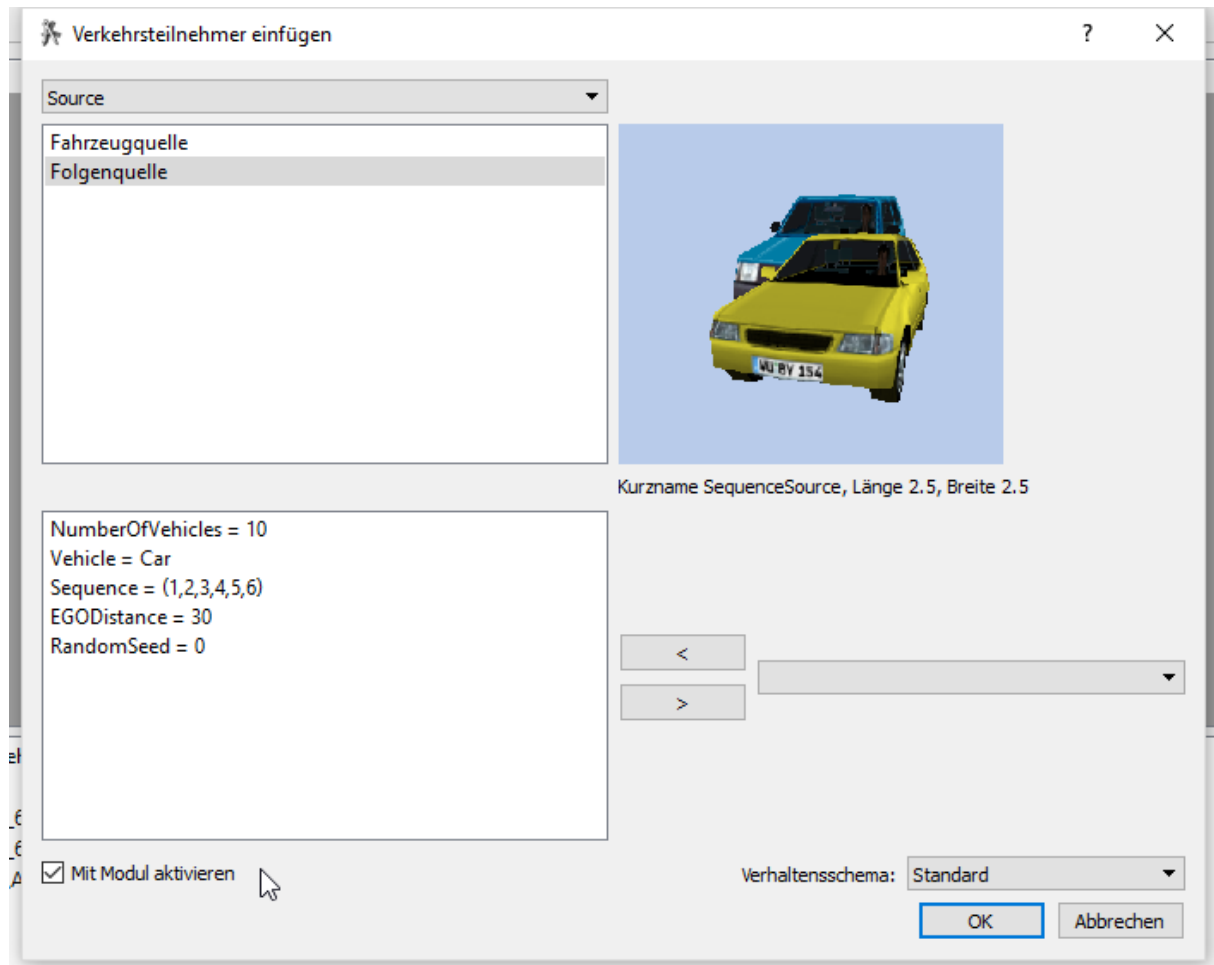
- **EGODistance:** Nähert sich das EGO – Fahrzeug der Quelle und wird der bei diesem Parameter angegebene Abstand (in m) zur Quelle unterschritten, dann deaktiviert sich die Quelle.
- **RandomSeed:** Die verwendeten Fahrzeuge werden zufällig aus allen verfügbaren Fahrzeugen ausgewählt. Die Zufallsfolge wurde mit dem Samen RandomSeed initialisiert
- **DeactivationRadius:** Der Radius d.h. die Entfernung eines Fahrzeugs zum EGO-Fahrzeugs muss einmal unterschritten sein und beim nächsten Überschreiten wird das zugehörige Fahrzeug deaktiviert.

Im Folgenden wird die Definition einer Quelle und Senke an einem Beispiel erläutert:

Die T-Kreuzung dieses Beispiels wird vom EGO – Fahrzeug von links nach rechts durchfahren. An der T-Kreuzung wird eine SequenceSource in die links abzweigende Nebenstraße auf die Spur l10 positioniert. Die Fahrzeuge biegen dann über l8 rechts ab und fahren nach l0 und kommen dem EGO – Fahrzeug entgegen. Dort wird eine Senke platziert, welche die Fahrzeuge der Quelle deaktiviert. Im späteren Verlauf soll das EGO – Fahrzeug die Quelle deaktivieren.

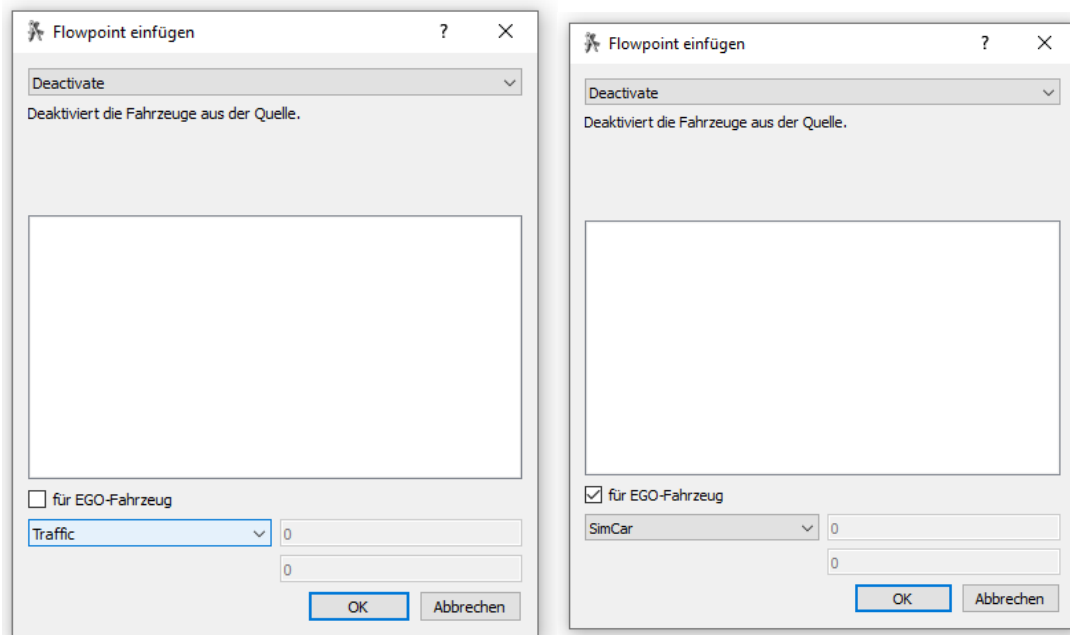


Die Quelle wird mit dem Werkzeug zum Einfügen von Verkehrsobjekten auf die Spur I10 platziert.



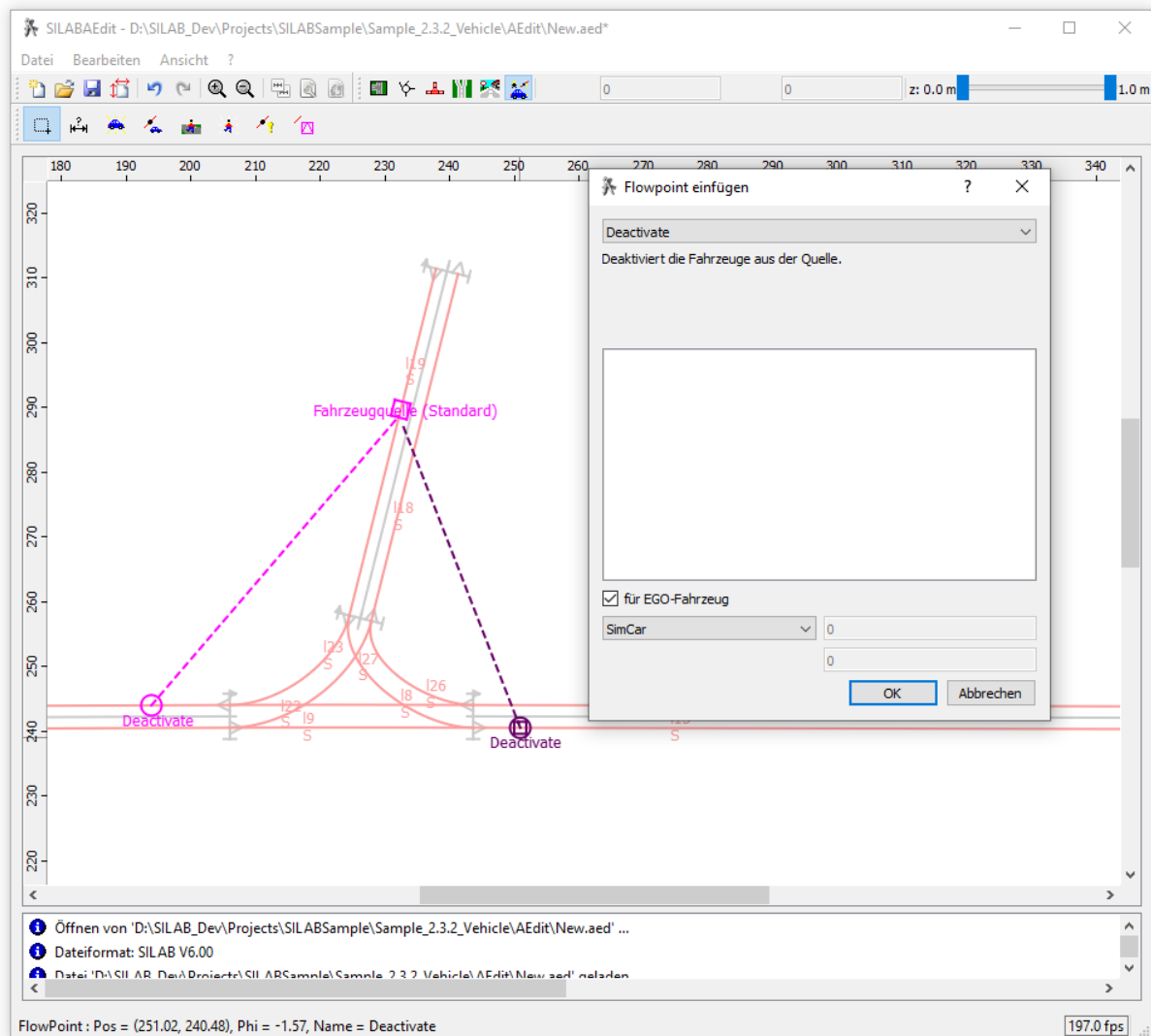
Mit dem Haken [] *Mit Modul aktivieren* wird die Quelle automatisch gestartet, wenn das Modul vom EGO – Fahrzeug befahren wird. Als Behaviour Scheme wird in der unteren Auswahlbox das Schema Standard gewählt. Für den Parameter der zeitlichen Abfolge der Fahrzeuge (Sequence) geben Sie die gewünschte Folge als Tupel ein. Bestätigen Sie die Einstellung Drücken von *<apply>*. Nun kann die Quelle positioniert werden.

Die Senke wird durch den privaten Flowpoint *Deactivate* realisiert. Beim Einfügen des Flowpoints klicken Sie zunächst auf die Quelle und wählen dann den Flowpoint *Deactivate* mit dem Kontext *Traffic* aus. (linker Dialog).



Das EGO – Fahrzeug soll im späteren Verlauf die Quelle deaktivieren. Hierzu fügen Sie beispielsweise auf die Spur I5 den Flowpoint *Deactivate* mit dem Kontext SimCar ein.

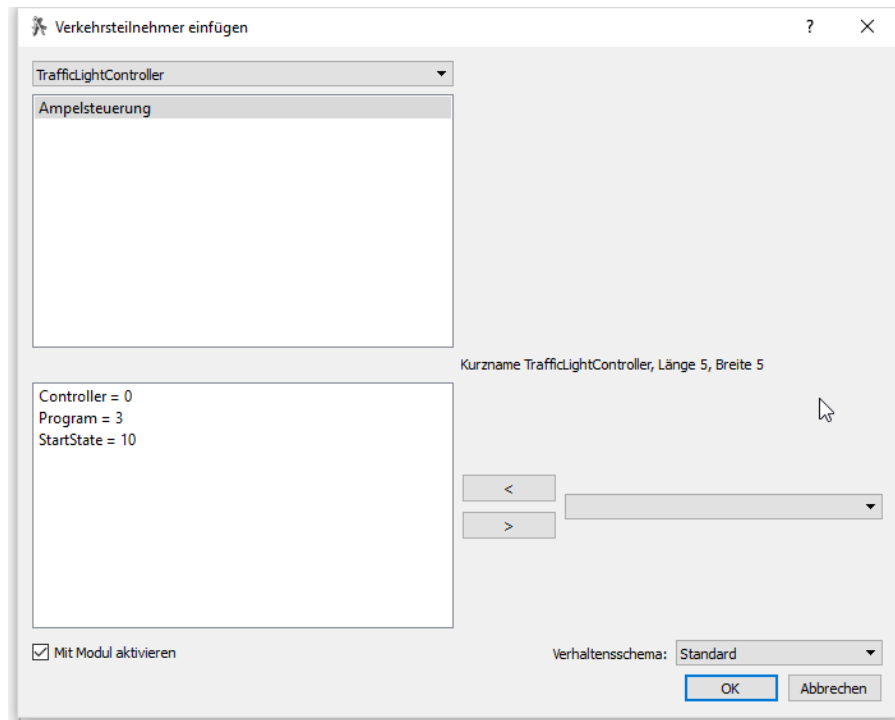
Das Szenario sieht nun wie folgt aus:



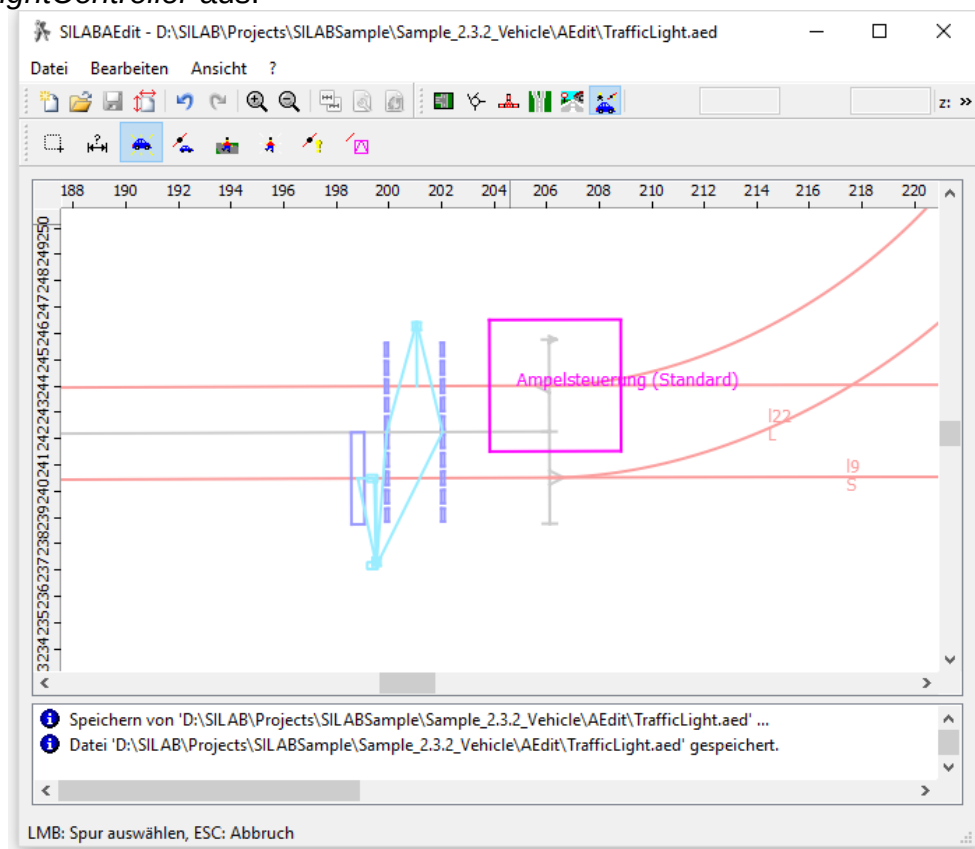
2.6 Ampelanlagen

Amplanlagen werden grundsätzlich in Area2 Streckenabschnitten mit dem grafischen Editor SILABAEEdit erzeugt. Eine detaillierte Beschreibung der Bedienung von SILABAEEdit finden Sie im Dokument „SILABAEEdit.pdf“. Im Folgenden wird die Vorgehensweise an einem Beispiel einer X-Kreuzung mit Fußgängerampel beschrieben.

In dem Editor SILABAEEdit werden zunächst in dem Decoration Layer die Ampelbäume in die Kreuzung nahe der Haltelinien eingefügt. Die Fußgängerampeln werden mit den entsprechenden Fußgängerfurten verbunden. Die Verkehrsteilnehmer werden später an der Position des Ampelbaumes (unabhängig von der Position der Haltelinien) bei einem roten Signal anhalten. Aus diesem Grund muss der Ampelbaum auf oder nahe genug an der Haltelinie positioniert werden. Für die Steuerung der Ampel wird ein *TrafficLightController* benötigt. Dieser wird wie ein Verkehrsteilnehmer in dem Kreuzungsbereich eingefügt.



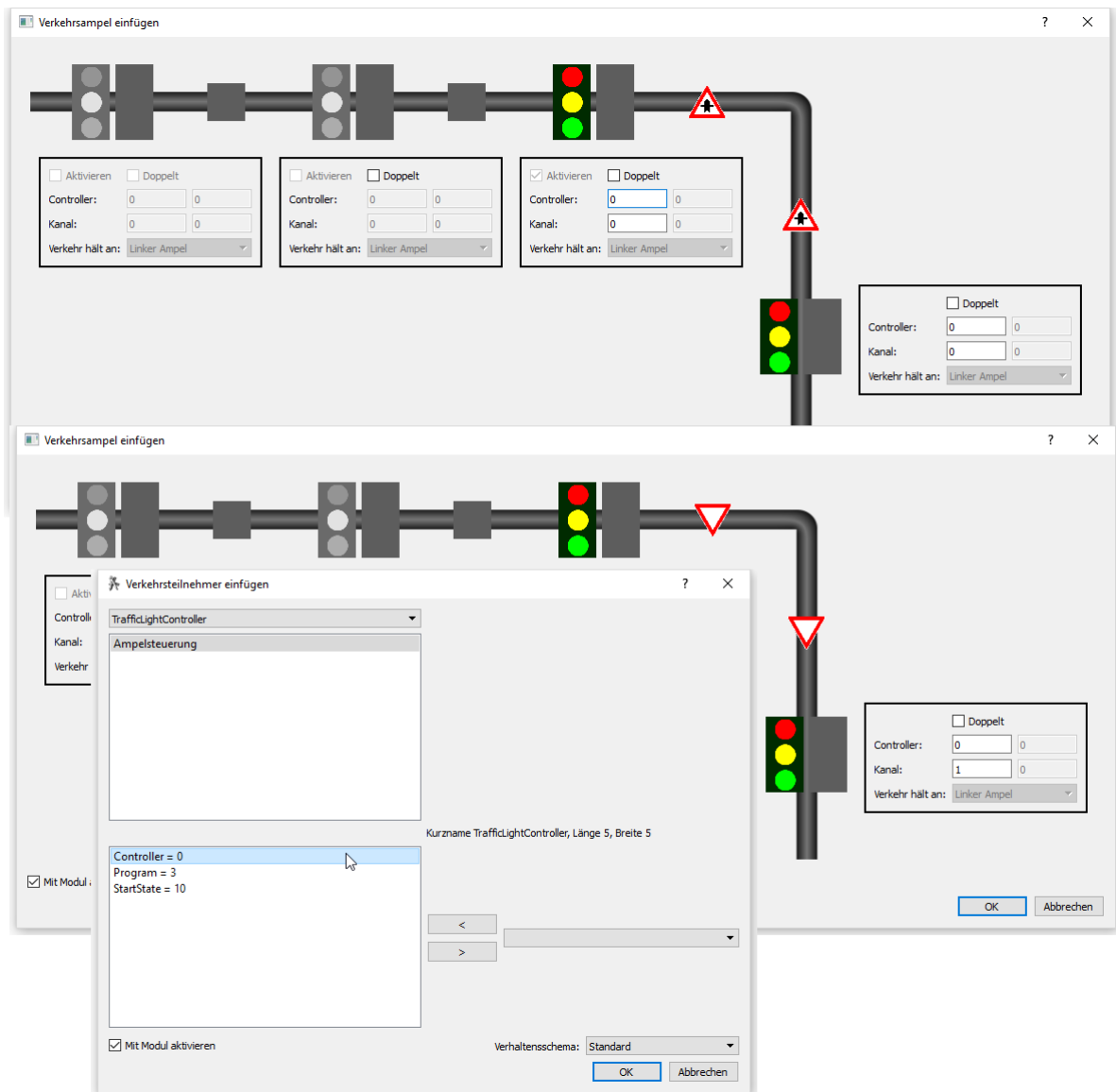
Hierzu wählen Sie statt *Vehicle* in der oberen Auswahlbox des Einfüge Dialogs den *TrafficLightController* aus.



Die Position des *TrafficLightControllers* spielt keine Rolle. In komplexeren Kreuzungen findet man den zu einer Ampelanlage gehörigen *TrafficLightController*

jedoch leichter wieder, wenn sich dieser in der Nähe der zugehörigen Ampelbäumen befindet.

Der Parameter *Controller* innerhalb des *TrafficLightControllers* muss auf die zugehörigen Ampelbäume und Fußgängerampeln abgestimmt werden. In diesem Beispiel wird bei den Ampelbäumen und dem *TrafficLightController* immer die Nummer 0 verwendet. Der Parameter *Channel* eines Ampelbaumes wird entsprechend der Zugehörigkeit des Ampelbaumes zur Straßenführung gesetzt.



Die Zuordnungen der *Channels* werden der Beschreibung des ausgewählten Ampelprogramms 2 entnommen (siehe auch Abschnitt 5.1.3.1).

Bei der Hauptstraße wird der *Channel* 0 verwendet und bei der Nebenstraße der *Channel* 1. Die Fußgängerampel der Hauptstraße erhält den *Channel* 2 und die der Nebenstraße den *Channel* 3.

Der Startzustand 10 des Programms schaltet die Ampelanlage der Hauptstraße zunächst auf Rot und die der Nebenstraße auf Grün.

2.7 Beispiele

2.7.1 Vorausfahrendes Fahrzeug auf einem Course Streckenabschnitt

Im folgenden Beispiel wird hinter einer Kurve ein vorausfahrendes Fahrzeug aufgesetzt. Die Strecke besteht aus zwei Course Instanzen, einer Kreuzung und einer weiteren Course Instanz zur Weiterführung der Umleitungsstrecke. Der Übersicht halber wird in diesem Beispiel auf Ausgestaltung der Course Instanzen verzichtet. Der Sichtschutz der Eingangskurve bedarf jedoch einer dichteren Bepflanzung, um die Aufsetzposition des vorausfahrenden Fahrzeuges zu verdecken.

```
define Module CarFollow
{
    ...
    define Course ct1
    {
        Type = Urban;
        Bend (200, -230);
        Straight(30);#230m
        HeightProfile = {
            ( 0, 50, 800),
            ( 3, 150, 700),
            ( 0, 30, 600)
        };
        ...
    };
    define Course ct2
    {
        Type = Urban;
        Straight(30);
        Bend (10, 60);
        Straight(22);
        Bend (8, -70);
        Straight(40);
        Bend (200, 130);
        Straight (300);
        Bend (70, -90);
        Straight(20);#400m
        HeightProfile = {
            ( 0, 20, 490),
            ( 5, 65, 550),
            (-4, 100, 750),
            (-1, 115, 800),
            ( 0, 400, 690)
        };
    };
};
Traffic TRF
{
    TrafficFlow Oncoming
    {
        ...
    };
    VehicleX CarFollow
    {
        Vehicle = OpelZafira;
        Colour = brown;
        Dirt = 0.2;
        Position = (C1,190,1);
        BehaviourScheme = Standard;
        StartSpeed = 13.8;
        TargetSpeed = 27.8;
        Flowpoints =
        {
            (C1, 1,1,SimCar ,Activate),
            (C2, 40,1,Traffic ,TargetSpeed,22.2),
            (C2,200,1,Traffic ,TargetSpeed,27.8),
        }
    }
}
```

```

                                (C2,350,1,Traffic ,Right)
                                };
                                };
};
include parts/Urban_ctT_Area2.cfg
ct1 C1;
ct2 C2;
ctT CT;
ct2 C3;
Connections = {
    C1.Begin <-> Port1,
    C1.End   <-> C2.Begin,
    C2.End   <-> CT.Port1,
    CT.Port2 <-> Port2,
    CT.Port3 <-> C3.Begin,
    C3.End   <-> Port3
};
};

```

Das vorausfahrende Fahrzeug wird hinter einer Kurve im Abstand von ca. 190 m vor dem EGO – Fahrzeug mit einer Zielgeschwindigkeit von 100 km/h (ca. 27.8 m/s) aufgesetzt. Die Startgeschwindigkeit beträgt 50 km/h (13.8 m/s), so dass das Vorderfahrzeug zunächst noch beschleunigt, wenn sich das EGO – Fahrzeug nähert. Am Anfang der zweiten Course Instanz wird die Zielgeschwindigkeit des Vorderfahrzeuges auf 80 km/h geändert. Gegen Ende der zweiten Course Instanz beschleunigt das Vorderfahrzeug wieder auf 100 km/h. Das Vorderfahrzeug wird, bedingt durch den letzten Flowpoint, angewiesen, an der nächsten Gelegenheit rechts abzubiegen. Diese Gelegenheit ergibt sich bei der nachfolgenden Kreuzung CT.

2.7.2 Plötzliches Bremsen eines vorausfahrenden Fahrzeugs

Auf einer Landstraße fährt vor dem EGO – Fahrzeug ein langsames Fahrzeug. Wenn der Fahrer zu dicht auf das Vorderfahrzeug auffährt, dann soll dieses plötzlich bremsen und so eine Kollision provozieren.

Da dieses Verhalten nicht mit einem vordefinierten Vorhaltensmodell realisiert werden kann, muss es mit einzelnen Befehlen für den Verkehrsteilnehmer definiert werden. In diesem Fall wird das sehr flexible Verhaltensmodell **CommandAction** (siehe Abschnitt 5.3.6.6) verwendet, um das Bremsmanöver in Abhängigkeit vom Abstand auszulösen.

Die Befehlsfolge für den Bremsvorgang lautet:

1. Verzögerung -1.5m/s^2
2. 2.5 Sekunden warten
3. Beschleunigung von 1m/s^2 (also wieder anfahren / beschleunigen)
4. 2.5 Sekunden warten

```

VehicleX Brake
{
    Vehicle = Car;
    Position = (C1,250,1);
    Behaviour = {
        (CommandAction,2,
            TRFC_AX,-1.5,
            TRFC_DELAY,2500,
            TRFC_AX,1,
            TRFC_DELAY,2500),
        (EXSTD1, 5, 25)
    };
    Flowpoints =
    {
        (C1,100,1,SimCar,Activate),
        (C1,110,1,SimCar,Synchronize,-10),
        (C1,250,1,Traffic,ActivateCommandAction,2),
        (C1,260,1,Traffic,DistanceLess,60,5000),
        (C1,650,1,Traffic,DeactivateCommandAction,2),
        (C1,650,1,Traffic,TargetSpeed,15),
        (C1,680,1,Traffic,Right)
    };
};

```

Diese Situation wird auf einer ca. 700m langen Landstraßenstrecke (Instanzname C1) aufgebaut. Nach dem Landstraßenstück folgt eine Kreuzung, an der das vorausfahrende Fahrzeug rechts abbiegen kann. Das EGO – Fahrzeug fährt an dieser Kreuzung geradeaus.

Der Ablauf ist folgendermaßen gestaltet:

- Mit dem ersten Flowpoint wird das Fahrzeug 150m vor dem EGO – Fahrzeug hinter einer Kurve aufgesetzt.
- Nach einer Strecke von 10m wird dessen Geschwindigkeit mit der Geschwindigkeit des EGO – Fahrzeuges synchronisiert. Danach fährt das Fahrzeug 10 m/s langsamer als das EGO – Fahrzeug.
- Schon beim Aufsetzen des Verkehrsteilnehmers wird das Verhaltensmodell **CommandAction** aktiviert und bei Streckenmeter 260 von dem Verkehrsteilnehmer scharf geschaltet. Dieses Modell wird nun die beschriebene Befehlsfolge abarbeiten, wenn der Abstand zum EGO – Fahrzeug weniger als 60 m beträgt (1. Parameter von **DistanceLess**). Der zweite Parameter (4000) sorgt dafür, dass das der **CommandAction** nachgeschaltete Verhaltensmodell 4 Sekunden lang blockiert bleibt, wenn die Befehlsfolge gesendet wird.
- Bei Streckenmeter 500 deaktiviert der Verkehrsteilnehmer das Verhaltensmodell **CommandAction**. Wenn also bis zu diesem Zeitpunkt das Bremsmanöver noch nicht eingetreten ist, dann wird es auch nicht mehr eintreten.
- Bei Streckenmeter 650 beschleunigt der Verkehrsteilnehmer auf 15 m/s, um das Szenario zu verlassen.
- Der letzte Flowpoint veranlasst den Verkehrsteilnehmer an der nächsten Gelegenheit rechts abzubiegen.

2.7.3 Kopplung eines vorausfahrenden Fahrzeugs

Auf einer Landstraßenstrecke soll ein vorausfahrendes Fahrzeug immer so schnell fahren wie das EGO – Fahrzeug. Diese Kopplung kann mit dem Verhaltensmodell **SimCarLocked** (siehe Abschnitt 5.3.6.4) realisiert werden. Hierzu muss das Verhaltensmodell in die hierarchische Menge der Verhaltensmodelle eingegliedert werden.

Im folgenden Beispiel wurde das Modell allen anderen Verhaltensmodellen vorangestellt.

```
Vehicle InFront
{
    Vehicle = PorscheMacan;
    Colour = silver;
    Dirt = 0.0;
    Position = (C1,300,1);
    Behaviour =
    {
        (SimCarLocked,200),
        (HazardFree, 29, 29, 5, 3, 8, -10)
    };
    Flowpoints =
    {
        (C1,100,1,SimCar,Activate),
        (C1,101,1,SimCar,LockDistance,100),
        (C1,102,1,SimCar,AutoBrakeLightControl,0),
        (C1,103,1,SimCar,BrakeLight,0),

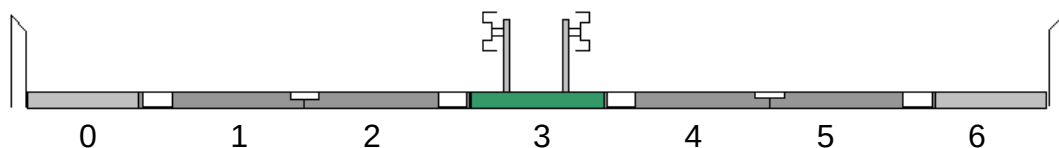
        (C1,500,1,SimCar,LockDistance,50),
        (C1,500,1,SimCar,BrakeLight,1),
        (C1,600,1,SimCar,BrakeLight,0),
        (C1,600,1,SimCar,Unlock)
    };
};
```

Das Szenario folgt diesem Ablauf:

- Mit dem ersten Flowpoint wird das Vorderfahrzeug in 200 m Entfernung aufgesetzt.
- Mit den nächsten Flowpoints wird zunächst das Fahrzeug auf 100 m Abstand gebracht (**LockDistance**) und dafür gesorgt, dass seine Bremslichter nicht aufleuchten.
- Das Vorderfahrzeug wird sich nun zurückfallen lassen, bis die Distanz erreicht wurde. Danach wird es jede Geschwindigkeitsänderung des EGO - Fahrzeugs mitmachen um den Abstand konstant zu halten.
- Überfährt das EGO – Fahrzeug den Streckenmeter 500, dann wird der Abstand zum vorausfahrenden Fahrzeug weiter auf 50 m reduziert. Diesmal werden die Bremsleuchten eingeschaltet, um den Abbremsvorgang dem Fahrer des EGO – Fahrzeuges mitzuteilen.
- Bei Streckenmeter 600 wird die Kopplung aufgehoben und das Vorderfahrzeug wieder auf 29 m/s beschleunigt.

2.7.4 Stau

In diesem Beispiel wird ein Stau bzw. zähfließender Verkehr auf einem Autobahnstück realisiert. Der verwendete Querschnitt hat sechs Spuren, die Fahrspuren in die eigene Richtung haben die Spurnummern 4 und 5:



Das Szenario findet auf drei aufeinanderfolgenden Course Streckenabschnitten mit jeweils einer Länge von 2km statt. Im dritten Course Streckenabschnitt (CJam) findet der eigentliche Stau statt.

```
define Module TrafficJam
{
    ...
    define Course ctA
    {
        NodeID = 1001;
        Type = Highway;
        Straight (1000);
        Straight (1000);
        HeightProfile =
        {
            (0.0, 500.0, 7000),
            (1.0, 1000.0, 7000),
            (0.0, 500.0, 7000)
        };
        ...
    };
    define Course ctB
    {
        NodeID = 1002;
        Type = Highway;
        Bend (1000, 2000);
        Bend (1000, -2000);
        HeightProfile =
        {
            (0.0, 250, 7000),
            (1.0, 500.0, 7000),
            (-1.5, 1000.0, 7000),
            (0.0, 250, 7000)
        };
        ...
    };

    define Course ctJam
    {
        NodeID = 1003;
        Type = Highway;
        Bend (1000, 5000);
        Bend (1000, -5000);
        HeightProfile =
        {
            (0.0, 500, 7000),
            (0.5, 500.0, 7000),
            (-2.0, 300.0, 2000),
            (0.0, 200.0, 7000),
            (1.5, 200.0, 7000),
            (0.0, 300, 7000)
        };
        ...
    };

    include parts\HighwayExit_Area2.cfg
    ctA      CA;
    ctB      CB;
    ctJam    CJam;
    ctExit   CExit;
    Connections =
    {
        Port1 <-> CA.Begin,
        CA.End <-> CB.Begin,
        CB.End <-> CJam.Begin,
        CJam.End <-> CExit.Port1,
        CExit.Port2 <-> CA.Begin,
        CExit.Port3 <-> Port2
    };
};
```

Das Modul besteht aus drei Course Streckeninstanzen `CA`, `CB` und `CJam`, die in dieser Reihenfolge hintereinandergeschaltet wurden. `CA` und `CB` werden genutzt um die Fahrzeuge vor dem EGO – Fahrzeug aufzubauen und den Stau in der Instanz `CJam` vorzubereiten. Bei der Instanz `CExit` handelt es sich um eine Autobahnausfahrt über die das EGO – Fahrzeug die Situation verlassen wird.

Die Höhenprofile der Course Streckenabschnitte dienen dazu weiter entfernt aufgesetzte Verkehrsteilnehmer zu verdecken.

Für die Verkehrsdefinition werden der Übersichtlichkeit halber drei Verkehrsabschnitte verwendet:

- Der Abschnitt `Lane5` beschreibt den Verkehr für die rechte Fahrspur (Spur 5).
- Der Abschnitt `Lane4` beschreibt den Verkehr für die linke Fahrspur (Spur 4).
- Der Abschnitt `Oncomming` beschreibt die Ausschmückung mit Quellen für die linken Gegenspuren (Spur 1 und 2)

Auf der rechten Fahrspur werden nacheinander insgesamt 11 Einzelfahrzeuge vom Typ `Car` aufgesetzt, so dass sich die rechte Spur vor dem EGO – Fahrzeug langsam füllt. Vor diesen Fahrzeugen werden gegen Mitte der Instanz `CB` vier Staufahrzeuge in die Courseinstanz `CJam` aufgesetzt. Diese vier Fahrzeuge bekommen über Flowpoints an den unterschiedlichsten Positionen in der Instanz `CJam` von dem EGO – Fahrzeug neue Zielgeschwindigkeiten zugewiesen. Diese Geschwindigkeiten gehen bis 0.001 m/s runter – aber nie in den Stillstand.

```
TrafficLane5
{
    VehicleX r0
    {
        Vehicle = Car;
        Position = (CA, 500, 5);
        Behaviour = {(HazardFree, 15, 27, 5, 3, 8, -10)};
        Flowpoints ={(CA, 1, 5, SimCar,Activate)};
    };
    VehicleX r1
    {
        Vehicle = Car;
        Position = (CA, 800, 5);
        Behaviour = {(HazardFree, 15, 27, 5, 3, 8, -10)};
        Flowpoints ={(CA, 1, 5, SimCar,Activate)};
    };
    VehicleX r2
    {
        Vehicle = Car;
        Position = (CA, 1160, 5);
        Behaviour = {(HazardFree, 30, 27, 5, 3, 8, -10)};
        Flowpoints ={(CA, 560, 5, SimCar,Activate)};
    };
    VehicleX r3
    {
        Vehicle = MercedesActros;
        Colour = white;
        Dirt = 0.0;
        Position = (CA, 1500, 5);
        Behaviour = {(HazardFree, 30, 30, 5, 3, 8, -10)};
        Flowpoints ={(CA, 700, 5, SimCar,Activate)};
    };
    VehicleX r4
    {
        Vehicle = Car;
        Position = (CA, 1800, 5);
        Behaviour = {(HazardFree, 32, 32, 5, 3, 8, -10)};
        Flowpoints ={(CA, 800, 5, SimCar,Activate)};
    };
    VehicleX r5
    {
        Vehicle = Car;
        Position = (CA, 1800, 5);
        Behaviour = {(HazardFree, 33, 33, 5, 3, 8, -10)};
        Flowpoints ={(CA, 900, 5, SimCar,Activate)};
    };
    VehicleX r6
    {
        Vehicle = MercedesActros;
        Colour = blue;
        Dirt = 0.0;
        Position = (CB, 100, 5);
        Behaviour = {(HazardFree, 33, 33, 5, 3, 8, -10)};
        Flowpoints ={(CA, 1000, 5, SimCar,Activate)};
    };
    VehicleX r7
    {
        Vehicle = Car;
        Position = (CB, 200, 5);
        Behaviour = {(HazardFree, 33, 33, 5, 3, 8, -10)};
        Flowpoints ={(CA, 1000, 5, SimCar,Activate)};
    };
    VehicleX r8
    {
        Vehicle = Car;
        Position = (CB, 400, 5);
        Behaviour = {(HazardFree, 33, 33, 5, 3, 8, -10)};
        Flowpoints ={(CA, 1000, 5, SimCar,Activate)};
    };
    VehicleX r9
    {
        Vehicle = Car;
        Position = (CB, 700, 5);
        Behaviour = {(HazardFree, 33, 33, 5, 3, 8, -10)};
    }
}
```

```

        Flowpoints = {(CA, 1000, 5, SimCar, Activate)};
    };
    VehicleX r10
    {
        Vehicle = Car;
        Position = (CB, 1000, 5);
        Behaviour = {(HazardFree, 33, 33, 5, 3, 8, -10)};
        Flowpoints = {(CA, 1000, 5, SimCar, Activate)};
    };
    VehicleX Jam_Truck_Right
    {
        Vehicle = MercedesActros;
        Colour = red;
        Dirt = 0.0;
        Position = (CJam, 280, 5);
        Behaviour = {(HazardFree, 15, 15, 5, 3, 8, -10)};
        Flowpoints =
        {
            (CB, 1200, 5, SimCar, Activate),
            (CJam, 200, 5, SimCar, TargetSpeed, 9),
            (CJam, 300, 5, SimCar, TargetSpeed, 5),
            (CJam, 600, 5, SimCar, TargetSpeed, 0.003),
            (CJam, 625, 5, SimCar, TargetSpeed, 20),
            (CJam, 700, 5, SimCar, TargetSpeed, 3.5),
            (CJam, 730, 5, SimCar, TargetSpeed, 11.5),
            (CJam, 800, 5, SimCar, TargetSpeed, 0.001),
            (CJam, 809, 5, SimCar, TargetSpeed, 1.5),
            (CJam, 840, 5, SimCar, TargetSpeed, 0.2),
            (CJam, 862, 5, SimCar, TargetSpeed, 3.5),
            (CJam, 920, 5, SimCar, TargetSpeed, 0.5),
            (CJam, 952, 5, SimCar, TargetSpeed, 8.5),
            (CJam, 1100, 5, SimCar, TargetSpeed, 0.02),
            (CJam, 1115, 5, SimCar, TargetSpeed, 3.5),
            (CJam, 1145, 5, SimCar, TargetSpeed, 30.5),
            (CJam, 1215, 5, SimCar, TargetSpeed, 1.5),
            (CJam, 1345, 5, SimCar, TargetSpeed, 30.5),
            (CJam, 1395, 5, SimCar, TargetSpeed, 1.5),
            (CJam, 1630, 5, SimCar, TargetSpeed, 10),
            (CJam, 1730, 5, SimCar, TargetSpeed, 0.20),
            (CJam, 1760, 5, SimCar, TargetSpeed, 2),
            (CJam, 1799, 5, SimCar, TargetSpeed, 20)
        };
    };
    VehicleX Jam_Car1_Right
    {
        Vehicle = Car;
        Position = (CJam, 200, 5);
        Behaviour = {(HazardFree, 15, 15, 5, 3, 8, -10)};
        Flowpoints =
        {
            (CB, 1200, 5, SimCar, Activate),
            (CJam, 200, 5, SimCar, TargetSpeed, 5),
            (CJam, 600, 5, SimCar, TargetSpeed, 0.4),
            (CJam, 630, 5, SimCar, TargetSpeed, 20),
            (CJam, 910, 5, SimCar, TargetSpeed, 25)
        };
    };
    VehicleX Jam_Car2_Right
    {
        Vehicle = Car;
        Position = (CJam, 150, 5);
        Behaviour = {(HazardFree, 15, 15, 5, 3, 8, -10)};
        Flowpoints =
        {
            (CB, 1200, 5, SimCar, Activate),
            (CJam, 200, 5, SimCar, TargetSpeed, 5),
            (CJam, 600, 5, SimCar, TargetSpeed, 0.4),
            (CJam, 630, 5, SimCar, TargetSpeed, 20),
            (CJam, 920, 5, SimCar, TargetSpeed, 25)
        };
    };
    VehicleX Jam_Car3_Right
    {
        Vehicle = Car;
        Position = (CJam, 100, 5);
    }

```

```
Behaviour = {(HazardFree, 15, 15, 5, 3, 8, -10)};
Flowpoints =
{
    (CB, 1200, 5, SimCar, Activate),
    (CJam, 200, 5, SimCar, TargetSpeed, 5),
    (CJam, 600, 5, SimCar, TargetSpeed, 0.4),
    (CJam, 630, 5, SimCar, TargetSpeed, 20),
    (CJam, 930, 5, SimCar, TargetSpeed, 25)
};
};
```

Auf der Spur 4 werden in dem folgenden Verkehrsabschnitt 8 Einzelfahrzeuge nacheinander vor dem EGO – Fahrzeug aufgesetzt und gegen Mitte von _{CB} ein Staufahrzeug vor den Einzelfahrzeugen.

```
TrafficEinzelfahrzeuge_Spur4
{
    VehicleX 10
    {
        Vehicle = Car;
        Position = (CA, 1, 4);
        Behaviour = {(HazardFree, 55, 55, 5, 3, 8, -10)};
        Flowpoints ={(CA, 250, 5, SimCar,Activate)};
    };
    VehicleX 11
    {
        Vehicle = Car;
        Position = (CA, 20, 4);
        Behaviour = {(HazardFree, 50, 45, 5, 3, 8, -10)};
        Flowpoints ={(CA, 300, 5, SimCar,Activate)};
    };
    VehicleX 12
    {
        Vehicle = Car;
        Position = (CA, 10, 4);
        Behaviour = {(HazardFree, 50, 45, 5, 3, 8, -10)};
        Flowpoints ={(CA, 300, 5, SimCar,Activate)};
    };
    VehicleX 13
    {
        Vehicle = Car;
        Position = (CA, 10, 4);
        Behaviour = {(HazardFree, 50, 45, 5, 3, 8, -10)};
        Flowpoints ={(CA, 400, 5, SimCar,Activate)};
    };
    VehicleX 14
    {
        Vehicle = Car;
        Position = (CA, 20, 4);
        Behaviour = {(HazardFree, 50, 45, 5, 3, 8, -10)};
        Flowpoints ={(CA, 400, 5, SimCar,Activate)};
    };
    VehicleX 15
    {
        Vehicle = Car;
        Position = (CA, 400, 4);
        Behaviour = {(HazardFree, 50, 44, 5, 3, 8, -10)};
        Flowpoints ={(CA, 1000, 5, SimCar,Activate)};
    };
    VehicleX 16
    {
        Vehicle = Car;
        Position = (CA, 100, 4);
        Behaviour = {(HazardFree, 40, 44, 5, 3, 8, -10)};
        Flowpoints ={(CA, 1000, 5, SimCar,Activate)};
    };
    VehicleX 17
    {
        Vehicle = Car;
        Position = (CA, 10, 4);
        Behaviour = {(HazardFree, 40, 44, 5, 3, 8, -10)};
        Flowpoints ={(CA, 1000, 5, SimCar,Activate)};
    };
};
```

```

VehicleX Jam_Left
{
    Vehicle = Car;
    Position = (CJam, 200, 4);
    Behaviour = {(HazardFree, 15, 16, 5, 3, 8, -10)};
    Flowpoints =
    {
        (CB, 1200, 5, SimCar, Activate),
        (CJam, 200, 5, SimCar, TargetSpeed, 9),
        (CJam, 300, 5, SimCar, TargetSpeed, 5),
        (CJam, 600, 5, SimCar, TargetSpeed, 0.003),
        (CJam, 625, 5, SimCar, TargetSpeed, 20),
        (CJam, 700, 5, SimCar, TargetSpeed, 3.5),
        (CJam, 730, 5, SimCar, TargetSpeed, 13.5),
        (CJam, 800, 5, SimCar, TargetSpeed, 0.001),
        (CJam, 809, 5, SimCar, TargetSpeed, 1.5),
        (CJam, 840, 5, SimCar, TargetSpeed, 0.2),
        (CJam, 862, 5, SimCar, TargetSpeed, 3.5),
        (CJam, 920, 5, SimCar, TargetSpeed, 0.5),
        (CJam, 942, 5, SimCar, TargetSpeed, 8.5),
        (CJam, 1100, 5, SimCar, TargetSpeed, 0.02),
        (CJam, 1115, 5, SimCar, TargetSpeed, 3.5),
        (CJam, 1145, 5, SimCar, TargetSpeed, 30.5),
        (CJam, 1215, 5, SimCar, TargetSpeed, 1.5),
        (CJam, 1345, 5, SimCar, TargetSpeed, 30.5),
        (CJam, 1395, 5, SimCar, TargetSpeed, 1.5),
        (CJam, 1630, 5, SimCar, TargetSpeed, 12),
        (CJam, 1730, 5, SimCar, TargetSpeed, 0.20),
        (CJam, 1760, 5, SimCar, TargetSpeed, 2),
        (CJam, 1799, 5, SimCar, TargetSpeed, 20),
        (CJam, 1850, 5, SimCar, TargetSpeed, 45)
    };
};
};

```

Für die Ausschmückung der Gegenspur werden noch zwei TrafficFlow an das EGO – Fahrzeug geheftet, welche Fahrzeuge auf der Spur 1 und 2 erzeugen.

```

Traffic Oncoming
{
    TrafficFlow Lane1
    {
        Source SL1
        {
            Position =(SimCar, -900, 1);
            Vehicle ={(7, Car , (HazardFree, 20, 20, 5, 3, 8, -10))};
            Parameters = (Gauss, 2,1, 451,512);
        };
        Drain DL11
        {
            Position = (SimCar, 920, 1);
        };
        Flowpoints=
        {
            (CA, 2, 5, SimCar, ActivateSource, (SL1)),
            (CA, 1, 5, SimCar, ActivateDrain, (DL1))
        };
    };
};

```

```

TrafficFlow Lane2
{
    Source SL2
    {
        Position =(SimCar, -900, 2);
        Vehicle ={(5, Car, (HazardFree, 30, 30, 5, 3, 8, -10))};
        Parameters = (Gauss, 2,1, 451,512);
    };
    Drain DL2
    {
        Position = (SimCar, 920, 2);
    };
    Flowpoints=
    {
        (CA, 2, 5, SimCar, ActivateSource, (SL2)),
        (CA, 1, 5, SimCar, ActivateDrain, (DL2))
    };
};

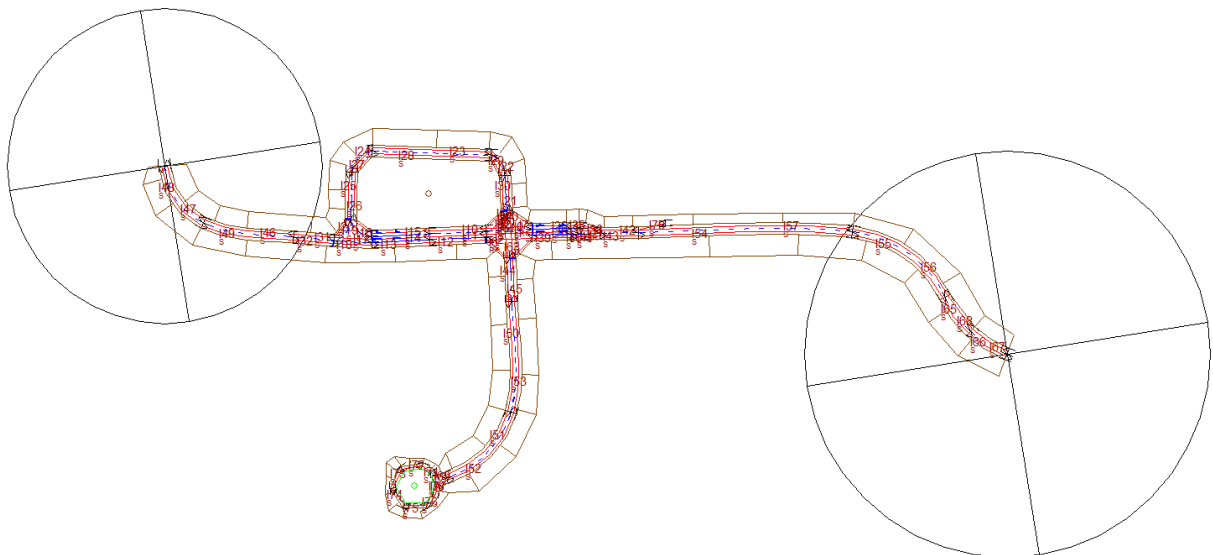
};

```

2.7.5 Vorausfahrendes und abbiegendes Fahrzeug in einer Area2

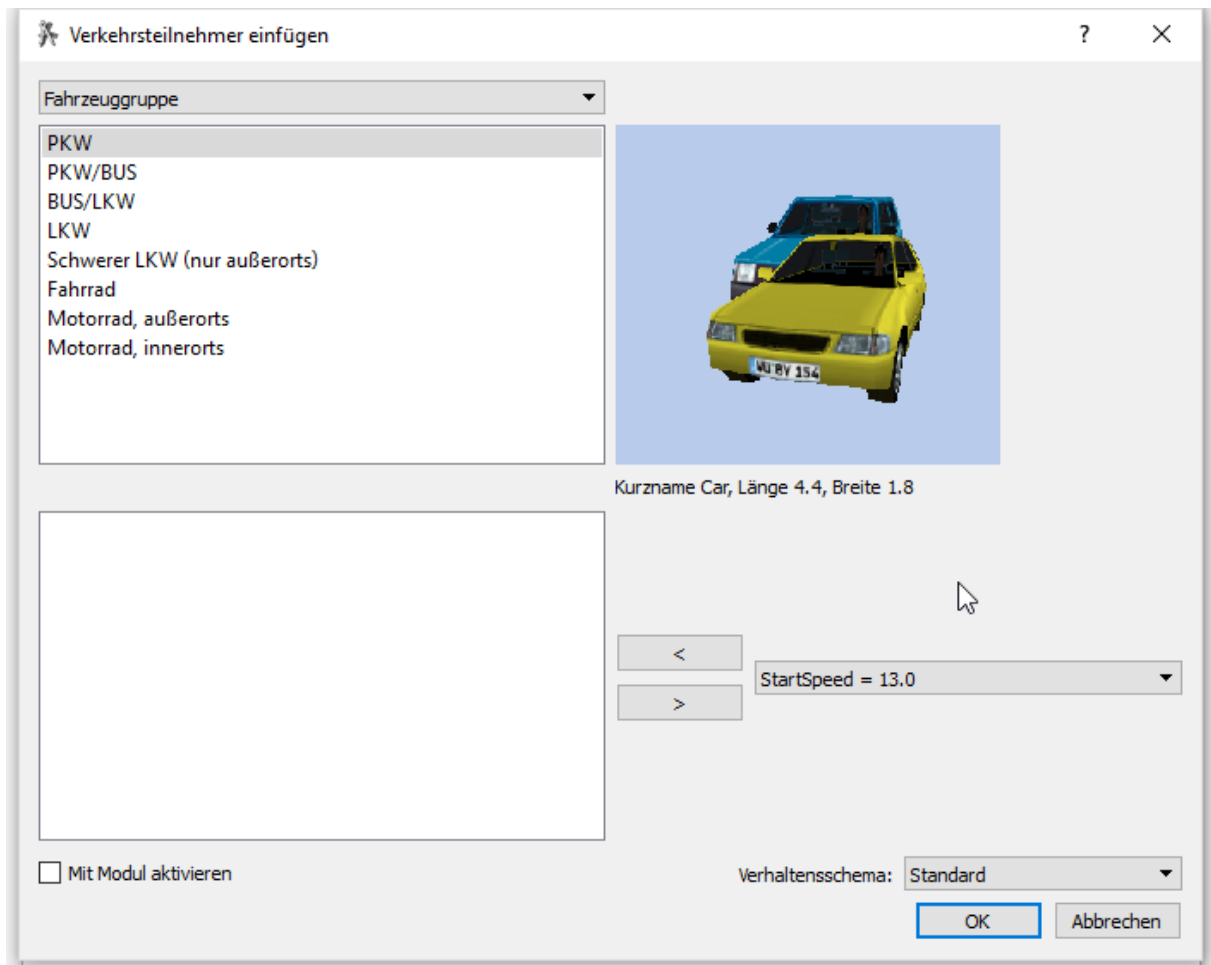
Dieses Beispiel beschreibt ein Area2 Modul mit zwei Verkehrsteilnehmern. Der erste Verkehrsteilnehmer biegt links ab und der zweite an einer anderen Stelle rechts. Die Ha7uptdurchfahrtsrichtung für den EGO – Fahrer ist bei diesem Modul immer geradeaus.

Damit dieses Szenario als Modul mit anderen Szenario-Modulen kombinierbar ist, benötigt die Straßengeometrie einen Sichtschutz an der Einfahrt und an der Ausfahrt. Hinter diesem Sichtschutz können die modulgebundenen Fahrzeuge für den EGO – Fahrer verdeckt aufgesetzt werden. Der Sichtschutz der Ausfahrt dient dazu, das nachfolgende Szenario vollständig zu verdecken. Ein Sichtschutz kann leicht mit aufeinander abgestimmten Kurven und Häuserzeilen realisiert werden. Die Ein- und Ausgangsports des Area2 Moduls erhalten dann auf den jeweiligen Sichtschutz angepasste Sichtbarkeitsradien (siehe auch Dokument „SILABAEEdit.pdf“).

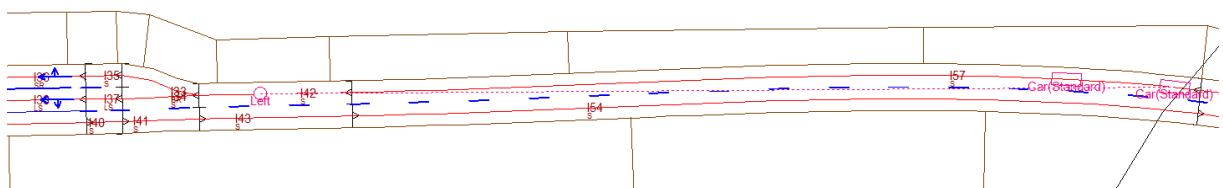


Das Modul wird von rechts nach links durchfahren. Die beiden Verkehrsteilnehmer können direkt an der Grenze des rechten Sichtbarkeitsradius aufgesetzt werden.

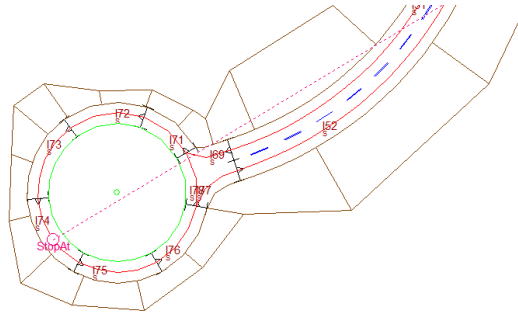
In dem Dialog zur Erzeugung der Verkehrsobjekte wird der Haken *[] Activate together with module* gesetzt, damit die Verkehrsteilnehmer aktiviert werden, sobald das EGO – Fahrzeug in das Modul einfährt. Hierbei gilt zu bedenken, dass der EGO – Fahrer in dieses Innenstadtmodul mit ca. 50 km/h hineinfahren wird. Damit die beiden Verkehrsteilnehmer auf das EGO – Fahrzeug treffen, müssen diese langsamer fahren als das EGO – Fahrzeug. Aus diesem Grund wird die Startgeschwindigkeit der beiden Verkehrsteilnehmer auf 0 gesetzt.



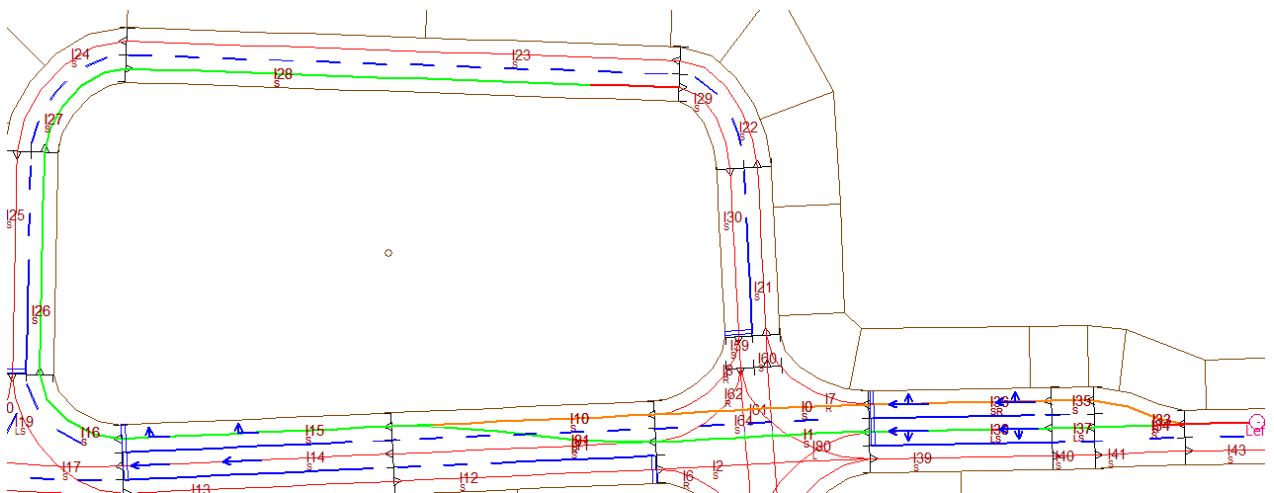
Damit der erste Verkehrsteilnehmer nach links abbiegt, muss er einen Flowpoint ‚Left‘ noch vor der Verzweigung der Fahrspuren überfahren können.



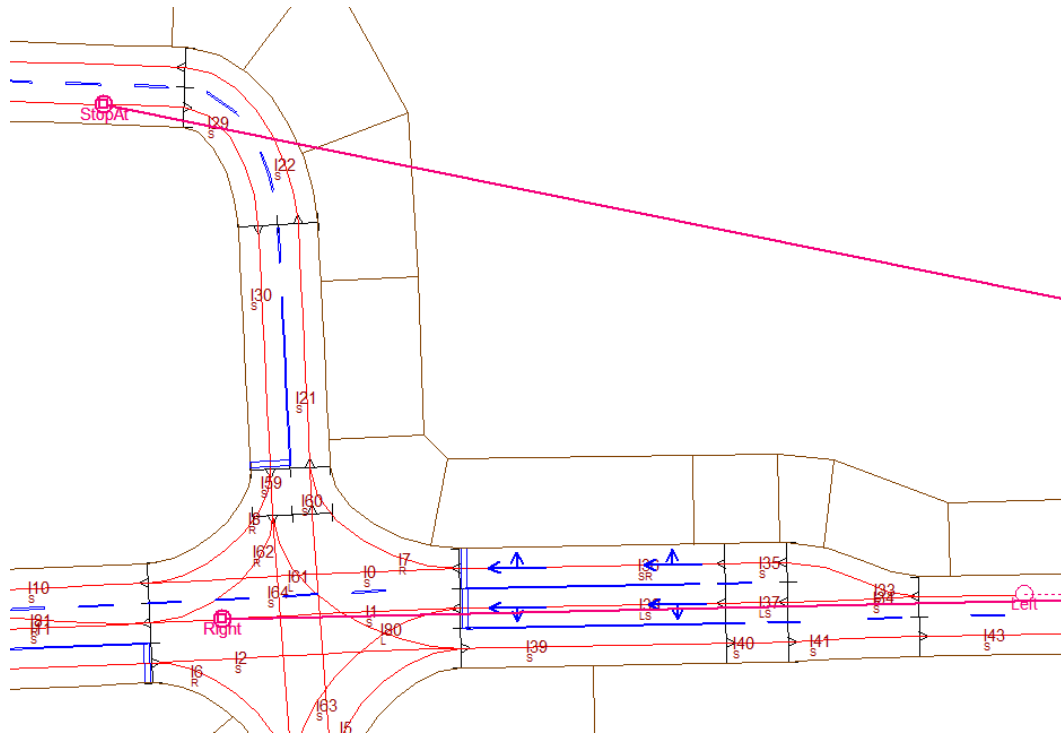
Wenn der Verkehrsteilnehmer diesen Flowpoint überfährt, wird er den Spuren folgen, die mit einem Heading „left“ versehen wurden. An der ersten Kreuzung wird er in den unteren Seitenarm hineinfahren, der in einem Kreisel endet. Damit der Verkehrsteilnehmer nicht wieder zurück in die Kreuzung fährt, sollten Sie ihn im Kreisel mit dem Flowpoint *StopAt* anhalten lassen.



Um den zweiten Verkehrsteilnehmer an der zweiten Kreuzung rechts abbiegen zu lassen, gibt es in der Geometrie zwei Wege:



Der Verkehrsteilnehmer könnte den orangen oder den grünen Weg (die Farben beziehen sich auf Markierungen in obigem Bild) nehmen. In diesem Fall ist der grüne Weg die bessere Wahl. Somit muss der Flowpoint *Right* auf I1 in der ersten Kreuzung gesetzt werden. Auch hier sollte der Verkehrsteilnehmer nach der zweiten Kreuzung in I28 gestoppt werden.



Die Linien der Flowpoints zum Verkehrsteilnehmer zeigen an, dass es sich um private Flowpoints eines Verkehrsteilnehmers handelt.

3 VERKEHRSDEFINITION IN MODULEN MIT GEMISCHTEN COURSE UND AREA2 STRECKENABSCHNITTEN

Mit den bisher vorgestellten Konzepten können Sie Verkehr

- in einem Szenario-Modul definieren, dessen Straßennetzwerken komplett aus Streckenabschnitten vom Typ Course besteht.
- in einer Area2 definieren, wenn Sie diese Area2 mit SILABAEEdit als Modul exportieren. Das exportierte Modul enthält als einzigen Streckenabschnitt Ihre Area2 (beispielsweise einen Stadtteil).

In vielen Fällen ist es jedoch nötig, dass Sie in einem Modul Course und Area2 Streckenabschnitte kombinieren. Beispiele hierfür sind Landstraßen mit Abzweigungen oder Autobahnen mit zugehörigen Auf-/Abfahrten. Auch dort ist es natürlich möglich, Verkehrsteilnehmer aufzusetzen und zu steuern.

Halten Sie sich an folgende Vorgehensweise:

1. Definieren Sie Ihren Modul-Typ (z.B. mit dem Namen „MyModuleType“) im Skript fügen Sie hierzu diesen Abschnitt ein:

```
define Module MyModuleType
{
};
```

2. Definieren und instanziiieren Sie die Course Streckenabschnitte innerhalb Ihres Modultyps.
3. Fügen Sie einen Abschnitt **Traffic** *<any name>* { ... }; in den Modul-Typ ein und definieren Sie dort mit den Methoden aus den Abschnitten 2.3.1 und 2.4.1 Verkehrsteilnehmer und Flowpoints für die Course Streckenabschnitte:

```
define Module MyModuleType
{
    # <insert types and instances of Course segments here>

    # traffic definition
    Traffic TRFOncourses
    {
        # <define VehicleX, TrafficFlow etc.here, including
        # corresponding flowpoints>
    };
};
```

4. Entwerfen Sie die Area2 Streckenabschnitte mit SILABAEEdit. Definieren Sie in SILABAEEdit Verkehrsteilnehmer und fügen Sie Flowpoints zu ihrer Steuerung ein. Bitte beachten Sie dabei folgendes: Wenn Sie Verkehrsteilnehmer mit der Option „Activate together with module start“ verwenden, funktioniert das nur, wenn sich die Area2 beim Betreten des Moduls bereits in der Sichtbarkeit des Fahrers befindet. Sicherer ist, ortsgebundene Aktivierungen von Verkehrsteilnehmern zu verwenden.
5. Exportieren Sie die Area2 Streckenabschnitte als Modul. Wählen Sie im Export-Dialog von SILABAEEdit jeweils den gleichen Namen als „Module type name“, „Area2 type name“ und „Area2 instance“.
6. Angenommen, Sie haben drei Area2 Streckenabschnitte, die Sie in einem Modul verwenden möchten. Für diese haben Sie die Namen „A1“, „A2“ und „A3“

gewählt. Binden Sie in Ihrem Modul-Typ die von SILABAEEdit exportierten Area2 Typen ein:

```
define Module MyModuleType
{
    # <insert types and instances of Course segments here>

    # types of Area2 segments
    include A1_Area2.cfg
    include A2_Area2.cfg
    include A3_Area2.cfg

    # traffic definition
    Traffic TRFOnCourses
    {
        # <define VehicleX, TrafficFlow etc.here, including
        # corresponding flowpoints>
    };
};
```

7. Instanzieren Sie die Area2 Typen. Verwenden Sie dabei den Instanznamen, den Sie beim Export vergeben haben:

```
define Module MyModuleType
{
    # <insert types and instances of Course segments here>

    # types of Area2 segments
    include A1_Area2.cfg
    include A2_Area2.cfg
    include A3_Area2.cfg

    # instances of Area2 segments
    A1 A1;
    A2 A2;
    A3 A3;

    # traffic definition
    Traffic TRFOnCourses
    {
        # <define VehicleX, TrafficFlow etc.here, including
        # corresponding flowpoints>
    };
};
```

8. Definieren Sie ein Straßennetzwerk, indem Sie die Course Instanzen und Area2 Instanzen verknüpfen (s. „Referenz Datenbasis SCNX.pdf“).

```
define Module MyModuleType
{
    # <insert types and instances of Course segments here>

    # types of Area2 segments
    include A1_Area2.cfg
    include A2_Area2.cfg
    include A3_Area2.cfg

    # instances of Area2 segments
    A1 A1;
    A2 A2;
    A3 A3;

    # road network
    Connections =
    {
        # <build up road network here>
    };
};
```

```

# traffic definition
Traffic TRFOncourses
{
    # <define VehicleX, TrafficFlow etc.here, including
    # corresponding flowpoints>
};
};

```

9. Binden Sie die von SILABAEEdit exportierten Verkehrsdefinitionen in ihrem Modul-Typ ein:

```

define Module MyModuleType
{
    # <insert types and instances of Course segments here>

    # types of Area2 segments
    include A1_Area2.cfg
    include A2_Area2.cfg
    include A3_Area2.cfg

    # instances of Area2 segments
    A1 A1;
    A2 A2;
    A3 A3;

    # road network
    Connections =
    {
        # <build up road network here>
    };

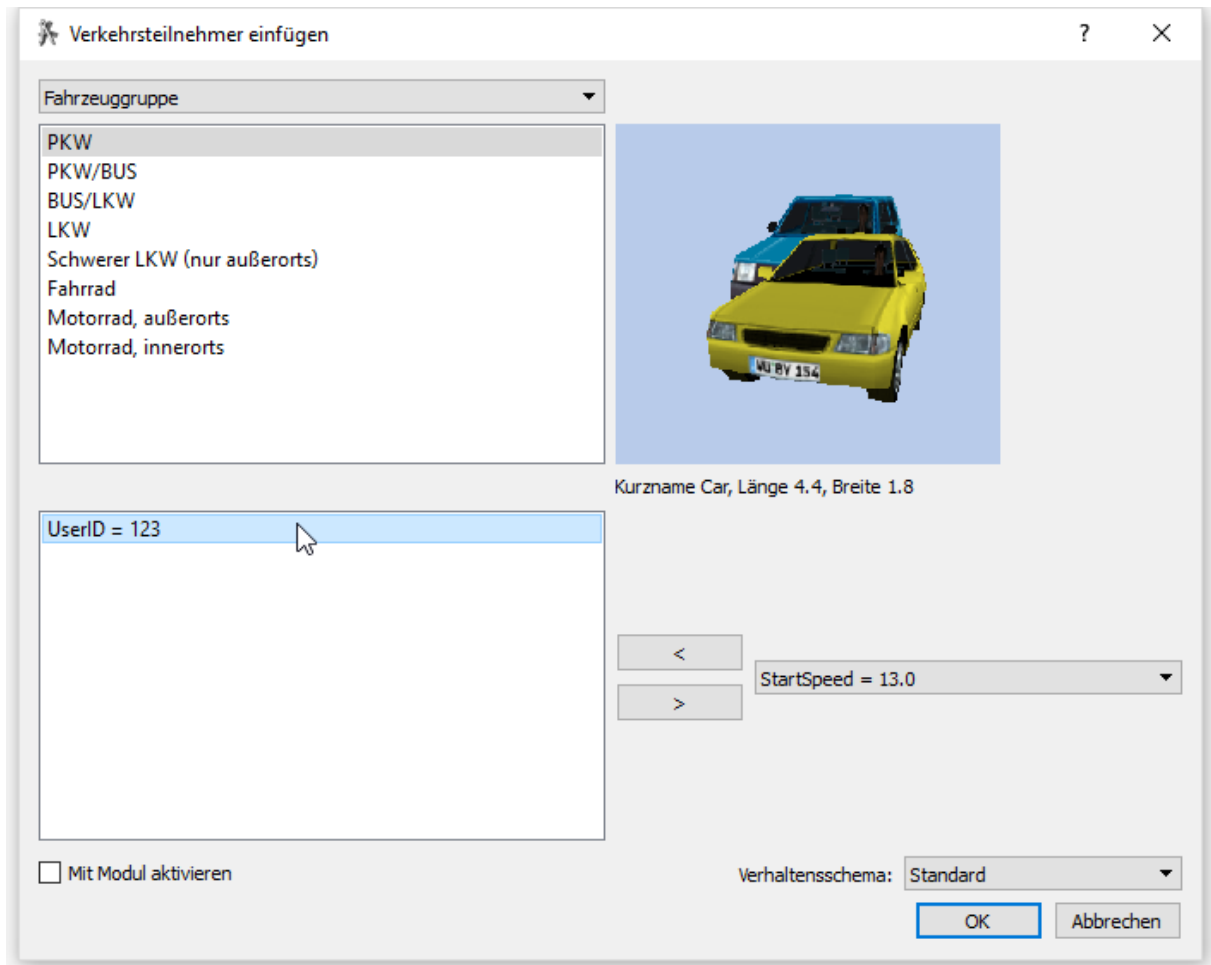
    # traffic definition
    Traffic TRFOncourses
    {
        # <define VehicleX, TrafficFlow etc.here, including
        # corresponding flowpoints>
    };
    include A1_TRF.cfg
    include A2_TRF.cfg
    include A3_TRF.cfg
};
};

```

10. Erweitern Sie Ihre Verkehrsdefinition aus Schritt 3 um Flowpoints, mit denen Sie Verkehrsteilnehmer, die in einer Area2 aufgesetzt wurden, in Course Abschnitten steuern. Näheres hierzu erfahren Sie gleich unten.
11. Erweitern Sie Ihre Verkehrsdefinition in den Area2 Streckenabschnitt (mit SILABAEEdit) um Flowpoints, um Verkehrsteilnehmer zu steuern, die in Course Streckenabschnitten aufgesetzt wurden. Vergessen Sie bitte nicht, veränderte Area2s mit den gleichen Einstellungen wie in Punkt 5 erneut zu exportieren. Im Skript brauchen Sie aufgrund des Exports nichts mehr zu ändern.

Erläuterungen zu Punkt 10:

Um in einer Area2 aufgesetzte Verkehrsteilnehmer mit Flowpoints zu steuern, die auf Course Streckenabschnitten liegen, müssen sie zunächst dem Verkehrsteilnehmer eine eindeutige UserID zuweisen.



Fügen Sie hierzu den Parameter UserID dem Verkehrsteilnehmer hinzu und ändern Sie den Wert in dem Eingabefeld. Übernehmen Sie den Wert mit der Taste *[apply]*.

Jetzt können Sie den Verkehrsteilnehmer über öffentliche Flowpoints in den Course - Abschnitten mit den Kontexten **SimCarUserID** und **TrafficUserID** steuern:

```
define Module MyModuleType
{
    # <insert types and instances of Course segments here>
    define Course c1
    {
        ...
    };
    # types of Area2 segments
    include A1_Area2.cfg
    include A2_Area2.cfg
    include A3_Area2.cfg

    # instances of Area2 segments
    A1 A1;
    A2 A2;
    A3 A3;
    c1 C1;

    # road network
    Connections =
    {
        # <build up road network here>
    };
};
```

```

# traffic definition
Traffic TRFOnCourses
{
    # <define VehicleX, TrafficFlow etc.here, including
    # corresponding flowpoints>

    Flowpoints =
    {
        (C1,100,1,TrafficUserID,123,TargetSpeed,35),
        (C1,1350,1,SimCarUserID,123,Deactivate)
    };

};
include A1_TRF.cfg
include A2_TRF.cfg
include A3_TRF.cfg
};

```

Im obigen Beispiel wird der Verkehrsteilnehmer mit der **UserID** = 123 zunächst bei Streckenmeter 100 auf der Spur 1 den Flowpoint **TargetSpeed** auslösen und seine eigene Geschwindigkeit auf 35 m/s erhöhen. Bei Streckenmeter 1350 wird er dann von dem EGO – Fahrzeug deaktiviert.

Erläuterungen zu Punkt 11:

Um in einem Course aufgesetzte Verkehrsteilnehmer mit Flowpoints einer Area2 zu steuern, setzen Sie zunächst den Parameter **UserID** innerhalb der Definition des Verkehrsteilnehmers auf einen eindeutigen Wert (z.B. 125).

```

define Module MyModuleType
{
    # <insert types and instances of Course segments here>
    define Course c1
    {
        ...
    };

    # types of Area2 segments
    include A1_Area2.cfg
    include A2_Area2.cfg
    include A3_Area2.cfg

    # instances of Area2 segments
    A1 A1;
    A2 A2;
    A3 A3;

    # road network
    Connections =
    {
        # <build up road network here>
    };

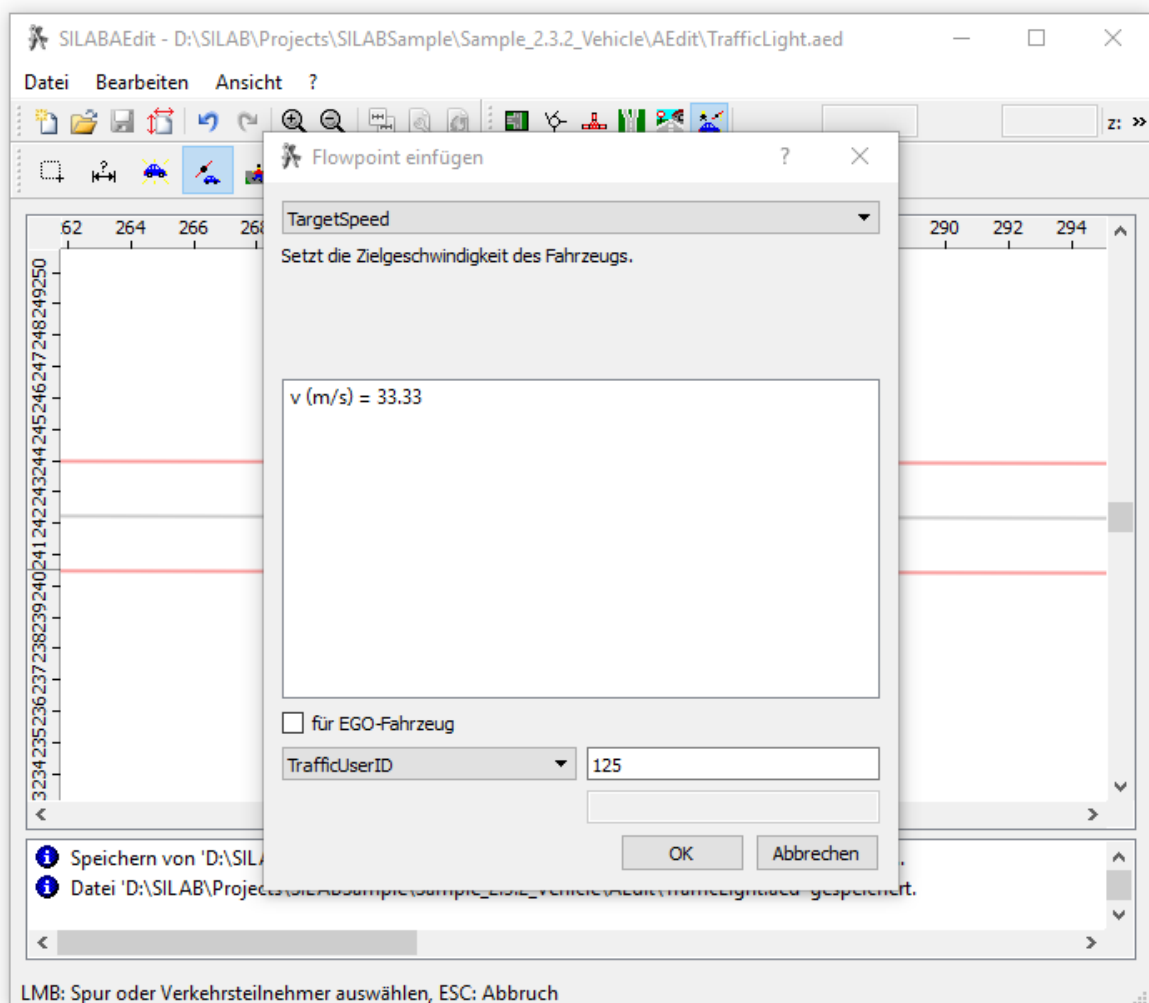
    # traffic definition
    Traffic TRFOnCourses
    {
        # <define VehicleX, TrafficFlow etc.here, including
        # corresponding flowpoints>
        VehicleX MyVehicle
        {
            Vehicle = Car;
            Position = (C1, 200, 1);
            BehaviourScheme = Standard;

```

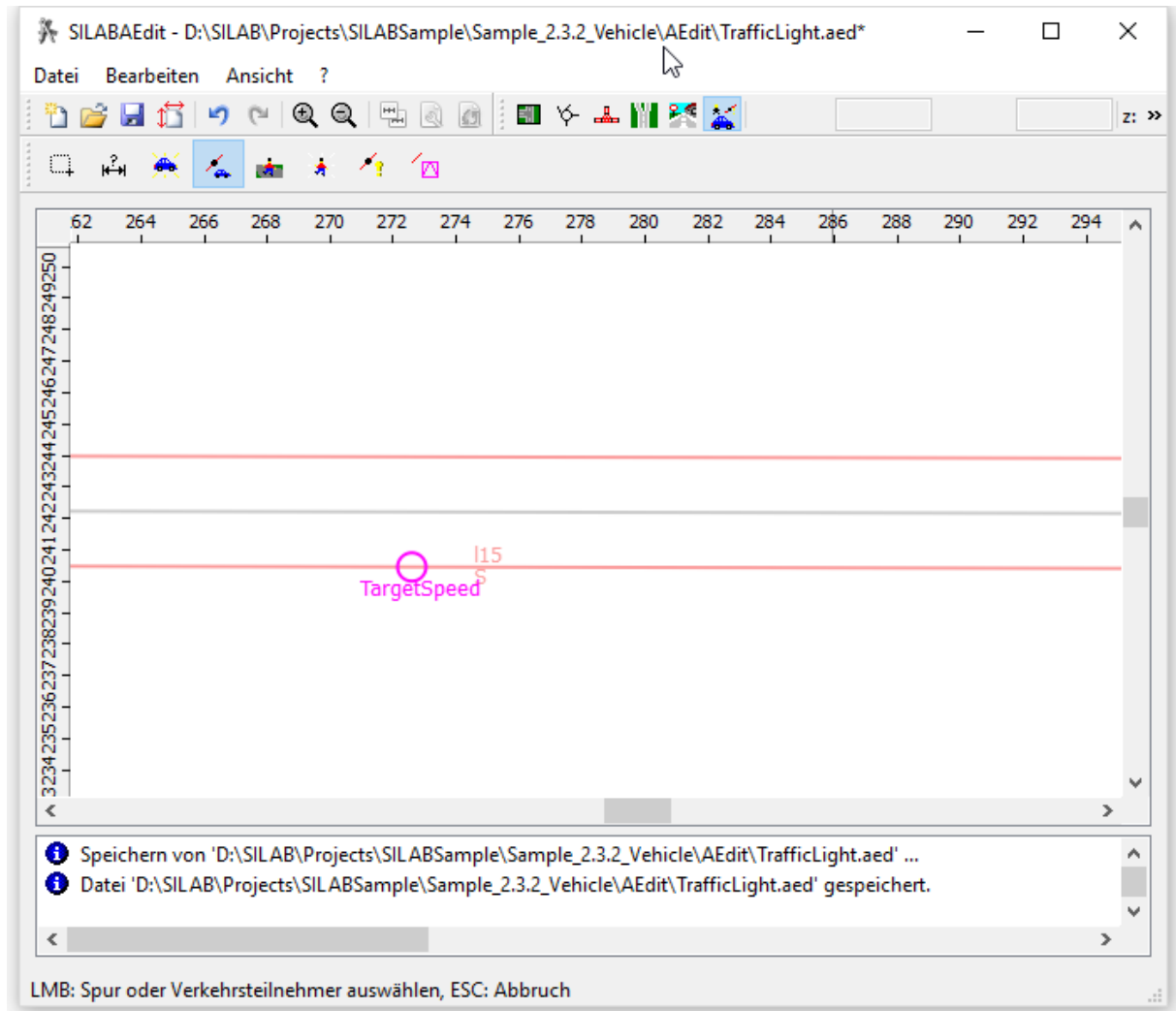
```
UserID = 125;  
Flowpoints =  
{  
    (ModuleTime, 0, Activate)  
};  
};  
include A1_TRF.cfg  
include A2_TRF.cfg  
include A3_TRF.cfg  
};
```

Nun können Sie in beispielsweise in der Area A1 öffentliche Flowpoints für diesen Verkehrsteilnehmer setzen.

Ein öffentlicher Flowpoint wird in dem Editor SILABAEEdit mit dem Werkzeug zum Erzeugen von Flowpoints direkt auf einer Spur gesetzt, ohne zuvor einen bestimmten Verkehrsteilnehmer auszuwählen. Positionieren Sie das Fadenkreuz direkt auf der Spur und öffnen Sie den Dialog zum Einfügen eines Flowpoints durch einen linken Mausklick.



Wählen Sie in der oberen Auswahlbox den Flowpoint und in der unteren Auswahlbox den Kontext *TrafficUserID*, wenn der Verkehrsteilnehmer den Flowpoint auslösen soll oder *SimCarUserID*, falls der Flowpoint durch das EGO – Fahrzeug ausgelöst werden soll. Im nun freigeschalteten Eingabefeld tragen Sie die vergebene **UserID** ein. In diesem Beispiel ist das der Wert 125.



Der öffentliche Flowpoint hat keine Verbindungslinie zu einem Verkehrsteilnehmer und steht alleine an der eingefügten Position. Trifft nun der Verkehrsteilnehmer *MyVehicle* mit der gesetzten **UserID** 125 auf diesen Flowpoint, dann wird er diesen auslösen. Öffentliche Flowpoints mit den Kontexten *TrafficUserID* und *SimCarUserID* und einer anderen **UserID** wird *MyVehicle* ignorieren.

Optimierungen:

Wenn Sie feststellen, dass Ihr Modul aufgrund seiner hohen Komplexität „nicht flüssig läuft“ (d.h. die am Ende von Abschnitt 2.2 beschriebenen Probleme macht), dann können Sie auch hier den Mechanismus der *UsedRoadUserTable* (vgl. Abschnitt 2.2)

verwenden. Erweitern Sie hierzu den Abschnitt der generellen Verkehrsdefinition wie folgt (Namen der Area2 Streckensegmente wie in den Beispielen oben):

```
SILAB TRF
{
    include SILAB_TRFX_600.cfg
    UsedRoadUserTable MyModuleCourse
    {
        # <insert list of vehicle types of Course segments here>
    };
    # include UsedRoadUserTables of Area2 segments (generated by
    # SILABAEEdit)
    include A1_TRFTable.cfg
    include A2_TRFTable.cfg
    include A3_TRFTable.cfg
    ...
};
```

4 MODULÜBERGREIFENDER VERKEHR

Ein modulübergreifender Verkehrsteilnehmer wird zunächst in einem bestimmten Szenario-Modul erzeugt. Im Gegensatz zu modulgebundenen Verkehrsteilnehmern kann er jedoch Modulgrenzen überfahren und in anderen Modulen mit dem Testfahrer interagieren.

4.1 Generelle Verkehrsdefinitionen

Die Fahrzeugtypen, die für die modulübergreifenden Verkehrsteilnehmer zur Verfügung stehen sollen, müssen in den generellen Verkehrsdefinitionen angegeben werden. Fügen Sie hierzu einen gesonderten Abschnitt *<global vehicles>* (s. Grammatik 16) ein:

Grammatik 16 Available Vehicle Types for Global Traffic

```

<global vehicles> →
    GlobalTraffic <any name>
    {
        Vehicle =
        {
            (,)(<number>, <name of group of vehicle types>)
        };
    };

```

Dabei gibt *<number>* an, wie viele Fahrzeuge aus einer Gruppe für globale Verkehrsteilnehmer zur Verfügung stehen. Der *<name of group of vehicle types>* entspricht den Namen, die Sie in der *<road user table>* für die Gruppen vergeben haben. Bitte beachten Sie, dass also für den globalen Verkehr keine bestimmten Einzelfahrzeuge festgelegt werden können. Es werden immer Fahrzeuge aus der angegebenen Gruppe gezogen (maximal *<number>* viele).

Im folgenden Beispiel werden dem modulübergreifenden Verkehr 60 Fahrzeuge der Gruppe Car und 10 Fahrzeuge der Gruppe Truck zur Verfügung gestellt. Beide Gruppen sind in der im Lieferumfang enthaltenen Datei „SILAB_TRFX_600.cfg“ definiert.

```

SILAB TRF
{
    include SILAB_TRFX_600.cfg

    GlobalTraffic MyGlobalTrafficVehicles
    {
        Vehicle =
        {
            (60,Car),
            (10,Truck)
        };
    };
};

```

4.2 Course Streckenabschnitte

4.2.1 GlobalConstVehicleX

Ein einzelner Verkehrsteilnehmer des modulübergreifenden Verkehrs wird durch ein **GlobalConstVehicleX** repräsentiert. Die Definition ist der des modulgebundenen **VehicleX** ähnlich (s. Abschnitt 2.3.1). Auf die einzelnen Abschnitte wird daher an dieser Stelle nicht nochmal eingegangen.

Grammatik 17 Global Traffic Participant

```
< GlobalConstVehicle >→
  GlobalConstVehicleX <instance name of vehicle>
  {
    Vehicle = Car | Truck | Van | Bike | Motorbike;
    (0,1)UserID = <index>;
    <initial position (→Grammatik 8 Seite 18)>
    <behaviour (→Grammatik 9 Seite 18)>
    (0,1)<flowpoints (→Grammatik 10 Seite 25) >
    (0,1)MergeDefault = <1 ... 10 (→Abschnitt 5.3.4, Seite 133)>
  };
```

Bitte beachten Sie, dass als *<vehicle type>* nur Klassen wie Car, Truck und Bike verwendet werden dürfen, wenn diese auch in dem globalen Verkehrsabschnitt definiert wurden.

Neben einzelnen Verkehrsteilnehmern stehen für Course Streckenabschnitte auch globale **Verkehrsquellen** zur Verfügung. Eine globale Verkehrsquelle ähnelt einer Fahrzeugquelle und erzeugt modulübergreifende Verkehrsteilnehmer.

Es gibt zwei unterschiedliche Typen von globalen Verkehrsquellen. Die Quellen vom Typ **GlobalIX** bewegen sich mit dem EGO-Fahrzeug, während Quellen vom Typ **GlobalSourceX** fest auf der Strecke verankert sind.

4.2.2 GlobalIX

Verkehrsquellen des Typs **GlobalIX** erzeugen globale Verkehrsteilnehmer die relativ zum EGO-Fahrzeug d.h. immer in konstantem Abstand nach vorne oder nach hinten aufgesetzt werden. Diese Verkehrsteilnehmer können auf allen Spuren, die im Querschnitt des Course als befahrbar (siehe "Referenz Datenbasis SCNX") markiert worden sind aufgesetzt werden. Alternativ kann auch explizit eine bestimmte befahrbare Spur angegeben werden, auf der die Verkehrsteilnehmer aufgesetzt werden.

Grammatik 18 Moving GlobalSource

```

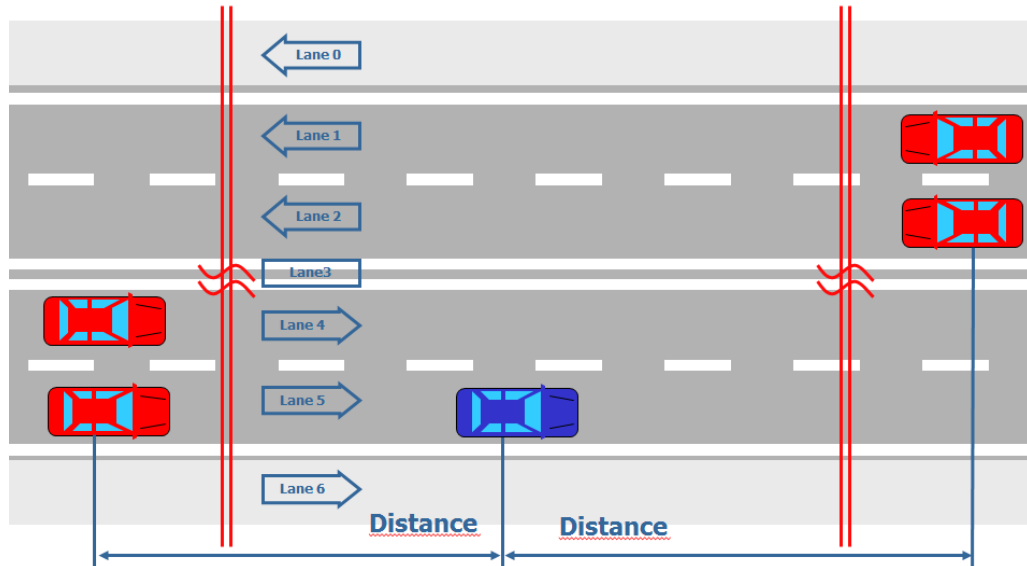
< Moving GlobalSource >→
  GlobalX < Instanzname Quelle>
  {
    Distance = <value>;
    DistanceDeactivation = <value>;
    ClearanceDistance = <value>;
    <behaviour (→Grammatik 9 Seite 18)>
    (0,1)pTruck = <value>;
    (0,1)LaneIDX = <value>;
    (0,1)TimeStep = <value>;
    Flowpoints = {
      (<Instanzname Course>, <Streckenmeter>, <Spurnummer>,
        Activate),
      ...,
      (<Instanzname Course >, <Streckenmeter>, <Spurnummer>,
        Deactivate)
    };
  };

```

Folgendes wird in den einzelnen Abschnitten festgelegt:

- **Distance:** Abstand in Meter des Aufsetzpunktes relativ zum EGO – Fahrzeug. Der Abstand wird entlang der jeweiligen Fahrspur gemessen und zwar *entgegen* der Fahrtrichtung der Spur. Auf der eigenen Spur werden folglich die modulübergreifenden Verkehrsteilnehmer hinter dem EGO – Fahrzeug aufgesetzt und auf der Gegenseite vor dem EGO – Fahrzeug.
- **DistanceDeactivation:** Sobald die aufgesetzten Fahrzeuge einen Abstand in Meter größer als die angegebene Distanz zum EGO – Fahrzeug besitzen, werden sie deaktiviert. Der Defaultwert ebenso wie der Minimalwert ist 110% der Wertes von **Distance**.
- **ClearanceDistance:** Ein neues Fahrzeug wird auf einer Spur erst dann aufgesetzt, wenn sich kein Fahrzeug näher als der angegebene Wert in Meter um den Aufsetzpunkt herum befindet.
- **<Behaviour>:** Definition des Verhaltens.
- **pTruck:** beschreibt die Wahrscheinlichkeit [0, 1], dass ein LKW als Verkehrsteilnehmer aufgesetzt wird. Wird dieser Parameter ausgelassen, dann werden ausschließlich Fahrzeuge vom Typ PKW aufgesetzt. Die Quelle durchsucht die in der Tabelle GlobalTraffic (siehe Grammatik 16) verwendeten Fahrzeuggruppen selbstständig nach LKW und PKW Typen. Ist keine entsprechender Typ verfügbar wird kein Fahrzeug aufgesetzt.
- **LaneIDX:** Normalerweise setzt die Quelle die Fahrzeuge auf alle Spuren, welche als befahrbar markiert wurden, ein. Wenn diese Markierung auf dem Streckenabschnitt nicht zur Verfügung steht oder die Fahrzeuge nur auf einer bestimmten Spur aufgesetzt werden sollen, muss dieser Parameter angegeben werden.

- **TimeStep**: beschreibt den zeitlichen Abstand in Sekunden zwischen zwei Fahrzeugen. Bei fehlender Spurangabe wird jedesmal zufällig eine Spur gewählt. Voraussetzung für das Aufsetzen ist das Einhalten der **ClearanceDistance**.



In dem obigen Beispiel einer Autobahnstrecke wurden nur Spur 1, Spur 2, Spur 4 und Spur 5 als befahrbar markiert. Das blaue Fahrzeug stellt das EGO – Fahrzeug dar und die roten Fahrzeuge die Aufsetzpositionen der von der Quelle eingesetzten Fahrzeuge dar.

Bitte beachten Sie:

Wird die Quelle mit dem EGO – Fahrzeug in eine Area2 Instanz überführt, dann ist sie inaktiv solange sich das EGO – Fahrzeug innerhalb der Area2 Instanz befindet. Somit werden innerhalb der Area2 keine Fahrzeuge aufgesetzt.

Beispiel:

```
GlobalX GlobalXExample
{
    Behaviour = {(EXSTD1, 13.8, 13.8, 1, 10)};
    Distance = 200;
    DistanceDeactivation = 400;
    pTruck = 0.0;
    LaneIDX = 2;
    TimeStep = 0.5;
    ClearanceDistance = 20;
    Flowpoints =
    {
        (Start, 5, 2, SimCar, Activate),
        (Start, 20, 2, Traffic, TargetSpeed, 0),
        (Start, 50, 2, Traffic, UserID, 10001),
        (Start, 110, 2, SimCar, Deactivate)
    };
};
```

4.2.3 GlobalSourceX

Verkehrsquellen des Typs GlobalSourceX erzeugen globale Verkehrsteilnehmer die immer an einer festen Position innerhalb des Streckennetzes aufgesetzt werden.

Grammatik 19 Fixed GlobalSource

```

< Fixed GlobalSource >→
  GlobalSourceX < Instanzname Quelle>
  {
    (0,1)Sequence = (<value>, (0...*)[,<value>]);
    <behaviour (→Grammatik 9 Seite 18)>
    NumberOfVehicles = <value>;
    Vehicle = Car/Bike/Truck/Motorbike
    Behaviour or BehaviourScheme
    (0,1)Distance = <value>;
    EGODistance = <value>;
    (0,1)pTruck = <value>;
    DeactivationRadius
    <initial position(→Grammatik 8 Seite 18)>
    Flowpoints = {
      (<Instanzname Course>, <Streckenmeter>, <Spurnummer>,
        Activate),
      ...,
      (<Instanzname Course >, <Streckenmeter>, <Spurnummer>,
        Deactivate)
    };
  };

```

Die Quellen werden in SILABAedit mit dem Werkzeug zum Einfügen eines Verkehrsobjektes in die Area2 eingesetzt. Wählen Sie in der oberen Auswahlbox hierfür Source aus. Danach stehen zwei Arten von Quellen zur Verfügung:

- **Source:** Diese Quelle setzt Fahrzeuge mit einem konstanten Abstand zueinander an der Position der Quelle ein. Die folgenden Parameter lassen sich verändern:
 - **<Behaviour>:** Definition des Verhaltens.
 - **Vehicle:** Hier wird ein Typ von Fahrzeugen ausgewählt. Zur Verfügung stehen die Typen Car, Bike, Truck oder Motorbike.
 - **NumberOfVehicles:** Anzahl der Fahrzeuge die diese Quelle nutzt.
 - **Distance:** Die Distanz gibt die Strecke in Meter an, die das letzte aufgesetzte Fahrzeug zurückgelegt haben muß, bevor ein neues Fahrzeug aufgesetzt wird. Der Default-Wert ist 10 Meter. Der Parameter **Distance** wird ignoriert sobald eine **Sequence** angegeben ist. Minimum 8 m
 - **EGODistance:** Sobald sich das EGO-Fahrzeug näher näher als die angegebene Distanz in Meter zur Quelle befindet, wird die Quelle deaktiviert.
 - **pTruck:** beschreibt die Wahrscheinlichkeit, dass ein LKW als Verkehrsteilnehmer aufgesetzt wird. Wird dieser Parameter ausgelassen, dann werden ausschließlich Fahrzeuge vom Typ PKW aufgesetzt. Die Quelle

- durchsucht die in der Tabelle GlobalTraffic (siehe Grammatik 16) verwendeten Fahrzeuggruppen selbstständig nach LKW und PKW Typen.
- **DeactivationRadius:** Der Radius d.h. die Entfernung eines Fahrzeugs zum EGO-Fahrzeugs muss einmal unterschritten sein und beim nächsten Überschreiten wird das zugehörige Fahrzeug deaktiviert.
 - **SequenceSource:** Die SequenceSource wird in Courses ausgewählt durch die Spezifikation des Parameters Sequence und im SILABAedit durch die Auswahl der SequenceSource im Auswahldialog. Diese Quelle setzt die Fahrzeuge in einer frei definierbaren Folge von zeitlichen Abständen (in Sekunden) auf. Diese Quelle kann über die folgenden Parameter verändert werden:
 - **<Behaviour>**: Definition des Verhaltens.
 - **Vehicle:** Hier wird ein Typ von Fahrzeugen ausgewählt. Zur Verfügung stehen die Typen Car, Bike, Truck oder Motorbike.
 - **NumberOfVehicles:** Anzahl der Fahrzeuge die diese Quelle nutzt.
 - **EGODistance:** Sobald sich das EGO-Fahrzeug näher näher als die angegebene Distanz in Meter zur Quelle befindet, wird die Quelle deaktiviert.
 - **pTruck:** beschreibt die Wahrscheinlichkeit, dass ein LKW als Verkehrsteilnehmer aufgesetzt wird. Wird dieser Parameter ausgelassen, dann werden ausschließlich Fahrzeuge vom Typ PKW aufgesetzt. Die Quelle durchsucht die in der Tabelle GlobalTraffic (siehe Grammatik 16) verwendeten Fahrzeuggruppen selbstständig nach LKW und PKW Typen.
 - **DeactivationRadius:** Der Radius d.h. die Entfernung eines Fahrzeugs zum EGO-Fahrzeugs muss einmal unterschritten sein und beim nächsten Überschreiten wird das zugehörige Fahrzeug deaktiviert.
 - **Sequence:** Die Folge der Abstände wird als Tupel angegeben. Das Tupel (2.0 , 4.0 , 3.0 , 5.3, 3.3) setzt das zweite Fahrzeug 2 Sekunden nach dem ersten auf, das dritte 4 Sekunden nach dem zweiten usw. Die Folge wiederholt sich, wenn der letzte Wert des Tupels erreicht wurde. Ist der Parameter Sequence angegeben wird der Parameter **Distance** ignoriert.

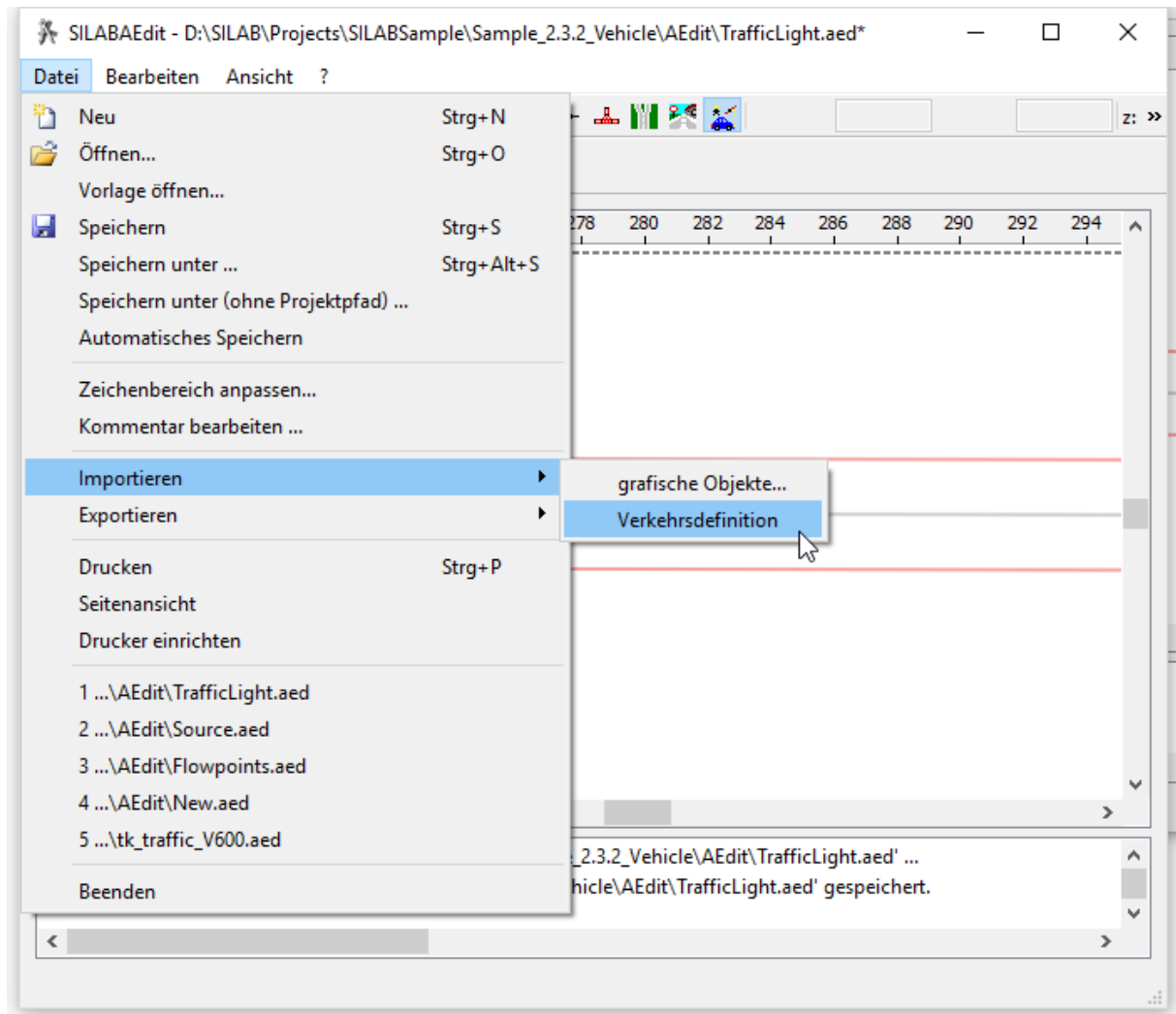
Beispiel:

```
GlobalSourceX GlobalSourceXExample
{
    Position = (Start, 100, 2);
    BehaviourScheme = Standard;
    NumberOfVehicles = 4;
    Sequence = (2.0,4.0,3.0,5.3,3.3);
    Distance = 10;
    EGODistance = 10;
    pTruck = 0;
    Flowpoints = {
        (Start, 5, 2, SimCar, Activate),
        (Start, 20, 2, Traffic, UserID, 200),
        (Start, 30, 2, Traffic, TargetSpeed, 13.8),
        (Start, 105, 2, SimCar, Deactivate)
    };
};
```

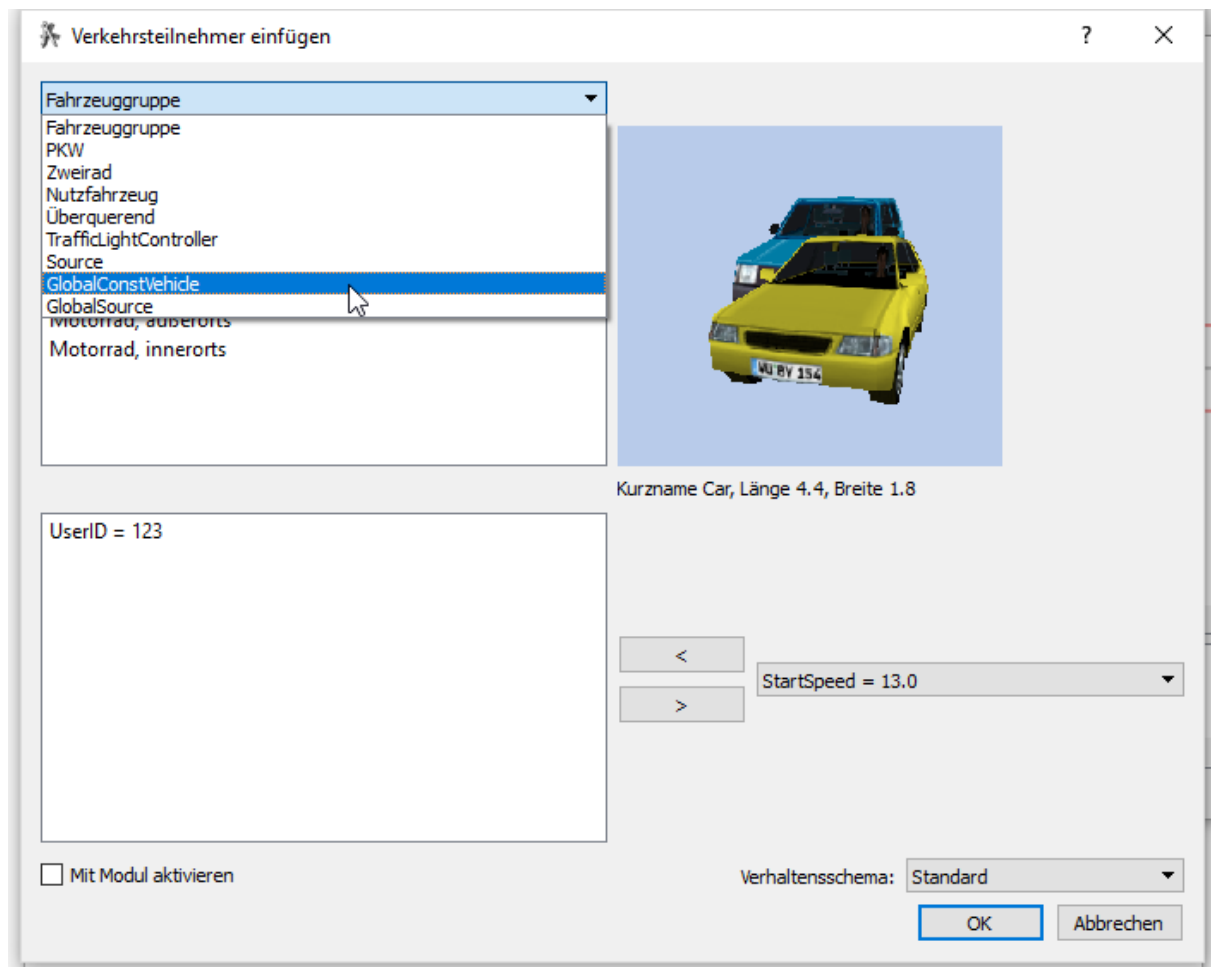
4.3 Area2 Streckenabschnitte

Auch in Streckenabschnitten vom Typ Area2 können Sie globale Verkehrsteilnehmer definieren. Verwenden Sie hierzu den grafischen Editor SILABAEEdit, dessen Bedienung im Dokument „SILABAEEdit.pdf“ beschrieben ist. Im Folgenden wird das Einfügen eines globalen Verkehrsteilnehmers anhand eines Beispiels erläutert.

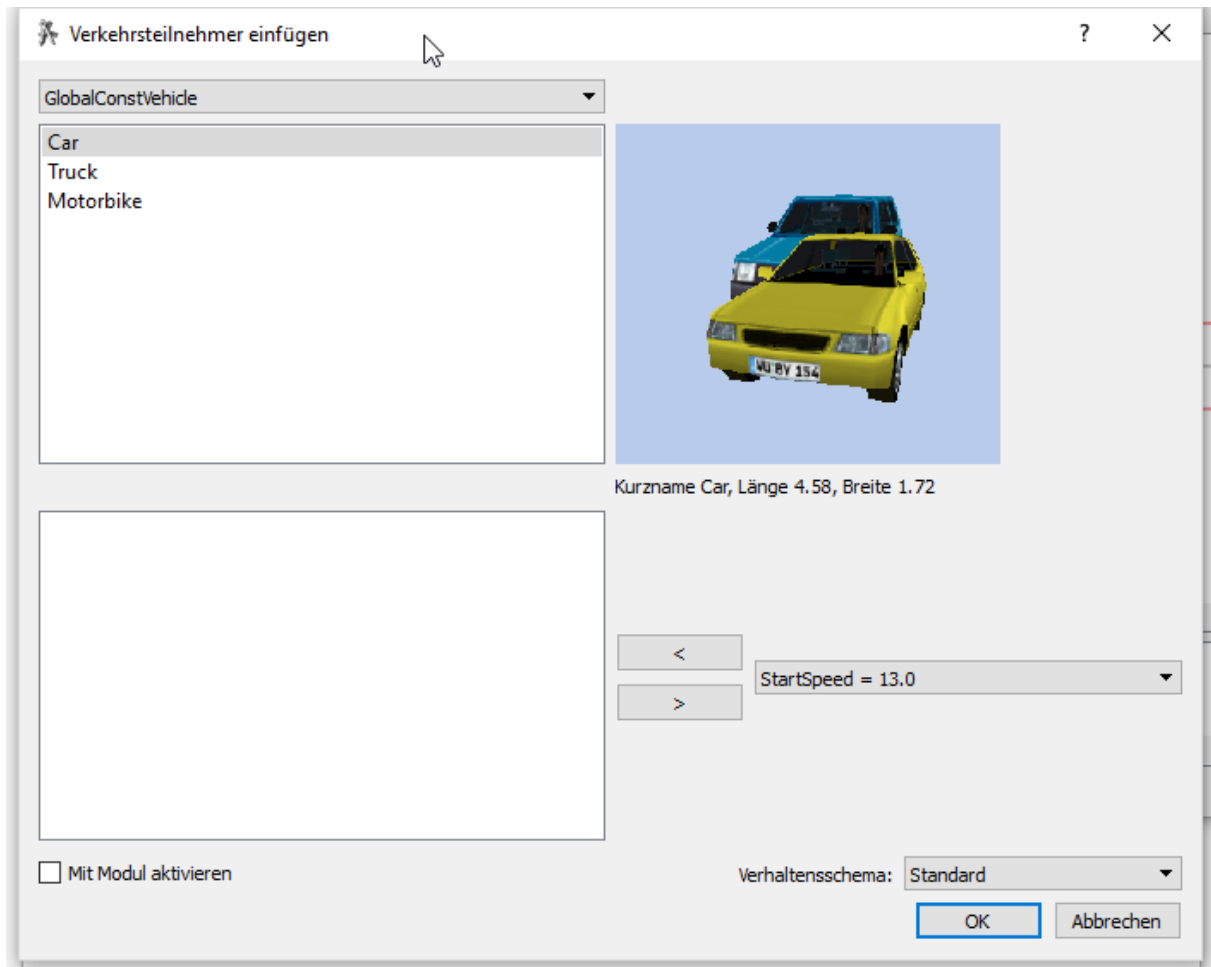
Zunächst müssen Sie die Definition der GlobalConstVehicle einfügen. Hierzu importieren Sie die Datei SILAB/lib/Aedit/GlobalTraffic.cfg über den Menüpunkt *File->Import->Traffic definitions*



Wählen Sie nun im folgenden Dialog die Datei `SILAB/lib/Aedit/GlobalTraffic.cfg` aus und bestätigen Sie die Auswahl mit dem Button *[ok]*.



Nach dem Import der Datei stehen im Dialog zum Einfügen eines Verkehrsteilnehmers die Type *GlobalConstVehicle* und *GlobalSource* sowie die zugehörigen Flowpoints zur Verfügung.
Die Mechanismen zur Erzeugung der *GlobalConstVehicle* sind analog zu der Erzeugung eines modulgebundenen Verkehrsteilnehmers.



Als Verkehrsteilnehmer stehen die beiden Gruppen *Car* und *Truck* zur Verfügung. Damit Sie diese Gruppen verwenden können, müssen Sie in der generellen Verkehrsdefinition (siehe Abschnitt 4.1) genügend Verkehrsteilnehmer in den Gruppen *Car* und *Truck* definiert haben. Mit der folgenden Definition stehen beispielsweise 60 PKW (Gruppe *Car*) und 10 LKW (Gruppe *Truck*) für die Verwendung als *GlobalConstVehicle* bzw. als *GlobalSource* zur Verfügung:

```
SILAB TRF
{
    include SILAB_TRFX_600.cfg

    GlobalTraffic MyGlobalTrafficVehicles
    {
        Vehicle =
        {
            ...
            (60,Car),
            (10,Truck)
        };
    };
};
```

Die vom Typ *GlobalSource* zur Verfügung gestellten Quellen funktionieren analog zu den in Abschnitt 2.5.2 eingeführten Quellen. Im Unterschied zu den modulgebundenen Quellen erzeugen diese *GlobalSource* Quellen als Verkehrsteilnehmer entsprechend *GlobalConstVehicleX* statt *VehicleX*. Die Anzahl dieser Verkehrsteilnehmer muss ebenfalls in der obigen generellen Verkehrsdefinition berücksichtigt werden.

Bitte beachten Sie:

- Stehen nicht genügend modulübergreifende Verkehrsteilnehmer in der generellen Verkehrsdefinition zur Verfügung, dann erscheinen die zu viel aktivierten Verkehrsteilnehmer nicht, unabhängig davon, ob diese aus einer Quelle stammen oder als einzelne Verkehrsteilnehmer definiert wurden.
- Jeder zur Verfügung gestellte Verkehrsteilnehmer erhöht die benötigte Rechenzeit. Daher muss die obige Tabelle auf den tatsächlichen Bedarf der Gesamtstrecke angepasst werden.

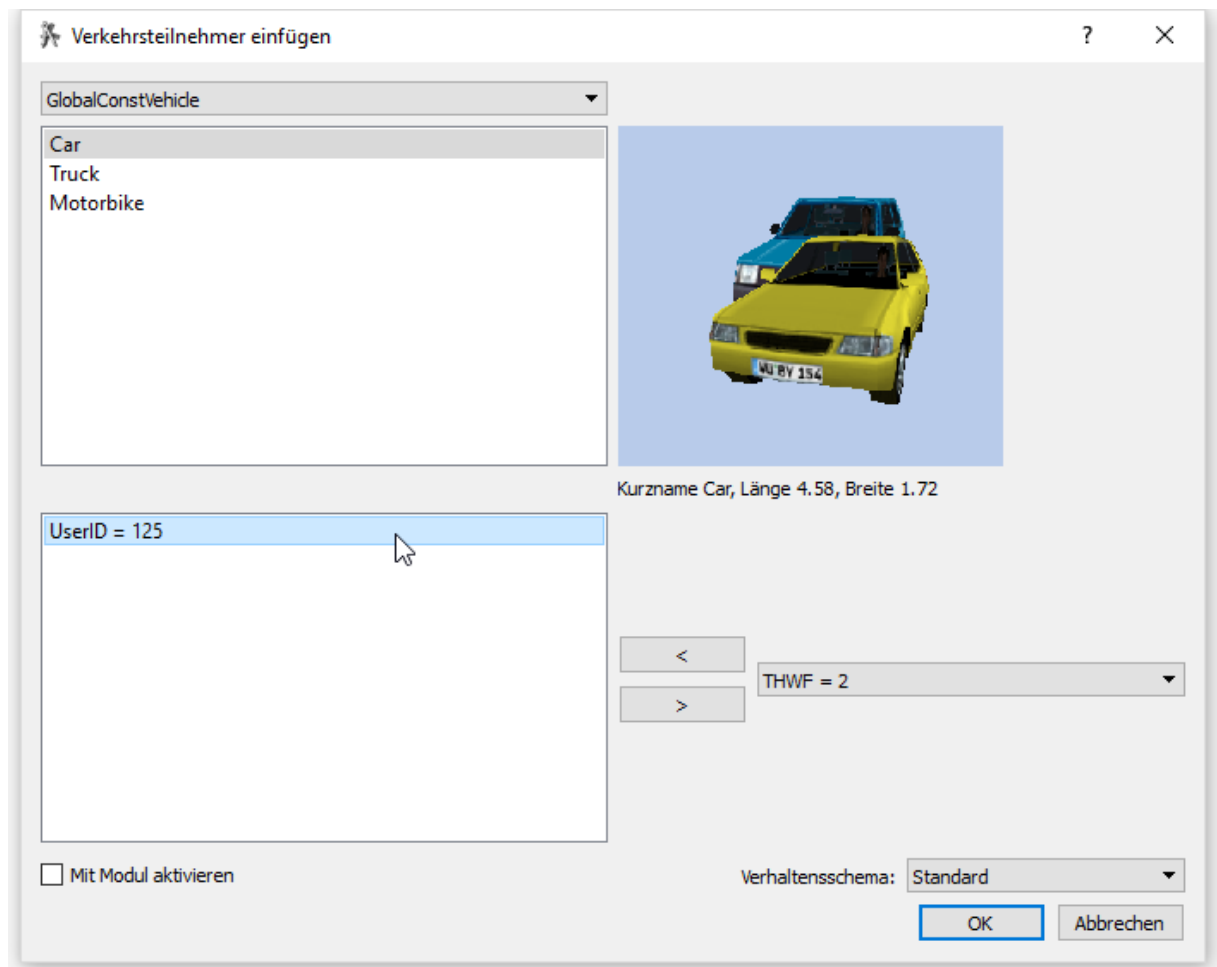
Die *GlobalSource* werden in SILABAedit mit dem Werkzeug zum Einfügen eines Verkehrsobjektes in die Area2 eingesetzt. Wählen Sie in der oberen Auswahlbox hierfür *GlobalSource* aus. Danach stehen zwei Arten von Quellen zur Verfügung:

- **Source:** Diese Quelle setzt Fahrzeuge mit einem konstanten Abstand zueinander an der Position der Quelle ein.
- **SequenceSource:** Diese Quelle setzt die Fahrzeuge in einer frei definierbaren Folge von zeitlichen Abständen (in Sekunden) auf.

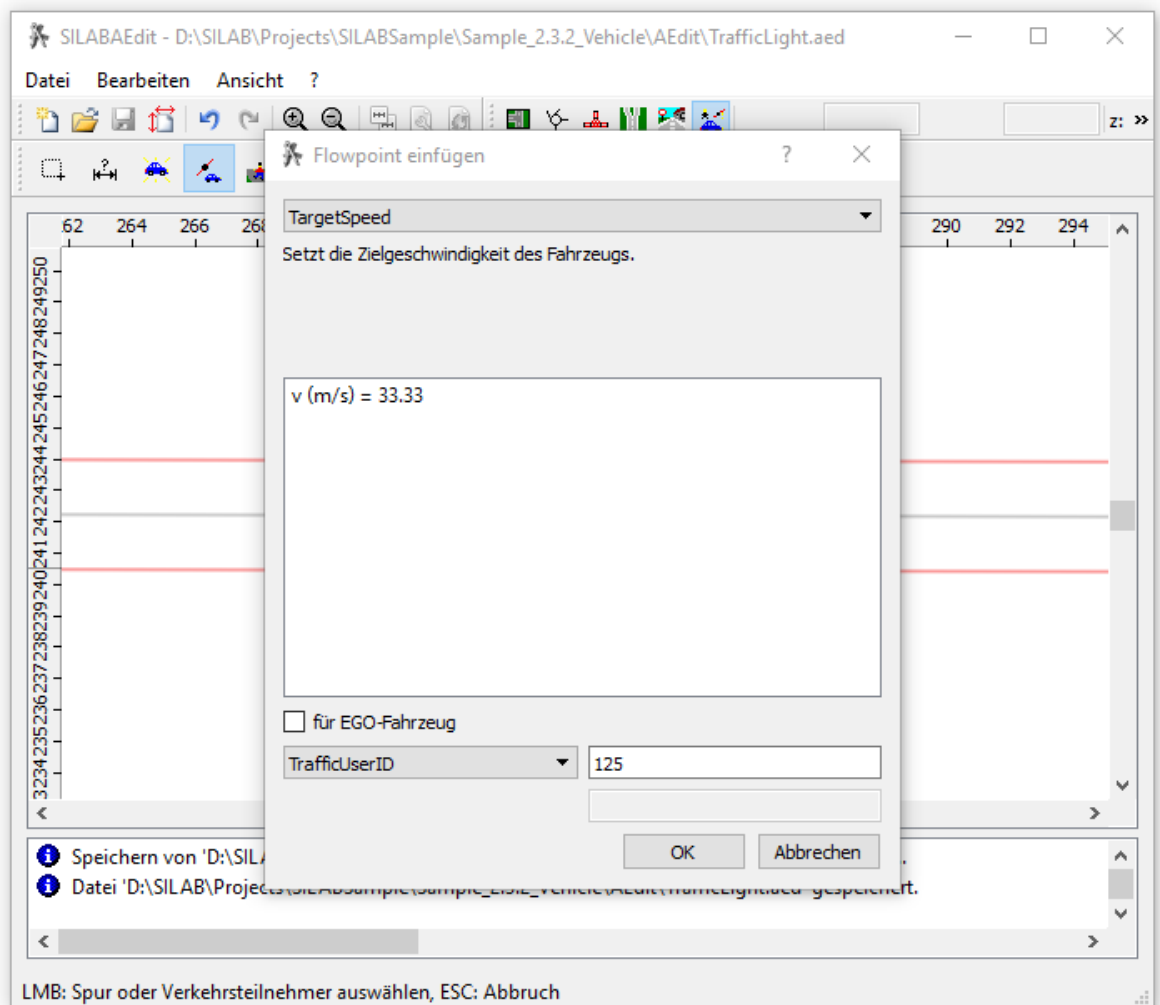
4.4 Verhaltenssteuerung

Der Mechanismus, um modulgebundene Verkehrsteilnehmer innerhalb des Moduls, in dem Sie definiert wurden, zu aktivieren, zu deaktivieren und zu steuern entspricht genau dem, mit dem Sie auch modulgebundene Verkehrsteilnehmer steuern.

- In Courses (Skript) : Setzen Sie entsprechende Flowpoints entweder direkt im **GlobalConstVehicle** Abschnitt des Verkehrsteilnehmers (s. Grammatik 17). Wenn Sie den Verkehrsteilnehmer auf der Ebene des Abschnitts **Traffic** (s. Grammatik 4) über öffentliche Flowpoints steuern möchten, dann weisen Sie bei der Definition dem **GlobalConstVehicle** mit dem Parameter **UserID** = *<index>* eine eindeutige Zahl zu. Verwenden Sie nun bei den öffentlichen Flowpoints zur Adressierung des Verkehrsteilnehmers den Kontext **TrafficUserID,<index>** bzw. **SimCarUserID,<index>** mit dem gleichen Wert.
- In Area2s (SILABAEdit): Importieren Sie zunächst, wie in Abschnitt 4.3 beschrieben, die Definitionsdatei für die modulübergreifenden Verkehrsteilnehmer. Danach steht das **GlobalConstVehicle** auch in SILABAEdit zur Verfügung. Wenn das **GlobalConstVehicle** über öffentliche Flowpoints (beispielsweise im nachfolgenden Modul) gesteuert werden soll, dann fügen Sie dem **GlobalConstVehicle** den Parameter **UserID** hinzu und setzen diesen auf einen eindeutigen Wert:



Private Flowpoints für das **GlobalConstVehicle** definieren Sie, indem Sie beim Einfügen eines Flowpoints zunächst das entsprechende **GlobalConstVehicle** im Editor anklicken und dann den Flowpoint positionieren. Bei öffentlichen Flowpoints verwenden Sie den Kontext **SimCarUserID** bzw. **TrafficUserID** mit dem passenden Parameter (im obigen Beispiel 125) um gezielt das **GlobalConstVehicle** zu steuern:



In anderen Modulen (also nicht den „Ursprungs-Modulen“ der Verkehrsteilnehmer), funktioniert die Steuerung wie folgt:

- In Courses (Skript): Setzen Sie öffentliche Flowpoints auf der Ebene des Abschnitts **Traffic** (s. Grammatik 4). Verwenden Sie zur Adressierung des Verkehrsteilnehmers den Kontext **TrafficUserID,<index>** bzw. **SimCarUserID,<index>** mit dem gleichen Wert den Sie bei der Definition im **Global-ConstVehicle** Abschnitt mit **UserID = <index>** zugewiesen haben. Hierbei ist wichtig, dass dieser Wert über alle verwendeten Module einer Karte hinweg eindeutig ist.
- In Area2s (SILABAEEdit): Setzen Sie die öffentlichen Flowpoints mit dem Kontext **SimCarUserID** bzw. **TrafficUserID** im Editor SILABAEEdit und verwenden Sie als zusätzlichen Parameter im Dialog zur Erzeugung des Flowpoints den Wert **<index>** des **UserID = <index>** Eintrags.

4.5 Verkehr in Modulübergängen

Da modulgebundener Verkehr bei vielen Szenario-Modulen an den Modulgrenzen für den Fahrer unauffällig aufgebaut und wieder abgebaut werden muss (die Fahrzeuge würden sonst plötzlich auftauchen und wieder verschwinden), entstehen bei Modulein- und ausfahrten zwangsläufig Streckenabschnitte, in denen sich kein Verkehrsteilnehmer befindet (s. hierzu auch Abschnitt 1.2.2). Globale Verkehrsteilnehmer eignen sich sehr gut, um mit ihnen solche Streckenabschnitte „auszuschmücken“. In der Regel möchte man jedoch, dass diese ausschmückenden Verkehrsteilnehmer das eigentliche Verkehrsszenario eines Moduls, das unter Verwendung des modulgebundenen Verkehrs definiert wurde, nicht stören. Um sie also rechtzeitig, d.h. während oder nach dem Aufbau des modulgebundenen Verkehrs wieder „los zu werden“, gibt es einen speziellen Mechanismus. Er basiert darauf, ausschmückende Verkehrsteilnehmer rechtzeitig abbiegen oder einparken zu lassen.

Natürlich muss dies das Straßennetzwerk eines Moduls in seinem Einfahrtsbereich unterstützen. Module haben also gewisse Kapazitäten, ausschmückende Verkehrsteilnehmer aufzunehmen und verschwinden zu lassen. Angenommen, ein erstelltes Modul „MyModule2“ hat die Kapazität, drei ausschmückende Verkehrsteilnehmer aufzunehmen. Der Fahrer befindet sich im Modul „MyModule1“ und ist auf dem Weg zum anschließenden „MyModule2“. Der im Folgenden beschriebene Mechanismus ermöglicht folgendes: Obwohl im aktuellen Modul „MyModule1“ im Bereich seiner Ausfahrt fünf modulübergreifende Verkehrsteilnehmer definiert sind, werden aufgrund der Kapazität von „MyModule2“ nur drei davon erzeugt.

Gehen Sie wie folgt vor, um den beschriebenen Mechanismus einzusetzen (Situation und Bezeichnungen wie in obigem Beispiel):

Verkehrsdefinition in „MyModule1“:

- In einem **Course Streckenabschnitt**: Fügen Sie im Skript die theoretisch erzeugbaren modulübergreifenden Verkehrsteilnehmer (gemäß Grammatik 17) mit folgenden Aktivierungs-Flowpoints ein (mit „...“ gekennzeichnete Stellen sind natürlich entsprechend zu ergänzen):

```
GlobalConstVehicleX Global1
{
    ...
    Flowpoints = { (... ,SimCar,Global_Activate,1) };
};
GlobalConstVehicleX Global2
{
    ...
    Flowpoints = { (... ,SimCar,Global_Activate,2) };
};
GlobalConstVehicleX Global3
{
    ...
    Flowpoints = { (... ,SimCar,Global_Activate,3) };
};
GlobalConstVehicleX Global4
{
    ...
    Flowpoints = { (... ,SimCar,Global_Activate,4) };
};
GlobalConstVehicleX Global5
{
    ...
    Flowpoints = { (... ,SimCar,0,Global_Activate,5) };
};
```

- Beachten Sie bitte die aufsteigende Nummerierung hinter `Global_Activate`.
- In einem **Area2 Streckenabschnitt**: Importieren Sie bitte zunächst die Datei „\SILAB\lib\AEdit\GlobalTraffic.cfg“ mit dem Menübefehl *File → Import → Traffic definition*. Damit steht das **GlobalConstVehicle** als Verkehrsteilnehmer zur Verfügung. Fügen Sie die ausschmückenden Verkehrsteilnehmer als solche **GlobalConstVehicle** ein. Achten Sie darauf im Selektionsmenü Verhaltensschema: „City“ auszuwählen, da nur dieses Schema in Area2 Streckenabschnitten die Steuerung globalen Verkehrs ermöglicht. Aktivieren Sie das **GlobalConstVehicle** mit dem Flowpoint

GlobalActivate,<aufsteigende Nummer>

Auch hier ist wichtig, dass Sie die ausschmückenden Verkehrsteilnehmer aufsteigend nummerieren.

Verkehrsdefinition in „MyModule2“:

- In einem **Course Streckenabschnitt**: Definieren Sie auf Ebene des Abschnitts **Traffic** (s. Grammatik 4) je nach Kapazität des Streckenabschnitts und an den entsprechenden Stellen folgende Flowpoints:
`Global_Straight`: das Fahrzeug kann geradeaus weiter fahren
`Global_Parking`: das Fahrzeug wählt die rechte Spur um dort zu parken.
Für diesen Flowpoint muss eine Park Spur mit dem Heading Right definiert werden, auf welcher der Verkehrsteilnehmer dann parken kann.
- In einem **Area2 Streckenabschnitt**: Fügen Sie mit SILABAEEdit je nach Kapazität und an den gewünschten Stellen folgende Flowpoints ein:

Global_Right, <aufsteigende Nummer>: Das Fahrzeug kann hier rechts abbiegen.

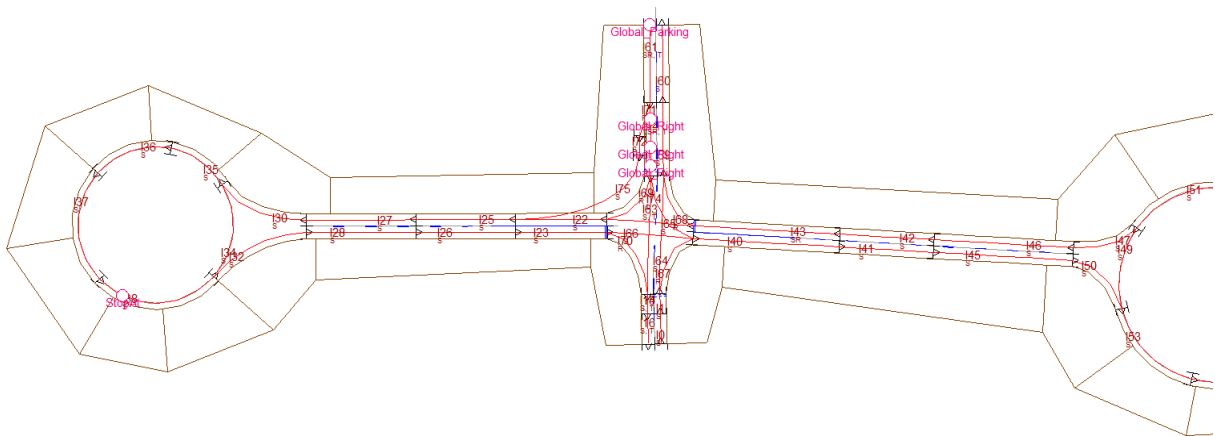
Global_Left, <aufsteigende Nummer>: Das Fahrzeug kann hier links abbiegen.

Global_Straight,<aufsteigende Nummer>: Das Fahrzeug kann geradeaus weiter fahren.

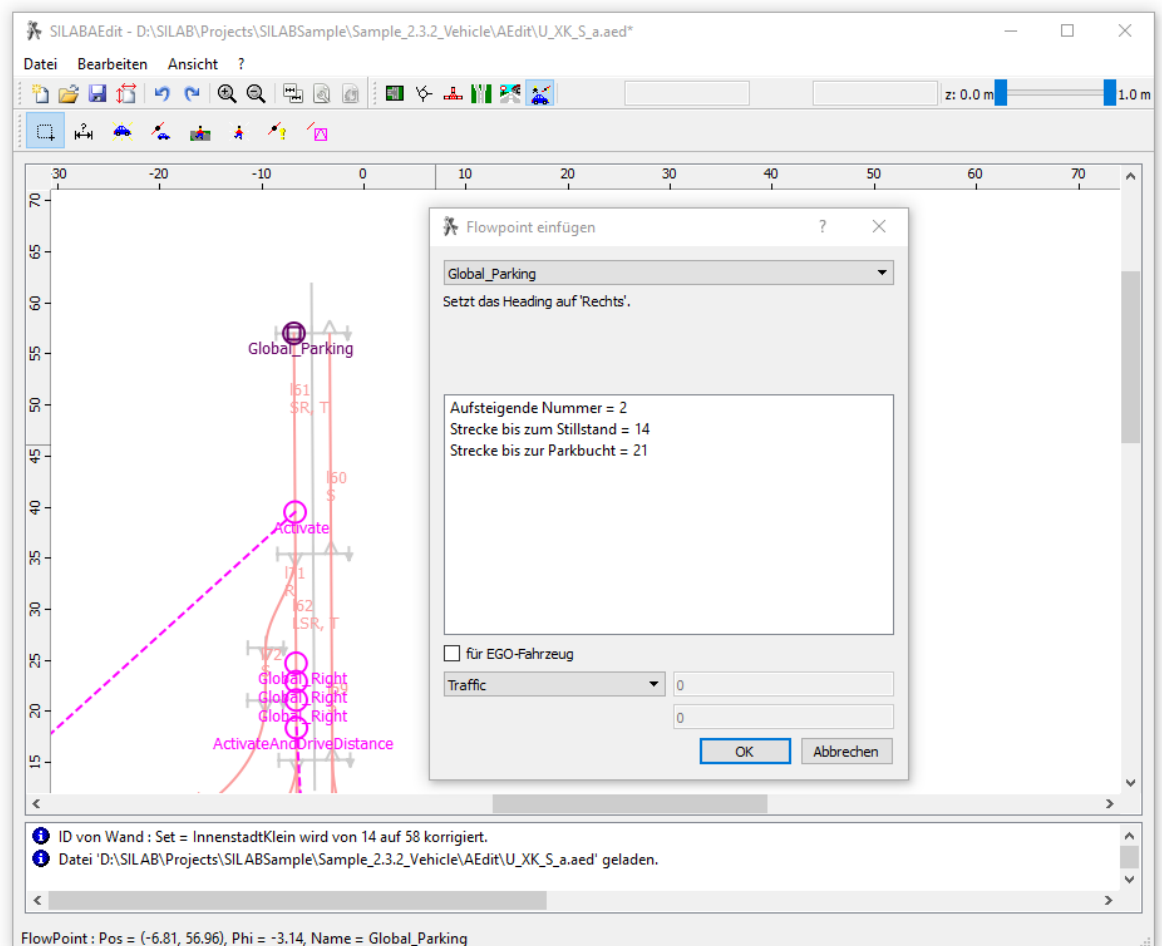
Global_Parking,<aufsteigende Nummer>: Das Fahrzeug wählt die rechte Spur um dort zu parken.

Beachten Sie, dass die geplanten Abbiegespuren entsprechende Headings haben. Auch die Spur zum Einparken muss das Heading „right“ haben. In obigem Beispiel hätten Sie für „MyModule2“ drei solche Flowpoints mit aufsteigenden Nummern von 1 bis 3 definiert. Die <aufsteigende Nummer> eines der Flowpoints bestimmt, für welchen der von „MyModule2“ aufgenommenen ausschmückenden Verkehrsteilnehmer aus „MyModule1“ er zuständig ist.

Im folgenden Beispiel wird ein kleines Übergangsmodul als Area2 definiert, welches vier Fahrzeuge aufnimmt und eines davon rechts parkt:

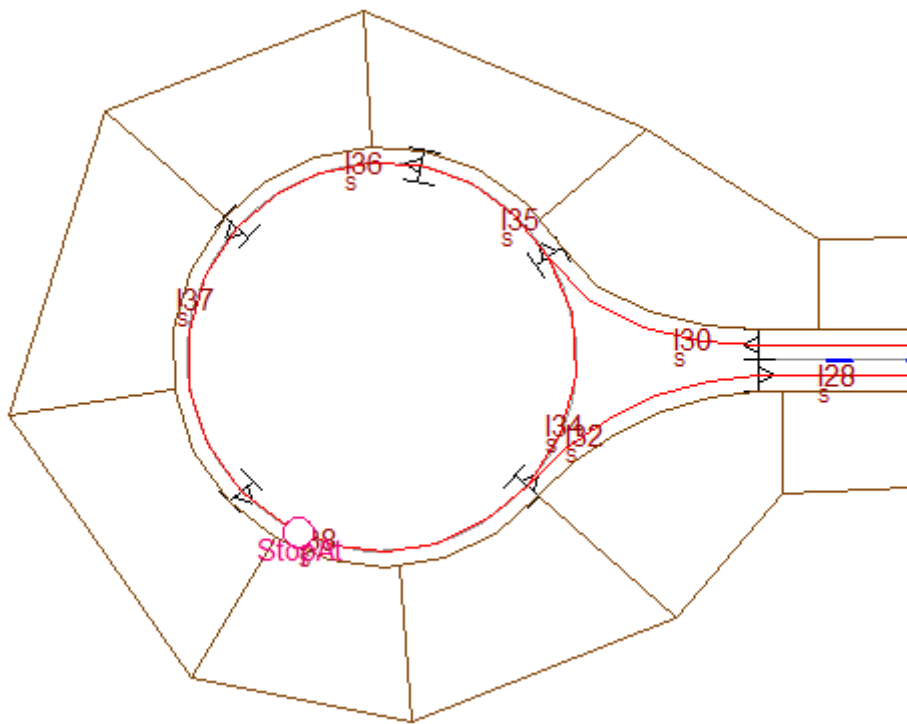


Die Einfahrt des Moduls ist der obere Port, die Ausfahrt der untere. Links und rechts der Kreuzung enden die abzweigenden Straßen in einem Kreisel.
Eine Vergrößerung der Einfahrt verdeutlicht die Technik der verwendeten öffentlichen Flowpoints:



Der oberste Flowpoint bezieht sich auf den Verkehrsteilnehmer, welcher im vorigen Modul mit **GlobalActivate** ,2 aktiviert wurde. Trifft dieser Verkehrsteilnehmer auf den

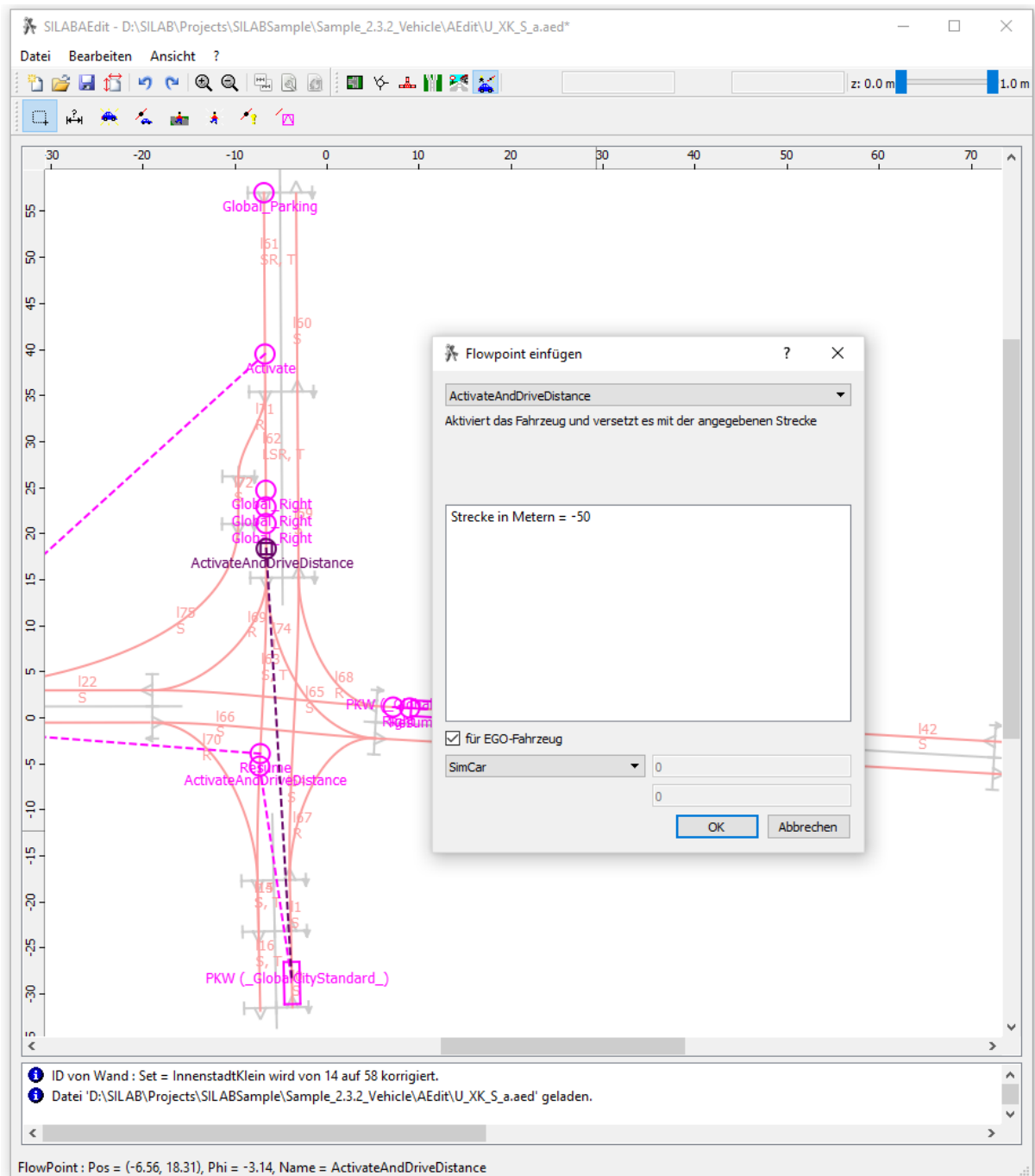
Flowpoint, dann fängt dieser nach 21m an zu blinken und selektiert die Fahrspur mit dem Heading „right“. Dies findet noch auf der Spur l61 statt. Danach wählt es die Spur l71 und hält nach 21 m + 14 m auf Spur l72 an. Über die Spur l71 (gesetztes Heading „right“) wird der Verkehrsteilnehmer auf den Bürgersteig geleitet. Die restlichen drei Flowpoints **Global_right** leiten die Verkehrsteilnehmer 1,3 und 4 nach rechts in den Seitenarm über Spur l69 ab.



Dort endet die Fahrt in dem anschließenden Kreisel an dem öffentlichen Flowpoint **StopAt**, der das Fahrzeug anhält.

Eine weitere Technik zur Ausgestaltung von Modulübergängen ist die Aktivierung und gleichzeitige Versetzung eines Verkehrsteilnehmers.

Wird ein **GlobalConstVehicle** mit dem Flowpoint **ActivateAndDriveDistance** aktiviert, dann wird der Verkehrsteilnehmer zunächst an dem definierten Aufsetzpunkt aufgesetzt. Noch bevor er sichtbar wird, wird es um die angegebene Distanz versetzt und dann aktiviert.



Wird die Distanz wie im obigen Beispiel auf einen negativen Wert gesetzt, dann fährt der Verkehrsteilnehmer zunächst die angegebene Strecke rückwärts entlang seiner Aufsetzspur. In diesem Fall fährt das Fahrzeug ca. 35m in das nachfolgende Modul hinein und erscheint dort. Wenn das nachfolgende Modul mit einem Sichtschutz (z.B. eine unübersichtliche Eingangskurve) ausgerüstet ist, dann erscheint der Verkehrsteilnehmer verdeckt im nachfolgenden Modul und kommt dem EGO –Fahrzeug aus entgegen. Da es sich um ein **GlobalConstVehicle** handelt, wird dieses dann auch nicht einfach verschwinden, wenn das EGO – Fahrzeug die Modulgrenze zum nachfolgenden Modul überfährt.

5 REFERENZEN

5.1 Generelle Verkehrsdefinitionen, SILAB_TRFX_600.cfg

5.1.1 Vordefinierte Behaviour Schemes

Die folgenden Behaviour Schemes sind bereits in der SILAB_TRFX_600.cfg vordefiniert.

```
BehaviourScheme Standard
{
    Behaviour =
    {
        (EXSTD1)
    };
};
BehaviourScheme Ambient
{
    Behaviour =
    {
        (EXSTD1)
    };
    ActivateOnDensity = 0.6;
};
BehaviourScheme NOP
{
    Behaviour =
    {
        (NOP)
    };
};
BehaviourScheme FreeDriving
{
    Behaviour =
    {
        (EXFD1, 14, 14, 1, -5)
    };
};
BehaviourScheme Emergency
{
    Behaviour =
    {
        (EXSTD2)
    };
};
BehaviourScheme LaneCrossing
{
    Behaviour =
    {
        (LaneCrossing, 7, -7, -1)
    };
};
BehaviourScheme City
{
    Behaviour =
    {
        (EXSTDGlobals)
    };
};
```

In Kapitel 5.3.5 und 5.3.5.6 finden Sie eine ausführliche Beschreibung der in den Behaviour Schemes enthaltenen Verhaltensmodelle.

5.1.2 Witterungsabhängige Geschwindigkeitsprofile

Einige Verhaltensmodelle können die Geschwindigkeitswahl von den Witterungseinflüssen abhängig machen. Hierzu können Tabellen für die Maximalgeschwindigkeit in Abhängigkeit der Witterungseinflüsse angegeben werden.

Grammatik 20 Weather dependant speed profiles

```

<weather dependant speed profiles > →
    <Token> = (
        (,) <environment value>,<v>
    );
<Token>→
    vMuLookup | vFogLookup | vHumidityLookup | vPrecipitationLookup
<environment value>→
    <value>
<v>→
    <value>

```

Für die Tabellen werden jeweils Paare von Umweltparametern und Maximalgeschwindigkeiten angegeben. Dabei wird der Wert des Umweltparameters in der Tabelle gesucht und zwischen benachbarten Angaben linear interpoliert.

Für Umweltwerte kleiner als der Minimalwert wird die entsprechende Minimalgeschwindigkeit angenommen. Für die Überschreitung des Maximalwertes des Umweltparameters gilt entsprechend der maximale Geschwindigkeitswert.

Es existiert jeweils eine Tabelle für die Umwelteinflüsse

- der Nebelstärke (**vFogLookup**).
- der Regendichte (**vPrecipitationLookup**).
- der Feuchtigkeit (**vHumidityLookup**).
- des Reibbeiwertes μ (**vMuLookup**).

Die Standardwerte der Tabellen entsprechen der folgenden Definition:

```

vMuLookup          = ( 0.0,   8.33,      # 30 km/h
                       0.2,   8.33,      # 30 km/h
                       0.4,  13.9,      # 50 km/h
                       0.6,  22.2,      # 80 km/h
                       0.8,  33.3,      # 120 km/h
                       1.0,  55.5 );    # 200 km/h

vFogLookup          = ( 0.0,   55.5,      # 200 km/h
                       0.8,  38.89,     # 140 km/h
                       0.9,  22.2,      # 80 km/h
                       1.0,  13.8 );    # 50 km/h

vHumidityLookup     = ( 0.0,   55.5,      # 200 km/h
                       0.24, 55.5,      # 200 km/h
                       0.8,  22.2,      # 80 km/h
                       1.04, 13.9 );    # 50 km/h

vPrecipitationLookup = ( 0.0,   55.5,      # 200 km/h

```

0.13, 38.89, # 140 km/h
 0.5, 22.2, # 80 km/h
 0.8, 13.7); # 50 km/h

Die Tabellen werden nochmals mit entsprechenden Faktoren verrechnet, welche bei den Verhaltensmodellparametern angegeben werden (siehe Kapitel 5.3.3.1).

5.1.3 Ampelprogramme

5.1.3.1 Vordefinierte Programme

Für die **TrafficlightController** im Editor SILABAEEdit können die hier beschriebenen Ampelprogramme verwendet werden. Findet sich unter den vordefinierten Programmen kein passendes für Ihre Ampelanlage, können Sie wie unter 5.1.3.2 beschrieben eigene Ampelprogramme für den TrafficlightController definieren.

5.1.3.1.1 Programm 1: schnelle Schaltzeiten ohne Fußgängerampel

Einfaches Schaltprogramm für eine Nebenstraße ohne Fußgängerampel.

Kanal 0: Verkehrsampel vorfahrtsberechtigter Straße

Kanal 1: Nebenstraße

Zu- stand	Nachfol- ger	Zeit	Kanal 0	Kanal 1
0	0	10.0s		
1	1	10.0s		
2	3	3.0s		
3	4	1.2s		
4	5	12.0s		
5	6	1.8s		
6	7	3.0s		
7	8	1.2s		
8	9	7.0s		
9	2	1.8s		

5.1.3.1.2 Programm 2: schnelle Schaltzeiten mit Fußgängerampel

Einfaches Schaltprogramm mit Fußgängerampel. Die Schaltzeiten von Programm 2 sind für die Fußgänger knapp bemessen. Für breitere Straßen bitte Programm 4 benutzen.

Kanal 0: Verkehrsampel vorfahrtsberechtigter Straße

Kanal 1: Verkehrsampel Nebenstraße

Kanal 2: Fußgängerampel vorfahrtsberechtigter Straße

Kanal 3: Fußgängerampel Nebenstraße

Kanal 4: gelbes Blinklicht für Fußgängerwarnung der vorfahrtsberechtigten Straße

Kanal 5: gelbes Blinklicht für Fußgängerwarnung der Nebenstraße

Zu- stand	Nachfol- ger	Zeit	Kanal 0	Kanal 1	Kanal 2	Kanal 3	Kanal 4	Kanal 5
0	0	10.0s						
1	1	10.0s						
2	3	3.0s						
3	4	1.2s						
4	5	8.0s						
5	6	4.0s						
6	7	1.8s						
7	8	3.0s						
8	9	1.2s						
9	10	4.0s						
10	11	3.0s						
11	2	1.8s						

5.1.3.1.3 Programm 3: nicht weiterschaltendes Programm mit Fußgängerampel

Das Programm bietet die Möglichkeit, eine Ampelsituation herzustellen, die nicht automatisch weiterschaltet. Dies wird erreicht, indem der Nachfolgezustand gleich dem aktuellen Zustand ist.

Kanal 0: Verkehrsampel vorfahrtsberechtigter Straße

Kanal 1: Verkehrsampel Nebenstraße

Kanal 2: Fußgängerampel vorfahrtsberechtigter Straße

Kanal 3: Fußgängerampel Nebenstraße

Kanal 4: gelbes Blinklicht für Fußgängerwarnung der vorfahrtsberechtigten Straße

Kanal 5: gelbes Blinklicht für Fußgängerwarnung der Nebenstraße

Zu- stand	Nachfol- ger	Zeit	Kanal 0	Kanal 1	Kanal 2	Kanal 3	Kanal 4	Kanal 5
0	0	10.0s						
1	1	10.0s						
2	2	10.0s						
3	3	10.0s						
4	4	10.0s						
5	5	10.0s						
6	6	10.0s						
7	7	10.0s						
8	8	10.0s						
9	9	10.0s						
10	10	10.0s						
11	11	10.0s						

5.1.3.1.4 Programm 4: langsame Schaltzeiten mit Fußgängerampel

Dieses Programm ermöglicht den Fußgängern, auch breitere Straßen zu überqueren, da die Schaltzeiten der Fußgängerampel deutlich länger sind als im Programm 2.

Kanal 0: Verkehrsampel vorfahrtsberechtigter Straße

Kanal 1: Verkehrsampel Nebenstraße

Kanal 2: Fußgängerampel vorfahrtsberechtigter Straße

Kanal 3: Fußgängerampel Nebenstraße

Kanal 4: gelbes Blinklicht für Fußgängerwarnung der vorfahrtsberechtigten Straße

Kanal 5: gelbes Blinklicht für Fußgängerwarnung der Nebenstraße

Zu- stand	Nachfol- ger	Zeit	Kanal 0	Kanal 1	Kanal 2	Kanal 3	Kanal 4	Kanal 5
0	0	10.0s						
1	1	10.0s						
2	3	3.0s						
3	4	2.2s						
4	5	16.0s						
5	6	4.0s						
6	7	1.8s						
7	8	3.0s						
8	9	1.2s						
9	10	15.0s						
10	11	6.0s						
11	2	1.8s						

5.1.3.1.5 Programm 5: gleichlange Schaltzeiten mit Fußgängerampel

Bei diesem Programm wird von der Schaltzeit her nicht zwischen Nebenstraße und vorfahrtsberechtigter Straße unterschieden.

Kanal 0: Verkehrsampel vorfahrtsberechtigter Straße

Kanal 1: Verkehrsampel Nebenstraße

Kanal 2: Fußgängerampel vorfahrtsberechtigter Straße

Kanal 3: Fußgängerampel Nebenstraße

Zu- stand	Nachfol- ger	Zeit	Kanal 0	Kanal 1	Kanal 2	Kanal 3
0	0	10.0s				
1	1	10.0s				
2	3	3.0s				
3	4	13.0s				
4	5	3.0s				
5	6	2.2s				
6	7	6.0s				
7	8	1.8s				
8	9	3.0s				
9	10	13.0s				
10	11	3.0s				
11	12	1.2s				
12	13	6.0s				
13	2	1.8s				

5.1.3.1.6 Programm 6: langsame Schaltzeiten mit Fußgängerampel

Spezialprogramm, bei dem alle Fußgängerampeln auf grün geschaltet werden, wenn alle Verkehrsampeln auf Rot sind. Dies ermöglicht den Fußgängern eine Überquerung der Straße.

Kanal 0: Verkehrsampel vorfahrtsberechtigzte Straße

Kanal 1: Verkehrsampel Nebenstraße

Kanal 2: Fußgängerampel vorfahrtsberechtigzte Straße

Kanal 3: Fußgängerampel Nebenstraße

Zu- stand	Nachfol- ger	Zeit	Kanal 0	Kanal 1	Kanal 2	Kanal 3
0	0	10.0s				
1	1	10.0s				
2	3	3.0s				
3	4	3.0s				
4	5	2.2s				
5	6	6.0s				
6	7	1.8s				
7	8	3.0s				
8	9	13.0s				
9	10	3.0s				
10	11	1.2s				
11	12	6.0s				
12	2	1.8s				

5.1.3.1.7 Programm 7: Fußgängerampel für Überwege

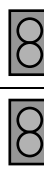
Einfaches Schaltprogramm für Fußgängerüberwege.

Kanal 0: Verkehrsampel

Kanal 1: alternative Verkehrsampel

Kanal 2: Fußgängerampel Überweg

Kanal 3: Fußgängerampel Überweg

Zu- stand	Nachfol- ger	Zeit	Kanal 0	Kanal 1	Kanal 2	Kanal 3
0	0	10.0s				
1	1	10.0s				
2	3	8.0s				
3	4	2.0s				
4	5	12.2s				
5	6	1.8s				
6	7	2.0s				
7	2	15.0s				

5.1.3.2 Benutzerdefinierte Programme

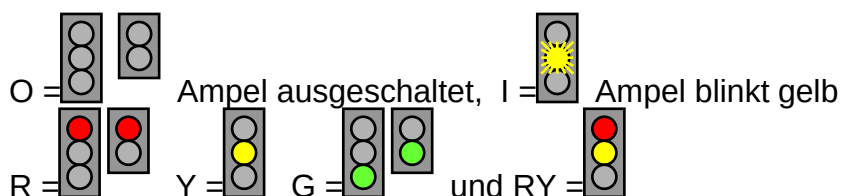
Grammatik 21 User defined programs for traffic lights

```

< user defined programs for traffic lights > →
    TrafficLight_<name> =
    {
        (<state>
    };
<state>→
    (<next state number>,<time in sec.>,( (<switching status channel 0 - 13>))
<switching status>→
    O | R | Y | G | RY | I

```

Die Ampelprogramme bestehen aus einer Menge von Programmezuständen. Jeder Programmezustand hat einen Nachfolgezustand, in dem nach Ablauf der angegebenen Schaltzeit geschaltet wird. Der Nachfolgezustand muss nicht zwingend ein anderer Programmezustand sein, es kann auch in den gleichen Zustand gewechselt werden. Die Programmezustände werden von 0 an beginnend in der Reihenfolge der Angabe durchnummeriert. Als letztes werden die Schaltzustände des Ampelcontrollers in einem Tupel beschrieben. Hier wird für jeden benötigten Kanal (0 – 13) einer der folgenden Ampelschaltzustände angegeben:



Als Beispiel folgt ein Ampelprogramm für die österreichische Ampelschaltung, bei der das Grünsignal am Ende der Grünphase zu blinken beginnt:

```

TrafficLight_Austria =
{
    ## (<State>,<Time sec>,<Channel 1>,<Channel 2>...)
    ## Switching state:
    ## O = Off
    ## R = Red
    ## Y = Yellow
    ## G = Green
    ## RY = Red and yellow
    ## I = yellow flasher
    ( 0,10.00,( O ,O ,O ,O )), # State 00
    ( 1,10.00,( O ,I ,O ,O )), # State 01

    ( 3, 3.00,( R ,R ,R ,R )), # State 02 , Mainstreet Red
    ( 4, 2.20,( RY,R ,R ,R )), # State 03 , Mainstreet Red/Yellow
    ( 5,16.00,( G ,R ,R ,G )), # State 04 , Mainstreet Green
    ( 6, 4.00,( G ,R ,R ,R )), # State 05 , Mainstreet Green flashing

```

```

( 7, 0.25, ( O ,R ,R ,R ) ), # State 06 , Mainstreet Green off ...
( 8, 0.25, ( G ,R ,R ,R ) ), # State 07 , Mainstreet Green flashing
( 9, 0.25, ( O ,R ,R ,R ) ), # State 08 , ... 4 times Green flashing
(10, 0.25, ( G ,R ,R ,R ) ), # State 09
(11, 0.25, ( O ,R ,R ,R ) ), # State 10
(12, 0.25, ( G ,R ,R ,R ) ), # State 11
(13, 0.25, ( O ,R ,R ,R ) ), # State 12
(14, 0.25, ( G ,R ,R ,R ) ), # State 13 , Mainstreet end of green
(15, 2.00, ( Y ,R ,R ,R ) ), # State 14 , Mainstreet Yellow
(16, 3.00, ( R ,R ,R ,R ) ), # State 15 , Mainstreet Red
(17, 2.00, ( R ,RY ,R ,R ) ), # State 16
(18,15.00, ( R ,G ,G ,R ) ), # State 17
(19, 6.00, ( R ,G ,R ,R ) ), # State 18
(20, 0.25, ( R ,O ,R ,R ) ), # State 19
(21, 0.25, ( R ,G ,R ,R ) ), # State 20
(22, 0.25, ( R ,O ,R ,R ) ), # State 21
(23, 0.25, ( R ,G ,R ,R ) ), # State 22
(24, 0.25, ( R ,O ,R ,R ) ), # State 23
(25, 0.25, ( R ,G ,R ,R ) ), # State 24
(26, 0.25, ( R ,O ,R ,R ) ), # State 25
(27, 0.25, ( R ,G ,R ,R ) ), # State 26
(2 , 1.80, ( R ,Y ,R ,R ) ) # State 27
};

```

In diesem Beispiel wurde der Kanal 0 für die Hauptverkehrsstraße belegt. Kanal 1 ist die Nebenstraße, Kanal 2 die Fußgängerampel für die Hauptstraße, Kanal 3 betrifft die Fußgängerampel für die Nebenstraße.

Programmzustand 0 schaltet nicht weiter, da hier die Ampelanlage ausgeschaltet ist. Programmzustand 1 schaltet auch nicht weiter: hier blinkt die Nebenstraße gelb, um auf die nicht in Betrieb genommen Ampelanlage aufmerksam zu machen.

Ab Zustand 2 schaltet die Anlage Ihr Programm durch und springt dann von Programmzustand 27 wieder auf den Zustand 2.

Wird ein benutzerdefiniertes Programm in dem **TrafficlightController**, verwendet, dann muss dies zusätzlich noch per Flowpoint *ProgramState* aktiviert werden.

Mit der Zuweisung **Program = (<Programmname1>, ..., <Programmname4>);** können dem **TrafficlightController** bis zu vier benutzerdefinierte Programme zugewiesen werden. Das erste Programm erhält die interne Programmnummer 20, das zweite die 21 bis zur 23. Wird ein benutzerdefiniertes Programm verwendet, dann muss noch die entsprechende Programmnummer mit einem Flowpoint aktiviert werden.

Im folgenden Beispiel werden dem **TrafficlightController** das obige Ampelprogramm für die österreichische Ampelsteuerung (TrafficLight_Austria) und noch ein weiteres benutzerdefiniertes Programm namens TrafficLight_Germany zugewiesen.

```

TrafficlightController TL_Austria
{
    Program = (Austria,Germany);
    ...
    Flowpoints =
    {
        (ModuleTime,0,Programstate,20,10) ,
        (... ,SimCar,ProgramState,21,7)
    };
};

```

Das österreichische Programm erhält die Programmnummer 20 und das zweite angegebene Programm die 21. Mit dem ersten Flowpoint wird die Ampelschaltung auf Programm 20 (also das österreichische Programm) geschaltet. Im weiteren Verlauf wird das zweite Programm (Nummer 21, Programm TrafficLight_Germany) durch das EGO – Fahrzeug aktiviert.

5.1.4 RoadUserTable

Im Folgenden wird beschrieben, aus welchen Fahrzeugen sich die vordefinierten Fahrzeuggruppen zusammensetzen. Bitte beachten Sie: Die vordefinierten Fahrzeuggruppen werden regelmäßig überarbeitet und die darin enthaltenen Fahrzeuge durch neuere (möglichst vergleichbare) Modelle ersetzt.

5.1.4.1 RoadUserTable aus „SILAB_TRFX_600.cfg“

Folgende einzelne Fahrzeugtypen und Fahrzeuggruppen sind in der Datei „SILAB_TRFX_600.cfg“ definiert:

Einzelfahrzeuge

 Alfa147 Länge: 4,172 m Breite: 1,916 m	 AudiA4 Länge: 4,66 m Breite: 2,064 m	 AudiA6 Länge: 4,58 m Breite: 1,856 m
 BMW645 Länge: 4,906 m Breite: 1,98 m	 BMWi8 Länge: 4,65 m Breite: 2,102 m	 BMWR1200 Länge: 2,208 m Breite: 0,94 m

 Chrysler300 Länge: 5,06 m Breite: 2,054 m	 CitroenC4 Länge: 4,29 m Breite: 1,982 m	 Citybike_L Länge: 1,87 m Breite: 0,82 m
 Citybike_M Länge: 1,8 m Breite: 0,66 m	 Citybike_S Länge: 1,62 m Breite: 0,76 m	 ClaasAxion Länge: 5,116 m Breite: 2,86 m
 ClaasLexion Länge: 6,4 m Breite: 6,48 m	 Fiat500 Länge: 3,572 m Breite: 1,885 m	 FiatPanda Länge: 3,652 m Breite: 2,036 m
 GarbageTruck Länge: 7,552 m Breite: 2,448 m	 HondaCivic Länge: 4,689 m Breite: 2,04 m	 HyundaiIX35 Länge: 4,344 m Breite: 2,088 m

 Hyundail40 Länge: 4,763 m Breite: 1,979 m	 HyundaiVeloster Länge: 4,225 m Breite: 1,89 m	 IvecoDaily_TowTruck Länge: 6,988 m Breite: 2.226 m
 IvecoDaily_Pickup Länge: 6,768 m Breite: 2.226 m	 KTMRC125 Länge: 2,05 m Breite: 0,72 m	 LiebherrLTM1055_crane Länge: 9,92 m Breite: 3,38 m
 LexusLS430 Länge: 4,85 m Breite: 2,074 m	 MAN_TGL Länge: 8,468 m Breite: 3,056 m	 MAN_TGM_Firetruck Länge: 9,34 m Breite: 3,38 m

 MAN_TGX Länge: 5,652 m Breite: 3,04 m	 MAN_TGX_Curtainsider Länge: 12,354 m Breite: 2,546 m	 MAN_TGX_Flatbed Länge: 12,44 m Breite: 2,38 m
 MAN_TGX_Tanker Länge: 11,8 m Breite: 2,56 m	 MercedesActros Länge: 21,376 m Breite: 2,86 m	 MercedesC Länge: 4,692 m Breite: 2,04 m
 MercedesC_Taxi Länge: 4,692 m Breite: 2,04 m	 MercedesC_Polizei Länge: 4,692 m Breite: 2,04 m	 MercedesE Länge: 4.86 m Breite: 1,97 m

 MercedesIntegro Länge: 12,228 m Breite: 2,88 m	 MerchedesSprinter Länge: 5,992 m Breite: 2,42 m	 MercedesSprinter_ Ambulance Länge: 6,39 m Breite: 2,68 m
 MercedesSprinter_Post Länge: 6 m Breite: 2,52 m	 MitsubishiASX Länge: 4,314 m Breite: 2,008 m	 Mountainbike Länge: 1,8 m Breite:0,68 m
 Nissan350Z Länge: 4,73 m Breite: 2,1 m	 Oldbike Länge: 1,77 m Breite: 0,6 m	 OpelCorsa Länge: 3,94 m Breite: 2,088 m

 OpelInsignia Länge: 4,92 m Breite: 2,16 m	 OpelMeriva Länge: 4,27 m Breite: 2,14 m	 OpelZafira Länge: 4,66 m Breite: 1,062 m
 ParcelService Länge: 6,864 m Breite: 2,68 m	 Peugeot508 Länge: 4,81 m Breite: 2,1m	 PiaggioFly Länge: 1,839 m Breite: 7.16 m
 PontiacAztek Länge: 4,596 m Breite: 2,102 m	 PorscheMacan Länge: 4,66 m Breite: 1,996 m	 Racebike Länge: 1,71 m Breite: 0,48 m

 RangeRover Länge: 4,89 m Breite: 2,08 m	 RenaultTwingo Länge: 3,73 m Breite: 2,02 m	 Sidewalkbike Länge: 0.872 m Breite: 0,42 m
 RoadSweeper Länge: 4,77 m Breite: 2,378 m	 SolarisUrbino Länge: 12.114 m Breite: 2,97 m	 ToyotaCelica Länge: 4,04 m Breite: 1,86 m
 VWCamper Länge: 4,492 m Breite: 1,93 m	 VWGolf Länge: 4,264 m Breite: 2,054 m	 VWPassat Länge: 4,71 m Breite: 2,1 m

 WildBoar_fast Länge: 1,676 m Breite: 0,515 m	 WildBoar_slow Länge: 1,676 m Breite: 0,515 m	
---	--	--

Gruppe: Car

HyundaiIX35 (red)
Fiat500 (blue)
OpelCorsa (black)
OpelInsignia (yellow)
OpelCorsa (white)
MercedesC (taxi)
VW Golf (silver)
Alfa147 (grey)
FiatPanda (black)
AudiA4 (grey)
CitroenC4 (beige)

MercedesE (black)
HondaCivic (grey)
HyundaiI40 (silver)
Peugeot508 (brown)
PorscheMacan (grey)
MitsubishiASX (blue)
OpelMeriva (green)
BMW645 (grey)
OpelZafira (black)
HyundaiVeloster (blue)
VWPassat (white)

Gruppe: CarVan

HyundaiIX35 (red)
Fiat500 (grey)
MercedesSprinter (silver)
IvecoDaily_pickup (yellow)
OpelMeriva (brown)
VW Golf (red)
Peugeot508 (green)
MercedesC (black)
IvecoDaily_pickup (orange)
FiatPanda (black)
PorscheMacan (black)

CitroenC4 (beige)
HondaCivic (black)
OpelInsignia (green)
VWCamper (white)
Alfa147 (red)
OpelZafira (green)
MercedesSprinter (white)
RenaultTwingo (blue)
AudiA4 (grey)

Gruppe: VanTruck

MercedesSprinter (white)
MAN_TGL (blue)
MercedesSprinter (silver)
MAN_TGL (red)
MAN_TGL (white)
MercedesSprinter (white)
MAN_TGL (green)
MercedesSprinter (black)

Gruppe: Truck

MAN_TGL (blue)
MAN_TGL (green)
MAN_TGL (red)

MAN_TGL(white)

Gruppe: **TruckMotorway**

MAN_TGX_Tanker(yellow)
MAN_TGL(beige)
MAN_TGX_Curtainsider(silver)
MAN_TGX_Flatbed(orange)
MAN_TGL(red)
MAN_TGX_Flatbed(mint)

Gruppe: **Van**

IvecoDaily_pickup(blue)
MercedesSprinter(white)
MAN_TGM_fire_truck(white)

Gruppe: **Motorbike**

KTMRC125(black)
BMW1200(red)
BMW1200(silver)
KTMRC125(red)
BMW1200(blue)

Gruppe: **MotorbikeCity**

OldBike(black)
Piaggiofly(red)
Piaggiofly(white)
OldBike(red)
Piaggiofly(yellow)

Gruppe: **Bike**

Citybike_L(green)
Citybike_M(blue)
Racebike(red)
Mountainbike(black)

Das Fahrzeug mit der besonderen Bezeichnung ‚Dummy‘ wird verwendet sobald in einer Strecke auf einen nicht definierten Fahrzeugnamen verwiesen wird.

Die möglichen Farben sind im Abschnitt **ColourTable** in der Datei „SILAB_TRFX_600.cfg“ definiert. Diese wird im nachfolgenden Abschnitt detailliert erläutert.

5.1.4.2 ColourTable aus „SILAB_TRFX_600.cfg“

In der Datei „SILAB_TRFX_600.cfg“ ist eine Tabelle in Form einer ColourTable definiert. Diese Farben können den Fahrzeugen entsprechend zugewiesen werden (siehe Abschnitt 2.5.1 für Fahrzeuge in Quellen sowie Abschnitt 5.1.4.3 für Einzelfahrzeuge).

Grammatik 22 ColourTable

ColourTable Colours

```
{  
    (0...*)<colour name> = (red value, green value, blue value);  
};
```

Beispiel:

```
ColourTable Colours  
{  
    black = (0.1, 0.1, 0.1);  
    white = (1.0, 1.0, 1.0);  
    taxi = (0.9, 0.82, 0.71);  
    beige = (0.91, 0.87, 0.82);  
    silver = (0.7, 0.7, 0.7);  
};
```

Liste der vordefinierten Farben in SILAB:



white (1.0, 1.0, 1.0)

black (0.1, 0.1, 0.1)

taxi (0.9, 0.82, 0.71)

beige 0.91, 0.87, 0.82



red (0.67, 0.12, 0.10)

ruby (0.54, 0.07, 0.07)

wine (0.13, 0.07, 0.10)

raspberry (0.45, 0.027, 0.137)



green (0.11, 0.22, 0.16)

emerald (0.2, 0.47, 0.33)

meadow (0.38, 0.6, 0.23)

neongreen (0, 0.71, 0.10)



yellow (1, 0.80, 0)

sun (0.98, 0.74, 0.12)

orange (1, 0.5, 0)

dawn (0.93, 0.36, 0)



blue (0, 0.09, 0.27)

darkblue (0.09, 0.19, 0.32)

sky (0.16, 0.45, 0.72)

sea (0.13, 0.4, 0.49)



brown (0.4, 0.2, 0.1)

nut (0.28, 0.15, 0.11)

copper (0.61, 0.38, 0.17)

clay (0.55, 0.29, 0.19)



5.1.4.3 Benutzerdefinierte RoadUserTable

Werden für einzelne Strecken besondere Fahrzeuge benötigt, die in der Standard RoadUserTable der Datei SILAB_TRFX_600.cfg nicht enthalten sind, kann eine eigene **RoadUserTable** definiert werden.

Grammatik 23 road user table

```

< road user table > →
    RoadUserTable <any name>
    {
        (0...*)<vehicle type> = <vehicle model>;
        (0...*)<vehicle group type> =
            { (1...*)<vehicle model> (colour name, dirt value)};
    };
<vehicle model> →
    (1)<Token>(0...*)[.<Token>]
  
```

In der **RoadUserTable** können Einzelfahrzeuge und Fahrzeuggruppen definiert werden. Im folgenden Beispiel werden in der **RoadUserTable** *MyRoadUser* die Fahr-

zeuggruppe `MyTruck` und ein Einzelfahrzeug `MyVehicle` definiert mit entsprechendem Verschmutzungsgrad aus dem Wertebereich `[0, 1]`. Die Farben der Gruppenfahrzeuge werden über die **ColourTable** gesetzt.

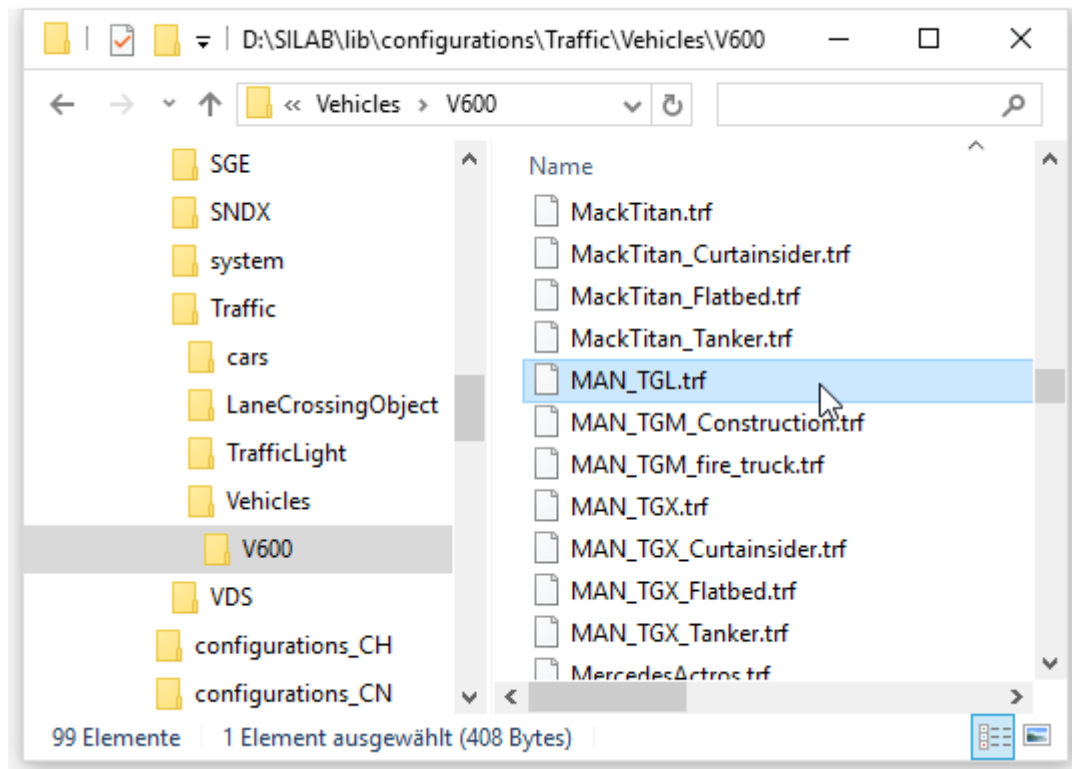
```

RoadUserTable MyRoadUser
{
    MyTruck =
    {
        Vehicles.V600.MAN_TGL(yellow, 0.4),
        Vehicles.V600.MAN_TGL(black, 0.0),
        Vehicles.V600.MAN_TGL(blue, 0.2),
        Vehicles.V600.MAN_TGL(red, 0.6)    };
    MyVehicle = Vehicles.V600.MAN_TGL;
};

ColourTable Colours
{
    black = (0.1, 0.1, 0.1);
    white = (1.0, 1.0, 1.0);
    taxi = (0.9, 0.82, 0.71);
    beige = (0.91, 0.87, 0.82);
    silver = (0.7, 0.7, 0.7);
    grey = (0.5, 0.5, 0.5);
    graphite = (0.24, 0.24, 0.24);
    iron = (0.25, 0.29, 0.33);
    red = (0.67, 0.12, 0.10);
    ruby = (0.54, 0.07, 0.07);
    wine = (0.13, 0.07, 0.10);
    raspberry = (0.45, 0.027, 0.137);
    pink = (1, 0.27, 0.8);
};

```

Die rechte Seite der Zuweisung bzw. die Liste in der Gruppe bezieht sich auf Dateien, in denen die Informationen über die Fahrzeuge wie z.B. Fahrzeuggröße und Position des Schwerpunkts definiert werden. Die Punktnotation entspricht einer Verzeichnisstruktur erweitert um das Verzeichnis `SILAB\lib\configurations\Traffic\Vehicles\V600`



Der Eintrag `Vehicles.V600.MAN_TGL` referenziert somit die im obigen Screenshot selektierte Datei `SILAB\lib\configurations\Traffic\Vehicles\V600\MAN_TGL.trf`.

5.2 Grundlegendes Verhalten von Verkehrsteilnehmern

Alle Verkehrsteilnehmer besitzen unabhängig von den zugeordneten Verhaltensmodellen einige grundsätzliche Steuerungsmöglichkeiten. Diese werden über Flowpoints ausgelöst und erlauben folgende Befehle an die Verkehrsteilnehmer zu senden:

- Aktivieren und Deaktivieren eines Verkehrsteilnehmers
 - **Activate**

Aktivierung des Verkehrsteilnehmers. Bei einzelnen Verkehrsteilnehmern muss dieser Flowpoint vom EGO - Fahrzeug ausgelöst werden. Der Verkehrsteilnehmer erscheint unmittelbar an der definierten Position (siehe Grammatik 8). Diese Position sollte außerhalb der Sichtbarkeit des EGO-Fahrzeugs sein. Eine Aktivierung kann zeitlich oder örtlich erfolgen (siehe Abschnitt 2.4).

- **Deactivate**

Dieser Flowpoint deaktiviert den Verkehrsteilnehmer. Er verschwindet unmittelbar nach Ausführung des

Flowpoints. Daher ist darauf zu achten, dass der Verkehrsteilnehmer für den EGO-Fahrer nicht sichtbar ist.

- Spurwechsel – hierbei ist zu beachten, dass dies ein ganz einfacher Spurwechsel ist, der ohne Berücksichtigung weiterer Verkehrsteilnehmer durchgeführt wird. Ein Spurwechsel, der andere Verkehrsteilnehmer beachtet, ist ebenfalls möglich, z.B. mit dem Verhaltensmodell **LaneChangeAvoidance** (siehe Abschnitt 5.3.6.2).
 - **ChangeLane**, *<lane>*, *<time / distance>*, *<mode>*, *<direction>*

Leitet einen Spurwechsel auf die Zielspur *<lane>* ein. Der Spurwechsel ist in der angegebenen *<time [s]/ distance [m]>* (je nach *<mode>*) abgeschlossen. Der *<mode>* wird vorgegeben.
<mode> = 0 : Weg
<mode> = 1 : Zeit.
 Der Parameter *<direction>* gibt an, ob ein Spurwechsel nach rechts oder links erfolgt (-1 = links, 1 = rechts).
- Richtungswahl für das Fahren auf abbiegenden Spuren
 - **Right**, [*<probability>*]
 Bei Aktivierung des Flowpoints wird die Fahrspur mit dem Heading **Right** bevorzugt gewählt. Ist eine *<probability>* angegeben (Wert zwischen 0=nie bis 1=immer) so wird zum Zeitpunkt der Aktivierung des Flowpoints über einen Zufallszahlengenerator entschieden, ob die Aktion übernommen wird. Ist kein Heading angegeben, dann wird immer die Richtung **Straight** gewählt, sofern verfügbar.
 - **Left**, [*<probability >*]
 Bei Aktivierung des Flowpoints wird die Fahrspur mit dem Heading **Left** bevorzugt gewählt. Ist eine *<probability>* angegeben (Wert zwischen 0=nie bis 1=immer) so wird zum Zeitpunkt der Aktivierung des Flowpoints über einen Zufallsgenerator entschieden, ob die Aktion übernommen wird.
 - **Straight**, [*<probability >*]
 Bei Aktivierung des Flowpoints wird die Fahrspur mit dem Heading **Straight** bevorzugt gewählt. Ist eine *<probability>* angegeben (Wert zwischen 0=nie bis 1=immer) wird zum Zeitpunkt der Aktivierung des Flowpoints über einen Zufallsgenerator entschieden, ob die Aktion übernommen wird.

Zu beachten ist, dass das gewählte Heading so lange aktiv bleibt, bis ein neues Heading zugewiesen wird. Daher ist es empfehlenswert, nach jedem Abbiegen das Heading erneut auf **Straight** zu setzen.
- Änderung der Lichtsignale der Fahrzeuge
 - **AutoBrakeLightControl**, *<value>*
 Wird *<value>* auf 0 gesetzt (Defaultwert = 1), dann werden bei negativen Beschleunigungen die Bremslichter nicht mehr automatisch aktiviert.

- **AutoIndicatorLightControl**, *<value>*
<value> = 1: es wird automatisch geblinkt, wenn die Wunschrichtung Links bzw. Rechts ist und die selektierte Fahrspur das gleiche Heading aufweist. Wert = 0: kein automatisches Setzen der Blinker.
- **Brakelight**, *<value>*
<value> = 1: Die Bremslichter leuchten.
<value> = 0: Die Bremslichter sind aus. Dieser Flowpoint wird ignoriert, wenn **AutoBrakeLightControl** auf 1 gesetzt ist.
- **Light**, *<value>*
<value> = 1: Einschalten des Abblendlichts.
<value> = 0: Ausschalten des Abblendlichts.
- **IndicatorLeft**, *<Wert>*
<value> = 1: Linken Blinker setzen.
<value> = 0: Linken Blinker ausschalten.
Dieser Flowpoint hat nur Wirkung, wenn **AutoIndicatorLightControl** auf 0 gesetzt ist.
- **IndicatorRight**, *<Wert>*
<value> = 1: Rechten Blinker setzen.
<value> = 0: Rechten Blinker ausschalten.
Dieser Flowpoint hat nur Wirkung, wenn **AutoIndicatorLightControl** auf 0 gesetzt ist.
- Status von Einsatzfahrzeugen
 - Privilege**, *<value>*
<value> = 0: Das Blaulicht sowie das Martinshorn sind deaktiviert.
<value> = 1: Das Blaulicht ist aktiviert und das Martinshorn deaktiviert
<value> = 2: Das Blaulicht und das Martinshorn sind aktiviert.
Das Martinshorn ist auf Stufe1.
<value> = 3: Das Blaulicht und das Martinshorn sind aktiviert.
Das Martinshorn ist auf Stufe2 eingestellt.
Stufe 2 heißt je nach Simulatoreinstellung ein anderer Ton und eine erhöhte Lautstärke.
- Farb- und Schmutzdarstellung
 - colourdirt**, *<red value>*, *<green value>*, *<blue value>*, *<dirt value>*

Ändern der Farbe und des Verschmutzungsgrad eines Fahrzeugs.
Der Wertebereich aller angegebenen Werte liegt im Intervall [0, 1].

Ein Beispiel für das grundlegende Verhalten von Verkehrsteilnehmern:

```
Vehicle Lighting      #
{
    Vehicle = Car;
    Position = (UE2,300,1);
    Behaviour = {... };

    Flowpoints =
    {
        (UE2, 10, 1, SimCar,Activate),
        (UE2, 1200, 1, Traffic, AutoIndicatorLightControl, 0),
        (UE2, 1210, 1, Traffic, IndicatorLeft, 1),
        (UE2, 1220, 1, Traffic, AutoIndicatorLightControl, 1),
        (UE2, 1300, 1, Traffic, ChangeLane, 1, 50, 0),
        (UE2, 2101, 1, Traffic, AutoBrakeLightControl,0), ...
        (UE2, 2100, 1, Traffic, Right)
    };
};
```

In dem Streckenabschnitt UE2 wird der Verkehrsteilnehmer aktiviert, wenn das EGO-Fahrzeug den Streckenmeter 10 passiert. Das Aufsetzen erfolgt in 290m Entfernung, damit es der EGO-Fahrer nicht plötzlich erscheinen sieht. Bei Streckenmeter 1200 wird der Blinker im Fahrzeug eingeschaltet. Hundert Meter weiter wird ein Spurwechsel durchgeführt. Nach dem Überholvorgang soll das Fahrzeug bei nächster Gelegenheit rechts abbiegen.

5.3 Verhaltensmodelle

5.3.1 Überblick

Die folgende Tabelle gibt einen Überblick über die von TRFX zur Verfügung gestellten Verhaltensmodelle, deren Einsatzzweck und Funktionsweise.

Modellname	Verwendungszweck	Einschränkungen
EXSTD1 (siehe Abschnitt 5.3.5.1) Wird im BehaviourScheme Standard und Ambient verwendet	Einsatzbereich: Course / Area2. Standardverhaltensmodell für Area2 Abschnitte.	Keine Reaktion auf Fahrzeuge mit Sonder- und Wegerechten. Keine Beachtung der Ge- schwindigkeitsschilder, Rechts vor Links Regel und Vorfahrts- regel außer Ampelanlagen. Da dieses Modell recheninten- siv ist, sollten in komplexen Course Szenarien mit hoher Verkehrsdichte Modelle bevor- zugt werden, die speziell für Course Abschnitte entwickelt wurden.
EXSTD2 (siehe Abschnitt 5.3.5.2) Wird im BehaviourScheme Emergency verwendet	Einsatzbereich: Course / Area2 Erweiterung des EXSTD1 um ein einfaches Verhalten im Falle des Einsatzes von Sonder- und Wege- rechten. Dieses Modell kann in diesem Fall an die Seite fahren.	Nur einfache Reaktion auf Fahr- zeuge mit Sonder- und Wege- rechten. Keine Beachtung der Ge- schwindigkeitsschilder, Rechts vor Links Regel und Vorfahrts- regel außer Ampelanlagen. Da dieses Modell recheninten- siv ist, sollten in komplexen Course Szenarien mit hoher Verkehrsdichte Modelle bevor- zugt werden, die speziell für Course Abschnitte entwickelt wurden.
EXSTDEMG (siehe Abschnitt 5.3.5.4)	Einsatzbereich: Course / Area2 Erweiterung des EXSTD2 um die Gassenbildung im Falle des Ein- satzes von Sonder- und Wege- rechten.	Keine Beachtung der Ge- schwindigkeitsschilder, Rechts vor Links Regel und Vorfahrts- regel außer Ampelanlagen. Da dieses Modell recheninten- siv ist, sollten in komplexen Course Szenarien mit hoher Verkehrsdichte Modelle bevor- zugt werden, die speziell für Course Abschnitte entwickelt wurden.
EXSTD3 (siehe Abschnitt 5.3.5.3)	Einsatzbereich: Course / Area2 Erweiterung des EXSTD1 um ein IDM Modell für Area2 Kreuzungs- bereiche.	Keine Beachtung der Ge- schwindigkeitsschilder, Rechts vor Links Regel und Vorfahrts- regel außer Ampelanlagen. Da dieses Modell recheninten- siv ist, sollten in komplexen Course Szenarien mit hoher Verkehrsdichte Modelle bevor- zugt werden, die speziell für Course Abschnitte entwickelt wurden.

Modellname	Verwendungszweck	Einschränkungen
EXSTDGlobalS (siehe Abschnitt 5.3.5.5) Wird im BehaviourScheme City verwendet	Einsatzbereich: Course / Area2 Komplexes Verhaltensmodell basierend auf dem Intelligent Driver Model, das somit eine kontinuierliche und realistische Verzögerung beim Abbremsen auf Objekte liefert. Darüber hinaus beachtet es die meisten Verkehrsregeln. Berücksichtigte Verkehrsregeln: • Rechts vor Links, • Vorfahrtsschilder, • Vorfahrt achten, • Stoppschilder, • Ampeln, • sowie Geschwindigkeitsbegrenzungen	Wenig Einflussmöglichkeiten auf das Verhalten des Verkehrsteilnehmers
LaneCrossing (siehe Abschnitt 5.3.5.6) Wird im Behaviour Scheme LaneCrossing verwendet	Einsatzbereich: Course / Area2. Verhaltensmodell für querende ‚Verkehrsteilnehmer‘ wie beispielsweise ein Wildschwein oder ein quer über die Straße hüpfender Ball.	Nur Einsatz für querende Verkehrsteilnehmer sinnvoll.
HazardAvoidance2 (siehe Abschnitt 5.3.6.1)	Einsatzbereich: Course. HazardAvoidance2 stellt eine Kollisionsvermeidung basierend auf dem Modell EXIDM dar. Das Ergebnis ist eine kontinuierliche und realistische Verzögerung beim Abbremsen auf Objekte	Dieses Modell führt keine Beschleunigungen durch, daher wird als nachfolgendes Modell für einen Verkehrsteilnehmer noch ein Verhaltensmodell HazardFree benötigt, damit der Verkehrsteilnehmer eine entsprechende Wunschgeschwindigkeit erhält. - Komplexere Verkehrsregeln wie z.B. Recht-vor-Links, Stoppschilder werden nicht berücksichtigt, daher empfiehlt es sich dieses Modell in Courses einzusetzen.
LaneChangeAvoidance (siehe Abschnitt 5.3.6.2)	Einsatzbereich: Course. Unterstützung bei einem Spurwechsel auf Überlandstrecken.	Dieses Modell führt nur den Spurwechsel durch. Damit der Verkehrsteilnehmer fahren kann, muss es mit anderen Modellen kombiniert werden wie beispielsweise HazardFree.

Modellname	Verwendungszweck	Einschränkungen
SignalSpeed (siehe Abschnitt 5.3.6.3)	Einsatzbereich: Course. Setzen der Geschwindigkeit aus Eingängen der DPUTRFX oder mit parametrisierten Signalgeneratoren.	Dieses Modell berücksichtigt keine anderen Verkehrsteilnehmer. Es kann mit HazardFree / HazardAvoidance2 kombiniert werden um auf andere Verkehrsteilnehmer zu bremsen. In diesem Fall kann es zu Abweichungen im Geschwindigkeitsprofil kommen.
SimCarLocked (siehe Abschnitt 5.3.6.4)	Einsatzbereich: Course. Dieses Verhaltensmodell regelt einen Verkehrsteilnehmer auf einen bestimmten Abstand zum EGO-Fahrzeug.	Dieses Modell berücksichtigt keine anderen Verkehrsteilnehmer, es sollte daher z.B. mit HazardFree kombiniert werden. SimCarLocked bremst bei Erreichen des eingestellten Abstands schlagartig ab. Sollte dieses Verhalten störend sein, dann ist SimCarLocked2 die bessere Wahl.
SimCarLocked2 (siehe Abschnitt 5.3.6.5)	Einsatzbereich: Course. SimCarLocked2 entspricht im Wesentlichen dem Verhaltensmodell SimCarLocked. Allerdings stellt SimCarLocked sanft (differenzierbare Geschwindigkeit) den Abstand zum EGO-Fahrzeug her.	Siehe Einschränkungen des Verhaltensmodells SimCarLocked.
CommandAction (siehe Abschnitt 5.3.6.6)	Einsatzbereich: Course / Area2. Senden von Kommandos an den Verkehrsteilnehmer. Mit diesem Verhaltensmodell können Verhaltensweisen erzeugt werden, die einen starren Ablauf besitzen unter den beschriebenen Auslösebedingungen einsetzen.	Dieses Modell berücksichtigt keine anderen Verkehrsteilnehmer. Aus diesem Grund wird es häufig mit HazardAvoidance2 / HazardFree kombiniert. Bei der Kombination mit komplexeren Modellen (z.B. EXSTD1) wird die CommandAction diesem Modell vorangestellt.

Modellname	Verwendungszweck	Einschränkungen
EXWDM (siehe Abschnitt 5.3.6.7)	Einsatzbereich: Course. EXWDM entspricht im Wesentlichen dem EXIDM-Modell. Der entscheidende Unterschied ist, dass EXWDM zwischen verschiedenen Stadien wie z.B. Bremsen, Folgenfahren oder Annähern unterscheidet und keine kontinuierliche Beschleunigung aufweist.	Für das EXWDM Modell gelten die gleichen Einschränkungen wie für das EXIDM Modell. Einsatzgebiet sind auch hier hauptsächlich Autobahnen und Landstraßen. Für kleinere Kreuzungen kann dieses Modell ebenfalls genutzt werden. Ampelgeregelte Kreuzungen ohne abbiegenden Gegenverkehr werden korrekt behandelt.
EXIDM (siehe Abschnitt 5.3.6.8)	Einsatzbereich: Course. EXIDM simuliert einen Verkehrsteilnehmer, der mit kontinuierlichen Beschleunigungen längere Zeit anderen Fahrzeugen folgen soll. Insbesondere auf Autobahnen ist dieses Modell sehr gut geeignet um ein flexibles Fahren darzustellen. Überholvorgänge werden von diesem Modell automatisch durchgeführt. In Kreuzungen reagiert es lediglich auf rote Ampeln, andere Verkehrsregeln werden nicht befolgt.	EXIDM eignet sich nicht für einen Verkehrsteilnehmer der zu jeder Zeit ein klar definiertes Verhalten zeigen soll, da aufgrund des komplexen Modells ein flexibles aber damit auch dynamisches Verhalten modelliert wird. Ein exakter Abstand zum Vordermann wird ebenso wenig eingehalten wie eine feste Geschwindigkeit. Für beides werden in diesem Modell nur Schranken spezifiziert.
HazardFree (siehe Abschnitt 5.3.6.9)	Einsatzbereich: Course. Das Modell HazardFree ist eine Kombination eines HazardAvoidance und eines FreeDriving-Modells, beides basiert in diesem Fall auf dem IDM-Modell (siehe hierzu auch EXIDM). Spurwechsel werden von diesem Modell nicht unterstützt. Der Vorteil dieses Modells ist, dass alle Beschleunigungen und Verzögerungen kontinuierlich sind und sehr realistisch dargestellt werden.	Dieses Modell verbindet HazardAvoidance und FreeDriving und Sie haben keine Möglichkeit zwischen diesen beiden Modellen andere Modelle zu nutzen. Insbesondere ein CommandAction- oder ein SimCar-Locked-Modell wird gelegentlich zwischen diese beiden geschaltet. In solchen Fällen müssen sie auf die einzelnen Modelle HazardAvoidance2 und HazardFree zurückgreifen. In allen anderen Fällen können sie an deren Stelle das Modell HazardFree nutzen, das von den kontinuierlichen Beschleunigungen des IDM-Modells profitiert. Komplexere Verkehrsregeln wie z.B. Recht-vor-Links, Stoppschilder werden nicht berücksichtigt, daher empfiehlt es sich dieses Modell in Courses einzusetzen.

Über die im obigen Abschnitt aufgeführten Verhaltensmodelle hinaus gibt es einige weitere obsoleete Verhaltensmodelle. Aus Kompatibilitätsgründen sind diese Modelle noch im Lieferumfang von SILAB enthalten.

Liste der obsoleten Modelle:

Modellname	Verwendungszweck	Einschränkungen
HazardAvoidance (siehe Abschnitt 5.3.7.1)	Einsatzbereich: Course. Das Modell HazardAvoidance stellt ein Modell zur Vermeidung von Kollisionen dar. Das Abbremsen erfolgt mittels sehr starker abrupter Verzögerungen. Empfohlen wird daher das Modell HazardAvoidance2, die eine kontinuierliche Verzögerung beim Abbremsen auf Hindernisse realisiert. Dieses Modell kann bedingt auch in Area2 eingesetzt werden, da es auch Ampelgeregelte Kreuzungen durchfahren kann. Auf alle Hindernisse im Scanbereich wird gebremst.	Dieses Modell führt keine Beschleunigungen durch, daher wird als nachfolgendes Modell für einen Verkehrsteilnehmer noch ein Verhaltensmodell FreeDriving benötigt, damit der Verkehrsteilnehmer eine entsprechende Wunschgeschwindigkeit erhält. - Komplexere Verkehrsregeln wie z.B. Recht-vor-Links, Stopp-Schilder werden nicht berücksichtigt, daher empfiehlt es sich dieses Modell in Courses einzusetzen.
FreeDriving (siehe Abschnitt 5.3.7.2)	Einsatzbereich: Course. Dieses Modell regelt auf eine entsprechende Wunschgeschwindigkeit. Die Beschleunigungs- und Verzögerungswerte können dabei spezifiziert werden.	Da dieses Modell keine weiteren Verkehrsteilnehmer berücksichtigt wird es in den meisten Fällen in Kombination mit dem Modell HazardFree / HazardAvoidance2 eingesetzt.

5.3.2 Definition von Verhaltensmodellen für Verkehrsteilnehmer

In manchen Fällen ist es notwendig, von den vordefinierten Behaviour Schemes (siehe Abschnitt 5.1.1) abzuweichen und eine eigene Menge an Verhaltensmodellen für einen Verkehrsteilnehmer zu erstellen. Dies funktioniert folgendermaßen:

Grammatik 24 behaviour models

< behavior models > →

Behaviour = {_(,) <name of a behaviour model> };

<name of a behaviour model > →

(<name> (0...*) [, <tuple-parameter>])

< Parameter > →

<value>

Die Menge wird aus einzelnen Verhaltensmodellen zusammengestellt, z.B.:

```
Behaviour =
{
    (HazardAvoidance2,1,2),
    (SignalSpeed,Data,1)
};
```

Im obigen Beispiel besteht die Menge aus dem Verhaltensmodell **HazardAvoidance2** und **SignalSpeed**. Das Modell mit der höchsten Priorität steht im ersten

Tupel innerhalb der Menge und das Modell mit der niedrigsten Priorität im letzten Tupel der Menge. Innerhalb des Tupels steht zuerst der Name des Verhaltensmodells gefolgt von einer optionalen Liste von Parametern. Die Bedeutung der einzelnen Parameter finden Sie in der Beschreibung der jeweiligen Verhaltensmodelle in den Abschnitten 5.3.5 und 5.3.5.6.

Im obigen Beispiel wird für die **HazardAvoidance2** der Parameter *<MinDistF>* auf 2.0m und der Parameter *<THWF>* auf 1.0s gesetzt. Bei der **SignalSpeed** wird der Parameter *<Signal>* auf Data (Geschwindigkeit wird über ein Eingang der DPUTRF gesetzt) und der Parameter *<Kanal>* auf 1 gesetzt.

Das Modell mit der höchsten Priorität wird in jedem Simulationsschritt als erstes ausgeführt. Wenn dieses Modell im aktuellen Simulationsschritt den Verkehrsteilnehmer nicht beeinflussen möchte, gibt es die Kontrolle über den Verkehrsteilnehmer noch im gleichen Simulationsschritt an das nächste Modell weiter. Greift ein Modell ein, dann werden alle nachfolgenden Modelle blockiert und in diesem Simulationsschritt nicht mehr ausgeführt.

5.3.3 Allgemeine Parameter der Verhaltensmodelle

In den nachfolgenden Abschnitten werden einzeln definierte Parameter beschrieben, welche von mehreren Verhaltensmodellen gemeinsam verwendet werden. Bei der Beschreibung der Verhaltensmodelle sind diese durch den Zusatz [general] gekennzeichnet.

Das besondere an diesen Parametern ist, dass diese innerhalb der Definition eines Verkehrsteilnehmers direkt zugewiesen werden können, wie im Folgenden am Beispiel des Parameters **TargetSpeed** dargestellt:

```
SILAB SCN
{
  ...
  define Module DEMO1
  {
    ...
    ModuleID = 701;

    Traffic Urban
    {
      VehicleX Fast
      {
        ...
        BehaviourScheme = Standard;
        TargetSpeed = 33.0;
      };
    };
  };
  ...
};
```

Bei jedem Parameter ist angegeben in welchen Verhaltensmodellen dieser Anwendung findet.

5.3.3.1 Parameter zur Längsgeschwindigkeit

SpeedFactor [1.0]	Multiplikator für die Zielgeschwindigkeit, wenn diese in Abhängigkeit der Entwurfsgeschwindigkeit des Streckenabschnitts gewählt wird. Wird keine Zielgeschwindigkeit (TargetSpeed) angegeben, dann wählt ein Verkehrsteilnehmer seine Geschwindigkeit als Produkt aus der Entwurfsgeschwindigkeit und dem SpeedFactor. [general] Verwendbar in Verhaltensmodell: EXSTDGlobalS
TargetSpeed [27.7]	Zielgeschwindigkeit im m/s. Wird eine Zielgeschwindigkeit festgelegt, ist automatisch die Verwendung der Entwurfsgeschwindigkeit unter Berücksichtigung des SpeedFactors deaktiviert. [general] Verwendbar in Verhaltensmodell: EXSTD1, EXSTD2, EXSTD3, EXSTDEMG, EXSTDGlobalS, FreeDriving, HazardFree, EXIDM, EXWDM
StartSpeed [27.7]	Startgeschwindigkeit in m/s [general] Verwendbar in Verhaltensmodell: EXSTD1, EXSTD2, EXSTD3, EXSTDEMG, EXSTDGlobalS, FreeDriving, HazardFree, EXIDM, EXWDM
TargetSpeedMul [1.0]	Faktor für Geschwindigkeit auf der rechten Fahrspur. Dieser Parameter kann z.B. verwendet werden, um auf der rechten Spur einer Autobahn eine reduzierte Zielgeschwindigkeit einzustellen. [general] Verwendbar in Verhaltensmodell: EXIDM, EXSTDGlobalS,

5.3.3.2 Parameter zur Abstandsregelung

THWF [2.0]	Abstand zum Vorfahrzeug in s [general] Verwendbar in Verhaltensmodell: EXSTD1, EXSTD2, EXSTD3, EXSTDEMG, EXSTDGlobalS, HazardAvoidance, HazardAvoidance2, HazardFree, EXIDM, EXWDM
MinDistF [1.5]	Mindestabstand zum Vorderfahrzeug in m [general] Verwendbar in Verhaltensmodell: EXSTD1, EXSTD2, EXSTD3, EXSTDEMG, EXSTDGlobalS, HazardAvoidance, HazardAvoidance2, HazardFree, EXIDM, EXWDM

Parameter zum lateralen Abstandsverhalten

LaneChangeDistance [60.0]	Dauer des Spurwechsels in m [general] Verwendbar in Verhaltensmodell: EXWDM, EXIDM, EXSTDGlobalS,
----------------------------------	---

LaneChangeDeadTime [10.0] Wartezeit zwischen zwei Spurwechseln in s
[general]
Verwendbar in Verhaltensmodell:
EXIDM, EXSTDGlobalS,

5.3.3.3 Parameter zur streckenabhängigen Geschwindigkeitsanpassung

AYCurvature [22.0] Akzeptierte Querbewegung in Abhängigkeit der
Straßenkrümmung in m/s^2 [general]
Verwendbar in Verhaltensmodell:
EXSTDGlobalS

5.3.3.4 Faktoren für die maximale Geschwindigkeit (siehe 5.1.2) in Abhängigkeit von Umweltparameter

EnvironmentFogMul [-1.0] der Nebelstärke [general]
EnvironmentPrecipitationMul [-1.0] der Regendichte [general]
EnvironmentHumidityMul [-1.0] der Feuchtigkeit [general]
EnvironmentMuMul [-1.0] des Reibbeiwertes μ [general]
Verwendbar in Verhaltensmodell: EXSTD1, EXSTD3, EXSTDGlobalS.

Bei der Angabe des Faktors -1.0 (Defaultwert) wird der entsprechende
Umwelteinfluss nicht berücksichtigt.

5.3.3.5 Parameter zur Gassenbildung

Damit ein Verkehrsteilnehmer eine Gassenbildung durchführt, muss dieser sich im
Einflussbereich eines Einsatzfahrzeuges befinden. Die Größe des Einflussbereichs
ist abhängig vom Signalstand des Einsatzfahrzeuges und den Parametern Privile-
geSize1-3 der DPUTRFX:

Signalstand	Radius des Einflussbereichs	Defaultwert
Aus	0m	0 m
Nur Blaulicht	DPUTRFX.PrivilegeSize1	25 m
Martinshorn 1 + Blaulicht	DPUTRFX.PrivilegeSize2	40 m
Martinshorn 2 + Blaulicht	DPUTRFX.PrivilegeSize3	60 m

ClearTrafficTime [2.0 (0.0 ... 3.0)] Reaktionszeit in s für die Gassenbildung.
Verwendbar in Verhaltensmodell:
EXSTDGlobalS

ClearTrafficTimeMul [1.0] Faktor für die Reaktionszeit der
Verkehrsteilnehmer
Verwendbar in Verhaltensmodell:
EXSTDGlobalS

ClearTrafficResumeTime [10.0] Wartezeit in s für das Schließen der Gasse
nachdem sich der Verkehrsteilnehmer

	außerhalb des Einflussbereiches des Einsatzfahrzeuges befindet.
	Verwendbar in Verhaltensmodell: EXSTDGlobalS
ClearTrafficResumeMul [1.0]	Faktor für die Wartezeit ClearTrafficResumeTime Verwendbar in Verhaltensmodell: EXSTDGlobalS
ClearTrafficBias [1.0]	Wahrscheinlichkeit für die Ablehnung ¹ der Reaktionszeitverlängerung durch den Parameter ClearTrafficBiasOffset. Verwendbar in Verhaltensmodell: EXSTDGlobalS
ClearTrafficBiasMul [1.0]	Faktor für ClearTrafficBias. Verwendbar in Verhaltensmodell: EXSTDGlobalS
ClearTrafficBiasOffset [5.0]	Reaktionszeitverlängerung in s, falls eine Verlängerung mit ClearTrafficBias eintritt. Verwendbar in Verhaltensmodell: EXSTDGlobalS
ClearTrafficDx [2.0 (2.0 ... 2.1)]	Distanz in m, welche der Verkehrsteilnehmer nach vorne fährt, um die Gassenbildung zu ermöglichen. Verwendbar in Verhaltensmodell: EXIDM, EXSTDGlobalS
ClearTrafficDxMul [1.0]	Faktor für ClearTrafficDx. Verwendbar in Verhaltensmodell: EXIDM, EXSTDGlobalS
ClearTrafficVx [0.8 (0.8 ... 1.2)]	Geschwindigkeit, mit der der Verkehrsteilnehmer die Distanz ClearTrafficDx zurücklegt. Verwendbar in Verhaltensmodell: EXIDM, EXSTDGlobalS
ClearTrafficDy [2.1 (2.0 ... 2.4)]	Lateraler Versatz ² der Gassenbildung in m Verwendbar in Verhaltensmodell: EXIDM, EXSTDGlobalS
ClearTrafficDyMul [1.0]	Faktor1 für ClearTrafficDy Verwendbar in Verhaltensmodell: EXIDM, EXSTDGlobalS
ClearTrafficDyMul2 [1.0]	Faktor2 für ClearTrafficDy Verwendbar in Verhaltensmodell: EXIDM, EXSTDGlobalS

¹ Das Verhaltensmodell zieht eine Zufallszahl zwischen 0.0 und 1.0. Ist diese Zufallszahl größer als der Parameter ClearTrafficBias • ClearTrafficBiasMul, dann wird der Parameter ClearTrafficBiasOffset auf die Reaktionszeit aufaddiert.

² Positive Werte führen nach rechts und negative Werte nach links. Der tatsächliche Versatz ergibt sich aus ClearTrafficDy • ClearTrafficDyMul • ClearTrafficDyMul2. Damit kann z.B. die Gassenbreite einerseits durch die Fahrzeugbreite und andererseits durch eine globale Option beeinflusst werden.

ClearTrafficVy [1.6 (1.0 ... 1.6)] Quergeschwindigkeit in m/s, mit der der laterale Versatz gefahren wird.
Verwendbar in Verhaltensmodell:
EXIDM, EXSTDGlobals

5.3.3.6 Verhaltensmodellunabhängiger Parameter zur Verkehrsdichtenregelung

ActivateOnDensity [0.0] Aktivierung, wenn die globale Verkehrsdichte (Eingang TrafficDensity der DPU TRFX) größer ist als dieser Parameter. Sinnvollerweise wird dieser Parameter als Intervall angegeben um eine zufällige Verkehrsdichte zu erreichen. Der Defaultwert ist jedoch 0.0, womit alle Verkehrsteilnehmer aktiviert werden.

Dieser Parameter wird von jedem Verkehrsteilnehmer (unabhängig von dessen Verhaltensmodell) ausgewertet. Damit können Verkehrsteilnehmer definiert werden, die nur bei hohen Verkehrsdichten aktiviert werden.

5.3.4 Setzen von Defaultparametern für Verhaltensmodelle

Im Beispiel aus Abschnitt 5.3.2 werden die Parameter der einzelnen Verhaltensmodelle in der **Behaviour**-Zuweisung gesetzt, z.B. *<MinDistF>* und *<THWF>* des **HazardAvoidance2**-Modells als erster und zweiter Parameter in der mit Kommata getrennten Liste. Dabei ist die Reihenfolge entscheidend. In Abschnitt 5.3.3 wurde gezeigt wie man allgemeine Parameter zuweisen kann.

Eine weitere Möglichkeit, die Parameter eines Verhaltensmodells zu beeinflussen, gibt es in dem Abschnitt der generellen Verkehrsdefinitionen (siehe Grammatik 5).

Grammatik 25 Default parameters for behaviour models

```

< default parameters for behaviour models> →
    Behaviour <name>
    {
        (0...*)<parameter(→Kapitel 5.3.3 Seite 129)> = <value|range>;
    };
<name> →
    Default | Default<1...10> | <name of a behaviour model>
<value> →
    floating-point number for example 1.3
<range> →
    <value> ... <value>

```

In der obigen Grammatik werden Mengen beschrieben, welche Zuweisungen von Startwerten für die in den Abschnitten 5.3.3, 5.3.5 und 5.3.5 beschriebenen Parametern enthalten.

An dieser Stelle existieren drei Möglichkeiten für die Vorgabe der Startwerte:

1. Wird die Menge mit **Behaviour Default** eingeleitet, dann gelten die Startwerte für alle verwendeten Verhaltensmodelle.
2. Wird *<name>* bei der Definition einer Menge durch den Namen eines Verhaltensmodells ersetzt, dann gelten die in der Menge aufgeführten Startwerte für jede Verwendung dieses Modells.
3. Wird für *<name>* bei der Definition einer Menge eines der Token **Default1**, **Default2** ... **Default10** verwendet, dann können die Startwerte mit dem Parameter **MergeDefault = 1 ... 10** (siehe Grammatik 7) aus einer dieser zehn möglichen Mengen übernommen werden.

Folgende Reihenfolge gilt bei der Abarbeitung der Zuweisung eines Parameters:

1. Parameter aus **Behaviour Default**
2. Parameter aus **Behaviour <name>**
3. Parameter aus der Parameterliste (Tupel) des Verhaltensmodells (siehe Grammatik 9)
4. Parameter aus Verhaltensschema (siehe Grammatik 9)
5. Parameter aus der Verkehrsteilnehmerdefinition (siehe Grammatik 7)
6. Parameter aus der Menge **Behaviour Default1 - Behaviour Default10** welche mit dem Parameter **MergeDefault = <number>** zugewiesen wurden

Mit dem folgenden Beispiel wird die Verwendung der Default Parameter für Verhaltensmodelle verdeutlicht:

```
SILAB TRF
{
  Behaviour Default
  {
    TargetSpeed = 27.6;
    THWF = 2.0;
    MinDistF = 1.0;
  };

  Behaviour EXSTD2
  {
    TargetSpeed = 13.6;
    MinDistF = 2.0;
  };

  Behaviour Default1
  {
    TargetSpeed = 8.33;
  };

  Behaviour Default2
  {
    TargetSpeed = 11.11;
    THWF = 1.0;
  };
  ...
};
```

In diesem Beispiel wurde eine Menge mit **Behaviour Default** definiert, welche für alle Verkehrsteilnehmer den Startwert der Zielgeschwindigkeit auf 100 km/h, den Ab-

stand zum Vorausfahrenden auf 2 Sekunden und den Mindestabstand zum Vorausfahrenden auf 1.0 m setzt. Die Menge **Behaviour EXSTD2** setzt die Zielgeschwindigkeit für alle Verkehrsteilnehmer, die das Verhaltensmodell EXSTD2 verwenden. Dieses Modell wird beispielsweise in dem Verhaltensschema Emergency verwendet (siehe Abschnitt 5.1.1). Die Mengen **Behaviour Default1** und **Behaviour Default2** setzen die Zielgeschwindigkeit auf 30 bzw. 40 km/h.

```
SILAB SCN
{
  ...
  define Module BehaviourDemo
  {
    ...
    Traffic Demo
    {
      VehicleX Vehicle1
      {
        ...
        BehaviourScheme = Standard;
      };
      VehicleX Vehicle2
      {
        ...
        BehaviourScheme = Standard;
        TargetSpeed = 22.0;
      };
      VehicleX Vehicle3
      {
        ...
        BehaviourScheme = Standard;
        MergeDefault = 1;
      };
      VehicleX Vehicle4
      {
        ...
        BehaviourScheme = Emergency;
        MergeDefault = 2;
      };
    }
  }
}
```

Nach den oben aufgeführten Auswertungsregeln gelten nun für die oben definierten Verkehrsteilnehmer die folgenden Startwerte:

Vehicle1 übernimmt die Werte aus der Menge Default:

TargetSpeed = 13.6, THWF = 2.0, MinDistF = 1.0

Vehicle2 übernimmt die Werte aus der Menge Default und überschreibt die Zielgeschwindigkeit:

TargetSpeed = 22.0, THWF = 2.0, MinDistF = 1.0

Vehicle3 übernimmt die Werte aus der Menge Default.

Mit dem Parameter **MergeDefault = 1** werden dann noch die Werte aus der Menge **Default1** übernommen:

TargetSpeed = 8.33, THWF = 2.0, MinDistF = 1.0

Vehicle4 übernimmt die Werte aus der Menge Default. Da hier das Verhaltensschema Emergency verwendet wird, werden auch die Werte aus der Menge EXSTD2

übernommen. Mit dem Parameter **MergeDefault = 2** werden dann noch die Werte aus der Menge **Default2** übernommen:

TargetSpeed = 11.11, THWF = 1.0, MinDistF = 2.0

5.3.5 Verhalten der Verkehrsteilnehmer

Die Verkehrssimulation von SILAB bietet eine große Menge von Verhaltensmodellen an, mit denen simulierte Verkehrsteilnehmer gesteuert und beeinflusst werden können. Die nachfolgenden Modelle sind sowohl für Courses- als auch für Area2-Abschnitte vorgesehen. In diesem Abschnitt wird eine andere Beschreibung der Flowpoints verwendet. Die Beschreibung entspricht der Darstellung des SILABAEEdit, die Argumente des Flowpoints stehen der Reihe nach untereinander. In Klammern ist immer der Default-Wert angegeben, der im SILABAEEdit voreingestellt ist für den jeweiligen Parameter.

5.3.5.1 Verhaltensmodell: EXSTD1

Das Verhaltensmodell EXSTD1 stellt eine Kombination des HazardAvoidance- und des FreeDriving-Modells dar. Dieses Modell ist für die Area2-Kreuzungsbereiche optimiert.

Tupel-Parameter

- | | |
|------------------|---|
| 1. <StartSpeed> | Geschwindigkeit bei der Aktivierung des Verkehrsteilnehmers (in m/s) [general]. |
| 2. <TargetSpeed> | Zielgeschwindigkeit während der Fahrt (in m/s) [general] |
| 3. <MinDistF> | Abstand, der nicht unterschritten werden darf (in m) [general] |
| 4. <THWF> | Sekundenabstand zum vorausfahrenden Fahrzeug [general] |

Flowpoints

Stop

Stoppt sofort das Fahrzeug. Das Fahrzeug kann dann mit einem der ResumeXXX Befehle wieder losfahren.

StopAt

<i>Distanz (X:m)</i>	<i>[10.0]</i>	<i>Anhalteweg in X-Richtung</i>
<i>Distanz (Y:m)</i>	<i>[0.0]</i>	<i>seitlicher Versatz am Haltepunkt</i>

Hält das Fahrzeug erst in X-Metern an. Dabei nimmt das Fahrzeug noch einen lateralen Weg entsprechend dem Y-Parameter an.

StopAtIfSimCarSpeedGreater

<i>Distanz (X:m)</i>	<i>[10.0]</i>	<i>Anhalteweg in X-Richtung</i>
<i>Distanz (Y:m)</i>	<i>[0.0]</i>	<i>seitlicher Versatz am Haltepunkt</i>
<i>v (m/s)</i>	<i>[13.89]</i>	<i>Geschwindigkeit des EGO-Fahrzeugs</i>

Hält das Fahrzeug in X-Metern mit dem Y-Versatz an, falls zum Zeitpunkt des Auslösens die Geschwindigkeit des EGO-Fahrzeugs größer ist als der angegebene Parameter.

StopAtIfSimCarSpeedLess

Distanz (X:m) [10.0] *Anhalteweg in X-Richtung*
Distanz (Y:m) [0.0] *seitlicher Versatz am Haltepunkt*
v (m/s) [13.89] *Geschwindigkeit des EGO-Fahrzeugs*
 Hält das Fahrzeug in X-Metern mit dem Y-Versatz an, falls zum Zeitpunkt des Auslösens die Geschwindigkeit des EGO-Fahrzeugs kleiner ist als der angegebene Parameter.

Resume

Das Fahrzeug fährt wieder los, wenn es mit einem der StopXXX Befehle angehalten wurde.

ResumeIfSimcarSpeedLess

v (m/s) [13.89] *Geschwindigkeit des EGO-Fahrzeugs*
 Das Fahrzeug fährt wieder los, wenn die Geschwindigkeit des EGO-Fahrzeugs unter der angegebenen Geschwindigkeit liegt.

ResumeIfUserChannel

Kanal [1] *Kanal der DPUTRFX*
Aktionsradius [20] *Aktionsradius in Metern*
 Das Fahrzeug fährt wieder los, wenn der Eingang der DPUTRFX mit der Kanalnummer *Kanal* größer 0.5 ist und sich das EGO-Fahrzeug noch im Aktionsradius des Flowpoints befindet. Befindet es sich außerhalb des angegebenen Radius um den Flowpoint und die Bedingung trifft nicht ein, dann wird der Verkehrsteilnehmer kein Resume ausführen.

SetVelocity

v (m/s) [13.89] *Geschwindigkeit des EGO-Fahrzeugs*
 Setzt die Geschwindigkeit des Fahrzeugs neu.

Synchronize

Differenzgeschwindigkeit (m/s) [0] *Geschwindigkeitsdifferenz*
 Synchronisiert die Geschwindigkeit des Fahrzeugs auf die Geschwindigkeit des EGO-Fahrzeuges. Es addiert dabei die Geschwindigkeitsdifferenz auf die Geschwindigkeit des EGO-Fahrzeuges und nimmt die Summe als eigene Geschwindigkeit

SynchronizeRel

Differenzgeschwindigkeit (%) [0] *Geschwindigkeitsdifferenz*
 Synchronisiert die Geschwindigkeit des Fahrzeugs relativ auf die Geschwindigkeit des EGO-Fahrzeuges.

Resetup

Setzt das Fahrzeug erneut auf den Startpunkt auf.

Activate

Aktiviert das Fahrzeug.

Deactivate

Deaktiviert das Fahrzeug.

Light

Zustand (0/1) [0] Lichtzustand
Schaltet das Licht ein oder aus.

BrakeLight

Zustand (0/1)	[0]	Lichtzustand
Schaltet das Bremslicht an oder aus. Dazu muss jedoch zuerst die Kontrolle des Bremslichtes dem Fahrzeug mit <i>AutoBrakeLightControl</i> entzogen werden.		

AutoBrakeLightControl

Zustand (0/1) [0] 1 = Fahrzeug übernimmt die Kontrolle
 0 = externe Kontrolle

Regelt die Kontrolle über das Bremslicht.

AutoIndicatorLightControl

Zustand (0/1) [0] 1 = Fahrzeug übernimmt die Kontrolle
 0 = externe Kontrolle

Regelt die Kontrolle über den Blinker.

IndicatorLeft

Zustand (0/1) **[0]** **Lichtzustand**
Schaltet das linke Blinklicht an oder aus. Dazu muss jedoch zuerst die Kontrolle des Blinkers dem Fahrzeug mit *AutoIndicatorLightControl* entzogen werden.

IndicatorRight

Zustand (0/1) **[0]** **Lichtzustand**
Schaltet das rechte Blinklicht an oder aus. Dazu muss jedoch zuerst die Kontrolle des Blinkers dem Fahrzeug mit *AutoIndicatorLightControl* entzogen werden.

Right

Wahrscheinlichkeit (%) [0]
Das Fahrzeug biegt mit der angegebenen Wahrscheinlichkeit bei der nächsten Gelegenheit rechts ab, indem es das Heading Right.

Left

Wahrscheinlichkeit (%) [0]
Das Fahrzeug biegt mit der angegeben Wahrscheinlichkeit bei der nächsten Gelegenheit links ab, indem es das Heading Left wählt.

Straight

Wahrscheinlichkeit (%) [0]
Das Fahrzeug wählt bei der nächsten Gelegenheit wieder die Geradeaus-Spur.

TargetSpeed

v (m/s)	[13.89]	Geschwindigkeit des Fahrzeugs
Setzt die Zielgeschwindigkeit des Fahrzeuges neu.		

TargetSpeedAndAcceleration

v (m/s)	[13.89]	Geschwindigkeit des Fahrzeugs
a (m/s ²)	[5.0]	Beschleunigung des Fahrzeugs
a (m/s ²)	[2.0]	Verzögerung des Fahrzeugs

Setzt die Zielgeschwindigkeit des Fahrzeuges neu. Die Zielgeschwindigkeit wird mit der angegebenen Beschleunigung erreicht, wenn die aktuelle Geschwindigkeit geringer ist. Ist die aktuelle Geschwindigkeit höher als die Zielgeschwindigkeit, dann wird mit der angegebenen Verzögerung die Zielgeschwindigkeit erreicht.

NewSetup

<i>max. Distanz SetupPoint <-> EGO (m)</i>	[30.00]
<i>max. Distanz FlowPoint <-> EGO (m)</i>	[20.00]

Setzt das Fahrzeug auf den zusätzlich gewählten Aufsetzpunkt wieder auf, wenn die angegebenen Distanzen zum EGO-Fahrzeug nicht unterschritten wurden. Mit der Distanz SetupPoint <-> EGO soll verhindert werden, dass das Fahrzeug plötzlich vor dem EGO-Fahrzeug erscheint. Die Distanz FlowPoint <-> EGO soll verhindern, dass das Fahrzeug plötzlich verschwindet, falls das EGO-Fahrzeug zu nah am Flowpoint ist. In diesem Fall wird der Flowpoint ignoriert.

Weitere für dieses Modell gültige allgemeine Parameter (siehe Abschnitt 5.3.3):

Umweltparameter.

5.3.5.2 Verhaltensmodell: EXSTD2

Das Verhaltensmodell EXSTD2 stellt eine Erweiterung des Modells EXSTD1 dar. Es wurde entwickelt für Verkehrsteilnehmer, die in Szenarien für Fahrten mit Sonder- und Wegerechte nicht unmittelbar für den EGO – Fahrer in einem Szenario von Bedeutung sind. Neben dem Standardverhalten von EXSTD1 reagiert es auf einen weiteren Flowpoint.

Parameter im Tupel

- | | |
|------------------|---|
| 1. <StartSpeed> | Geschwindigkeit bei der Aktivierung des Fahrzeugs (in m/s) [general]. |
| 2. <TargetSpeed> | Zielgeschwindigkeit während der Fahrt (in m/s) [general] |
| 3. <MinDistF> | Abstand, der nicht unterschritten werden darf (in m) [general] |
| 4. <THWF> | Sekundenabstand zum vorausfahrenden Fahrzeug [general] |

Flowpoints**StopOnEmergency**

<i>Distanz (X:m)</i>	[10.0]	<i>Anhalteweg in X-Richtung</i>
<i>Distanz (Y:m)</i>	[0.0]	<i>seitlicher Versatz am Haltepunkt</i>

Hält das Fahrzeug in X-Metern mit dem Y-Versatz an, falls zum Zeitpunkt des Auslösens der Verkehrsteilnehmer sich im Bereich eines Fahrzeuges

mit Sonder- und Wegerechten befindet. Der Verkehrsteilnehmer kann nach dem Anhalten wieder mit Resume zur Weiterfahrt gebracht werden.

5.3.5.3 Verhaltensmodell: EXSTD3

Das Verhaltensmodell EXSTD3 stellt eine Erweiterung des Modells EXSTD1 dar. Es beinhaltet ein HazardAvoidance- und ein FreeDriving-Modell auf Basis des IDM-Modells (siehe auch Verhaltensmodells EXIDM in Abschnitt 5.3.6.8). Im Gegensatz zu dem Modell EXIDM beinhaltet das Modell EXSTD nicht das Spurwechselmodell MOBIL.

Das Modell ist für die Area2-Kreuzungsbereiche optimiert und beinhaltet identische Parameter wie das Modell EXSTD1, damit dieses leicht ersetzt werden kann. Zusätzliche Parameter ermöglichen weitere Eingriffsmöglichkeiten in das Modell. Diese sind optional und müssen nicht gesetzt werden.

Parameter im Tupel

1. *<StartSpeed>* Geschwindigkeit bei der Aktivierung des Fahrzeugs (in m/s) [general].
2. *<TargetSpeed>* Zielgeschwindigkeit während der Fahrt (in m/s) [general].
3. *<MinDistF>* Abstand, der nicht unterschritten werden darf (in m) [general].
4. *<THWF>* Sekundenabstand zum vorausfahrenden Fahrzeug (in s) [general].
5. *<LaneChangeDistance>* Gefahrene Strecke während der ein Spurwechsel durchgeführt wird (in m). (Default = 50m)
6. *<LaneChangeRightOnEmergency>* Soll das Fahrzeug bei einem Einsatzfahrzeug nach rechts ausweichen können? (0 = nein / 1 = ja). (Defaultwert = 1)
7. *<LaneChangeLeftOnEmergency>* Soll das Fahrzeug bei einem Einsatzfahrzeug nach links ausweichen können? (0 = nein / 1 = ja). (Defaultwert = 1)
8. *<MaxAcceleration>* Maximale Beschleunigung des Fahrzeugs in m/s^2 . (Defaultwert = 20m/s^2)
9. *<MaxDeceleration>* Maximale Verzögerung des Fahrzeugs in m/s^2 . Ist dieser Wert zu groß, dann kann der gewünschte Sicherheitsabstand unter Umständen nicht eingehalten werden. (Defaultwert = -50m/s^2)
10. *<type of acceleration>* Die Art wie das Fahrzeug auf die Wunschgeschwindigkeit beschleunigt. Höhere Werte führen zu einer stärkeren Beschleunigung. (Defaultwert = 1.5).
11. *<type of deceleration>* Die Art wie das Fahrzeug auf ein Hindernis Bremsst. Höhere Werte führen zu einem früheren Abbremsen mit geringerer Maximalverzögerung. (Defaultwert = 4)

12. <Delta>

Beschleunigungsexponent des IDM Modells (4.0).
Siehe auch Beschreibung des Modells EXIDM.

Flowpoints**Stop**

Stoppt sofort das Fahrzeug. Das Fahrzeug kann dann mit einem der ResumeXXX Befehle wieder losfahren.

StopAt

Distanz (X:m) [10.0] *Anhalteweg in X-Richtung*
Distanz (Y:m) [0.0] *seitlicher Versatz am Haltepunkt*
 Hält das Fahrzeug erst in X-Metern an. Dabei nimmt das Fahrzeug noch einen lateralen Weg entsprechend dem Y-Parameter an.

StopAtIfSimCarSpeedGreater

Distanz (X:m) [10.0] *Anhalteweg in X-Richtung*
Distanz (Y:m) [0.0] *seitlicher Versatz am Haltepunkt*
v (m/s) [13.89] *Geschwindigkeit des EGO-Fahrzeugs*
 Hält das Fahrzeug in X-Metern mit dem Y-Versatz an, falls zum Zeitpunkt des Auslösens die Geschwindigkeit des EGO-Fahrzeugs größer ist als der angegebene Parameter.

StopAtIfSimCarSpeedLess

Distanz (X:m) [10.0] *Anhalteweg in X-Richtung*
Distanz (Y:m) [0.0] *seitlicher Versatz am Haltepunkt*
v (m/s) [13.89] *Geschwindigkeit des EGO-Fahrzeugs*
 Hält das Fahrzeug in X-Metern mit dem Y-Versatz an, falls zum Zeitpunkt des Auslösens die Geschwindigkeit des EGO-Fahrzeugs kleiner ist als der angegebene Parameter.

Resume

Das Fahrzeug fährt wieder los, wenn es mit einem der StopXXX Befehle angehalten wurde.

ResumeIfSimcarSpeedLess

v (m/s) [13.89] *Geschwindigkeit des EGO-Fahrzeugs*
 Das Fahrzeug fährt wieder los, wenn die Geschwindigkeit des EGO-Fahrzeugs unter der angegebenen Geschwindigkeit liegt.

ResumeIfUserChannel

Kanal [1] *Kanal der DPUTRFX*
Aktionsradius [20] *Aktionsradius in Metern*
 Das Fahrzeug fährt wieder los, wenn der Eingang der DPUTRFX mit der Kanalnummer *Kanal* größer 0.5 ist und sich das EGO-Fahrzeug noch im Aktionsradius des Flowpoints befindet. Befindet es sich außerhalb des angegebenen Radius um den Flowpoint und die Bedingung trifft nicht ein, dann wird der Verkehrsteilnehmer kein Resume ausführen.

SetVelocity

v (m/s) [13.89] *Geschwindigkeit des EGO-Fahrzeugs*
Setzt die Geschwindigkeit des Fahrzeugs neu.

Synchronize

Differenzgeschwindigkeit (m/s) [0] *Geschwindigkeitsdifferenz*
Synchronisiert die Geschwindigkeit des Fahrzeugs auf die Geschwindigkeit des EGO-Fahrzeugs. Es addiert dabei die Geschwindigkeitsdifferenz auf die Geschwindigkeit des EGO-Fahrzeugs und nimmt die Summe als eigene Geschwindigkeit

SynchronizeRel

Differenzgeschwindigkeit (%) [0] *Geschwindigkeitsdifferenz*
Synchronisiert die Geschwindigkeit des Fahrzeugs relativ auf die Geschwindigkeit des EGO-Fahrzeugs.

Resetup

Setzt das Fahrzeug erneut auf den Startpunkt auf.

Activate

Aktiviert das Fahrzeug.

Deactivate

Deaktiviert das Fahrzeug.

Light

Zustand (0/1) [0] *Lichtzustand*
Schaltet das Licht ein oder aus.

BrakeLight

Zustand (0/1) [0] *Lichtzustand*
Schaltet das Bremslicht an oder aus. Dazu muss jedoch zuerst die Kontrolle des Bremslichtes dem Fahrzeug mit *AutoBrakeLightControl* entzogen werden.

AutoBrakeLightControl

Zustand (0/1) [0] *1 = Fahrzeug übernimmt die Kontrolle*
0 = externe Kontrolle
Regelt die Kontrolle über das Bremslicht.

AutoIndicatorLightControl

Zustand (0/1) [0] *1 = Fahrzeug übernimmt die Kontrolle*
0 = externe Kontrolle
Regelt die Kontrolle über den Blinker.

IndicatorLeft

Zustand (0/1) [0] *Lichtzustand*
Schaltet das linke Blinklicht an oder aus. Dazu muss jedoch zuerst die Kontrolle des Blinkers dem Fahrzeug mit *AutoIndicatorLightControl* entzogen werden.

IndicatorRight

Zustand (0/1) [0] *Lichtzustand*

Schaltet das rechte Blinklicht an oder aus. Dazu muss jedoch zuerst die Kontrolle des Blinkers dem Fahrzeug mit *AutoIndicatorLightControl* entzogen werden.

Right

Wahrscheinlichkeit (%) [0]

Das Fahrzeug biegt mit der angegebenen Wahrscheinlichkeit bei der nächsten Gelegenheit rechts ab, indem es das Heading Right wählt.

Left

Wahrscheinlichkeit (%) [0]

Das Fahrzeug biegt mit der angegebenen Wahrscheinlichkeit bei der nächsten Gelegenheit links ab, indem es das Heading Left wählt.

Straight

Wahrscheinlichkeit (%) [0]

Das Fahrzeug wählt bei der nächsten Gelegenheit wieder die Geradeaus-Spur.

TargetSpeed

v (m/s) [13.89] Geschwindigkeit des Fahrzeugs

Setzt die Zielgeschwindigkeit des Fahrzeuges neu.

NewSetup

max. Distanz SetupPoint <-> EGO (m) [30.00]

max. Distanz FlowPoint <-> EGO (m) [20.00]

Setzt das Fahrzeug auf den zusätzlich gewählten Aufsetzpunkt wieder auf, wenn die angegebenen Distanzen zum EGO-Fahrzeug nicht unterschritten wurden. Mit der Distanz SetupPoint <-> EGO soll verhindert werden, dass das Fahrzeug plötzlich vor dem EGO-Fahrzeug erscheint. Die Distanz FlowPoint <-> EGO soll verhindern, dass das Fahrzeug plötzlich verschwindet, falls das EGO-Fahrzeug zu nah am Flowpoint ist. In diesem Fall wird der Flowpoint ignoriert.

Weitere für dieses Modell gültige allgemeine Parameter (siehe Abschnitt 5.3.3):

Umweltparameter.

5.3.5.4 Verhaltensmodell: EXSTDEMG

Das Verhaltensmodell EXSTDEMG stellt eine Erweiterung des Modells EXSTD2 dar. Es wurde entwickelt für die Gassenbildung für Einsatzfahrten. Neben dem Standardverhalten von EXSTD2 reagiert es auf weitere Flowpoints.

Parameter im Tupel

1. <StartSpeed> Geschwindigkeit bei der Aktivierung des Verkehrsteilnehmers [general].
2. <TargetSpeed> Zielgeschwindigkeit [general]
3. <MinDistF> Abstand, der nicht unterschritten werden darf (in m) [general]
4. <THWF> Sekundenabstand zum vorausfahrenden Fahrzeug [general]

Flowpoints

ClearTraffic

<i>Reaktionszeit (s)</i>	<i>[10.0]</i>	Verzögerung der Gassenbildung
<i>Seitenabstand (m)</i>	<i>[1.7]</i>	seitlicher Versatz am Haltepunkt
<i>Seitengeschw. (m/s)</i>	<i>[1.7]</i>	Quergeschwindigkeit
<i>Vorwärtsfahrt (m)</i>	<i>[2.0]</i>	Strecke für die Gassenbildung
<i>Geschwindigkeit (m/s)</i>	<i>[1.0]</i>	Geschwindigkeit für die obige Strecke
<i>Standzeit (s)</i>	<i>[30.0]</i>	Nach dieser Standzeit schließt sich die Gasse wieder

Befindet sich der Verkehrsteilnehmer im Einflussbereich eines Einsatzfahrzeuges, dann beginnt er mit der Gassenbildung.

ClearTrafficPause

Mit diesem Flowpoint kann die Gassenbildung pausiert werden.

5.3.5.5 Verhaltensmodell: EXSTDGlobalS

Das Verhaltensmodell **EXSTDGlobalS** basiert auf dem Intelligent Driver Model (siehe hierzu auch Abschnitt 5.3.6.8) und liefert somit kontinuierliche Beschleunigungen sowie Verzögerungen.

Bei dem Modell **EXSTDGlobalS** handelt es sich um ein sehr komplexes Verhaltensmodell, das selbständig Verkehrsregeln und Verkehrszeichen erkennt und auch beachtet. Daher ist es besonders geeignet für komplexe Kreuzungen bei denen viele Interaktionen mit anderen Verkehrsteilnehmern zu beachten sind.

Folgende Verkehrsregeln/-zeichen werden ausgewertet und gemäß Ihrer Bedeutung umgesetzt:

- **Rechts vor Links,**
- **Vorfahrtsschilder,**
- **Vorfahrt achten,**
- **Stoppschilder,**
- **Ampeln,**
- **sowie Geschwindigkeitsbegrenzungen.**

Eine Besonderheit stellt die Vorfahrtsregelung „Rechts vor Links“ dar. Damit diese korrekt umgesetzt werden kann, müssen alle beteiligten Verkehrsteilnehmer das Verhaltensmodell **EXSTDGlobalS** besitzen.

Bei der Erstellung der Szenarien ist zu beachten, dass nicht nur die Schilder sondern auch die zugehörigen Verkehrsregeln eingefügt werden müssen (siehe hierzu Dokumentation SILABAEEdit – Einfügen von Schildern als Verkehrsregel).

Ebenso reagiert dieses Modell auf die Umweltparameter entsprechend den in 5.3.3.4 beschriebenen Parametern.

Das Modell wird im Behaviour Scheme **City** verwendet.

Parameter im Tupel

1. *<StartSpeed/TargetSpeed>*

Geschwindigkeit bei der Aktivierung des

- | | |
|----------------------------------|--|
| | Verkehrsteilnehmers, sowie Zielgeschwindigkeit (in m/s) [general]. |
| 2. <i><TargetSpeedMul></i> | Multiplikator für die rechte Fahrspur [general] |
| 3. <i><THWF></i> | Sekundenabstand zum vorausfahrenden Fahrzeug (in s) [general]. |
| 4. <i><Amax></i> | Maximale Beschleunigung die das Fahrzeug (in m/s ²) |
| 5. <i></i> | Komfortable Bremsverzögerung |
| 6. <i><MinDistF></i> | Abstand, der nicht unterschritten werden darf (in m) [general] |

Weitere für dieses Modell gültige allgemeine Parameter (siehe Abschnitt 5.3.3):

AYCurvature, SpeedFactor, Parameter zum lateralen Abstandsverhalten, Umweltparameter, Parameter zur Gassenbildung.

Flowpoints:

Global_Right, Global_Left, Global_Straight, Global_Parking

Diese Flowpoints dienen der Steuerung des Ausschmückungsverkehrs wie in Abschnitt 4.5 beschrieben.

5.3.5.6 Verhaltensmodell: LaneCrossing

Das Modell **LaneCrossing** dient dazu Objekte z.B. ein Wildschwein oder einen Ball quer über eine Straße zu bewegen.

Das Objekt wird an einer bestimmten Position auf der Straße aufgesetzt. An dieser Position quert das Objekt mit dem Modell **LaneCrossing** die Straße

Parameter im Tupel

- | | |
|----------------------------------|--|
| 1. <i><start distance></i> | Lateraler Abstand des Objekts zu Beginn (in m). Positive Werte geben Positionen rechts der Straße an, negative Werte Positionen links der Straße. (Defaultwert = 10) |
| 2. <i><end distance></i> | Lateraler Abstand bis zu dem das Objekt bewegt wird. (in m). Danach bleibt das Objekt an dieser Position stehen. (Defaultwert = -30) |
| 3. <i><velocity></i> | Geschwindigkeit mit der das Objekt bewegt wird (in m/s). Negative Werte führen eine Bewegung nach links, positive eine Bewegung nach rechts aus. Immer gesehen in der Entwurfsrichtung. (Defaultwert = 2); |

Die Geschwindigkeit bleibt während der Ausführung des Modells konstant und kann nicht verändert werden. Über den allgemeinen Parameter StartSpeed kann diese in der Definition des Objekts angepasst werden.

Meist wird links und rechts der Straße eine Verdeckung z.B. ein Busch platziert, damit ein Fahrer das Objekt überraschend hinter diesem hervorkommen sieht.

Das Objekt dreht sich nach dem Aufsetzen zunächst in die Bewegungsrichtung, daher wird auch aus diesem Grund empfohlen das Objekt verdeckt aufzusetzen.

Beispiel für ein querendes Wildschwein:

```
VehicleX WildBoar
{
    Vehicle = WildBoar;
    Position = (Start, 25, 1);
    Behaviour = {(LaneCrossing, -5.0, 5.0, -2)};
    StartSpeed = -10;
    Flowpoints =
    {
        (Start, 2, 1, SimCar, Activate)
    };
};
```

In dem Beispiel wird ein Wildschein definiert, dass auf dem Course Start nach 25m aufgesetzt wird. Bei Streckenmeter 2 wird dieses vom EGO-Fahrzeug aktiviert und bewegt sich dann quer über die Straße von links nach rechts. Mit dem Parameter StartSpeed kann Geschwindigkeit des Behaviours überschrieben werden.

5.3.6 Verhalten der Verkehrsteilnehmer speziell in Course-Instanzen

In diesem Abschnitt werden die Verhaltensmodelle beschrieben, die für die Verwendung in Streckenabschnitten des Typs Course optimiert sind. Auf allgemeine Parameter (siehe Kapitel 5.3.4), die von einem bestimmten Verhaltensmodell berücksichtigt werden, wird jeweils durch den Zusatz [general] bei dem jeweiligen Parameter hingewiesen.

5.3.6.1 Verhaltensmodell: HazardAvoidance2

HazardAvoidance2 ist eine Erweiterung des obsolten Modells HazardAvoidance und verhindert das Auffahren auf einen vorausfahrenden Verkehrsteilnehmer. Dazu wird das IDM-Modell verwendet, um auf Vorausfahrende zu reagieren (siehe auch Verhaltensmodells EXIDM in Abschnitt 5.3.6.7).

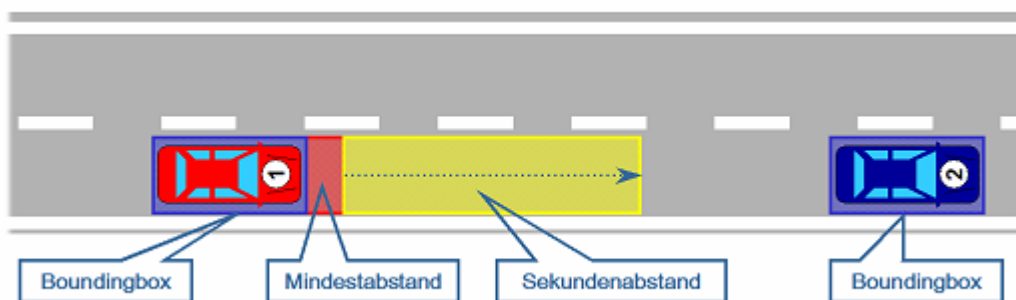


Abbildung 4 Vorausschau der Hazardavoidance2

Parameter im Tupel

1. <MinDistF> Abstand, der nicht unterschritten werden darf (in m) [general]
2. <THWF> Sekundenabstand zum vorausfahrenden Fahrzeug (in s) [general]
3. <maxacceleration>

- | | |
|-------------------------------------|---|
| | Maximale Beschleunigung des Fahrzeugs in m/s^2 . (Default = 20m/s^2) |
| 4. <i><maxdeceleration></i> | Maximale Verzögerung des Fahrzeugs in m/s^2 . Durch die Festlegung der maximalen Verzögerung kann es dazu kommen, dass der gewünschte Sicherheitsabstand unter Umständen nicht eingehalten werden kann. (Default = -50m/s^2) |
| 5. <i><acceleration type></i> | Die Art wie das Fahrzeug auf die Wunschgeschwindigkeit beschleunigt. Höhere Werte führen zu einer stärkeren Beschleunigung. (Default = 1.5). |
| 6. <i><deceleration type></i> | Die Art wie das Fahrzeug auf ein Hindernis bremst. Höhere Werte führen zu einem früheren Abbremsen mit geringerer Maximalverzögerung. (Default = 4) |
| 7. <i><Delta></i> | Beschleunigungsexponent des IDM Modells (4.0). Siehe auch Beschreibung des Modells EXIDM. |

Es ist zu beachten, dass aus Gründen der Vereinheitlichung der erste Parameter den Stillstandabstand spezifiziert und der zweite Parameter den zusätzlichen Sekundenabstand beim Fahren im Gegensatz zum obsoleten Modell HazardAvoidance.

Abstandsregelung:

Die Abstandsregelung erfolgt über eine Verzögerung, die sich aus Differenz zwischen der eigenen und der Geschwindigkeit des vorausfahrenden Fahrzeugs ergibt.

Flowpoints:

HazardAvoidance2, *<MinDistF>*, *<THWF>*, *<maxacceleration>*, *<maxdeceleration>*
Setzt die Parameter der HazardAvoidance2 neu.

5.3.6.2 Verhaltensmodell: LaneChangeAvoidance

Mit der LaneChangeAvoidance kann ein Spurwechsel ohne Kollision durchgeführt werden. Dazu wird der Spurwechselwunsch eines Verkehrsteilnehmers einige Zeit aufrechterhalten. In dieser Zeit wird durch einen seitlichen, trapezförmigen Scanbereich der Raum zwischen der jetzigen Fahrzeuglage und der Ziel Spur überwacht, d.h. der Scanbereich beginnt am Rande der Boundingbox des Fahrzeugs und endet an der Idealspur der Zielfahrspur. Der Spurwechselwunsch wird solange aufrechterhalten, bis entweder ein Spurwechsel ohne Kollision innerhalb des trapezförmigen Scanbereiches möglich ist oder die Zeit für einen Spurwechselwunsch abgelaufen ist.

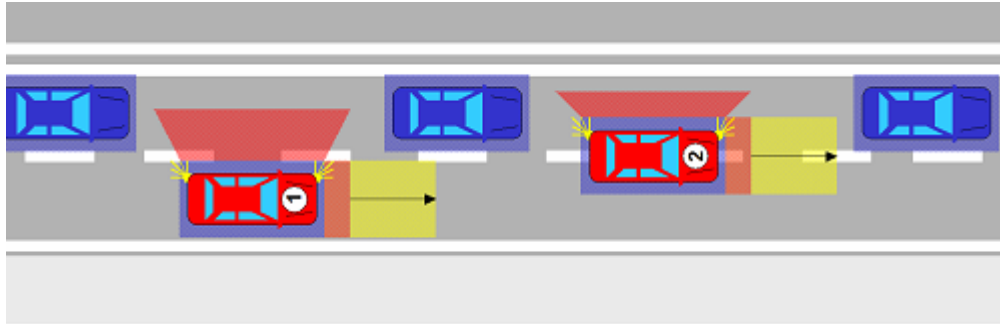


Abbildung 5 Seitlicher Scanbereich der LaneChangeAvoidance

Parameter im Tupel

- | | |
|--|---|
| 1. <code><front distance></code> | Scanabstand nach vorne (in m) |
| 2. <code><back distance></code> | Scanabstand nach hinten (in m) |
| 3. <code><time></code> | Zeitvorgabe für den Spurwechselwunsch (in s) |
| 4. <code><multiplier></code> | Multiplikator für die HazardAvoidance |
| 5. <code><blocking mode></code> | 1 = die nachfolgenden Verhaltensmodelle werden während dem Spurwechsel blockiert |
| 6. <code><speedfactor></code> | Multiplikator für die Geschwindigkeitswahl. Wird dieser Parameter angegeben, dann wird die aktuelle Zielgeschwindigkeit des Fahrzeuges auf die Geschwindigkeit des vorausfahrenden Fahrzeugs multipliziert mit diesem Faktor gesetzt um das vorausfahrende Fahrzeug zu überholen. Wird z.B. 1.25 angegeben, dann überholt der Verkehrsteilnehmer mit 100 km/h einen vorausfahrenden Verkehrsteilnehmer der 80 km/h fährt. |

Flowpoints

ChangeLaneRequest, `<target lane>`, `<length [m] or [s]>`, `<mode>`

Startet den Spurwechselwunsch auf die angegebene Ziel Spur.

Im Wegmodus (`<mode> = 0`) wird der Spurwechsel in der unter `<length [m]>` angegebenen Strecke abgeschlossen. Im Zeitmodus (`<mode> = 1`) ist der Spurwechsel in der unter `<length in s>` angegeben Zeitdauer abgeschlossen.

Der Spurwechsel wird nicht mehr gestartet, wenn die Zeit `<time>` abgelaufen ist.

ForgetLaneChangeRequest, `<time>`

Setzt den Spurwechselwunsch wieder zurück.

5.3.6.3 Verhaltensmodell: SignalSpeed

Das SignalSpeed-Modell generiert ein periodisches Geschwindigkeitssignal für das Fahrzeug mit einem Sinus- Dreieck- oder Rechteckgenerator.

Alternativ zu den Signalformen Sinus, Dreieck und Rechteck kann auch ein Dateneingang (*<signal>* = **Data**) der DPUTRF als Quelle für den Geschwindigkeitsverlauf und den Verlauf der lateralen Spurbabweichung verwendet werden.

Parameter im Tupel für die Signaltypen sine, triangle und square

1. *<signal>* sine | triangle | square
2. *<amplitude>* Amplitude des Signals in m/s.
3. *<period>* Periode in Millisekunden.
4. *<offset>* Offset der Periode in Millisekunden um eine Phasenverschiebung zu einem anderen, synchron aktiviertem Verhaltensmodell zu erreichen.
5. *<signaloffset>* Dieser Wert wird zu dem generierten Signal hinzuaddiert.
6. *<gradient>* Die Steigung des Geschwindigkeitssignals wird über diesen Parameter begrenzt (in m/s²)
7. *<repetitions>* Dieser Parameter legt fest, wie viele Wiederholungen eines Signals durchlaufen werden sollen, bevor das konstante Signaloffset dauerhaft anliegt. Der Default Wert ist 0 und bedeutet, dass das gewählte Signal endlos wiederholt wird. Gezählt wird jeder Nulldurchgang eines Signals.

Parameter im Tupel für den Signaltyp Data

1. *<signal>* Data
2. *<channel1>* Kanal für die Geschwindigkeit in m/s.
3. *<channel2 >* Kanal für die laterale Spurbabweichung. Wird dieser Kanal nicht mit angegeben, dann wird die laterale Spurbabweichung konstant auf 0 gehalten.

Flowpoints

SignalSpeedActive Dieser Flowpoint startet ein Signal der spezifizierten Form. Bei einem Data-Signal ist der Default-Wert 1.0 (d.h. *true*) und das Signal liegt sofort an. Bei den anderen Signalen ist der Defaultwert 0.0, d.h. der Signalgenerator muss erst über einen Flowpoint aktiviert werden.

SignalSpeedAmplitude Setzen der Amplitude eines Signals in m/s

SignalSpeedPeriod Setzen der Periode eines Signals in ms

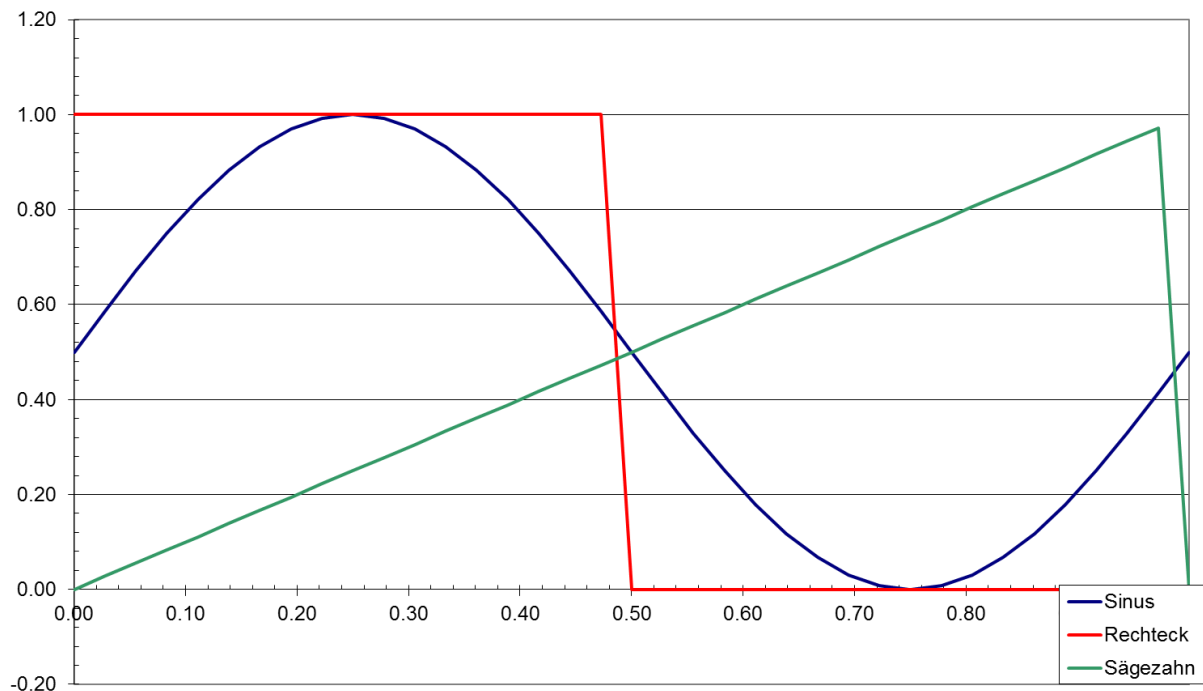
SignalSpeedPeriodOffs Setzen des Offsets einer Periode in m/s

SignalSpeedSpeedOffs Setzen des Geschwindigkeitsoffsets in m/s

SignalSpeedSlope Setzen der Steigung mit der das Signal den Offset annimmt in m/s²

SignalSpeedRepetitions Anzahl der Wiederholungen

Periodische Signalformen



Alle periodischen Signalformen werden mit der folgenden Formel generiert:

$$Signal_{out} = \left(Signal\left(\frac{t}{Period}\right) + Offset \cdot Amplitude \right) + Signaloffset$$

Beim Start des Verhaltensmodells wird die aktuelle Geschwindigkeit des Fahrzeugs mit der angegebenen Steigung linear erhöht bzw. erniedrigt bis die Basisgeschwindigkeit (Parameter *Signaloffset*) erreicht ist. Anschließend startet die entsprechende Signalform.

Externe Daten als Signal

Wird als Signalform der Typ Data gewählt, dann werden die Daten aus dem über Parameter 2 bestimmten Eingang **UserData<channel1>_In** der DPUTRF als Geschwindigkeit gesetzt. Bei der Aktivierung des Modells wird zur Steuerung des Datenflusses der Ausgang **UserData<channel1>_Out** auf 1 gesetzt, solange das Modell aktiv ist. Wurde als Parameter 3 ein Wert größer 0 angegeben, wird dem Fahrzeug der Wert des Eingangs **UserData<channel2>_In** als laterale Spurbabweichung übergeben.

Mit folgender Verschaltung

```
DPUTraffic TRF
{
    Computer = {MAIN};
    Index = 6;
};
DPUSampleFilePlayer TrafficData
{
```

```
        Computer = {MAIN};
        Index = 5;
        FileName = "speedprofile.asc";
        SampleID1 = 1;
        SampleID2 = 2;
    };
    Connections =
    {
        TrafficData.Out1 <-> TRF.UserData1_In,
        TrafficData.Out2 <-> TRF.UserData2_In,
        TRF.UserData1_Out <-> TrafficData.Start
    };
...

...
Traffic TRF
{
    ...
    Vehicle samplecontrolled
    {
        ...
        BehaviourMachine = {(SignalSpeed,Data,1,2)};
        ...
    };
    ...
};
```

wird die DPUSampleFilePlayer gestartet, sobald das Modell aktiviert, deren Kanal1 auf den Wert 1 und Kanal 2 auf den Wert 2 gesetzt wurde.

Die DPUSampleFilePlayer hat eine Datendatei geladen, deren erste Spalte die Geschwindigkeit und die zweite Spalte die laterale Spurabweichung enthält. Diese Daten spielt die DPUSampleFilePlayer in die Kanäle 1 und 2 ein.

5.3.6.4 Verhaltensmodell: SimCarLocked

Dieses Verhaltensmodell heftet das Fahrzeug in einem definierten Abstand an das EGO - Fahrzeug. Auf diesen Abstand kann zusätzlich eine Sinusschwingung addiert werden. Wird als Abstand eine negative Distanz angegeben, dann wird das Fahrzeug hinter dem EGO - Fahrzeug eingeregelt, bei einer positiven Distanz vor dem EGO - Fahrzeug.

Parameter im Tupel

1. *<distance [m]>* Der Abstand der zum EGO-Fahrzeug eingenommen werden soll (Default-Wert = 0).

Flowpoints:

LockDistance, *<distance [m]>*, [*<amplitude of sine>*, *<period of sine [ms]>*]

Mit diesem Flowpoint wird das Fahrzeug an das EGO - Fahrzeug geheftet. Die Distanz wird von dem Fahrzeug eingeregelt. Werden die beiden Parameter für eine Amplitude und Periode noch angegeben, dann schwingt der Abstand sinusförmig.

SetLockDistance, *<distance [m]>*

Dieser Flowpoint ändert die aktuellen Parameter.

LockDistanceRel, *<distance [m]>*, *<TailGateDistance [m]>*, *<DeltaRelSpeed [m/s]>*

Das Fahrzeug wird auf die *distance* geheftet, wenn der Abstand zum EGO - Fahrzeug größer ist als die *TailGateDistance*. Die Maximale Geschwindigkeit, mit der der Abstand nun ausgeregelt wird, berechnet sich aus *DeltaRelSpeed* multipliziert mit der EGO - Fahrzeuggeschwindigkeit.

LockDistanceAbs, *<distance [m]>*, *<TailGateDistance [m]>*, *<DeltaAbsSpeed [m/s]>*

Das Fahrzeug wird auf die *Distanz* gelockt, wenn der Abstand zum EGO - Fahrzeug größer ist als die *TailGateDistance*. Der Abstand wird nun mit maximal *DeltaAbsSpeed* ausgeregelt.

Sinusdistance, *<amplitude [m]>*, *<period [s]>*

Setzt die Parameter für die Sinusschwingung.

Unlock Hebt die Abstandskopplung wieder auf.

DistanceInc, *<offset>*

Vergrößert (bzw. verkleinert bei negativem Vorzeichen) den aktuellen Abstand um den übergebenen Wert.

5.3.6.5 Verhaltensmodell: SimCarLocked2

Das Verhaltensmodell **SimCarLocked2** stellt eine Erweiterung des Verhaltensmodells **SimCarLocked** dar. Zusätzlich zu den bestehenden Möglichkeiten bietet dieses Modell die Option, dass der Verkehrsteilnehmer den Abstand zum EGO-Fahrzeug fließend annimmt und nicht schlagartig die Geschwindigkeit an dem gewünschten Abstand ändert.

Parameter im Tupel

1. *<distance [m]>* Der Abstand, der zum EGO-Fahrzeug eingenommen werden soll (Default-Wert = 0).
2. *<time distance [s]>* Über *<time distance>* wird ein Mindestsekundenabstand eingestellt, den das Verhaltensmodell zum EGO-Fahrzeug einhält. Diese wird auch dann nicht unterschritten wenn eine geringere *<distance>* angegeben wird (Default-Wert = 0).
3. *<min speed [m/s]>* Über diesen Parameter kann eine Mindestgeschwindigkeit eingestellt werden die der Verkehrsteilnehmer nicht unterschreiten kann. (Default-Wert = 0).

Sind keine Parameter im Tupel angegeben, dann wartet das Verhaltensmodell auf einen Flowpoint und erlaubt einem nachfolgendem Verhaltensmodell den Verkehrsteilnehmer zu steuern. Sind die Parameter angegeben, dann fährt der Verkehrsteilnehmer auf den angegebenen Abstand an das EGO-Fahrzeug heran.

Flowpoints

LockDistance, *<distance [m]>*, *<deceleration distance [m]>*, *<amplitude [m]>*, *<period [ms]>*

Mit diesem Flowpoint wird das Fahrzeug an das EGO - Fahrzeug geheftet. Die Distanz *<distance>* wird von dem Fahrzeug eingeregelt. Werden die beiden Parameter für eine Amplitude und Periode noch mit angegeben, dann schwingt der Abstand sinusförmig.

Die *<deceleration distance>* gibt an, ab welcher Entfernung die Geschwindigkeit des Fahrzeugs der des EGO - Fahrzeugs angeglichen werden soll. Die Entfernung ist dabei ein reiner Abstand, der nicht gerichtet ist. Defaultwert für die *<deceleration distance>* ist 5m.

<distance> spezifiziert den Abstand des Fahrzeugs zum EGO - Fahrzeug in Meter. Positive Werte lassen das Fahrzeug vor dem EGO - Fahrzeug herfahren, negative hinter dem EGO – Fahrzeug.

<amplitude> legt den Abstand fest, um den das Fahrzeug die Distanz variiert z.B. lässt der Eintrag 1, das Fahrzeug um +/-1m variieren. Die Sinusperiode gibt an wie viele Sekunden es dauert bis ein Fahrzeug eine komplette Schwingung durchgeführt hat d.h. von *<distance>* zu (*<distance>* + *<amplitude>*), dann zu (*<distance>* - *<amplitude>*) und wieder zu *<distance>*.

SetLockDistance, *<distance [m]>*, *<deceleration distance [m]>*, *<amplitude [m]>*, *<period [ms]>*

Dieser Flowpoint ändert die aktuellen Parameter (siehe LockDistance).

LockDistanceRel, *<distance [m]>*, *<deceleration distance [m]>*, *<range [m]>*, *<deltarelspeed [%]>*

Das Fahrzeug wird auf die *<distance>* gelockt, wenn der Abstand zum EGO-Fahrzeug kleiner ist als *<range>*. Der Abstand zum EGO-Fahrzeug wird dabei absolut betrachtet, d.h. es spielt keine Rolle ob nach vorne oder nach hinten. Der Default-Wert für *<range>* ist 100m und es sind nur positive Werte zugelassen. Die Maximale Geschwindigkeit, mit der der Abstand ausgeregelt wird, berechnet sich aus *<deltarelspeed>* multipliziert mit der Geschwindigkeit des EGO-Fahrzeugs (Default-Wert gleich 0.4). Der Parameter *<deceleration distance>* ist unter **LockDistance** erklärt.

LockDistanceAbs, *<distance[m]>*, *<deceleration distance [m]>*, *<range [m]>*, *<deltaabsspeed [m/s]>*

Das Fahrzeug wird auf *<distance>* gelockt, wenn der Abstand zum EGO - Fahrzeug kleiner ist als *<range>*. Der Abstand wird mit maximal *<deltaabsspeed>* ausgehend von der Geschwindigkeit des EGO-Fahrzeugs ausgeregelt. *<deltaabsspeed>* ist auf positive Werte beschränkt. Der Verkehrsteilnehmer kann außerdem nur positive Geschwindigkeiten annehmen. Der Parameter *<deceleration distance>* ist unter **LockDistance** erklärt.

SinusDistance, *<amplitude [m]>*, *<period [s]>*

Setzt die Parameter für die Sinusschwingung neu.

Unlock Hebt die Abstandskopplung wieder auf.

DistanceInc, *<difference to current distance [m]>*

Vergrößert (bzw. verkleinert bei negativem Vorzeichen) den aktuellen Abstand um den übergebenen Wert.

5.3.6.6 Verhaltensmodell: CommandAction

Die CommandAction schickt ein Kommando an das Fahrzeug, wenn die im Flowpoint kodierte Bedingung eintritt. Dieses Kommando wird dann vom Fahrzeug ausgeführt. Da das Kommando eventuell von einem anderen (nachfolgenden) Verhaltensmodell wieder überschrieben werden kann, existiert bei den meisten Flowpoints die Möglichkeit, die nachfolgenden Verhaltensmodelle eine gewisse Zeit (letzter Parameter **TRFC_DELAY** in Millisekunden) zu blockieren. Ein Verhaltensmodell mit einer höheren Priorität kann nicht blockiert werden, deshalb sollte die CommandAction vor diesen Verhaltensmodellen in der Liste stehen.

Parameter im Tupel

1. *<number>* eine beliebige natürliche Zahl
- 2...n *<commands>* Folge von Fahrzeugkommandos und deren Parameter

Ein Fahrzeug kann mehrere **CommandAction** - Verhaltensmodelle besitzen, die zu unterschiedlichen Zeitpunkten aktiv sind. Deshalb muss jeder **CommandAction** eine Nummer (*<number>*) zugewiesen werden.

Kommandos

TRFC_SETRIGHT | TRFC_SETLEFT | TRFC_SETSTRAIGHT

Erzwingen einer Spur Wahl bei der nächstmöglichen Mehrfachauswahl von Fahrspuren mit entsprechenden Headings.

TRFC_ONOFF, *<0|1>*

Deaktivierung des Fahrzeugs.

TRFC_SHOW, *<0|1>*

Mit diesem Kommando kann das Fahrzeug versteckt oder wieder angezeigt werden. Ein verstecktes Fahrzeug fährt weiter und andere Verkehrsteilnehmer reagieren weiterhin auf das Fahrzeug.

TRFC_VX, *<speed [m/s]>*

Setzt die Geschwindigkeit neu. Der Verkehrsteilnehmer übernimmt diese Geschwindigkeit im nächsten Simulationsschritt.

TRFC_AX, *<acceleration[m/s²]>*

Setzt die Beschleunigung neu. Der Verkehrsteilnehmer übernimmt diese Beschleunigung im nächsten Simulationsschritt.

TRFC_DY, *<value [m]>*

Setzt den lateralen Abstand dieses Fahrzeugs von der Idealspur neu. Der Versatz wird sofort angenommen.

TRFC_DX, *<value [m]>*

versetzt das Fahrzeug um das angegebene Argument in Längsrichtung.

TRFC_VY, *<value [m/s]>*

Das Fahrzeug bewegt sich mit der angegebenen Quergeschwindigkeit bis zum maximalen Abstand von der Idealspur. Der Wert gibt die Geschwindigkeit in [m/s] an. Negative Werte bewegen das Fahrzeug nach links bis zum Wert TRFC_DYLIMITL, positive Werte bewegen das Fahrzeug nach rechts bis zum Wert TRFC_DYLIMITR. Zu beachten ist hierbei, dass der

aktuelle Versatz vor der Änderung mittels TRFC_VY innerhalb des Intervalls [TRFC_DYLIMITL, TRFC_DYLIMITR] liegt. Ist dies nicht der Fall, springt das Fahrzeug auf den nächsten Wert innerhalb des Intervalls.

TRFC_DYLIMITL, <min limit of DY [m]>

Minimalwert des Spurabstandsintervall. Negative Werte bezeichnen Abstände nach links, positive Werte Abstände nach rechts.

TRFC_DYLIMITR, <max limit of DY [m]>

Maximalwert des Spurabstandsintervall. Negative Werte bezeichnen Abstände nach links, positive Werte Abstände nach rechts.

TRFC_LATDOFS, <value [m]>

Offset für den lateralen Versatz.

TRFC_LATDMUL, <multiplicator>

Multiplikator für den lateralen Abstand.

TRFC_FRONTLIGHT, <0|1>

Ein- / Ausschalten des Abblendlichtes.

TRFC_BRAKELIGHT, <0|1>

Ein- / Ausschalten des Bremslichtes. Um dieses Kommando nutzen zu können, muss zuerst **TRFC_AUTOBRAKELIGHTCONTROL** mit Parameter 0 gesendet werden, damit die Bremsleuchten nicht automatisch gesetzt werden.

TRFC_LEFTINDICATOR, <0|1>

Ein- / Ausschalten des linken Blinkers. Um dieses Kommando nutzen zu können, muss zuerst **TRFC_AUTOINDICATORLIGHTCONTROL** mit Parameter 0 gesendet werden, damit die Blinker nicht automatisch gesetzt werden.

TRFC_RIGHTINDICATOR, <0|1>

Ein- / Ausschalten des rechten Blinkers. Um dieses Kommando nutzen zu können, muss zuerst **TRFC_AUTOINDICATORLIGHTCONTROL** mit Parameter 0 gesendet werden, damit die Blinker nicht automatisch gesetzt werden.

TRFC_AUTOBRAKELIGHTCONTROL, <0|1>

Deaktivierung / Aktivierung der automatischen Bremslichtkontrolle bei negativen Geschwindigkeitsänderungen.

TRFC_AUTOINDICATORLIGHTCONTROL, <0|1>

Deaktivierung / Aktivierung der automatischen Blinkersetzung bei Abbiegespuren.

TRFC_CHANGELANEREQUEST, <lane>, <mode>, <Distanz in m/ Zeit in s>

Setzt einen Spurwechselwunsch, der von dem Verhaltensmodell **LaneChangeAvoidance** ausgeführt wird. Dieser leitet dann einen Spurwechsel auf die übergebene Ziel Spur <lane> ein. Der Spurwechsel ist in der angegebenen Zeit / Distanz (je nach Modus) abgeschlossen. Der Mode-

Parameter gibt an ob der Weg (*<mode> = 0*) oder die Zeit (*<mode> = 1*) spezifiziert wird.

TRFC_CHANGE_LANE, *<lane>*, *<mode>*, *<Distanz [m] / Zeit [s]>*

Leitet einen Spurwechsel auf die übergebene Ziel Spur ein. Der Spurwechsel ist in der angegebenen Zeit / Distanz (je nach Modus) abgeschlossen.

Der Mode-Parameter gibt an ob der Weg (*<mode> = 0*) oder die Zeit (*<mode> = 1*) spezifiziert wird.

TRFC_DELAY, *<time [ms]>*

Verzögert die Versendung der nachfolgenden Befehle an das Fahrzeug um die angegebene Zeit in Millisekunden.

Die nachfolgenden Kommandos simulieren defekte Lichter am Fahrzeug. Die eigentliche Funktion des entsprechenden Lichtes kann damit deaktiviert bzw. wieder aktiviert werden.

TRFC_RBACKLIGHT, *<0|1>*

Aktiviert / Deaktiviert die Funktion des rechten Rücklichtes.

TRFC_LBACKLIGHT, *<0|1>*

Aktiviert / Deaktiviert die Funktion des linken Rücklichtes.

TRFC_RFRONTLIGHT, *<0|1>*

Aktiviert / Deaktiviert die Funktion des rechten Abblendlichtes.

TRFC_LFRONTLIGHT, *<0|1>*

Aktiviert / Deaktiviert die Funktion des linken Abblendlichtes.

TRFC_RBRAKELIGHT, *<0|1>*

Aktiviert / Deaktiviert die Funktion des rechten Bremslichtes.

TRFC_LBRAKELIGHT, *<0|1>*

Aktiviert / Deaktiviert die Funktion des linken Bremslichtes.

Flowpoints

ActivateCommandAction, *<number>*

Aktivierung der CommandAction mit der angegebenen Nummer (*<number>*). Dabei ist zu beachten, dass alle Flowpoints von allen aktiven **CommandAction** Verhaltensmodellen ausgeführt werden. Deshalb sollte immer nur ein CommandAction Modell aktiviert sein.

DeactivateCommandAction, *<number>*

Deaktivierung der **CommandAction** angegebenen mit der angegebenen Nummer (*<number>*).

SpeedAndDistanceLess, *<speed [m/s]>*, *<distance [m]>*, *<time [ms]>* |

DistanceAndSpeedLess, *<distance [m]>*, *<speed [m/s]>*, *<time [ms]>*

Kommando wird einmalig gesendet, wenn die Geschwindigkeit unter der angegebenen Grenze und die euklidische Distanz des Fahrzeugs zum

EGO - Fahrzeug unter der angegebenen Distanz liegt. Der Parameter Zeit gibt wie oben beschrieben die Zeit an in der andere Verhaltensmodelle blockiert werden.

SpeedAndDistanceGreater, *<speed [m/s]>, <distance [m]>, <time [ms]>* |

DistanceAndSpeedGreater, *< distance [m]>, < speed [m/s]>, <time [ms]>*

Kommando wird einmalig gesendet, wenn die Geschwindigkeit über der angegebenen Grenze und die euklidische Distanz des Fahrzeugs zum EGO - Fahrzeug über der angegebenen Distanz liegt.

SpeedGreaterAndDistanceLess, *<speed [m/s]>, <distance [m]>, <time [ms]>* |

DistanceLessAndSpeedGreater, *< distance [m]>, < speed [m/s]>, <time [ms]>*

Kommando wird einmalig gesendet, wenn die Geschwindigkeit über der angegebenen Grenze und die euklidische Distanz des Fahrzeugs zum EGO - Fahrzeug unter der angegebenen Distanz liegt.

SpeedLessAndDistanceGreater, *<speed [m/s]>, <distance [m]>, <time [ms]>* |

DistanceGreaterAndSpeedLess, *< distance [m]>, < speed [m/s]>, <time [ms]>*

Kommando wird einmalig gesendet, wenn die Geschwindigkeit unter der angegebenen Grenze und die euklidische Distanz des Fahrzeugs zum EGO - Fahrzeug über der angegebenen Distanz liegt.

SpeedLess, *<speed [m/s]>, <time [ms]>*

Kommando wird einmalig gesendet, wenn die Geschwindigkeit unter der angegebenen Grenze liegt.

SpeedGreater, *<speed [m/s]>, <time [ms]>*

Kommando wird einmalig gesendet, wenn die Geschwindigkeit über der angegebenen Grenze liegt.

DistanceLess, *<distance [m]>, <time [ms]>*

Kommando wird einmalig gesendet, wenn die euklidische Distanz des Fahrzeugs zum EGO - Fahrzeug unter der angegebenen Distanz liegt.

DistanceGreater, *<distance [m]>, <time [ms]>*

Kommando wird einmalig gesendet, wenn die euklidische Distanz des Fahrzeugs zum EGO - Fahrzeug über der angegebenen Distanz liegt.

DistanceInFrontLess, *<distance [m]>, <time [ms]>*

Kommando wird einmalig gesendet, wenn die euklidische Distanz des Fahrzeugs zum EGO - Fahrzeug unter der angegebenen Distanz liegt und sich das Fahrzeug vor dem EGO-Fahrzeug befindet.

DistanceInFrontGreater, *<distance [m]>, <time [ms]>*

Kommando wird einmalig gesendet, wenn die euklidische Distanz des Fahrzeugs zum EGO - Fahrzeug über der angegebenen Distanz liegt und sich das Fahrzeug vor dem EGO-Fahrzeug befindet.

DistanceBackwardsLess, *<distance [m]>, <time [ms]>*

Kommando wird einmalig gesendet, wenn die euklidische Distanz des Fahrzeugs zum EGO - Fahrzeug unter der angegebenen Distanz liegt und sich das Fahrzeug hinter dem EGO-Fahrzeug befindet.

DistanceBackwardsGreater, *<distance [m]>*, *<time [ms]>*

Kommando wird einmalig gesendet, wenn die euklidische Distanz des Fahrzeugs zum EGO - Fahrzeug über der angegebenen Distanz liegt und sich das Fahrzeug hinter dem EGO-Fahrzeug befindet.

UserChannel, *<channel>*, *<time [ms]>*

Kommando wird einmalig geschickt, wenn der Eingang **UserData***<channel>*_In den Wert 0.5 überschreitet.

UserChannelGreater, *<channel>*, *<value>* *<time [ms]>*

Kommando wird einmalig geschickt, wenn der Eingang **UserData***<channel>*_In den angegebenen Wert *<value>* überschreitet.

UserChannelLess, *<channel>*, *<value>* *<time [ms]>*

Kommando wird einmalig geschickt, wenn der Eingang **UserData***<channel>*_In den angegebenen Wert *<value>* unterschreitet.

5.3.6.7 Verhaltensmodell: EXWDM

Das EXWDM Modell basiert auf dem Fahrzeugfolgemodell von Wiedemann³ in Kombination mit dem Spurwechselmodell MOBIL (Minimizing Overall Braking decelerations Induced by Lane changes) - siehe auch Verhaltensmodell EXIDM.

Das Modell nach Wiedemann ist ein Fahrzeug-Folgemodell, das realistisches Folgenfahren inklusive variierendem Abstand abbildet. Im Gegensatz zum Modell EXIDM besitzt das Modell nach Wiedemann keine kontinuierliche Beschleunigungsfunktion sondern ist auf einen Wenn-Dann-Entscheidungsbaum aufgebaut

Um das Modell in den Grundzügen zu erklären wird im folgenden Abschnitt kurz auf die Theorie des Modells eingegangen. Für weiterführende Informationen verweisen wir an dieser Stelle auf entsprechende Fachliteratur.

Fahrzeugfolgemodell Wiedemann

Das Modell von Wiedemann beschreibt das Fahren auf einer Spur ohne Spurwechsel und unterscheidet grundsätzlich zwischen unbeeinflusstem (ohne Interaktion mit anderen Fahrzeugen) und beeinflusstem Fahren (Interaktion mit anderen Fahrzeugen). Abbildung 6 stellt die Geschwindigkeitsdifferenz eines Fahrzeugs beim Annähern an ein Vorderfahrzeug in Abhängigkeit vom Abstand zu diesem Vorderfahrzeug dar.

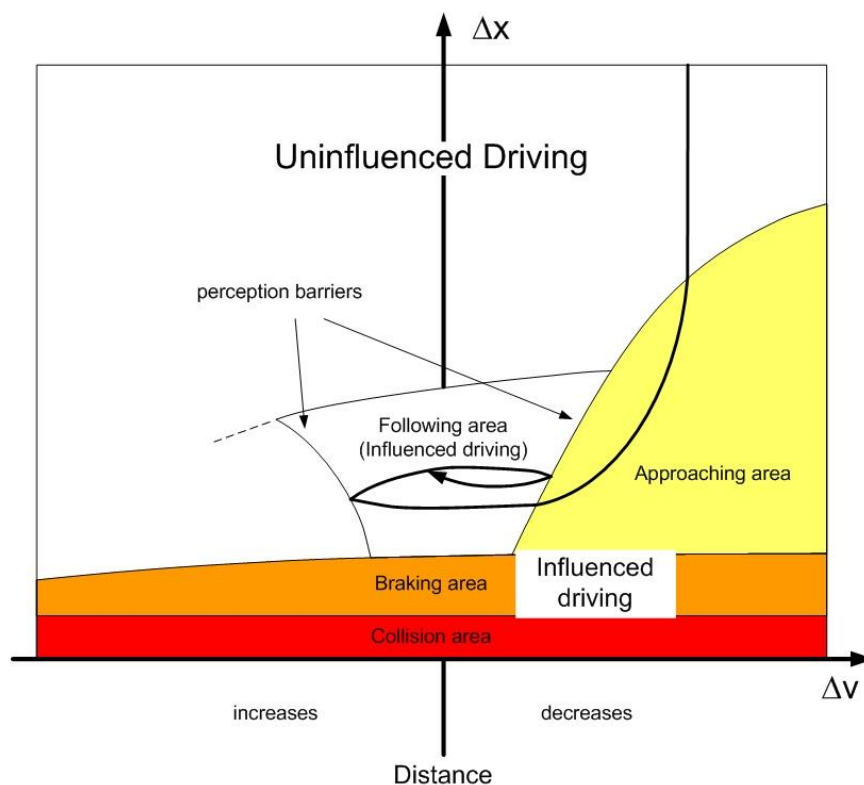


Abbildung 6: Wahrnehmung und Entscheidung bei Annähern und Folgen eines Vorderfahrzeugs

³ Siehe R. Wiedemann, [Simulation des Straßenverkehrsflusses](#). Schriftenreihe des Instituts für Verkehrswesen der Universität Karlsruhe, 1974.

Das Modell nach Wiedemann simuliert detailliert das freie Fahren und Folgefahren eines Fahrers, der nach Vorgabe einer Wunschgeschwindigkeit versucht, diese einzustellen und zu halten. Dabei wird ein Pendeln um die Zielgeschwindigkeit ebenso simuliert wie das Halten eines Abstandes zum Vordermann beim Folgefahren.

Eine weitergehende Erläuterung ist nachzulesen in der Arbeit von Wiedemann³.

Spurwechselmodell MOBIL

Das Spurwechselmodell basierend auf MOBIL⁴ untersucht zunächst ausgehend von der jeweiligen Position des Fahrzeugs, welche Spuren für Spurwechsel verwendet werden können.

In Courses wird die Befahrbarkeit durch die Linien zwischen den Spuren festgelegt, d.h. eine durchgezogene Linie zwischen zwei Spuren wird als Grenze interpretiert, die von MOBIL nicht überschritten werden darf. Ist hingegen keine Linie oder eine gestrichelte Linie zwischen den Spuren, dann verwendet MOBIL auch die benachbarte Spur. Auf diese Weise werden alle möglichen Spuren für ein Fahrzeug ermittelt.

MOBIL erlaubt Spurwechsel nur zwischen Spuren, die in dieselbe Richtung führen, d.h. das Überholen auf Gegenspuren wird dadurch nicht erfasst. Es wählt bei mehreren möglichen Spuren (mit gleicher Fahrtrichtung) unter Berücksichtigung des Rechtsfahrgebots zu jeder Zeit diejenige aus, die den Wünschen des Fahrers am besten entspricht und führt einen erforderlichen Spurwechsel selbstständig durch.

Bei der Auflistung der Parameter sind in Klammern die voreingestellten Standardwerte angegeben.

Parameter im Tupel

- | | | |
|----|----------------------------------|--|
| 1. | <code><StartSpeed></code> | Startgeschwindigkeit (Defaultwert = 27.8m/s) [general] |
| 2. | <code><TargetSpeed></code> | Zielgeschwindigkeit (Defaultwert = 27.8m/s) [general] |
| 3. | <code><MinDistF></code> | Mindestabstand zum Vorderfahrzeug beim Stillstand, der nicht unterschritten werden sollte ($1\text{m} + 2 \cdot \text{ZF1}[\text{m}]$). Der Wert entspricht dem AX-Parameter des Modells. Dieser Abstand wird auf den Sekundenabstand während der Fahrt addiert (Defaultwert = 1.5) [general]. |
| 4. | <code><THWF></code> | Gewünschter minimaler Folgeabstand zum Vorderfahrzeug. Dieser Wert entspricht dem Parameter BX des Modells, der aber in einen Sekundenabstand umgewandelt wurde. (Defaultwert = 2.0) [general] |
| 5. | <code><ZF1></code> | Eine ($\mu=0.5$, $\sigma=0.15$) normalverteilte Zufallsvariable, die das Sicherheitsbedürfnis des Fahrers widerspiegelt. 99% der Zufallswerte liegen im 3σ -Bereich, für die Fahrsimulation wurden die Werte auf diesen Bereich begrenzt. ZF1 wird zufällig gezogen, wenn kein Wert spezifiziert |

⁴ Siehe Treiber, M., Helbing, D.(2002). Mikrosimulation im Straßenverkehr mit einem einfachen Modell. In D. Tavanganrian, R. Grützner (Eds.), ASIM 2002 – Tagungsband, 16. Symposium Simulationstechnik, Rostock, 514ff.

wird, ansonsten wird der angegebene Wert verwendet. Ein Wert um 0.1 wird empfohlen. Dieser Wert legt u.a. fest, um welchen Prozentsatz der $\langle THWF \rangle$ eines Fahrers überschritten wird. Zu große Werte für diesen Parameter führen (je nach Kombination der Parameter) leicht zu Kollisionen mit einem Vorderfahrzeug. (Defaultwert = 0.1)

6. $\langle ZF2 \rangle$

Eine ($\mu=0.5$, $\sigma=0.15$) normalverteilte Zufallsvariable, die das Schätzvermögen des Fahrers widerspiegelt. 99% der Zufallswerte liegen im 3σ -Bereich, für die Fahrsimulation wurden die Werte auf diesen 3σ -Bereich begrenzt. $\langle ZF2 \rangle$ wird zufällig gezogen, wenn kein Wert spezifiziert wird, ansonsten wird der angegebene Wert verwendet. Bei einem Wert von 1.0 gibt es nur eine geringe Distanz um die das Folgefahren schwankt (Verkehrsteilnehmer besitzt ein gutes Schätzverhalten).

Bei einem Wert von 0.0 ergibt sich ein großer Bereich für das Folgefahren und dem Verkehrsteilnehmer wird ein schlechter Schätzwert für den Abstand zugewiesen. Zu kleine Werte für diesen Parameter führen (je nach Kombination der Parameter) leicht zu Kollisionen mit einem Vorderfahrzeug. (Default-Wert = 0.9)

7. $\langle ZF3 \rangle$

Eine ($\mu=0.5$, $\sigma=0.15$) normalverteilte Zufallsvariable, die die Unterschiede des Brems- und Beschleunigungsvermögens der Fahrzeuge widerspiegelt. Größere Werte bedeuten stärkere Beschleunigungen, kleinere Werte auch kleinere Beschleunigungen. 99% der Zufallswerte liegen im 3σ -Bereich, für die Fahrsimulation wurden die Werte auf diesen Bereich begrenzt. $ZF3$ wird zufällig gezogen, wenn kein Wert spezifiziert wird, ansonsten wird der angegebene Wert verwendet. Bei einem Wert von 1.0 wird die maximal mögliche Beschleunigung/Verzögerung genutzt. Allgemein gilt, dass der Wert $ZF3$ als Faktor in die genutzte Beschleunigung/Verzögerung eingeht. Zu geringe Werte führen leicht dazu, dass ein Fahrzeug nicht mehr rechtzeitig auf ein Vorderfahrzeug bremsen kann, sofern dieses in der Lage ist größere Verzögerungen umzusetzen. (Defaultwert = 0.9).

8. $\langle ZF4 \rangle$

Eine ($\mu=0.5$, $\sigma=0.15$) normalverteilte Zufallsvariable, die den untersten Schwellwert der feinsten noch möglichen Kontrolle des Bremspedals widerspiegelt. 99% der Zufallswerte liegen im 3σ -Bereich. Für die Fahrsimulation wurden die Werte auf den 3σ -Bereich begrenzt. $ZF4$ wird zufällig gezogen, wenn kein Wert angegeben ist, ansonsten wird der angegebene Wert verwendet. Dieser Wert

- geht ausschließlich in die Bestimmung des Parameters BNULL ein. (Default-Wert = 0.1)
9. <NZF> Eine ($\mu=0.5$, $\sigma=0.15$) normalverteilte Zufallsvariable, die zur Berechnung des Parameters EX verwendet wird. Aus dem Schätzvermögen (ZF2) eines Fahrers wird das durchschnittliche Abdriften beim Folgevorgang errechnet. Ein gutes Schätzvermögen impliziert normalerweise ein geringes Abdriften beim Folgevorgang. Durch NZF wird das Abdriften wieder erhöht. (Default-Wert = 0.05)
 10. <CX> CX ist im Modell von Wiedemann eine Hilfsvariable zur Berechnung der Wahrnehmungsschwelle für Geschwindigkeitsdifferenzen bei relativ großen Abständen ($SDV=((<DX>-<AX>)/<CX>)^2$). Die Größe <CX> berechnet sich aus $25 \cdot (1 + ZF1 + ZF2)$. Der Benutzer kann diesen Wert statisch überschreiben, dann wird <CX> nicht mehr aus den Hilfsvariablen bestimmt. Ansonsten wird <CX> wie oben beschrieben aus <ZF1> und <ZF2> errechnet.
 11. <EX> <EX> ist im Modell von Wiedemann eine Hilfsvariable zur Berechnung der oberen Grenze für das Abtriften beim Folgevorgang ($SDX = <AX> + <EX> \cdot (<BX> - <AX>)$). Die Größe <EX> berechnet sich aus $2 - <ZF2> + <NZF>$. Der Benutzer kann diesen Wert statisch überschreiben, dann wird <EX> nicht mehr aus den Hilfsvariablen bestimmt. Ansonsten wird <EX> wie oben angegeben aus <ZF2> und <NZF> errechnet.
 12. <BNULL> <BNULL> bezeichnet den untersten Schwellwert, der als Beschleunigung bzw. Verzögerung wahrgenommen wird. Er berechnet sich folgendermaßen: $\pm <BNULL> = 0.2 \cdot (<ZF4> + <NZF>)$. Der Benutzer kann diesen Wert statisch überschreiben, dann wird <BNULL> nicht mehr aus den Hilfsvariablen bestimmt. Ansonsten wird <BNULL> wie oben angegeben aus <ZF4> und <NZF> errechnet.
 13. <DMAX> Maximale Entfernung bei der auf ein Hindernis frühestens reagiert werden kann. Diese errechnet sich bei Wiedemann aus $150 + 50 \cdot (<ZF1> + <ZF2>)$. Der Benutzer kann diesen Wert statisch überschreiben, dann wird <DMAX> nicht mehr aus den Hilfsvariablen bestimmt. Ansonsten wird <DMAX> wie oben angegeben aus <ZF1> und <ZF2> errechnet.
 14.
 Der Parameter
 bezeichnet die Schwelle zwischen Motorverzögerung und (zusätzlicher) Verzögerung mit der Bremse. Er setzt sich folgendermaßen zusammen: $
 = -0.5 - 1.5 \cdot <ZF3>$. Der Benutzer kann diesen

- Wert statisch überschreiben, dann wird `<BR>` nicht mehr aus den Hilfsvariablen bestimmt. Ansonsten wird `<BR>` wie oben angegeben aus `<ZF3>` errechnet.
15. `<TimeStep>` Das Folgemodell von Wiedemann berechnet in jeder Sekunde die gewünschte Beschleunigung oder Verzögerung des Fahrers. Um eine realistische Simulation zu gewährleisten, wird in SILAB auch nur alle `<TimeStep>` [ms] die neue Beschleunigung bzw. Verzögerung berechnet und wird anschließend für `<TimeStep>` ms von dem Verkehrsteilnehmer umgesetzt. Ausnahme hierbei ist natürlich der Fall, dass auf ein Vorderfahrzeug gebremst werden muss, dann wird auch in kürzeren Zeitabständen reagiert. (Defaultwert = 500ms)
16. `<ACCMMax>` Das Folgemodell wählt zum Aufnehmen der Wunschgeschwindigkeit eine Beschleunigung kleiner gleich `<ACCMMax>`. Es sind ausschließlich positive Werte erlaubt. (Defaultwert = 10.0m/s).
17. `<ACCMin>` Das Folgemodell wählt zum Abbremsen und zur Kollisionsvermeidung eine negative Beschleunigung größer gleich `<ACCMin>`. Es sind ausschließlich negative Werte erlaubt. (Defaultwert = -12m/s)
18. `<p>` Das Spurwechselmodell MOBIL besitzt einen Höflichkeitsfaktor `<p>`. Dieser Faktor beschreibt den Egoismus des Modells beim Spurwechsel. Der Wert 0 spezifiziert ein rein egoistisches Modell, d.h. wenn das Fahrzeug nach einem Spurwechsel höher beschleunigen kann als vorher wird ein Wechselwunsch geäußert. Die möglicherweise dadurch verursachten negativen Beschleunigungen anderer Verkehrsteilnehmer werden nicht berücksichtigt. Ein Wert größer gleich 1 spezifiziert ein sehr altruistisches Verhalten d.h. der Fahrer vermeidet auch negative Beschleunigungen bei anderen Verkehrsteilnehmern und verzichtet dafür auf eigene Beschleunigungsvorteile. Werte zwischen 0 und 0.6 werden in der Literatur als realistisches Verhalten bezeichnet. (Defaultwert = 0.4)
19. `<athres>` Spurwechselschranke, die einen Spurwechsel eingrenzt. Das Spurwechselmodell MOBIL vergleicht die Beschleunigungen nach einem (fiktiven) Spurwechsel mit den aktuellen Beschleunigungen (ohne Spurwechsel). Der Parameter `athres` ist ein Offset, der negativ für einen Wechsel gewertet wird und beschreibt somit, dass ein Spurwechsel erst ab einem gewissen Beschleunigungsvorteil (der größer als `<athres>` sein muss) erfolgt. Dieser Wert

- sollte im Normalfall zwischen 0.1m/s^2 und 1m/s^2 gewählt werden. (Defaultwert = 0.2m/s^2)
20. `<bsave>` Maximal sichere Verzögerung in m/s^2 für ein nachfolgendes Fahrzeug. Dieses übergeordnete Sicherheitskriterium stellt sicher, dass ein nachfolgendes Fahrzeug keinesfalls mit einer höheren negativen Beschleunigung als `<bsave>` abbremsen muss. Dieser Wert sollte im Normalfall zwischen 4m/s^2 und 2m/s^2 gewählt werden. (Defaultwert = 4.0m/s^2)
21. `<BiasRight>` Beschleunigungsschranke für rechte Spur. Das Spurwechselmodell MOBIL vergleicht die Beschleunigungen nach einem (fiktiven) Spurwechsel nach rechts mit den aktuellen Beschleunigungen im nächsten Zeitschritt. Der Parameter `<biasright>` ist ein Offset, der positiv für die rechte Spur gewertet wird (bei Spurwechselberechnungen nach rechts) und beschreibt somit einen Vorteil für die rechte Spur. Daraus resultiert ein entsprechendes Rechtsfahrgebot. Dieser Wert sollte im Normalfall zwischen 0.1m/s^2 und 2m/s^2 gewählt werden. (Defaultwert = 0.1m/s^2)
22. `<WaitForBlinkTime>` Diese Zeit in ms muss ein Wunsch für einen Spurwechsel bestanden haben, damit ein Blinken in die entsprechende Richtung erfolgt. (Defaultwert = 500ms)
23. `<LaneChangeBlinkTime>` Diese Zeit in ms muss ein Blinken erfolgen, damit ein entsprechender Spurwechsel erfolgen kann. (Defaultwert = 1000ms)
24. `<LaneChangeWaitTime >` Diese Zeit in ms muss ein Fahrzeug nach einem Spurwechsel warten, bevor es erneut einen Spurwechsel durchführt (Defaultwert = 5000ms).
25. `<LaneChangeMinSpeedDiff>` Diese Differenz zur Zielgeschwindigkeit in m/s muss mindestens vorliegen, damit ein Spurwechsel ausgeführt wird. (Defaultwert = 4m/s).
26. `<LaneChangeDistance>` Distanz in Meter in der ein Spurwechsel minimal durchgeführt wird. Üblicherweise wird ein Spurwechsel über 2s hinweg durchgeführt, bei geringen Geschwindigkeiten wird jedoch mindestens der angegebene Weg zurückgelegt. (Defaultwert = 10m) [general].
27. `<LaneChangeDistNLMin>` Ein sinnvoller Spurwechsel sollte immer einen gewissen Platzgewinn nach vorne mit sich bringen. Dieser wird

- durch den Parameter `<LaneChangeDistNLMin [m]>` spezifiziert. Dadurch können sehr viele unnötige Spurwechsel gerade bei niedrigen Geschwindigkeiten vermieden werden. (Defaultwert = 10m)
28. `<MobilEnable>` Das Spurwechselmodell MOBIL wird hiermit aktiviert (1) / deaktiviert (0). (Voreinstellung: aktiviertes Spurwechselmodell).
29. `Reserved` Dieser Parameter ist reserviert und es sollte immer der Wert 0 verwendet werden.
30. `<MinScanTHW>` Der Sekundenabstand der von einem Fahrzeug aus mindestens nach vorne gescannt wird. (Defaultwert = 3sec)

Das Verhaltensmodell wechselt die Fahrspur nur dann, wenn es eine befahrbare Spur in die gleiche Richtung findet.

Dies wird in Streckenabschnitten des Typs `Course` in dem Querschnittsprofil definiert (siehe auch *Referenz Datenbasis SCNX* → *Streckentyp Course / Querschnittsprofil*). In Streckenabschnitten des Typs `Area2` wird nach benachbarten Spuren gesucht (siehe hierzu *Handbuch SILABAEEdit*).

Beispiel für die Definition eines Verkehrsteilnehmers mit einem Modell nach Wiedemann:

```
VehicleX WDMVehicle
{
  Vehicle = PKW;
  Position = (Course1, 60, 0);
  ID = 10;
  BehaviourScheme = WDM_Standard;
  Flowpoints =
  {
    # Vehicle should follow right heading on the road
    (el, 380, 1, SimCar, Activate),
    (alar, 17, 0, Traffic, Right, 1.0),
    (albr, 350, 1, SimCar, Deactivate)
  };
  TimeStep = 500;
  # TargetSpeed is chosen between 27.8 and 30.9
  TargetSpeed = 27.8...30.9;
  # Definition of StartSpeed
  StartSpeed = 15.0;
  # p: politeness factor
  p = 0.3;
  # athres: Threshold for lane changing
  athres = 0.2;
  # b_save: maximum safe deceleration
  bsave = 4.0;
  # Following the german rule to keep on the right side of the road
  BiasRight = 0.1;
  # distance for changing lane
  LaneChangeDistance = 10.0;
  # Deactivating MOBIL model for lane changing
  MOBILEnable = 0;
};
```

5.3.6.8 Verhaltensmodell: EXIDM

Das EXIDM Modell basiert auf dem Intelligent-Driver Model (IDM⁵) in Kombination mit dem MOBIL Spurwechselmodell. Im einfach gehaltenen IDM Modell gibt es im Gegensatz zum Wiedemann-Modell (siehe 5.3.6.7) keine diskreten Fahrdynamikbereiche und damit keine verschachtelten Wenn-Dann-Bedingungen. Die Beschleunigung des Fahrzeuges hängt vielmehr direkt von der Position, Geschwindigkeit und Beschleunigung des Vorderfahrzeuges ab.

In Klammern sind die Standardwerte für die Modellparameter angegeben.

Parameter im Tupel

1. `<TargetSpeed/StartSpeed>` Start- und Zielgeschwindigkeit (27.8 m/s) (entspricht IDM Parameter v_0) [general]
2. `<TargetSpeedMul>` Geschwindigkeitsmultiplikator für die rechte Spur (1.0) [general]
3. `<THWF>` Vorausschau (2.0 s) (entspricht IDM Parameter T) [general]
4. `<a>` Maximale Beschleunigung (1.0 m/s²) mit der der Verkehrsteilnehmer beschleunigt wird. [entspricht IDM Parameter a]
5. `<b>` Komfortable Bremsverzögerung (3.0 m/s²) (entspricht IDM Parameter b). Dies ist die maximale Verzögerung mit der auf ein stehendes Hindernis gebremst wird.
6. `<MinDistF>` Mindestabstand zum Vorderfahrzeug (1.5 m), der beim Stillstand eingehalten wird. Während des Fahrens wird der Sekundenabstand `<THWF>` zuzüglich dieses Abstands eingehalten (entspricht IDM Parameter s_0). [general]
7. `<delta>` Beschleunigungsexponent des IDM Modells (4.0) Dieser Parameter steuert den Beschleunigungsverlauf auf die jeweilige Zielgeschwindigkeit. Größere Werte führen zu höherer Beschleunigung auf die Zielgeschwindigkeit, kleinere Werte zu geringeren Beschleunigungen.
8. `<p>` Das Spurwechselmodell MOBIL besitzt einen Höflichkeitsfaktor `<p>`. Dieser Faktor beschreibt den Egoismus des Modells beim Spurwechsel. Der Wert 0 spezifiziert ein rein egoistisches Modell, d.h. wenn das Fahrzeug nach einem Spurwechsel höher beschleunigen kann als vorher wird ein Wechselwunsch geäußert. Die möglicherweise dadurch verursachten negativen Beschleunigungen anderer Verkehrsteilnehmer werden nicht berücksichtigt. Ein

⁵ Siehe M. Treiber and D. Helbing, [Realistische Mikrosimulation von Straßenverkehr mit einem einfachen Modell](#), 16. Symposium "Simulationstechnik ASIM 2002" Rostock, 10.09 -13.09.2002, edited by Djamshid Tavangarian and Rolf Grützner pp. 514--520.

- Wert größer gleich 1 spezifiziert ein sehr altruistisches Verhalten d.h. der Fahrer vermeidet auch negative Beschleunigungen bei anderen Verkehrsteilnehmern und verzichtet dafür auf eigene Beschleunigungsvorteile. Werte zwischen 0 und 0.6 werden in der Literatur als realistisches Verhalten bezeichnet. (Defaultwert = 0.3)
9. *<athres>* Spurwechselschranke, die einen Spurwechsel eingrenzt. Das Spurwechselmodell MOBIL vergleicht die Beschleunigungen nach einem (fiktiven) Spurwechsel mit den aktuellen Beschleunigungen (ohne Spurwechsel). Der Parameter a_{thres} ist ein Offset, der negativ für einen Wechsel gewertet wird und beschreibt somit, dass ein Spurwechsel erst ab einem gewissen Beschleunigungsvorteil (der größer als *<athres>* sein muss) erfolgt. Dieser Wert sollte im Normalfall zwischen 0.1m/s^2 und 1m/s^2 gewählt werden. (Defaultwert = 0.2m/s^2)
10. *<bsave>* Maximal sichere Verzögerung in m/s^2 für ein nachfolgendes Fahrzeug. Dieses übergeordnete Sicherheitskriterium stellt sicher, dass ein nachfolgendes Fahrzeug keinesfalls mit einer höheren negativen Beschleunigung als $-\text{<bsave>}$ abbremsen muss. Dieser Wert sollte im Normalfall zwischen 4m/s^2 und 2m/s^2 gewählt werden. (Defaultwert = 4.0m/s^2)
11. *<gapMin>* Mindestabstand [m] den ein Fahrzeug nach einem Spurwechsel zum neuen Vorderfahrzeug und Hinterfahrzeug einhalten muss. Ist dies nicht gewährleistet wird kein Spurwechsel durchgeführt. (Defaultwert = 2m).
12. *<BiasRight>* Beschleunigungsschranke für rechte Spur. Das Spurwechselmodell MOBIL vergleicht die Beschleunigungen nach einem (fiktiven) Spurwechsel nach rechts mit den aktuellen Beschleunigungen im nächsten Zeitschritt. Der Parameter *<biasright>* ist ein Offset, der positiv für die rechte Spur gewertet wird (bei Spurwechselberechnungen nach rechts) und beschreibt somit einen Vorteil für die rechte Spur. Daraus resultiert ein entsprechendes Rechtsfahrgebot. Dieser Wert sollte im Normalfall zwischen 0.1m/s^2 und 2m/s^2 gewählt werden. (Defaultwert = 0.1m/s^2)
13. *<LaneChangeDistance>*Weg in dem ein Spurwechsel durch den Verkehrsteilnehmer durchgeführt wird (Defaultwert = 10 m). [general]

14. **<LaneChangeDeadTime>** Wartezeit nach einem Spurwechsel (Defaultwert = 5.0s). Nach einem Spurwechsel muss diese Zeit verstreichen bevor erneut ein Spurwechsel erfolgen kann. [general]

Weitere für dieses Modell gültige allgemeine Parameter (siehe Abschnitt 5.3.3):
 AimPointDist, ClearTrafficTime, ClearTrafficVy, ClearTrafficVx

Das Verhaltensmodell wechselt die Fahrspur nur dann, wenn es eine benachbarte Spur findet, auf die gewechselt werden darf. Dabei werden die Fahrbahnmarkierungen berücksichtigt. Durchgehende Fahrbahnmarkierungen werden als Wechselverbot interpretiert, fehlende sowie gestrichelte Markierungen werden als benachbarte Spuren interpretiert, zwischen denen das Modell wechseln darf. Die Markierungen werden in dem Querschnittsprofil des Courses definiert (siehe auch *Referenz Datenbasis SCNX* → *Streckentyp Course / Querschnittsprofil*). Wird als zweiter Parameter (Geschwindigkeitsmultiplikator) ein Wert kleiner 1.0 angegeben, so erreicht das Modell die Zielgeschwindigkeit nur auf der linken Spur. Auf den rechten Spuren (wenn also eine befahrbare Spur links von der aktuell befahrenen Spur gefunden wird) wird die Zielgeschwindigkeit mit dem Multiplikator verrechnet.

5.3.6.9 Verhaltensmodell: HazardFree

Das Modell **HazardFree** ist eine Kombination von Modellen zur HazardAvoidance und FreeDriving basierend auf dem IDM-Modell (siehe auch Verhaltensmodells EXIDM in Abschnitt 5.3.6.7). Das Modell versucht, eine Zielgeschwindigkeit zu erreichen, ist aber auch in der Lage, realistisch hinter einem vorausfahrenden Fahrzeug herzufahren. Dabei erfolgt der Übergang der Beschleunigungen zwischen Folgefahren und freiem Fahren fließend.

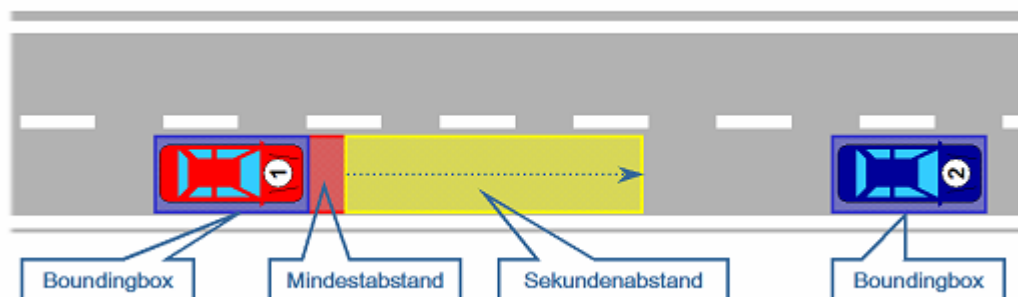


Abbildung 7 Vorausschau der HazardFree

Parameter im Tupel

- | | |
|-------------------------------|---|
| 1. <StartSpeed> | Geschwindigkeit bei der Aktivierung des Fahrzeugs (in m/s) [general]. |
| 2. <TargetSpeed> | Zielgeschwindigkeit während der Fahrt (in m/s) [general]. |
| 3. <MinDistF> | Abstand, der nicht unterschritten werden darf (in m) [general] |
| 4. <THWF> | Sekundenabstand zum vorausfahrenden |

- | | |
|-------------------------------------|--|
| | Fahrzeug (in s) der während des Fahrens auf den Stillstandabstand addiert wird. [general] |
| 5. <i><MaxAcceleration></i> | Maximale Beschleunigung des Fahrzeugs in m/s^2 . (Default = 20m/s^2) |
| 6. <i><MaxDeceleration></i> | Maximale Verzögerung des Fahrzeugs in m/s^2 . Ist dieser Wert zu groß, dann kann der gewünschte Sicherheitsabstand unter Umständen nicht eingehalten werden. (Default = -50m/s^2) |
| 7. <i><acceleration type></i> | Die Art wie das Fahrzeug auf die Wunschgeschwindigkeit beschleunigt. Höhere Werte führen zu einer stärkeren Beschleunigung. (Default = 1.5). |
| 8. <i><deceleration type></i> | Die Art wie das Fahrzeug auf ein Hindernis bremst. Höhere Werte führen zu einem früheren Abbremsen mit geringerer Maximalverzögerung. (Default = 4) |
| 9. <i><delta></i> | Beschleunigungsexponent des IDM Modells (4.0). Siehe auch Beschreibung des Modells EXIDM. |

Abstandsregelung:

Die Abstandsregelung erfolgt über eine Verzögerung, die sich aus Differenz zwischen der eigenen und der Geschwindigkeit des vorausfahrenden Fahrzeugs ergibt.

Flowpoints:**AimPointTimeDistance,<THWF>**

Setzt den Sekundenabstand neu.

HazardFree ,<StartSpeed>, <TargetSpeed>, <MinDistF>, <THWF>, <MaxAcceleration>, <MaxDeceleration>

Setzt die Parameter der HazardFree neu.

Beispiel:

```
BehaviourScheme HazardFree
{
    Behaviourmachine =
    {
        (HazardFree, 33, 33, 10, 2, 8, -10, 1.5, 4, 4)
    };
};
```

5.3.7 Obsolete Verhaltensmodelle

In diesem Abschnitt werden obsolete Verhaltensmodelle beschrieben, die für die Verwendung in SILAB nicht mehr empfohlen werden. Beide genannten Modelle sind optimiert für Streckenabschnitte des Typs Course und sind aus Kompatibilitätsgründen noch im Lieferumfang von SILAB enthalten.

5.3.7.1 Verhaltensmodell: HazardAvoidance

Das Verhaltensmodell **HazardAvoidance** verhindert das Auffahren auf einen vorausfahrenden Verkehrsteilnehmer. Dazu wird als Parameter der gewünschte Abstand in Sekunden angegeben.

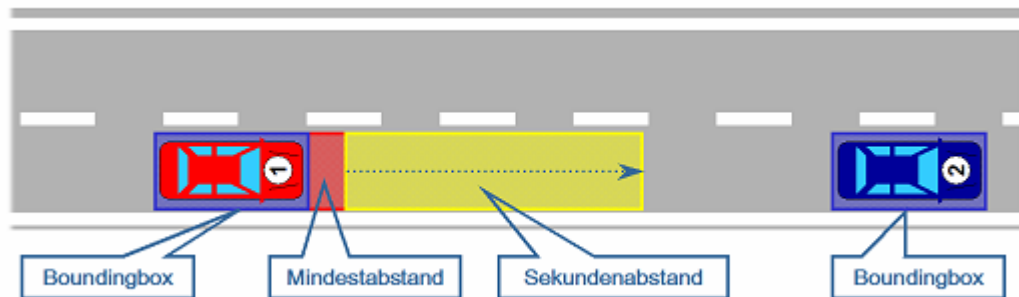


Abbildung 8 Vorausschau der Hazardavoidance

Parameter im Tupel

1. `<THWF>` Sekundenabstand zum vorausfahrenden Fahrzeug (in s) [general]
2. `<MinDistF>` Abstand, der nicht unterschritten werden darf (in m) [general]
3. `<maxdeceleration>` Maximale Verzögerung (in m/s^2)
4. `<maxacceleration>` Maximale Beschleunigung (in m/s^2)
5. `<multiplier>` Multiplikator für den Sekundenabstand im Falle eines Spurwechsels, der mit dem Modell LaneChangeAvoidance ausgelöst wurde.

Die Abstandsregelung der **HazardAvoidance** erfolgt über eine Verzögerung, die sich aus Differenz zwischen der eigenen und der Geschwindigkeit des vorausfahrenden Fahrzeuges ergibt. Der Abstand `<MinDistF>` wird in Sekunden angegeben.

Flowpoints:

HazardAvoidance, `<THWF>`, `<MinDistF>`, `<maxdeceleration>`, `<maxacceleration>`, `<multiplier>`

Setzt die Parameter der HazardAvoidance neu.

5.3.7.2 Verhaltensmodell: FreeDriving

Mit dem FreeDriving-Modell erhält der Verkehrsteilnehmer ein Wunschverhalten für die Geschwindigkeit.

Parameter im Tupel

1. `<StartSpeed>` Geschwindigkeit bei der Aktivierung des Verkehrsteilnehmers (in m/s) (Default = 27.7) [general].
2. `<TargetSpeed>` Zielgeschwindigkeit während der Fahrt (in m/s) (Default = 27.7) [general].

3. *<acceleration>* Maximale Beschleunigung, mit der die Wunschgeschwindigkeit erreicht werden soll in m/s^2 (Default = 5).
4. *<deceleration>* Maximale Verzögerung um die Wunschgeschwindigkeit durch Bremsen zu erreichen in m/s^2 (Default = -5).

Bei den Parametern Start- / Wunschgeschwindigkeit kann auch ein Intervall angegeben werden. Die Geschwindigkeit wird dann zufällig innerhalb des Intervalls ausgewählt.

Wird eine Geschwindigkeitsbegrenzung durch den Flowpoint **Speedlimit** gesetzt, dann wird diese Geschwindigkeitsbegrenzung eingehalten. Nach der Aufhebung durch den Flowpoint **EndSpeedlimit** wird wieder die ursprüngliche Wunschgeschwindigkeit erreicht.

Darüber hinaus gibt es noch die Möglichkeit, die Wunschgeschwindigkeit mit der Geschwindigkeit des EGO - Fahrzeuges über die Flowpoints **Synchronize** und **SynchronizeRel** zu synchronisieren.

Mit **Synchronize** wird die Wunschgeschwindigkeit auf die Geschwindigkeit des EGO - Fahrzeuges plus den angegebenen Geschwindigkeitsunterschied gesetzt, wenn der Abstand des EGO - Fahrzeugs die angegebene Distanz unterschreitet. Der Geschwindigkeitsunterschied wird bei **Synchronize** in m/s und bei **SynchronizeRel** in Prozent angegeben.

Nach dem Überfahren des Flowpoints **EndSynchronize** wird nicht mehr synchronisiert.

Flowpoints

Targetspeed, *<speed [m/s]>*, *<acceleration [m/s²]>*, *<deceleration [m/s²]>*
Setzen einer neuen Wunschgeschwindigkeit.

Speedlimit, *<speed [m/s]>*, *<distance [m]>*
Geschwindigkeitsbegrenzung auf die angegebene *Geschwindigkeit*. Diese Geschwindigkeit wird in der angegebenen *Distanz* zum Flowpoint erreicht.

EndSpeedlimit

Aufheben des Speedlimits. Der Verkehrsteilnehmer wird erneut die Wunschgeschwindigkeit annehmen.

Synchronize, *<distance [m]>*, *<speed difference [m/s]>*, *<deceleration [m/s²]>*
Synchronisation auf den *Geschwindigkeitsunterschied* in m/s mit der optional übergebenen Verzögerung

SynchronizeRel, *<distance in m>*, *< speed difference [%]>*, *<deceleration [m/s²]>*
Synchronisation auf den *Geschwindigkeitsunterschied* in % mit der optional übergebenen Verzögerung

EndSynchronize

Beendet die Synchronisation.

5.4 Generierung von Messwerten

Zur Datenaufzeichnung der Verkehrsteilnehmer stehen mehrere Möglichkeiten zur Verfügung. Die nachfolgende Tabelle verweist auf eine Reihe von DPUs und beschreibt deren Einsatzmöglichkeiten. Für eine detaillierte Beschreibung dieser DPUs wird auf die Einzeldokumente der DPUs verwiesen.

DPUName	Einsatzmöglichkeit
DPUTRFDataSingle	Diese DPU filtert aus simulierten Verkehrsteilnehmer gezielt einzelne Fahrzeuge mit bestimmter UserID bzw. TRFID heraus. Zu diesen Einzelfahrzeugen liefert diese DPU detaillierte Informationen. Diese DPU eignet sich um die Daten bestimmter einzelner Partner in definierten Situationen abzufragen.
DPUTRFTarget	Diese DPU liefert Informationen zu den nächsten Fahrzeugen in einem bestimmten Umkreis um den Fahrer (euklidischer Abstand). Diese DPU eignet sich für eine generelle Aufzeichnung der umgebenden Fahrzeuge des EGO-Fahrzeugs.
DPUTRFDataFull	Die DPUTRFDataFull stellt in Courses Informationen über die umgebenden Fahrzeuge um das EGO-Fahrzeug zur Verfügung. Die Fahrzeuge werden dabei nach Spurnachbarschaften und Fahrtrichtung kategorisiert. Aus diesem Grund ist ein Einsatz in Areas nicht sinnvoll.
DPUTRFDataSmall	Die DPUTRFDataSmall arbeitet analog zur DPUTRFDataFull, jedoch mit einer reduzierten Klassifizierung.
DPUATBTRFSensor	Diese DPU ist speziell für die Aufzeichnung der umgebenden Vorder- und Hinterfahrzeug (klassifiziert nach deren relativer Position) in Areas sowie in Courses entworfen.